Adjunctive Vasopressor Selection Decision Model

Code

AUTHOR
Seth Bauer

PUBLISHED
2026-10-09

1 Overview

The objective of this work is to create a decision model to augment adjunctive vasopressor selection in adults with septic shock. Patient encounters meeting clinical criteria for septic shock, whether Centers for Disease Control and Prevention Adult Sepsis Event (CDC ASE) or Third International Consensus Definition for Sepsis and Septic Shock (Sepsis-3) criteria, were previously identified. We will first divide the cohort of patients with septic shock from the Cleveland Clinic Ohio region into sub-cohorts based on second-line adjunctive vasopressor received (such as vasopressin or epinephrine). Using these sub-cohorts, we will create and calibrate separate treatment-specific clinical prediction models for adjunctive vasopressor response, defined as achievement of a mean arterial pressure (MAP) ≥65 mm Hg and a decrease in norepinephrine-equivalent vasopressor dose at six hours after adjunct response. The models will be derived using candidate predictors of patient characteristics, treatment context factors, and shock characteristics measured at first adjunctive vasopressor initiation. We will build and optimize prediction models within each sub-cohort with regression-based and machine learning algorithms for binary categorical outcomes. The prediction model with the best performance (based on predictive discrimination, calibration, decision curve analysis, and implementation readiness) in each sub-cohort will be considered the final model and carried forward.

We will then define for each patient the decision model-recommended adjunctive vasopressor based on the medication with the highest predicted likelihood of response for a given patient. To validate the model, we will conduct a discordance analysis using patients with septic shock from the Cleveland Clinic Florida region. For each patient, the model-recommended second-line adjunctive vasopressor therapy will be compared against the observed decision made in the standard process of care. Concordance will be defined as a patient given the second-line adjunctive vasopressor consistent with the decision model recommendation. Discordance will be defined as a patient given an adjunctive vasopressor differing from the medication recommended by the decision model. Each patient will be defined as receiving concordant or discordant therapy, creating counterfactual controls for model recommendations. The global effect of discordance will be defined as the odds ratio (estimated with logistic regression) for adjunctive vasopressor response comparing discordant decisions to concordant decisions.

2 Load Packages and Setup

▼ Code

```
knitr::opts_chunk$set(comment = NA)
options(width = 70)

library(broom); library(conflicted); library(DataExplorer); library(dcurves);
library(emmeans); library(fastshap); library(finetune); library(flextable);
library(forester); library(ggalluvial); library(GGally); library(ggdist);
library(ggh4x); library(ggrepel); library(ggsci); library(gt);
library(gtsummary); library(ivs); library(janitor); library(kableExtra);
library(naniar); library(nnet); library(officer);library(patchwork);
library(pmcalibration); library(pmsampsize); library(renv); library(report);
library(riskRegression); library(rms); library(scales); library(shapviz);
library(simputation); library(splines); library(splines2); library(survival);
library(survminer); library(vip); library(vroom); library(tidytext);
library(tidymodels); library(tidyverse)

theme_set(theme_bw())

conflict_prefer("count", "dplyr"); conflict_prefer("select", "dplyr");
conflict_prefer("filter", "dplyr"); conflict_prefer("recode", "dplyr");
conflict_prefer("summarize", "dplyr"); conflict_prefer("min", "base");
conflict_prefer("max", "base"); conflict_prefer("data.frame", "base");
conflict_prefer("sum", "base"); conflict_prefer("mean", "base");
conflict_prefer("describe", "rms"); conflict_prefer("Predict", "rms");
conflict_prefer("font", "flextable"); conflicts_prefer(vip::vi);
tidymodels_prefer()
```

3 Load Data

Read in the data from .Rds files created in other Quarto documents. Each source .Rds file utilized multiple input files or queries, which will not be iterated here. See Bauer 2025 and open source Quarto documents for development of the patient cohort and an initial norepinephrine failure prediction model, which contain the process steps to build the patient cohorts and .Rds files. Also load a file containing a lookup table of emergency department and hospital characteristics. The data sources for this document are:

TIBBLE	FILE (.RDS)	SOURCE QUARTO DOCUMENT (.QMD)
ne_incl	ne_failure_included	ne_failure_model
pressors_courses	vasopressors_courses_updated	ne_failure_model
dbt	vasopressors_courses_dbt	vasopressors
pressors_doses	vasopressors_long	vasopressors

TIBBLE	FILE (.RDS)	SOURCE QUARTO DOCUMENT (.QMD)
map	map	cv_sofa
hosp_char	Hospital_characteristics.csv	none (raw file)
body_weight	weight	weight
adt	adt_wide	adt
abx	abx_sep3	antimicrobials
surg	surgeries	surgeries
encounter_key	encounter_key.txt	none (raw file)
map	map	cv_sofa
scr	encounter_scr	kidney_function
rrt	encounter_rrt	kidney_function
esrd	esrd	kidney
platelets	plts	platelets
bili	bili	bilirubin
gcs	sofa_cns	cns_sofa
sed	sedatives	sedative_antipsychotic
antipsy	antipsychotics	sedative_antipsychotic
oxygenation	sofa_resp	resp_sofa-2
mv	vent_all	vent

TIBBLE	FILE (.RDS)	SOURCE QUARTO DOCUMENT (.QMD)
adv_resp_support	adv_resp_support	resp_sofa-2
pH	pH	arterial_pH
temperature	temperature	vitals
heart_rate	heart_rate	vitals
resp_rate	resp_rate	vitals
lactate	lactate_all	lactate
fluids	fluid_bolus	fluid_bolus
steroids	steroids	steroids
lvef	lvef	echo_parameters

Some .Rds files include all observations in the raw queries, but in this work only observations from patients with septic shock receiving norepinephrine as the first vasopressor (among other criteria) are of interest. Therefore, as applicable, the call below filters observations to those from eligible cases. This call also selects vectors (columns) of interest for this work, adjusts vector structure as needed, and drops unused levels of factor variables.

▼ Code

```
ne_incl <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/ne_failure_included.Rds")

pressors_courses <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/vasopressors_courses_updated.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id) |>
  droplevels()

dbt <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/vasopressors_courses_dbt.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id,
         medication == "dobutamine") |>
  droplevels()

pressors_doses <- read_rds(
  "/proj/DaltonLab/projects/sbauer/rds_files/vasopressors_long.Rds"
```

```
  ) |>
  filter(encounter_id %in% ne_incl$encounter_id)

map <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/map.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

body_weight <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/weight.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

adt <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/adt_wide.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id) |>
  droplevels()

hosp_char <-
  vroom("/proj/DaltonLab/projects/sbauer/data_raw/full_cohort/Hospital_characteristics.csv",
        col_types = c(hospital = "f", hosp_beds = "f", hosp_beds_cat = "f",
                      hosp_teaching_status = "f", hosp_state = "f"))

abx <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/abx_sep3.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

surg <-read_rds("/proj/DaltonLab/projects/sbauer/rds_files/surgeries.Rds")

encounter_key <-
  vroom("/proj/DaltonLab/projects/sbauer/data_raw/full_cohort/encounter_key.txt",
        col_types = c(EncounterID = "c")) |>
  clean_names()

scr <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/encounter_scr.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

rrt <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/encounter_rrt.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

esrd <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/esrd.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

platelets <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/plts.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

bili <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/bili.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

gcs <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/sofa_cns.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

sed <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/sedatives.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)
```

```r
antipsy <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/antipsychotics.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

oxygenation <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/sofa_resp.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

adv_resp <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/adv_resp_support.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id) |>
  rename(adv_resp_time = time)

mv <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/vent_all.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id) |>
  rename(mv_time = time, mv_date = date)

pH <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/pH.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

temperature <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/temperature.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

heart_rate <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/heart_rate.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

resp_rate <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/resp_rate.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

lactate <- read_rds("/proj/DaltonLab/projects/sbauer/rds_files/lactate_all.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

fluids <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/fluid_bolus.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

steroids <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/steroids.Rds") |>
  filter(encounter_id %in% ne_incl$encounter_id)

lvef <-
  read_rds("/proj/DaltonLab/projects/sbauer/rds_files/lvef.Rds")
```

4 Verify Tibble Structure

Next, check the structure of the imported tibbles to ensure the vectors are in the desired format for modeling. Annotations are only included below if the structure is not in the intended format.

▼ Code

```r
str(ne_incl, vec.len = 0)
```

```
tibble [25,982 × 92] (S3: tbl_df/tbl/data.frame)
 $ encounter_id                     : chr [1:25982]  ...
 $ event                            : num [1:25982] NULL ...
 $ event_type                       : Factor w/ 2 levels "0-no_event","1-failure": NULL ...
 $ surv_hours                       : num [1:25982] NULL ...
 $ any_adj                          : Factor w/ 2 levels "No Adjunct","Adjunct": NULL ...
 $ hours_to_first_adj               : num [1:25982] NULL ...
 $ mort_hosp                        : chr [1:25982]  ...
 $ hours_to_death                   : num [1:25982] NULL ...
 $ pressors_range                   : iv<dttm> [1:25982] [2014-03-26 13:05:00, 2014-03-28
11:34:01...
 $ pressors_dur_hours               : num [1:25982] NULL ...
 $ admission_time                   : POSIXct[1:25982], format:  ...
 $ discharge_time                   : POSIXct[1:25982], format:  ...
 $ criteria_met                     : Factor w/ 3 levels "Both","ASE only",..: NULL ...
 $ age                              : num [1:25982] NULL ...
 $ sex                              : Factor w/ 2 levels "Female","Male": NULL ...
 $ race_eth                         : Factor w/ 6 levels "Asian","Black",..: NULL ...
 $ admit_year                       : Factor w/ 13 levels "2012","2013",..: NULL ...
 $ sepsis_onset_hosp_beds_cat       : Factor w/ 3 levels "<200","200-499",..: NULL ...
 $ sepsis_onset_hosp_teaching_status: Factor w/ 2 levels "Non-teaching",..: NULL ...
 $ sepsis_onset_hosp_state          : Factor w/ 2 levels "Ohio","Florida": NULL ...
 $ comm_onset                       : Factor w/ 2 levels "Community","Hospital": NULL ...
 $ max_sofa                         : num [1:25982] NULL ...
 $ tot_sofa                         : num [1:25982] NULL ...
 $ surg_service_30d_prior_to_dc     : Factor w/ 31 levels "Bariatrics","Cardiology
discipline",..: NULL ...
 $ prior_surg_service               : Factor w/ 29 levels "Bariatrics","Cardiology
discipline",..: NULL ...
 $ sepsis_onset_unit_type           : Factor w/ 3 levels "ED","Ward","ICU": NULL ...
 $ bcx_collected                    : Factor w/ 2 levels "0","1": NULL ...
 $ abx_days                         : num [1:25982] NULL ...
 $ shock_onset_tot_sofa             : num [1:25982] NULL ...
 $ mv_before_shock                  : Factor w/ 2 levels "MV Before Shock",..: NULL ...
 $ sepsis_onset_time                : POSIXct[1:25982], format:  ...
 $ sepsis_window_start              : POSIXct[1:25982], format:  ...
 $ sepsis_window_end                : POSIXct[1:25982], format:  ...
 $ sepsis_onset_hospital            : Factor w/ 24 levels "Akron","Ashtabula",..: NULL ...
 $ sepsis_onset_icu_group           : Factor w/ 38 levels "ACMC ICU","AK CVICU",..: NULL ...
 $ sepsis_onset_covid               : Factor w/ 2 levels "Not Positive",..: NULL ...
 $ shock_onset_unit_type            : Factor w/ 3 levels "ED","Ward","ICU": NULL ...
 $ shock_onset_hospital             : Factor w/ 25 levels "Akron","Ashtabula",..: NULL ...
 $ shock_onset_icu_group            : Factor w/ 38 levels "ACMC ICU","AK CVICU",..: NULL ...
 $ shock_onset_time                 : POSIXct[1:25982], format:  ...
 $ wt_time                          : POSIXct[1:25982], format:  ...
 $ wt_kg                            : num [1:25982] NULL ...
 $ cerebrovascular_disease          : Factor w/ 2 levels "No","Yes": NULL ...
 $ hypertension                     : Factor w/ 2 levels "No","Yes": NULL ...
```

```
 $ heart_failure                   : Factor w/ 2 levels "No","Yes": NULL ...
 $ valvular_disease                : Factor w/ 2 levels "No","Yes": NULL ...
 $ chronic_pulmonary_disease       : Factor w/ 2 levels "No","Yes": NULL ...
 $ pulmonary_circulation_disease   : Factor w/ 2 levels "No","Yes": NULL ...
 $ liver_disease                   : Factor w/ 2 levels "No","Yes": NULL ...
 $ kidney_disease                  : Factor w/ 2 levels "No","Yes": NULL ...
 $ malignancy                      : Factor w/ 2 levels "No","Yes": NULL ...
 $ diabetes                        : Factor w/ 2 levels "No","Yes": NULL ...
 $ obesity                         : Factor w/ 2 levels "No","Yes": NULL ...
 $ alcohol_abuse                   : Factor w/ 2 levels "No","Yes": NULL ...
 $ elixhauser_index                : num [1:25982] NULL ...
 $ shock_onset_tx_unit             : Factor w/ 4 levels "ED/Ward","Medical/Cardiac ICU",..: NULL
...
 $ shock_onset_hosp_beds_cat       : Factor w/ 3 levels "≥500","200-499",..: NULL ...
 $ shock_onset_hosp_teaching_status : Factor w/ 2 levels "Teaching","Non-teaching": NULL ...
 $ shock_onset_hosp_state          : Factor w/ 2 levels "Ohio","Florida": NULL ...
 $ ssc_era                         : Factor w/ 4 levels "SSC 2008","SSC 2012",..: NULL ...
 $ prior_abx                       : Factor w/ 2 levels "Antibiotics",..: NULL ...
 $ prior_surg                      : Factor w/ 2 levels "No Surgery","Surgery": NULL ...
 $ rrt_before_shock                : Factor w/ 2 levels "No RRT Before Shock",..: NULL ...
 $ aki                             : Factor w/ 5 levels "No AKI","AKI Stage 1",..: NULL ...
 $ shock_onset_minus_24h           : POSIXct[1:25982], format:  ...
 $ shock_onset_scr                 : num [1:25982] NULL ...
 $ shock_onset_platelets           : num [1:25982] NULL ...
 $ shock_onset_bili                : num [1:25982] NULL ...
 $ shock_onset_minus_6h            : POSIXct[1:25982], format:  ...
 $ shock_onset_gcs                 : num [1:25982] NULL ...
 $ shock_onset_pf                  : num [1:25982] NULL ...
 $ shock_onset_pH                  : num [1:25982] NULL ...
 $ shock_onset_temp_c              : num [1:25982] NULL ...
 $ shock_onset_heart_rate          : num [1:25982] NULL ...
 $ shock_onset_resp_rate           : num [1:25982] NULL ...
 $ shock_onset_map                 : num [1:25982] NULL ...
 $ shock_onset_map_min             : num [1:25982] NULL ...
 $ shock_onset_minus_12h           : POSIXct[1:25982], format:  ...
 $ shock_onset_lactate             : num [1:25982] NULL ...
 $ shock_onset_lactate_max         : num [1:25982] NULL ...
 $ fluid_mL_12h_prior_shock        : num [1:25982] NULL ...
 $ fluid_mL_kg_12h_prior_shock     : num [1:25982] NULL ...
 $ shock_onset_steroid             : Factor w/ 2 levels "No Steroids",..: NULL ...
 $ first_adj                       : Factor w/ 18 levels "vasopressin",..: NULL ...
 $ death_time                      : POSIXct[1:25982], format:  ...
 $ event_3_level                   : Factor w/ 4 levels "-99","0","1",..: NULL ...
 $ event_3_level_type              : Factor w/ 4 levels "-99-no_known_event_unknown_death",..:
NULL ...
 $ surv_hours_3_level              : num [1:25982] NULL ...
 $ surv_obj                        : 'Surv' num [1:25982, 1:2] NULL ...
  ..- attr(*, "type")= chr  ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:2]  ...
```

```
 $ key_discharge_time               : POSIXct[1:25982], format:  ...
 $ ne_initial_dose_mcg_min          : num [1:25982] NULL ...
 $ ne_initial_dose_mcg_kg_min       : num [1:25982] NULL ...
```

The variables each have the intended structure, however some factor variables need re-leveled to place the intended baseline (reference) category first. This work will below in section Section 5.1.

▼ Code

```
str(pressors_courses, vec.len = 0)
```

```
tibble [25,982 × 33] (S3: tbl_df/tbl/data.frame)
 $ encounter_id        : chr [1:25982]  ...
 $ pressors_range      : iv<dttm> [1:25982] [2014-03-26 13:05:00, 2014-03-28 11:34:01...
 $ pressors_dur_hours  : num [1:25982] NULL ...
 $ first_pressor       : Factor w/ 1 level "norepinephrine": NULL ...
 $ multiple_first      : chr [1:25982]  ...
 $ norepinephrine_start: POSIXct[1:25982], format:  ...
 $ vasopressin_start   : POSIXct[1:25982], format:  ...
 $ phenylephrine_start : POSIXct[1:25982], format:  ...
 $ epinephrine_start   : POSIXct[1:25982], format:  ...
 $ dopamine_start      : POSIXct[1:25982], format:  ...
 $ norepinephrine_stop : POSIXct[1:25982], format:  ...
 $ vasopressin_stop    : POSIXct[1:25982], format:  ...
 $ phenylephrine_stop  : POSIXct[1:25982], format:  ...
 $ epinephrine_stop    : POSIXct[1:25982], format:  ...
 $ dopamine_stop       : POSIXct[1:25982], format:  ...
 $ adj_avp             : num [1:25982] NULL ...
 $ adj_epi             : num [1:25982] NULL ...
 $ adj_pe              : num [1:25982] NULL ...
 $ adj_da              : num [1:25982] NULL ...
 $ adj_ne              : num [1:25982] NULL ...
 $ time_to_adj_ne      : num [1:25982] NULL ...
 $ time_to_adj_avp     : num [1:25982] NULL ...
 $ time_to_adj_epi     : num [1:25982] NULL ...
 $ time_to_adj_pe      : num [1:25982] NULL ...
 $ time_to_adj_da      : num [1:25982] NULL ...
 $ adj_ne_order        : num [1:25982] NULL ...
 $ adj_avp_order       : num [1:25982] NULL ...
 $ adj_epi_order       : num [1:25982] NULL ...
 $ adj_pe_order        : num [1:25982] NULL ...
 $ adj_da_order        : num [1:25982] NULL ...
 $ any_adj             : Factor w/ 2 levels "No Adjunct","Adjunct": NULL ...
 $ first_adj           : Factor w/ 10 levels "vasopressin",..: NULL ...
 $ hours_to_first_adj  : num [1:25982] NULL ...
```

▼ Code

```
str(pressors_doses, vec.len = 0)
```

```
tibble [1,735,479 × 14] (S3: tbl_df/tbl/data.frame)
 $ encounter_id               : chr [1:1735479]  ...
 $ mar_taken_time             : POSIXct[1:1735479], format:  ...
 $ medication                 : Factor w/ 5 levels "dopamine","epinephrine",..: NULL ...
 $ titrate_action             : Factor w/ 3 levels "start","stop",..: NULL ...
 $ dose_mcg_min               : num [1:1735479] NULL ...
 $ dose_mcg_kg_min            : num [1:1735479] NULL ...
 $ dose_units_min             : num [1:1735479] NULL ...
 $ wt_kg                      : num [1:1735479] NULL ...
 $ nee_goradia                : num [1:1735479] NULL ...
 $ running_nee_goradia        : num [1:1735479] NULL ...
 $ nee_kotani                 : num [1:1735479] NULL ...
 $ running_nee_kotani         : num [1:1735479] NULL ...
 $ running_nee_goradia_mcg_min: num [1:1735479] NULL ...
 $ running_nee_kotani_mcg_min : num [1:1735479] NULL ...
```

▼ Code

```
str(map, vec.len = 0)
```

```
tibble [21,087,319 × 6] (S3: tbl_df/tbl/data.frame)
 $ encounter_id     : chr [1:21087319]  ...
 $ encounter_id_orig: chr [1:21087319]  ...
 $ patient_id       : chr [1:21087319]  ...
 $ flo_type         : chr [1:21087319]  ...
 $ map              : num [1:21087319] NULL ...
 $ map_time         : POSIXct[1:21087319], format:  ...
 - attr(*, "na.action")= 'omit' Named int [1:58722] NULL ...
  ..- attr(*, "names")= chr [1:58722]  ...
```

▼ Code

```
str(lactate, vec.len = 0)
```

```
tibble [734,505 × 6] (S3: tbl_df/tbl/data.frame)
 $ encounter_id      : chr [1:734505]  ...
 $ encounter_id_orig : chr [1:734505]  ...
 $ patient_id        : chr [1:734505]  ...
 $ lactate_taken_time: POSIXct[1:734505], format:  ...
 $ lactate_result    : num [1:734505] NULL ...
 $ lactate_type      : chr [1:734505]  ...
```

▼ Code

```
str(steroids, vec.len = 0)
```

```
tibble [218,294 × 8] (S3: tbl_df/tbl/data.frame)
 $ encounter_id        : chr [1:218294]  ...
 $ short_name          : Factor w/ 9 levels "beclomethasone",..: NULL ...
```

```
 $ epic_medication_name: chr [1:218294]  ...
 $ route               : Factor w/ 13 levels "CORPAK","G-TUBE",..: NULL ...
 $ mar_taken_time      : POSIXct[1:218294], format:  ...
 $ mar_sig             : chr [1:218294]  ...
 $ mar_dose_units      : Factor w/ 4 levels "g","mg","mg/hr",..: NULL ...
 $ mar_action          : Factor w/ 9 levels "Dual Sign Off",..: NULL ...
```

▼ Code

```
str(fluids, vec.len = 0)
```

```
tibble [158,944 × 9] (S3: tbl_df/tbl/data.frame)
 $ encounter_id        : chr [1:158944]  ...
 $ short_name          : chr [1:158944]  ...
 $ epic_medication_name: chr [1:158944]  ...
 $ route               : Factor w/ 1 level "INTRAVENOUS": NULL ...
 $ mar_taken_time      : POSIXct[1:158944], format:  ...
 $ mar_sig             : num [1:158944] NULL ...
 $ mar_dose_units      : Factor w/ 2 levels "g","mL": NULL ...
 $ mar_action          : Factor w/ 16 levels ".Self Admin",..: NULL ...
 $ vol_mL              : num [1:158944] NULL ...
```

▼ Code

```
str(scr, vec.len = 0)
```

```
tibble [520,441 × 4] (S3: tbl_df/tbl/data.frame)
 $ encounter_id  : chr [1:520441]  ...
 $ scr_taken_time: POSIXct[1:520441], format:  ...
 $ scr_result    : num [1:520441] NULL ...
 $ pta_scr       : num [1:520441] NULL ...
```

▼ Code

```
str(rrt, vec.len = 0)
```

```
tibble [89,406 × 4] (S3: tbl_df/tbl/data.frame)
 $ encounter_id  : chr [1:89406]  ...
 $ rrt_taken_time: POSIXct[1:89406], format:  ...
 $ rrt_type      : Factor w/ 3 levels "CRRT","IHD","PD": NULL ...
 $ new_rrt       : Factor w/ 2 levels "No","Yes": NULL ...
```

All tibbles have vectors in the structure desired for modeling.

Next, confirm that the `encounter_id`s are the same between `ne_incl` and `pressors_courses` as well as between `ne_incl` and `pressors_doses`.

▼ Code

```
setequal(ne_incl$encounter_id, pressors_courses$encounter_id)
```

```
[1] TRUE
```

▼ Code

```
setequal(ne_incl$encounter_id, pressors_doses$encounter_id)
```

```
[1] TRUE
```

5 Data Creation

Next, select vectors already in the tibble that will be used for modeling. Then identify and create vectors for exposure risk, covariates, and outcomes that will be used for modeling.

5.1 Select Vectors

Create a new tibble (`adj`) filtered to patients in the `ne_incl` tibble who received an adjunctive vasopressor. In the same call, select out columns (vectors) that will not be used for joining data between tibbles or for modeling, change the column ordering, rename one column, drop unused factor levels, and adjust the levels for some categorical variables.

▼ Code

```
adj <- ne_incl |>
  filter(event_3_level_type == "1-adjunct") |>
  select(-c(event, event_type, surv_hours, any_adj, surv_obj,
            surv_hours_3_level, hours_to_death, wt_time, wt_kg, prior_abx,
            mv_before_shock, surg_service_30d_prior_to_dc, prior_surg_service,
            shock_onset_minus_6h, shock_onset_minus_12h, shock_onset_minus_24h,
            event_3_level, event_3_level_type
            )
         ) |>
  rename(sepsis_onset_tot_sofa = tot_sofa) |>
  droplevels() |>
  mutate(criteria_met = fct_relevel(criteria_met, "Both", "ASE only"),
         race_eth = fct_recode(race_eth,
                               "Other, non-Hispanic" = "Asian",
                               "Black, non-Hispanic" = "Black",
                               "Hispanic or Latino" = "Hispanic or Latino",
                               "Multiracial, non-Hispanic" = "Multiracial",
                               "Other, non-Hispanic" = "Other",
                               "White, non-Hispanic" = "White"),
         race_eth = fct_relevel(race_eth,
                               "Black, non-Hispanic",
                               "Hispanic or Latino",
```

```
                          "Multiracial, non-Hispanic",
                          "Other, non-Hispanic",
                          "White, non-Hispanic"
                          ),
        shock_onset_hosp_beds_cat =
          fct_relevel(shock_onset_hosp_beds_cat, "≥500", "200-499"),
        shock_onset_hosp_teaching_status =
          fct_relevel(shock_onset_hosp_teaching_status, "Non-teaching"),
        shock_onset_tx_unit =
          fct_relevel(shock_onset_tx_unit, "Medical/Cardiac ICU"),
        ssc_era = fct_relevel(ssc_era, "SSC 2021")
        )
```

Now check the vector names in the `adj` tibble to confirm expectations are met.

▼ Code

```
names(adj)
```

```
 [1] "encounter_id"
 [2] "hours_to_first_adj"
 [3] "mort_hosp"
 [4] "pressors_range"
 [5] "pressors_dur_hours"
 [6] "admission_time"
 [7] "discharge_time"
 [8] "criteria_met"
 [9] "age"
[10] "sex"
[11] "race_eth"
[12] "admit_year"
[13] "sepsis_onset_hosp_beds_cat"
[14] "sepsis_onset_hosp_teaching_status"
[15] "sepsis_onset_hosp_state"
[16] "comm_onset"
[17] "max_sofa"
[18] "sepsis_onset_tot_sofa"
[19] "sepsis_onset_unit_type"
[20] "bcx_collected"
[21] "abx_days"
[22] "shock_onset_tot_sofa"
[23] "sepsis_onset_time"
[24] "sepsis_window_start"
[25] "sepsis_window_end"
[26] "sepsis_onset_hospital"
[27] "sepsis_onset_icu_group"
[28] "sepsis_onset_covid"
[29] "shock_onset_unit_type"
[30] "shock_onset_hospital"
[31] "shock_onset_icu_group"
```

```
[32] "shock_onset_time"
[33] "cerebrovascular_disease"
[34] "hypertension"
[35] "heart_failure"
[36] "valvular_disease"
[37] "chronic_pulmonary_disease"
[38] "pulmonary_circulation_disease"
[39] "liver_disease"
[40] "kidney_disease"
[41] "malignancy"
[42] "diabetes"
[43] "obesity"
[44] "alcohol_abuse"
[45] "elixhauser_index"
[46] "shock_onset_tx_unit"
[47] "shock_onset_hosp_beds_cat"
[48] "shock_onset_hosp_teaching_status"
[49] "shock_onset_hosp_state"
[50] "ssc_era"
[51] "prior_surg"
[52] "rrt_before_shock"
[53] "aki"
[54] "shock_onset_scr"
[55] "shock_onset_platelets"
[56] "shock_onset_bili"
[57] "shock_onset_gcs"
[58] "shock_onset_pf"
[59] "shock_onset_pH"
[60] "shock_onset_temp_c"
[61] "shock_onset_heart_rate"
[62] "shock_onset_resp_rate"
[63] "shock_onset_map"
[64] "shock_onset_map_min"
[65] "shock_onset_lactate"
[66] "shock_onset_lactate_max"
[67] "fluid_mL_12h_prior_shock"
[68] "fluid_mL_kg_12h_prior_shock"
[69] "shock_onset_steroid"
[70] "first_adj"
[71] "death_time"
[72] "key_discharge_time"
[73] "ne_initial_dose_mcg_min"
[74] "ne_initial_dose_mcg_kg_min"
```

The retained columns (vectors) are consistent with expectations.

5.2 Outcome: Adjunctive Vasopressor Response

The outcome of interest will be adjunctive vasopressor blood pressure response, defined as achieving MAP ≥65 mm Hg with a decrease from baseline in norepinephrine dosage at 6 hours after first adjunctive vasopressor initiation.

- The outcome will only be adjudicated for the first adjunctive agent (the “second-line” vasopressor).
- Patients with an unavailable (missing) MAP value at 6 hours (or within the 3 hours preceding [missing MAP in hours 3-6 after adjunct start]) will be adjudicated as not meeting response criteria. The impact of this component of the criteria will be explored.
- Because additional (“third-line”, etc.) adjunctive vasopressors may be initiated within the 6 hour window period, the norepinephrine-equivalent (NEQ) dose at 6 hours will be utilized instead of solely the norepinephrine dose. The NEQ will be based on the Kotani equivalence equation (Kotani 2023).

The outcome adjudication will be operationalized as follows.

1. Join closest MAP (and within 3 hours) to the `pressors_doses` tibble for each dose titration. This value will be used for the adjudication of the MAP value at 6 hours after adjunct start. Joining the MAP in this way will have two added advantages: a) availability of closest MAP at adjunct start (which will be identified in a subsequent section for use as a predictor), and b) allow for calculation of the ratio of MAP to norepinephrine-equivalent dose (MAP/NEQ) for each dose, which will be added at the end of the call for potential future use.

▼ Code

```
pressors_doses <- pressors_doses |>
  mutate(mar_taken_minus_3h = mar_taken_time - dhours(3)) |>
  left_join(map |>
              # Filter MAP values to plausible range for patient in shock
              filter(map >=15 & map <=120) |>
              select(encounter_id, map_time, map) |>
              # Arrange to have the higher MAP from invasive and non-invasive
              # devices that were taken at the exact same time
              arrange(encounter_id, map_time, desc(map)) |>
              distinct(encounter_id, map_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(mar_taken_time >= map_time),
                         mar_taken_minus_3h <= map_time
                         ),
            relationship = "many-to-one"
            ) |>
  rename(closest_map_time = map_time,
         closest_map = map) |>
  mutate(map_neq_goradia = if_else(running_nee_kotani != 0 &
                                     !is.na(closest_map),
                                   closest_map / running_nee_goradia,
                                   NA
```

```
                                ),
        map_neq_kotani = if_else(running_nee_kotani != 0 &
                                   !is.na(closest_map),
                                 closest_map / running_nee_kotani,
                                 NA
                                 )
        )

# Remove `map` tibble since it won't be used moving forward
rm(map)
```

2. Determine the date/time of the first adjunctive vasopressor start (`first_adj_time`) in the `pressors_courses` tibble and join this vector to the `adj` tibble.

▼ Code

```
pressors_courses <- pressors_courses |>
  mutate(first_adj_time = as.POSIXct(case_when(
    first_adj == "vasopressin" ~ vasopressin_start,
    first_adj == "epinephrine" ~ epinephrine_start,
    first_adj == "phenylephrine" ~ phenylephrine_start,
    first_adj == "dopamine" ~ dopamine_start,
    first_adj == "AVP + Epi" ~ vasopressin_start,
    first_adj == "AVP + PE" ~ vasopressin_start,
    first_adj == "Epi + PE" ~ epinephrine_start,
    first_adj == "AVP + DA" ~ vasopressin_start,
    first_adj == "AVP + Epi + PE" ~ vasopressin_start,
    first_adj == "Epi + DA" ~ epinephrine_start,
    is.na(first_adj) ~ NA,
    # Create "error" date/time to ensure all options captured in coding
    .default = as.POSIXct(1900-01-01)
    )))
```

Ensure coding didn't result in any error date/time

▼ Code

```
sum(pressors_courses$first_adj_time == as.POSIXct(1900-01-01), na.rm = TRUE)
```

```
[1] 0
```

No error resulted, therefore continue with the join.

▼ Code

```
adj <- adj |>
  left_join(pressors_courses |> select(encounter_id, first_adj_time),
            by = join_by(encounter_id),
            relationship = "one-to-one"
```

```
          ) |>
  relocate(c(first_adj_time, first_adj, hours_to_first_adj),
           .after = encounter_id
           )
```

3. To the `adj` tibble, join from the `pressors_doses` tibble the closest NEQ dose before the time of first adjunct start (but not before or equal to the time of first adjunct start because the NEQ at the time equal to the first adjunct start includes the initial adjunct dose). While this baseline value only includes the norepinephrine dosage, this will be called `adj_baseline_neq` for clarity. Also join the NEQ with units of mcg/min and the MAP/NEQ for completeness. These values will not be used other than for description of the cohort of included patients.

▼ Code

```
adj <- adj |>
  left_join(pressors_doses |>
              select(encounter_id, mar_taken_time, running_nee_kotani,
                     running_nee_kotani_mcg_min, map_neq_kotani
                     ) |>
              distinct(),
            by = join_by(encounter_id,
                         closest(first_adj_time > mar_taken_time)
                         ),
            relationship = "one-to-one"
            ) |>
  select(-mar_taken_time) |>
  rename(adj_baseline_neq = running_nee_kotani,
         adj_baseline_neq_mcg_min = running_nee_kotani_mcg_min,
         adj_baseline_map_neq = map_neq_kotani) |>
  relocate(adj_baseline_neq, .after = hours_to_first_adj)
```

Confirm no missing values for `adj_baseline_neq`.

▼ Code

```
sum(is.na(adj$adj_baseline_neq))
```

```
[1] 0
```

There are no missing values.

Review random sample of all cases to ensure expectations met (commented out for rendering).

▼ Code

```
set.seed(2026)
df <- adj |>
  select(encounter_id, first_adj, first_adj_time, adj_baseline_neq) |>
  slice_sample(n=10)
```

```
#View(df |> arrange(encounter_id))
#View(pressors_doses |> filter(encounter_id %in% df$encounter_id) |>
#       select(encounter_id, mar_taken_time, medication, titrate_action,
#              running_nee_kotani
#              ) |>
#       arrange(encounter_id)
#     )
rm(df)
```

Expectations are met, therefore continue.

Assess if any values of `adj_baseline_neq` are zero (doses must be greater than zero to indicate adjunctive vasopressor use).

▼ Code

```
sum(adj$adj_baseline_neq == 0)
```

```
[1] 369
```

There are 369 cases where the norepinephrine dosage documented immediately before adjunct initiation was zero. Review some of these cases to determine if there is an issue with the joining process or if they indicate true cessations of norepinephrine (commented out for rendering purposes).

▼ Code

```
df <- adj |> filter(adj_baseline_neq == 0) |> head(n=10)
#View(df)
#View(pressors_doses |> filter(encounter_id %in% df$encounter_id))
rm(df)
```

In the manual review most norepinephrine doses were truly documented as zero immediately prior to first “adjunct” vasopressor initiation. These cases should be subsequently excluded because they do not represent adjunctive vasopressor usage: the start of the non-norepinephrine vasopressor represents monotherapy with that medication.

There was one case, though, in the manual review that had a norepinephrine dosage of zero immediately preceding the adjunct start but at the same time of adjunct start there was a non-zero norepinephrine dosage documented. To account for these cases join the norepinephrine dosage at the exact time of first adjunct start, then replace the `adj_baseline_neq` dose of zero with the norepinephrine dosage at the exact time of first adjunct start. Also join `adj_baseline_neq_mcg_min` and calculate `adj_baseline_map_neq` for these cases with the alternative join criteria.

▼ Code

```
adj <- adj |>
  left_join(pressors_doses |>
```

```
            filter(medication == "norepinephrine") |>
            select(encounter_id, mar_taken_time, nee_kotani, closest_map,
                   dose_mcg_min) |>
            distinct(),
          by = join_by(encounter_id,
                       first_adj_time == mar_taken_time
                       ),
          relationship = "one-to-one"
          ) |>
  mutate(adj_baseline_neq = if_else(adj_baseline_neq == 0 & !is.na(nee_kotani),
                                    nee_kotani,
                                    adj_baseline_neq
                                    ),
         adj_baseline_neq_mcg_min = if_else(adj_baseline_neq == 0 &
                                              !is.na(dose_mcg_min) &
                                              dose_mcg_min != 0,
                                    dose_mcg_min,
                                    adj_baseline_neq_mcg_min
                                    ),
         adj_baseline_map_neq = if_else(is.na(adj_baseline_map_neq) &
                                          !is.na(nee_kotani) &
                                          nee_kotani != 0 &
                                          !is.na(closest_map),
                                        closest_map / nee_kotani,
                                        adj_baseline_map_neq
                                        )
         ) |>
  select(-c(nee_kotani, closest_map))

sum(adj$adj_baseline_neq == 0)
```

```
[1] 313
```

▼ Code

```
sum(adj$adj_baseline_neq_mcg_min == 0, na.rm = TRUE)
```

```
[1] 268
```

▼ Code

```
nrow(adj |> filter(is.na(adj_baseline_map_neq)))
```

```
[1] 686
```

4. To the `adj` tibble, join from the `pressors_doses` tibble the NEQ and MAP values from the time closest to 6 hours after first adjunct start (and also equal to or after initiation of the first adjunct). Also join the MAP/NEQ for completeness. Only joining values after initiation of the first adjunct

(not equal to) was considered, but since an adjunct contributes to the NEQ, if there are no subsequent NEQ values documented then non-response should be adjudicated.

▼ Code

```
adj <-
  left_join(adj |>
              mutate(first_adj_plus_6h = first_adj_time + dhours(6)),
            pressors_doses |>
              select(encounter_id, mar_taken_time,
                     running_nee_kotani, closest_map, map_neq_kotani
                     ) |>
              distinct(),
            by = join_by(encounter_id,
                         closest(first_adj_plus_6h >= mar_taken_time),
                         first_adj_time <= mar_taken_time
                         ),
            relationship = "one-to-one"
            ) |>
  select(-mar_taken_time) |>
  rename(adj_plus6h_neq = running_nee_kotani,
         adj_plus6h_map = closest_map,
         adj_plus6h_map_neq = map_neq_kotani
         ) |>
  relocate(c(adj_plus6h_neq, adj_plus6h_map), .after = adj_baseline_neq)
```

Check for missing values from the join.

▼ Code

```
sum(is.na(adj$adj_plus6h_neq))
```

```
[1] 0
```

▼ Code

```
paste0(round_half_up((sum(is.na(adj$adj_plus6h_neq))/nrow(adj))*100,0),"%")
```

```
[1] "0%"
```

▼ Code

```
sum(is.na(adj$adj_plus6h_map))
```

```
[1] 283
```

▼ Code

```
paste0(round_half_up((sum(is.na(adj$adj_plus6h_map))/nrow(adj))*100,0),"%")
```

```
[1] "3%"
```

There are no missing values for the NEQ and 282 (3%) observations have a missing value for MAP at 6 hours after first adjunct start. This will be evaluated as part of the adjunctive vasopressor response criteria.

5. Lastly, determine first adjunctive vasopressor response criteria as being met if MAP ≥65 mm hg at 6 hours after first adjunct start (`adj_plus6h_map`) **and** NEQ at 6 hours after first adjunct start (`adj_plus6h_neq`) is < baseline NEQ (`adj_baseline_neq`).
    - If `death_time` prior to baseline time plus 6 hours then adjudicate as not meeting criteria (death before 6 hours == non-response)
    - If NEQ at 6 hours is the same or higher than baseline NEQ then adjudicate as not meeting criteria. Notably, initiation of additional adjunctive vasopressors (third-line, etc.) between first adjunct start and 6 hours after first adjunct start are accounted for through the contribution of their doses to the NEQ component.
    - If MAP <65 mm Hg at 6 hours then adjudicate as not meeting criteria
    - If NEQ or MAP at 6 hours is missing then adjudicate as not meeting criteria (no evidence of lower NEQ or MAP at goal)
    - Response criteria met if MAP ≥65 mm Hg at 6 hours *and* NEQ at 6 hours is lower than baseline NEQ

▼ Code

```
adj <- adj |>
  mutate(
    response_type = factor(case_when(
      !is.na(death_time) & death_time <= first_adj_plus_6h ~ "0-no_resp_death",
      adj_plus6h_neq >= adj_baseline_neq ~ "0-no_resp_neq_same_or_higher",
      adj_plus6h_map < 65 ~ "0-no_resp_map_below_65",
      is.na(adj_plus6h_neq) ~ "2-resp_unclear_neq_missing",
      is.na(adj_plus6h_map) ~ "2-resp_unclear_map_missing",
      adj_plus6h_map >= 65 & adj_plus6h_neq < adj_baseline_neq ~ "1-response",
      .default = "Error"),
      levels = c("1-response", "0-no_resp_neq_same_or_higher",
                 "0-no_resp_map_below_65", "0-no_resp_death",
                 "2-resp_unclear_neq_missing", "2-resp_unclear_map_missing"
                 )
      ),
    response = factor(if_else(response_type == "1-response",
                              "Response",
                              "No Response"),
                      # Ensure level ordering is correct for modeling
                      levels = c("No Response", "Response")
      )
    )
```

Show levels of `response_type`, which also serves as a check to ensure all scenarios are accounted for (no "Error" appears).

▼ Code

```
adj |> tabyl(response_type) |> adorn_totals() |> adorn_pct_formatting()
```

```
                response_type    n percent
                   1-response 2785   31.9%
 0-no_resp_neq_same_or_higher 5097   58.4%
       0-no_resp_map_below_65  540    6.2%
              0-no_resp_death  247    2.8%
   2-resp_unclear_neq_missing    0    0.0%
   2-resp_unclear_map_missing   58    0.7%
                        Total 8727  100.0%
```

First, no "Error" appears, therefore the code worked as intended.

Second, the incidence of response (32%) is lower than in our previous work with vasopressin alone (40%) but is reasonably close. The difference is explainable because patients and criteria in the previous work were slightly different - in this work all patients received initial norepinephrine (previously all initial vasopressors were included), death between first adjunct start and 6 hours later was considered non-response (not considered in prior work), the vasopressor contribution of vasopressin (equated through the Kotani norepinephrine-equivalent dose [NEQ]) was considered in the response criteria (previously it was ignored), and the NEQ was required to be lower than baseline (not lower or equal to baseline since the vasopressin NEQ contribution was considered). Inclusion of the vasopressor contribution of vasopressin as part of the NEQ is likely the largest driver of the difference in response incidence between studies.

Third, only 58 observations (0.7%) were adjudicated as not meeting response criteria because a MAP value in the 3 to 6 hour period after first adjunct start was unavailable. If the alternative approach where missing MAP values are ignored (considered as meeting response because no evidence MAP was below goal) were to be utilized then the frequency of response would not change in a meaningful way (<1% difference). Therefore, will continue with the approach of considering missing MAP values as criteria not being met.

5.3 Patient Descriptors and Candidate Predictors

This sub-section will create and join case eligibility criteria and patient characteristics, some of which are candidate predictors of interest.

5.3.1 Biological and Context Characteristics

Some of the covariates (predictors) of interest are in the `adj` tibble already, but some additional covariates of interest need to be joined from other loaded tibbles.

5.3.1.1 Body Weight

Join body weight at first adjunct start from the `body_weight` tibble.

▼ Code

```
adj <- left_join(
  adj,
  body_weight |> select(encounter_id, wt_time, wt_kg) |> distinct(),
  by = join_by(encounter_id,
               closest(first_adj_time >= wt_time)
               ),
  relationship = "one-to-one"
  ) |>
  select(-wt_time) |>
  rename(first_adj_wt_kg = wt_kg)

# Remove `body_weight` tibble because it won't be used moving forward
rm(body_weight)
```

5.3.1.2 Treatment Unit

Join treatment unit (department), service, and ICU type (if applicable) at first adjunct start based on admission/discharge/transfer (ADT) data in the `adt` tibble.

▼ Code

```
adj <- adj |>
  left_join(adt |> select(encounter_id, in_effective_time, out_effective_time,
                first_adj_hospital = hospital,
                first_adj_department_name = department_name,
                first_adj_hospital_service = hospital_service,
                first_adj_icu_group = icu_group,
                first_adj_unit_type = unit_type),
            join_by(encounter_id,
                    closest(first_adj_time >= in_effective_time),
                    first_adj_time <= out_effective_time),
            relationship = "one-to-one") |>
            mutate(first_adj_unit_type = fct_relevel(first_adj_unit_type,
                                                     "ED", "Ward", "ICU")) |>
  select(-in_effective_time, -out_effective_time)
```

There are some observations with first adjunct start time occurring between unit-specific admission and discharge times in the `adt` tibble due to inconsistency in manual time stamps or back-dated documentation during transport between units. This results in some observations having NA for the characteristics because the join criteria are not met. The call below counts the number of observations having NA for the `first_adj_hospital` vector (as a representative example).

▼ Code

```r
sum(is.na(adj$first_adj_hospital))
```

```
[1] 10
```

There are 10 observations (patients) with missing `first_adj_hospital`. The call below replaces these NA values with the characteristics from the treatment unit preceding the `first_adj_time`.

▼ Code

```r
adj <- adj |>
  left_join(adt |>
              select(encounter_id, in_effective_time, out_effective_time,
                hospital, department_name, hospital_service, icu_group,
                unit_type),
            join_by(encounter_id,
                    closest(first_adj_time > out_effective_time)),
            relationship = "one-to-many") |>
  arrange(encounter_id, first_adj_time, desc(out_effective_time)) |>
  distinct(encounter_id, .keep_all = TRUE) |>
  mutate(first_adj_hospital =
           coalesce(first_adj_hospital, hospital),
         first_adj_department_name =
           coalesce(first_adj_department_name, department_name),
         first_adj_hospital_service =
           coalesce(first_adj_hospital_service, hospital_service),
         first_adj_icu_group = coalesce(first_adj_icu_group, icu_group),
         first_adj_unit_type = coalesce(first_adj_unit_type, unit_type)) |>
  select(-c(in_effective_time, out_effective_time, hospital, department_name,
            hospital_service, icu_group, unit_type))

# Remove `adt` tibble since it won't be used moving forward
rm(adt)
```

Join hospital characteristics.

▼ Code

```r
adj <- adj |>
  left_join(hosp_char |> select(hospital, hosp_beds_cat, hosp_teaching_status,
                                hosp_state),
            by = join_by(first_adj_hospital == hospital),
            relationship = "many-to-one"
  ) |>
  rename(first_adj_hosp_beds_cat = hosp_beds_cat,
         first_adj_hosp_teaching_status = hosp_teaching_status,
         first_adj_hosp_state = hosp_state
  ) |>
  relocate(c(first_adj_hosp_beds_cat, first_adj_hosp_teaching_status,
             first_adj_hosp_state
```

```
            ),
          .after = first_adj_hospital)

# Remove `hosp_char` tibble since it won't be used moving forward
rm(hosp_char)
```

The `adj` tibble now has a vector for the treatment location at the time of first adjunct start (`first_adj_unit_type`) but this is too broad of a categorization. Contrastingly, the `first_adj_icu_group` vector for ICU designations is too granular for external applicability. Therefore a new vector (`first_adj_tx_unit`) will be created that combines information from these vectors to generate categories.

▼ Code

```
adj <- adj |> mutate(first_adj_tx_unit = factor(case_when(
  first_adj_unit_type == "ED" | first_adj_unit_type == "Ward" ~ "ED/Rapid Response",
  first_adj_unit_type == "ICU" &
    grepl("MICU", first_adj_icu_group) ~ "Medical ICU",
  first_adj_unit_type == "ICU" &
    grepl("CCU", first_adj_icu_group) ~ "Cardiac ICU",
  first_adj_unit_type == "ICU" &
    grepl("SICU|CVICU", first_adj_icu_group) ~ "Surgical ICU",
  first_adj_unit_type == "ICU" ~ "Mixed Med/Surg ICU",
  .default = NA
  )))
```

Now show the distribution of values to ensure expectations are met.

▼ Code

```
adj |> tabyl(first_adj_tx_unit) |> adorn_pct_formatting() |>
  adorn_totals() |>
  rename("First Adjunct Start Treatment Unit" = "first_adj_tx_unit") |>
  kable(align = c('lcc')) |>
  kable_paper(full_width = FALSE)
```

FIRST ADJUNCT START TREATMENT UNIT	N	PERCENT
Cardiac ICU	779	8.9%
ED/Rapid Response	617	7.1%
Medical ICU	2957	33.9%
Mixed Med/Surg ICU	2284	26.2%
Surgical ICU	2090	23.9%
Total	8727	-

5.3.2 Adjunct Start Characteristics

Transform variables at the time of first adjunctive vasopressor start (baseline).

5.3.2.1 Antimicrobials

Antimicrobial administration within the 24 hours prior to first adjunct start. This will be adjudicated based on the `abx_range_start` and `abx_range_end` variables in the `abx` tibble. Antibiotic start must be prior to first adjunct start and the last dose of antibiotics as part of the course must be given within 24 hours prior to first adjunct start. This will account for antibiotic courses that are "ongoing" at first adjunct start.

▼ Code

```
adj <-
  left_join(
    adj |> mutate(first_adj_minus_24h = first_adj_time - dhours(24)),
    abx |> select(encounter_id, abx_range_start, abx_range_end),
    join_by(encounter_id,
            closest(first_adj_time >= abx_range_start),
            first_adj_minus_24h <= abx_range_end),
    relationship = "one-to-many") |>
  distinct(encounter_id, .keep_all = TRUE) |>
  mutate(first_adj_abx = factor(if_else(!is.na(abx_range_start),
                                        "Antibiotics", "No Antibiotics"
                                        ))) |>
  arrange(encounter_id, admission_time) |>
  select(-c(abx_range_start, abx_range_end, first_adj_minus_24h))

# Remove `abx` tibble because it won't be used moving forward
rm(abx)
```

5.3.2.2 Surgery

First, join the `patient_id` identifier from the `encounter_key` tibble to the `adj` tibble.

▼ Code

```
adj <- adj |>
  left_join(encounter_key |> select(episode_key, patient_id),
            join_by(encounter_id == episode_key),
            relationship = "one-to-many") |>
  distinct(encounter_id, .keep_all = TRUE)
```

Next adjudicate surgery within the 30 days prior to adjunct start.

▼ Code

```
adj <-
  left_join(
```

```
    adj |> mutate(first_adj_minus_30d = first_adj_time - ddays(30)),
    surg |> select(patient_id, surgery_start_time),
    join_by(patient_id,
            first_adj_minus_30d <= surgery_start_time,
            first_adj_time >= surgery_start_time),
    relationship = "many-to-many") |>
  arrange(encounter_id, surgery_start_time) |>
  distinct(encounter_id, .keep_all = TRUE) |>
  mutate(prior_surg = factor(if_else(!is.na(surgery_start_time),
                                     "Surgery", "No Surgery"
                                     ))) |>
  arrange(encounter_id, admission_time) |>
  select(-surgery_start_time, -first_adj_minus_30d)
# Remove tibbles that won't be used moving forward
rm(surg, encounter_key)
```

5.3.2.3 Acute Kidney Injury

Acute kidney injury (AKI) will be defined and staged per the KDIGO 2012 criteria. The urine output component will be omitted due to the lack of reliable data collection in hospitalized patients Alrawashdeh 2024.

AKI will be staged as follows:

- Stage 1: Serum creatinine (SCr) 1.5-1.9 times baseline OR ≥0.3 mg/dL increase from baseline
- Stage 2: SCr 2.0-2.9 times baseline
- Stage 3: SCr ≥3.0 times baseline OR increase in SCr to ≥4.0 mg/dL OR (new) initiation of renal replacement therapy

Baseline SCr will be determined as follows (Hoste 2015, Alfieri 2023):

- Median pre-morbid value from the 3 months prior to encounter/admission
- If pre-morbid SCr not available, minimum value during the encounter observed prior to first adjunct start

Patients with end-stage kidney disease (ESKD) or renal replacement therapy prior to admission will be adjudicated as **not** meeting criteria for Stage 3 AKI.

Retain only SCr values obtained prior to first adjunct start. Then determined baseline SCr value based on criteria above. The work to determine pre-morbid SCr value (`pta_scr`) in the `scr` tibble was done in the `kidney_function.qmd` Quarto document.

▼ Code

```
scr <-
  left_join(scr,
            adj |> select(encounter_id, first_adj_time),
            by = join_by(encounter_id),
            relationship = "many-to-one"
```

```
          ) |>
  mutate(scr_before_adj =
           if_else(scr_taken_time <= first_adj_time, "Yes", "No")
         ) |>
  filter(scr_before_adj == "Yes") |>
  select(-scr_before_adj) |>
  group_by(encounter_id) |>
  mutate(min_scr_before_adj = min(scr_result)) |>
  ungroup() |>
  mutate(baseline_scr = case_when(
    !is.na(pta_scr) ~ pta_scr,
    !is.na(min_scr_before_adj) ~ min_scr_before_adj,
    .default = NA
    ))
```

Join pertinent values from `scr` to `adj` tibble (notably, only join the closest SCr value obtained before first adjunct start).

▼ Code

```
adj <-
  left_join(adj,
            scr |> select(encounter_id, scr_taken_time, scr_result,
                          baseline_scr
                          ) |>
              distinct(encounter_id, scr_taken_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= scr_taken_time)),
            relationship = "one-to-one"
            )
rm(scr) # Remove `scr` tibble because it won't be used moving forward
```

Determine renal replacement therapy (RRT) before first adjunct start. The work to determine new RRT (`new_rrt`) in the `rrt` tibble was done in the `kidney_function.qmd` Quarto document.

▼ Code

```
rrt_before_adj <-
  left_join(rrt,
            adj |> select(encounter_id, first_adj_time),
            by = join_by(encounter_id),
            relationship = "many-to-one") |>
  mutate(rrt_before_adj =
           if_else(rrt_taken_time <= first_adj_time, "Yes", "No")
         ) |>
  filter(rrt_before_adj == "Yes") |>
  select(-c(rrt_before_adj))
```

```
rrt_before_adj_new <- rrt_before_adj |>
  filter(new_rrt == "Yes")
```

Create variables for RRT before adjunct (`rrt_before_adj`) and new RRT before adjunct (`new_rrt_before_adj`) within the `adj` tibble.

▼ Code

```
adj <- adj |>
  mutate(rrt_before_adj = factor(
           if_else(encounter_id %in% rrt_before_adj$encounter_id,
                   "RRT Before Adjunct", "No RRT Before Adjunct"
                   )),
         new_rrt_before_adj =
           if_else(encounter_id %in% rrt_before_adj_new$encounter_id,
                   "Yes", "No"
                   )
         )
# Remove `rrt_before_adj`, `new_rrt_before_adj`, and `rrt` tibbles because
# they won't be used moving forward
rm(rrt_before_adj, rrt_before_adj_new, rrt)
```

Now that all necessary variables are available, determine AKI presence and stage in the `adj` tibble. Data for administrative coding for ESKD was managed in the `kidney.qmd` Quarto document.

▼ Code

```
adj <- adj |>
  mutate(aki = factor(case_when(
    encounter_id %in% esrd$encounter_id ~ "ESKD",
    scr_result / baseline_scr >= 3 |
      scr_result >=4 |
      new_rrt_before_adj == "Yes" ~ "AKI Stage 3",
    (scr_result / baseline_scr >= 2) &
      (scr_result / baseline_scr < 3) ~ "AKI Stage 2",
    (scr_result / baseline_scr >= 1.5) &
      (scr_result / baseline_scr < 2) |
      scr_result - baseline_scr >= 0.3 ~ "AKI Stage 1",
    is.na(scr_result) ~ NA,
    .default = "No AKI"
     ))) |>
  mutate(aki = fct_relevel(aki, "No AKI")) |>
  rename(first_adj_aki = aki)
rm(esrd) # Remove `esrd` tibble because it won't be used moving forward
```

Show table of AKI staging. NA values are expected due to instances when an SCr value (`src_result`) is not available before first adjunct start.

▼ Code

```r
adj |> tabyl(first_adj_aki) |>
  adorn_pct_formatting() |>
  adorn_totals() |>
  rename("Acute Kidney Injury Stage" = "first_adj_aki") |>
  kable(align = c('lcc')) |>
  kable_paper(full_width = FALSE)
```

ACUTE KIDNEY INJURY STAGE	N	PERCENT	VALID_PERCENT
No AKI	3842	44.0%	44.7%
AKI Stage 1	1780	20.4%	20.7%
AKI Stage 2	823	9.4%	9.6%
AKI Stage 3	1621	18.6%	18.8%
ESKD	537	6.2%	6.2%
NA	124	1.4%	-
Total	8727	-	-

The code worked as intended and the counts align with expectations.

Retain only SCr values collected within 24 hours of first adjunct start (SCr values collected before first adjunct start were ensured in an above step).

▼ Code

```r
adj <- adj |>
  mutate(first_adj_minus_24h = first_adj_time - dhours(24),
         first_adj_scr =
           if_else(scr_taken_time >= first_adj_minus_24h, scr_result, NA)
         ) |>
  # Remove variables that won't be used moving forward
  select(-c(scr_taken_time, scr_result, baseline_scr, new_rrt_before_adj))
```

5.3.2.4 SOFA-2 Components and Calculation

In this section join components of and calculate the Sequential Organ Failure Assessment (SOFA)-2 score Ranzani, JAMA 2025. The SOFA-2 includes six organ systems each scored from 0 to 4 points, with a total score ranging from 0 to 24 (higher scores indicate worse organ dysfunction).

5.3.2.4.1 Brain

The Brain score centers on the Glasgow Coma Scale (GCS) score but includes a clause “For sedated patients, use the last recorded GCS before sedation. If the previous GCS is unknown, score 0”. Therefore sedative medication medication administration must be considered when joining the GCS value.

Join sedative medication administration (from the `sed` tibble) within the preceding 6 hours of the GCS value to the `gcs` tibble. Sedatives included in the `sed` tibble are dexmedetomidine, diazepam, fentanyl, hydromorphone, ketamine, lorazepam, midazolam, and propofol administered via any non-epidural route. The 6 hours duration preceding the GCS value was based on an approximation of the duration of these medications (recognizing some medications have a shorter duration).

▼ Code

```
gcs <- left_join(gcs |>
                   select(encounter_id, gcs_taken_time, gcs) |>
                   arrange(encounter_id, gcs_taken_time, gcs) |>
                   distinct(encounter_id, gcs_taken_time, .keep_all = TRUE) |>
                   mutate(gcs_minus_6h = gcs_taken_time - dhours(6)),
                 sed |> select(encounter_id, sed_taken_time = mar_taken_time),
                 by = join_by(encounter_id,
                              closest(gcs_taken_time >= sed_taken_time),
                              gcs_minus_6h <= sed_taken_time
                              ),
                 relationship = "many-to-many"
                 ) |>
  mutate(sed_prev_6h = if_else(!is.na(sed_taken_time), 1, 0)) |>
  arrange(encounter_id, gcs_taken_time, desc(sed_prev_6h)) |>
  distinct(encounter_id, gcs_taken_time, .keep_all = TRUE)
```

Join the closest GCS value from the `gcs` tibble that was obtained without a sedative medication administered in the preceding 6 hours, restricted to values obtained within 6 hours prior to first adjunct start.

▼ Code

```
adj <-
  left_join(adj |>
              mutate(first_adj_minus_6h = first_adj_time - dhours(6)),
            gcs |>
              filter(sed_prev_6h == 0) |>
              select(encounter_id, gcs_taken_time, gcs),
            by = join_by(encounter_id,
                         closest(first_adj_time >= gcs_taken_time),
                         first_adj_minus_6h <= gcs_taken_time
                         ),
            relationship = "one-to-one") |>
  rename(first_adj_gcs_no_sed = gcs) |>
  # Remove variable that won't be used moving forward
  select(-c(gcs_taken_time))
```

The Brain score also includes a clause “If receiving drug treatment for delirium (short- or long-term), score 1 point even if GCS is 15”. Therefore, join delirium medication administration within the preceding 12 hours. The `antipsy` tibble includes the anti-psychotic medications aripiprazole,

haloperidol, quetiapine, and ziprasidone, which may be used at the development site for treatment of delirium. Also join dexmedetomidine administration within the preceding 12 hours since it may be used for treatment of delirium as outlined in the 2025 Focused Update to the Clinical Practice Guidelines for the Prevention and Management of Pain, Anxiety, Agitation/Sedation, Delirium, Immobility, and Sleep Disruption in Adult Patients in the ICU Lewis, Crit Care Med 2025.

▼ Code

```
adj <-
  left_join(adj |>
              mutate(first_adj_minus_12h = first_adj_time - dhours(12)),
            antipsy |>
              select(encounter_id, antipsy_taken_time = mar_taken_time) |>
              distinct(),
            by = join_by(encounter_id,
                         closest(first_adj_time >= antipsy_taken_time),
                         first_adj_minus_12h <= antipsy_taken_time
                         ),
            relationship = "one-to-one") |>
  left_join(sed |>
              filter(short_name == "dexmedetomidine") |>
              select(encounter_id, dex_taken_time = mar_taken_time) |>
              distinct(),
            by = join_by(encounter_id,
                         closest(first_adj_time >= dex_taken_time),
                         first_adj_minus_12h <= dex_taken_time
                         ),
            relationship = "one-to-one"
            ) |>
  mutate(delirium_med_prev_12h =
           if_else(!is.na(antipsy_taken_time) | !is.na(dex_taken_time), 1, 0)
         ) |>
  select(-c(antipsy_taken_time, dex_taken_time))

# Remove tibbles that won't be used moving forward
rm(gcs, sed, antipsy)
```

5.3.2.4.2 Respiratory

5.3.2.4.2.1 PaO_2/FiO_2 and SpO_2/FiO_2 Ratio

The Respiratory score centers on the PaO_2 to FiO_2 ratio (P/F ratio). The SOFA-2 document includes a clause "Use the arterial oxygen saturation (SpO_2) to FIO_2 ratio only when the PaO_2:FIO_2 ratio is unavailable and when the SpO_2 is less than 98%." Therefore both the P/F ratio and SpO_2 to FIO_2 ratio (S/F ratio) will be joined. The PaO_2 is only available from an arterial blood gas, which is not commonly obtained outside of the ICU (such as in the ED or on the ward). As such, missingness is expected for this variable.

Join the ratio of P/F ratio and S/F ratio closest to first adjunct start and within 6 hours prior. Both PaO_2 to FiO_2 ratio (`p_f_ratio`) and SpO_2 to FiO_2 ratio (`s_f_ratio`) are available in the `oxygenation`

tibble. Notably, the `s_f_ratio` was only calculated if the SpO_2 was <98%; values >=98% produced a missing value (NA) for `s_f_ratio`.

▼ Code

```
adj <-
  left_join(adj,
            oxygenation |>
              filter(!is.na(p_f_ratio)) |>
              select(encounter_id, obs_time, p_f_ratio) |>
              arrange(encounter_id, obs_time, p_f_ratio) |>
              distinct(encounter_id, obs_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= obs_time),
                         first_adj_minus_6h <= obs_time
                         ),
            relationship = "one-to-one") |>
  select(-obs_time) |>
  left_join(oxygenation |>
              filter(!is.na(s_f_ratio)) |>
              select(encounter_id, obs_time, s_f_ratio) |>
              arrange(encounter_id, obs_time, s_f_ratio) |>
              distinct(encounter_id, obs_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= obs_time),
                         first_adj_minus_6h <= obs_time
                         ),
            relationship = "one-to-one") |>
  rename(first_adj_pf = p_f_ratio,
         first_adj_sf = s_f_ratio) |>
  select(-obs_time)

# Remove `oxygenation` tibble because it won't be used moving forward
rm(oxygenation)
```

Check for missing values in `first_adj_pf` and `first_adj_sf`. Missing values are expected due to inadequate documentation of oxygen administration (resulting in no FiO_2 available) or inability to calculate `s_f_ratio` from SpO_2 values ≥98% (which are generally considered unreliable on the basis of the sigmoidal shape of the hemoglobin-oxygen dissociation curve values).

▼ Code

```
round_half_up((sum(is.na(adj$first_adj_pf)) / nrow(adj)) * 100, 0)
```

```
[1] 38
```

▼ Code

```
round_half_up((sum(is.na(adj$first_adj_sf)) / nrow(adj)) * 100, 0)
```

```
[1] 16
```

The `adj` tibble has 38% of `first_adj_pf` values and 16% of `first_adj_sf` missing, which are reasonable for the reasons outlined above.

5.3.2.4.2.2 Advanced Ventilatory Support

The SOFA-2 score calculation outlines "advanced ventilatory support" as part of the respiratory sub-score, which is defined as "receipt of high-flow nasal cannula, continuous positive airflow pressure, bilevel positive airway pressure, noninvasive ventilation, invasive mechanical ventilation, or long-term home ventilation". Join advanced respiratory (ventilatory) support documented time, if the start of advanced support (`adv_resp_start`) is prior to `first_adj_time` and the end of advanced support (`adv_resp_end`) is after `first_adj_time`. Since multiple values of `adv_resp_start` are possible then arrange by time (to put latest `adv_resp_start` as first row with `desc()`) then use `distinct()` to keep only the latest `adv_resp_start` before first adjunct start.

▼ Code

```
adj <- left_join(adj,
                 adv_resp |> select(encounter_id, adv_resp_start, adv_resp_end),
                 join_by(encounter_id,
                         closest(first_adj_time >= adv_resp_start),
                         first_adj_time <= adv_resp_end),
                 relationship = "one-to-many") |>
  arrange(encounter_id, first_adj_time, desc(adv_resp_start)) |>
  distinct(encounter_id, .keep_all = TRUE) |>
  mutate(adv_resp_before_adj =
           factor(if_else(!is.na(adv_resp_start),
                          "Adv Resp Before Adjunct",
                          "No Adv Resp Before Adjunct"
                          )),
         adv_resp_before_adj =
           fct_relevel(adv_resp_before_adj, "No Adv Resp Before Adjunct")
         ) |>
  select(-c(adv_resp_start, adv_resp_end))

# Remove `adv_resp` tibble because it won't be used moving forward
rm(adv_resp)
```

5.3.2.4.3 Cardiovascular

The Cardiovascular score centers on MAP, vasopressor or inotrope administration, and vasopressor dosage. The MAP is only considered in patients not receiving a vasopressor or inotrope. Because all patients in the cohort will be receiving norepinephrine the MAP does not need to be considered for this component. Because norepinephrine will be the only vasopressor administered at first adjunct start, other vasopressors do not need to be considered.

The highest Cardiovascular score (4 points) also includes mechanical support and the SOFA-2 document includes a clause “Any type of mechanical cardiovascular support: eg, venoarterial ECMO, intra-aortic balloon pump, left ventricular assist device, microaxial flow pump.” The use of these mechanical circulatory support devices is not available in the extracted data, therefore they are not able to be considered.

The medication dobutamine is considered an inotrope and not a vasopressor, therefore it may be administered in the cohort at first adjunct start. Join dobutamine administration, if the start of dobutamine (`start_time`) is prior to `first_adj_time` and the end of dobutamine administration (`stop_time`) is after `first_adj_time`. Since multiple values of `start_time` are possible then arrange by time (to put latest `start_time` as first row with `desc()`) then use `distinct()` to keep only the latest `start_time` before first adjunct start.

▼ Code

```
adj <- adj |>
  left_join(dbt |> select(encounter_id, start_time, stop_time),
            join_by(encounter_id,
                    closest(first_adj_time >= start_time),
                    first_adj_time <= stop_time),
                relationship = "one-to-many") |>
  arrange(encounter_id, first_adj_time, desc(start_time)) |>
  distinct(encounter_id, .keep_all = TRUE) |>
  mutate(dbt_before_adj =
           factor(if_else(!is.na(start_time),
                          "Dobutamine Before Adjunct",
                          "No Dobutamine Before Adjunct"
                          ))
         ) |>
  select(-c(start_time, stop_time))

# Remove `dbt` tibble because it won't be used moving forward
rm(dbt)
```

5.3.2.4.4 Liver

Join the closest bilirubin laboratory value from the `bili` tibble, restricted to values obtained within 24 hours prior to first adjunct start.

▼ Code

```
adj <-
  left_join(adj,
            bili |>
              select(encounter_id, bili_taken_time, bili_result) |>
              arrange(encounter_id, bili_taken_time, bili_result) |>
              distinct(encounter_id, bili_taken_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= bili_taken_time),
```

```
                        first_adj_minus_24h <= bili_taken_time
                        ),
            relationship = "one-to-one") |>
  rename(first_adj_bili = bili_result) |>
  # Remove variable that won't be used moving forward
  select(-c(bili_taken_time))

# Remove `bili` tibble because it won't be used moving forward
rm(bili)
```

5.3.2.4.5 Hemostasis

Join the closest platelets laboratory value from the `platelets` tibble, restricted to values obtained within 24 hours prior to first adjunct start.

▼ Code

```
adj <-
  left_join(adj,
            platelets |>
              select(encounter_id, plts_taken_time, plts_result) |>
              arrange(encounter_id, plts_taken_time, plts_result) |>
              distinct(encounter_id, plts_taken_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= plts_taken_time),
                         first_adj_minus_24h <= plts_taken_time
                         ),
            relationship = "one-to-one") |>
  rename(first_adj_platelets = plts_result) |>
  # Remove variable that won't be used moving forward
  select(-c(plts_taken_time))

# Remove `platelets` tibble because it won't be used moving forward
rm(platelets)
```

5.3.2.4.6 SOFA-2 Calculation

Calculate the SOFA-2 score within each organ system, then sum to obtain the SOFA-2 at first adjunct start (`first_adj_sofa_2`). Because of the several clauses in the SOFA-2 document, the handling of these elements will be explicitly outlined below.

ORGAN SYSTEM	CLAUSE (DIRECT QUOTE)	MANAGEMENT
General	For missing values at day 1, the general recommendation is to score these as 0 points. This may vary for specific purposes (eg, bedside use, research, etc). For sequential scoring, for	Because first adjunct vasopressors may be started on day 1 of an encounter or subsequent days, only values obtained prior to first adjunct start were considered for scoring. The timing of values prior to first adjunct start corresponded to their typical

ORGAN SYSTEM	CLAUSE (DIRECT QUOTE)	MANAGEMENT
	missing data after day 1, it is to carry forward the last observation, the rationale being that nonmeasurement suggests stability.	frequency of obtainment and are outlined below. No values were carried forward and missing values will be scored as 0.
Brain	For sedated patients, use the last recorded GCS before sedation. If the previous GCS is unknown, score 0	The GCS without documented administration of sedation in the preceding 6 hours was joined for `first_adj_gcs`. If `first_adj_gcs` is NA, then this sub-score will be scored as 0
Brain	If receiving drug treatment for delirium (short- or long-term), score 1 point even if GCS is 15	Drug treatment for delirium within the preceding 12 hours was joined in `delirium_med_prev_12h`, including antipsychotics and dexmedetomidine
Respiratory	Use the arterial oxygen saturation (SpO_2) to FIO_2 ratio only when the PaO_2:FIO_2 ratio is unavailable and when the SpO_2 is less than 98%. Cutoffs: 0 points, greater than 300 mm Hg; 1 point, 300 mm Hg or less; 2 points, 250 mm Hg or less; 3 points, 200 mm Hg or less with ventilatory support; 4 points, 120 mm Hg or less with ventilatory support or ECMO.	Both PaO_2 to FiO_2 ratio (`p_f_ratio`) and SpO_2 to FiO_2 ratio (`s_f_ratio`) in the preceding 6 hours were joined. The `p_f_ratio` will be used preferentially, with the `s_f_ratio` only used if the `p_f_ratio` is unavailable. Notably, the `s_f_ratio` was only calculated if the SpO_2 was <98%; values >=98% produced a missing value (NA) for `s_f_ratio`.
Respiratory	Advanced ventilatory support is defined as receipt of high-flow nasal cannula, continuous positive airflow pressure, bilevel positive airway pressure, noninvasive ventilation, invasive mechanical ventilation, or long-term home ventilation.	Advanced respiratory support device administration at first adjunct start was joined as `adv_resp_before_adj`. This variable accounted for the start and end times of device administration to ensure administration at first adjunct start.
Respiratory	If used for respiratory failure, ECMO (all forms) should be scored 4 in the respiratory component (regardless of PaO_2:FIO_2ratio), but not in the cardiovascular component. If	ECMO use was unavailable in the dataset, therefore it cannot be considered for score calculation.

ORGAN SYSTEM	CLAUSE (DIRECT QUOTE)	MANAGEMENT
	used for cardiovascular indications (all forms), it should be automatically scored in both the cardiovascular and the respiratory systems.	
Cardiovascular	MAP, norepinephrine and epinephrine dosing, and inotrope and other vasopressor use are considered for scoring.	MAP is only used for scoring in patients not receiving a vasopressor or inotrope. All patients were receiving norepinephrine monotherapy at first adjunct start, therefore only norepinephrine dosing (`adj_baseline_neq`) was considered (MAP, dosing of epinephrine, and other vasopressor use were not considered). Administration of the inotrope dobutamine at first adjunct start was joined as `dbt_before_adj`. The use of other inotropes (e.g., milrinone) was not available in the dataset.
Cardiovascular	Vasopressor medication is only scored if given by continuous intravenous infusion for at least 1 hour	All patients received norepinephrine for at least 15 minutes prior to first adjunct start. Because first adjunct start may have been within 1 hour of norepinephrine start the minimum duration of 1 hour was not included in scoring.
Cardiovascular	Norepinephrine is usually dispensed as the salt (eg, hemitartrate or bitartrate). Dose should be expressed as the base.	All norepinephrine doses are expressed as the base in the dataset.
Cardiovascular	If dopamine is used as a single vasopressor, scoring is based on the following cutoffs: 2 points (<=20 µg/kg/min); 3 points (>20 to <=40 µg/kg/min); 4 points (>40 µg/kg/min).	No patient received dopamine as a single vasopressor in this study.
Cardiovascular	Any type of mechanical cardiovascular support: eg, venoarterial ECMO, intra-aortic balloon pump, left ventricular	Mechanical circulatory (cardiovascular) support is unavailable in the dataset, therefore it cannot be considered for score calculation.

ORGAN SYSTEM	CLAUSE (DIRECT QUOTE)	MANAGEMENT
	assist device, microaxial flow pump.	
Kidney	Serum creatinine and urine output are considered for scoring	Serum creatinine in the preceding 24 hours was joined as `first_adj_scr`. Urine output was not considered for scoring because of known poor documentation of hourly urine output within hospital
settings. This is consistent with the approach in prior work for calculating SOFA score in patients with sepsis. (Ginestra, Ann Thorac Soc 2022, Alrawashdeh, Crit Care Med 2024		
Kidney	Excludes patients receiving RRT exclusively for nonrenal causes (eg, removal of toxic products, bacterial toxins, cytokines).	Patients in the dataset received RRT for renal causes. Use of RRT before first adjunct start was joined as `rrt_before_adj`.
Kidney	For patients receiving intermittent RRT, score 4 points on days not receiving RRT until RRT use is terminated.	The `rrt_before_adj` variable includes continuation of intermittent RRT until first adjunct start.
Liver	Total bilirubin is considered for scoring	Bilirubin in the preceding 24 hours was joined as `first_adj_bili`.
Hemostasis	Platelet count is considered for scoring	Platelets in the preceding 24 hours was joined as `first_adj_platelets`.

First, calculate the SOFA-2 score at first adjunct start (`first_adj_sofa_2`) for reporting.

▼ Code

```
adj <- adj |>
  mutate(
    sofa2_brain = case_when(
      first_adj_gcs_no_sed <= 5 ~ 4,
      first_adj_gcs_no_sed <= 8 ~ 3,
```

```
  first_adj_gcs_no_sed <= 12 ~ 2,
  first_adj_gcs_no_sed <= 14 | delirium_med_prev_12h == 1 ~ 1,
  (first_adj_gcs_no_sed == 15 & delirium_med_prev_12h == 0) |
    is.na(first_adj_gcs_no_sed) ~ 0,
  #Error code if all possibilities not accounted for
  .default = -99
  ),
sofa2_resp = case_when(
  first_adj_pf <=75 &
    adv_resp_before_adj == "Adv Resp Before Adjunct" ~ 4,
  first_adj_pf <=150 &
    adv_resp_before_adj == "Adv Resp Before Adjunct" ~ 3,
  first_adj_pf <=225 ~ 2,
  first_adj_pf <=300 ~ 1,
  first_adj_pf >300 ~ 0,
  is.na(first_adj_pf) & first_adj_sf <=120 &
    adv_resp_before_adj == "Adv Resp Before Adjunct" ~ 4,
  is.na(first_adj_pf) & first_adj_sf <=200 &
    adv_resp_before_adj == "Adv Resp Before Adjunct" ~ 3,
  is.na(first_adj_pf) & first_adj_sf <=250 ~ 2,
  is.na(first_adj_pf) & first_adj_sf <=300 ~ 1,
  is.na(first_adj_pf) & first_adj_sf >300 ~ 0,
  is.na(first_adj_pf) & is.na(first_adj_sf) ~ 0,
  #Error code if all possibilities not accounted for
  .default = -99
  ),
sofa2_cv = case_when(
  adj_baseline_neq >0.4 |
    (adj_baseline_neq >0.2 &
       dbt_before_adj == "Dobutamine Before Adjunct"
     ) ~ 4,
  (adj_baseline_neq >0.2 &
     dbt_before_adj == "No Dobutamine Before Adjunct") |
    (adj_baseline_neq <=0.2 &
       dbt_before_adj == "Dobutamine Before Adjunct") ~ 3,
  adj_baseline_neq <=0.2 &
       dbt_before_adj == "No Dobutamine Before Adjunct" ~ 2,
  #Error code if all possibilities not accounted for
  .default = -99
  ),
sofa2_liver = case_when(
  first_adj_bili <= 1.2 | is.na(first_adj_bili) ~ 0,
  first_adj_bili <= 3 ~ 1,
  first_adj_bili <= 6 ~ 2,
  first_adj_bili <= 12 ~ 3,
  first_adj_bili > 12 ~ 4,
  #Error code if all possibilities not accounted for
  .default = -99
  ),
sofa2_kidney = case_when(
  #Need to adjudicate in this order to ensure correct scoring
```

```
      rrt_before_adj == "RRT Before Adjunct" ~ 4,
      first_adj_scr >3.5 ~ 3,
      first_adj_scr <=1.2 | is.na(first_adj_scr) ~ 0,
      first_adj_scr <=2 ~ 1,
      first_adj_scr <=3.5 ~ 2,
      #Error code if all possibilities not accounted for
      .default = -99
      ),
    sofa2_hemostasis = case_when(
      #Need to adjudicate in this order to ensure correct scoring
      first_adj_platelets <=50 ~ 4,
      first_adj_platelets <=80 ~ 3,
      first_adj_platelets <=100 ~ 2,
      first_adj_platelets <=150 ~ 1,
      first_adj_platelets >150 | is.na(first_adj_platelets) ~ 0,
      #Error code if all possibilities not accounted for
      .default = -99
      )
    )
```

Show distribution of SOFA-2 score components to ensure no score was calculated with the error code (-99) and values match expectations.

▼ Code

```
describe(adj |> select(sofa2_brain, sofa2_resp, sofa2_cv, sofa2_liver, sofa2_kidney,
         sofa2_hemostasis))
```

```
select(adj, sofa2_brain, sofa2_resp, sofa2_cv, sofa2_liver, sofa2_kidney, sofa2_hemostasis)

 6  Variables      8727  Observations
---------------------------------------------------------------------------
sofa2_brain
       n  missing distinct     Info     Mean  pMedian      Gmd
    8727        0        5    0.763   0.9167      0.5    1.327

Value          0     1     2     3     4
Frequency   5367  1142   734   546   938
Proportion 0.615 0.131 0.084 0.063 0.107
---------------------------------------------------------------------------
sofa2_resp
       n  missing distinct     Info     Mean  pMedian      Gmd
    8727        0        5    0.938    1.645      1.5    1.615

Value          0     1     2     3     4
Frequency   3023  1080  1680  1863  1081
Proportion 0.346 0.124 0.193 0.213 0.124
---------------------------------------------------------------------------
sofa2_cv
       n  missing distinct     Info     Mean  pMedian      Gmd
```

```
    8727        0        3    0.879     2.86        3    0.866

Value          2     3     4
Frequency   3498  2953  2276
Proportion 0.401 0.338 0.261
------------------------------------------------------------------------
sofa2_liver
       n  missing distinct     Info     Mean  pMedian      Gmd
    8727        0        5    0.674    0.574      0.5   0.8932

Value          0     1     2     3     4
Frequency   5973  1500   603   301   350
Proportion 0.684 0.172 0.069 0.034 0.040
------------------------------------------------------------------------
sofa2_kidney
       n  missing distinct     Info     Mean  pMedian      Gmd
    8727        0        5    0.942    1.457      1.5    1.443

Value          0     1     2     3     4
Frequency   2651  2263  1878  1047   888
Proportion 0.304 0.259 0.215 0.120 0.102
------------------------------------------------------------------------
sofa2_hemostasis
       n  missing distinct     Info     Mean  pMedian      Gmd
    8727        0        5     0.78    0.861      0.5    1.236

Value          0     1     2     3     4
Frequency   5205  1606   508   740   668
Proportion 0.596 0.184 0.058 0.085 0.077
------------------------------------------------------------------------
```

No score component was calculated with the error code and values match expectations, including `sofa2_cv` only ranging from values 2 to 4. Move on to calculation of the SOFA-2 score (`first_adj_sofa_2`) and show the distribution of the score to ensure expectations are met.

▼ Code

```
adj <- adj |>
  rowwise() |>
  mutate(first_adj_sofa_2 = sofa2_brain + sofa2_resp + sofa2_cv +
           sofa2_liver + sofa2_kidney + sofa2_hemostasis
         ) |>
  ungroup()
describe(adj$first_adj_sofa_2)
```

```
adj$first_adj_sofa_2
       n  missing distinct     Info     Mean  pMedian      Gmd
    8727        0       21    0.991    8.313        8      3.7
     .05      .10      .25      .50      .75      .90      .95
       3        4        6        8       10       13       14
```

```
lowest :  2  3  4  5  6, highest: 18 19 20 21 22
```

The minimum value of the score is 2 points, which matches expectations because the minimum value for the Cardiovascular component is 2 in this cohort. It is not unexpected to have a maximum value of 22 (instead of 24) at first adjunct start since this time point may be relatively early in a patient's septic shock disease progression.

Finally, calculate the SOFA-2 score without the cardiovascular component at first adjunct start (`first_adj_sofa_2_no_cv`) for use in modeling. This variable is needed because of the intent to incorporate norepinephrine dose at first adjunct start (`adj_baseline_neq`) as a variable in the model (to avoid collinearity between the two variables). Then show the distribution of `first_adj_sofa_2_no_cv`.

▼ Code

```
adj <- adj |>
  rowwise() |>
  mutate(first_adj_sofa_2_no_cv = sofa2_brain + sofa2_resp + sofa2_liver +
           sofa2_kidney + sofa2_hemostasis
         ) |>
  ungroup()
describe(adj$first_adj_sofa_2_no_cv)
```

```
adj$first_adj_sofa_2_no_cv
       n  missing distinct     Info     Mean  pMedian      Gmd
    8727        0       20     0.99    5.453      5.5    3.493
     .05      .10      .25      .50      .75      .90      .95
       1        2        3        5        7       10       11

0 (283, 0.032), 1 (465, 0.053), 2 (730, 0.084), 3 (1014, 0.116), 4
(1257, 0.144), 5 (1132, 0.130), 6 (948, 0.109), 7 (795, 0.091), 8
(679, 0.078), 9 (470, 0.054), 10 (361, 0.041), 11 (210, 0.024), 12
(150, 0.017), 13 (91, 0.010), 14 (68, 0.008), 15 (37, 0.004), 16 (24,
0.003), 17 (11, 0.001), 18 (1, 0.000), 19 (1, 0.000)
```

The distribution of `first_adj_sofa_2_no_cv`, with 0 as the lowest value and 19 as the maximum (since 20 would be the maximum possible value without the Cardiovascular component).

5.3.2.5 Mechanical Ventilation

Join mechanical ventilation documented time, if the start of mechanical ventilation (`mv_start`) is prior to `first_adj_time` and the end of mechanical ventilation (`mv_end`) is after `first_adj_time`. Since multiple values of `mv_start` are possible then arrange by time (to put latest `mv_start` as first row with desc()) then use distinct() to keep only the latest `mv_start` before first adjunct start.

▼ Code

```
adj <- left_join(adj,
                 mv |> select(encounter_id, mv_start, mv_end),
                 join_by(encounter_id,
                         closest(first_adj_time >= mv_start),
                         first_adj_time <= mv_end),
                 relationship = "one-to-many") |>
  arrange(encounter_id, first_adj_time, desc(mv_start)) |>
  distinct(encounter_id, .keep_all = TRUE) |>
  mutate(mv_before_adj =
           factor(if_else(!is.na(mv_start),
                          "MV Before Adjunct", "No MV Before Adjunct")),
         mv_before_adj =
           fct_relevel(mv_before_adj, "No MV Before Adjunct")
         ) |>
  select(-c(mv_start, mv_end))

# Remove `mv` tibble because it won't be used moving forward
rm(mv)
```

5.3.2.6 Arterial pH

Join arterial pH values closest to first adjunct start and within 6 hours prior. Measured pH values from an arterial blood gas will be used preferentially. However, arterial pH values are commonly unavailable in patients outside of the intensive care unit and venous pH is used as a reasonable clinical surrogate when a correction factor is applied Byrne 2014. Therefore, when a measured arterial pH value is unavailable, use a calculated arterial pH value from a venous sample.

Both measured arterial pH (`pH_result`) and calculated arterial pH from a venous sample (`pH_result_calc`) are available in the `pH` tibble. In the call below, join both the closest value for `pH_result` and `pH_result_calc` at first adjunct start, then populate the `first_adj_pH` variable with `pH_result` when available, if not `pH_result_calc` when available.

▼ Code

```
adj <-
  left_join(adj,
            pH |>
              select(encounter_id, pH_taken_time, pH_result) |>
              arrange(encounter_id, pH_taken_time, pH_result) |>
              distinct(encounter_id, pH_taken_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= pH_taken_time),
                         first_adj_minus_6h <= pH_taken_time
                         ),
            relationship = "one-to-one") |>
  select(-pH_taken_time) |>
  left_join(pH |>
              select(encounter_id, pH_taken_time, pH_result_calc) |>
              arrange(encounter_id, pH_taken_time, pH_result_calc) |>
```

```
              distinct(encounter_id, pH_taken_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= pH_taken_time),
                         first_adj_minus_6h <= pH_taken_time
                         ),
            relationship = "one-to-one") |>
  mutate(first_adj_pH = coalesce(pH_result, pH_result_calc)) |>
  # Remove variables that won't be used moving forward
  select(-c(pH_taken_time, pH_result, pH_result_calc))

# Remove `pH` tibble because it won't be used moving forward
rm(pH)
```

5.3.2.7 Temperature

Join the closest temperature value (`temp_c`) from the `temperature` tibble, restricted to values obtained within 6 hours prior to first adjunct start.

▼ Code

```
adj <-
  left_join(adj,
            temperature |>
              select(encounter_id, temp_time, temp_c) |>
              arrange(encounter_id, temp_time, temp_c) |>
              distinct(encounter_id, temp_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= temp_time),
                         first_adj_minus_6h <= temp_time
                         ),
            relationship = "one-to-one") |>
  rename(first_adj_temp_c = temp_c) |>
  # Remove variable that won't be used moving forward
  select(-c(temp_time))

# Remove `temperature` tibble because it won't be used moving forward
rm(temperature)
```

5.3.2.8 Heart Rate

Join the closest heart rate value (`hr`) from the `heart_rate` tibble, restricted to values obtained within 6 hours prior to first adjunct start. Since all patients had a perfusing heart rhythm at the time of first adjunct start, filter out `hr` values <20 bpm or >200 bpm prior to joining the data since these values are likely erroneous (confirmed on random review of values).

▼ Code

```
adj <-
  left_join(adj,
```

```
            heart_rate |>
              filter(hr >=20 & hr <=200) |>
              select(encounter_id, hr_time, hr) |>
              arrange(encounter_id, hr_time, hr) |>
              distinct(encounter_id, hr_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= hr_time),
                         first_adj_minus_6h <= hr_time
                         ),
            relationship = "one-to-one") |>
  rename(first_adj_heart_rate = hr) |>
  # Remove variable that won't be used moving forward
  select(-c(hr_time))

# Remove `heart_rate` tibble because it won't be used moving forward
rm(heart_rate)
```

5.3.2.9 Respiratory Rate

Join the closest respiratory rate value (`rr`) from the `resp_rate` tibble, restricted to values obtained within 6 hours prior to first adjunct start. Since all patients had a perfusing heart rhythm at the time of first adjunct start, filter out `rr` values <10 breaths per minute or >70 breaths per minute prior to joining the data since these values are likely erroneous (confirmed on random review of values).

▼ Code

```
adj <-
  left_join(adj,
            resp_rate |>
              filter(rr >=10 & rr <=70) |>
              select(encounter_id, rr_time, rr) |>
              arrange(encounter_id, rr_time, rr) |>
              distinct(encounter_id, rr_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= rr_time),
                         first_adj_minus_6h <= rr_time
                         ),
            relationship = "one-to-one") |>
  rename(first_adj_resp_rate = rr) |>
  # Remove variable that won't be used moving forward
  select(-c(rr_time))

# Remove `resp_rate` tibble because it won't be used moving forward
rm(resp_rate)
```

5.3.2.10 Mean Arterial Pressure (MAP)

Join the closest MAP value (`closest_map`) at first adjunct start, which was already adjudicated for each vasopressor dose above in the `pressors_doses` tibble. Note that this is the closest MAP value

within 3 hours of first adjunct start.

▼ Code

```
adj <-
  left_join(adj,
            pressors_doses |>
              select(encounter_id, mar_taken_time, closest_map),
            by = join_by(encounter_id,
                         first_adj_time == mar_taken_time),
            relationship = "one-to-many"
            ) |>
  distinct(encounter_id, .keep_all = TRUE) |>
  rename(first_adj_map = closest_map)
```

5.3.2.11 Lactate

Join the closest lactate lab value (`lactate_result`) from the `lactate` tibble, restricted to values obtained within 12 hours prior to first adjunct start. The rationale for selecting a window of 12 hours before first adjunct start (instead of a shorter window such as 6 hours used with other variables) is because relatively low values may not be followed serially resulting in sparse sampling.

▼ Code

```
adj <-
  left_join(adj |>
              mutate(first_adj_minus_12h = first_adj_time - dhours(12)),
            lactate |>
              select(encounter_id, lactate_taken_time, lactate_result) |>
              arrange(encounter_id, lactate_taken_time, lactate_result) |>
              distinct(encounter_id, lactate_taken_time, .keep_all = TRUE),
            by = join_by(encounter_id,
                         closest(first_adj_time >= lactate_taken_time),
                         first_adj_minus_12h <= lactate_taken_time
                         ),
            relationship = "one-to-one") |>
  rename(first_adj_lactate = lactate_result) |>
  # Remove variable that won't be used moving forward
  select(-c(lactate_taken_time))

# Remove `lactate` tibble because it won't be used moving forward
rm(lactate)
```

5.3.2.12 Fluid Boluses

Determine the cumulative volume of fluid boluses administered within the 12 hours prior to first adjunct start. First, join the `first_adj_time` and `first_adj_minus_12h` vectors from the `adj` tibble to the `fluids` tibble. Since this relationship is expected to have a "many-to-one" relationship, specify it in the call. Then, determine if the fluid administration date/time (`mar_taken_time`) was within the

outlined time range. Lastly, only retain observations meeting these criteria and remove vectors that will not be used moving forward.

▼ Code

```
fluids_adj <- fluids |>
  left_join(adj |> select(encounter_id, first_adj_time, first_adj_minus_12h),
            join_by(encounter_id),
            relationship = "many-to-one"
            ) |>
  mutate(within_prior_12h =
           if_else(mar_taken_time >= first_adj_minus_12h &
                     mar_taken_time <= first_adj_time, "Yes", "No")
         ) |>
  filter(within_prior_12h == "Yes")
```

Next, calculate the cumulative sum of the fluid bolus volume administered (`vol_mL`) per `encounter_id` in a new vector (`fluid_mL_12h_prior_adj`).

▼ Code

```
fluids_adj  <- fluids_adj  |>
  group_by(encounter_id) |>
  mutate(fluid_mL_12h_prior_adj = cumsum(vol_mL)) |>
  ungroup()
```

Lastly, join the largest value of the `fluid_mL_12h_prior_adj` vector per observation (representing the cumulative volume of fluid administered within the 12 hours prior to first adjunct start) to the `adj` tibble. Some patients (`encounter_id`s) were not present in the `fluids_adj` tibble because they did not have fluid boluses administered within the 12 hours prior to vasopressor start. In these cases the `fluid_mL_12h_prior_adj` vector will have a value of “NA” after the join. Replace these “NA” values with zero as part of the call.

▼ Code

```
adj <- left_join(adj,
                 fluids_adj  |>
                   arrange(encounter_id, desc(fluid_mL_12h_prior_adj)) |>
                   select(encounter_id, fluid_mL_12h_prior_adj) |>
                   distinct(encounter_id, .keep_all = TRUE),
                 join_by(encounter_id),
                 relationship = "one-to-one"
                 ) |>
  mutate(fluid_mL_12h_prior_adj = replace_na(fluid_mL_12h_prior_adj, 0),
         fluid_mL_kg_12h_prior_adj = fluid_mL_12h_prior_adj / first_adj_wt_kg
         ) |>
  rename(first_adj_fluid_mL = fluid_mL_12h_prior_adj,
         first_adj_fluid_mL_kg = fluid_mL_kg_12h_prior_adj
```

```
        )
# Remove fluids tibbles since they won't be used moving forward
rm(fluids, fluids_adj)
```

5.3.2.13 Steroids

Determine the presence of at least one parenteral or enteral dose of any glucocorticoid medication administered within the 24 hour period prior to first adjunct start. All observations in the `steroids` tibble represent parenteral or enteral administration of a glucocorticoid medication (also known as a "corticosteroid", or, simply "steroid").

First, join the `first_adj_time` and `first_adj_minus_24h` variables from the `adj` tibble to the `steroids` tibble.

▼ Code

```
steroids <- left_join(steroids,
                      adj |> select(encounter_id, first_adj_time,
                                    first_adj_minus_24h
                                    ) |>
                        distinct(),
                      join_by(encounter_id),
                      relationship = "many-to-one")
```

Next, assess steroid administration within the outlined time period (between 24 hours prior to first adjunct start and first adjunct start) based on the steroid MAR taken time.

▼ Code

```
steroids <- steroids |>
  mutate(during_window = if_else(mar_taken_time >= first_adj_minus_24h &
                                   mar_taken_time <= first_adj_time,
                                 "Yes", "No")) |>
  filter(during_window == "Yes")
```

Lastly, create a vector within the `adj` tibble for steroid administration during the outlined time period (`first_adj_steroid`) and remove the `steroid` tibble because it won't be used moving forward.

▼ Code

```
adj <- adj |>
  mutate(first_adj_steroid =
           factor(if_else(encounter_id %in% steroids$encounter_id,
                          "Steroids", "No Steroids"
                          ))
         )
rm(steroids)
```

5.3.2.14 LVEF

Join left ventricular ejection fraction (LVEF) within the 30 days prior to adjunct start. Ideally only the LVEF after norepinephrine initiation (and prior to adjunct start) would be used, but if this approach were to be used the missingness percentage would be unreasonable for an imputation model (85%, data not shown).

▼ Code

```
adj <-
  left_join(
    adj |> mutate(first_adj_minus_30d = first_adj_time - ddays(30)),
    lvef |> select(patient_id, observation_date, lvef),
    join_by(patient_id,
            closest(first_adj_time >= observation_date),
            first_adj_minus_30d <= observation_date),
    relationship = "one-to-many") |>
  arrange(encounter_id, desc(observation_date)) |>
  distinct(encounter_id, .keep_all = TRUE) |>
  arrange(encounter_id, admission_time) |>
  select(-first_adj_minus_30d, observation_date) |>
  rename(first_adj_lvef = lvef)

# Remove the `lvef` tibble since it won't be used moving forward
rm(lvef)
```

Show missingness percentage in the `first_adj_lvef` column.

▼ Code

```
round_half_up((sum(is.na(adj$first_adj_lvef))/nrow(adj))*100,0)
```

```
[1] 61
```

In all patients who received an adjunctive vasopressor regimen, 61% were missing an LVEF measurement within 30 days prior to adjunct start. This missingness percentage is not ideal, but at a reasonable level to move forward.

6 Sample Size Calculation

The prevalence of positive response to adjunctive vasopressors is unclear. Previous data using similar criteria ranged from 40-45% for vasopressin (Sacha 2018, Sacha 2022) and was 25% for epinephrine in one study (Ammar 2019). Yet, no studies exist for response to phenylephrine. Because phenylephrine and epinephrine are catecholamine analogs that activate the adrenergic alpha-1 receptor similar to norepinephrine, their adjunctive use may lead to less frequent blood pressure response. Therefore, for the sample size calculation will use a lower response prevalence estimate (30%).

For a binary categorical outcome (`response`), assuming outcome prevalence 0.3, expected C-statistic 0.75, shrinkage 0.9, and 15 candidate predictor parameters, calculate the minimum sample size using `pmsampsize()` from the `{pmsampsize}` package.

▼ Code

```
pmsampsize(type = "b",
           parameters = 15,
           prevalence = 0.3,
           cstatistic = 0.75
           )
```

```
Given input C-statistic = 0.75  & prevalence = 0.3
Cox-Snell R-sq = 0.1593

NB: Assuming 0.05 acceptable difference in apparent & adjusted R-squared
NB: Assuming 0.05 margin of error in estimation of intercept
NB: Events per Predictor Parameter (EPP) assumes prevalence = 0.3

           Samp_size Shrinkage Parameter CS_Rsq Max_Rsq Nag_Rsq   EPP
Criteria 1       771     0.900        15 0.1593   0.705   0.226 15.42
Criteria 2       383     0.819        15 0.1593   0.705   0.226  7.66
Criteria 3       323     0.900        15 0.1593   0.705   0.226  6.46
Final            771     0.900        15 0.1593   0.705   0.226 15.42

 Minimum sample size required for new model development based on user inputs = 771,
 with 232 events (assuming an outcome prevalence = 0.3) and an EPP = 15.42
```

Each derivation (first adjunctive vasopressor use) cohort will need at least 771 included patients to have adequate power to estimate a clinical prediction model (with the above assumptions).

7 Eligibility Criteria

The goal of this work is to predict second-line (first adjunctive) vasopressor response in patients with septic shock receiving norepinephrine. Eligibility criteria will be outlined below and subsequently applied in a code chunk.

1. Adults (age ≥18 years) with septic shock per clinical criteria (CDC Adult Sepsis Event and/or Sepsis-3)
    - Restricted to the first sepsis episode on the patient level
2. Norepinephrine as initial therapy
3. Norepinephrine duration ≥15 minutes
4. Norepinephrine monotherapy duration ≥15 minutes
    - Each of these first four criteria were implemented in the prior work to create the imported `ne_incl` tibble.

5. Adjunctive vasopressor receipt
    - Initiation of a non-norepinephrine vasopressor and documentation of a non-zero norepinephrine dosage at the time of non-norepinephrine vasopressor initiation
6. Second-line (first adjunctive) vasopressor administration of vasopressin, epinephrine, or phenylephrine
    - Although the use of adjunctive phenylephrine is not recommended for or against by the 2026 Surviving Sepsis Campaign guidelines, it is often used in clinical practice - particularly for patients with tachycardia/tachyarrhythmias on norepinephrine or as an alternative to vasopressin due to lower cost. Phenylephrine use notations in Surviving Sepsis Campaign guideline iterations that were released during the study time period are outlined below
    - 2008: Recommendation: “We suggest that epinephrine, phenylephrine, or vasopressin should not be administered as the initial vasopressor in septic shock.” (No recommendation regarding adjunct use.). Text body: “Phenylephrine is the adrenergic agent least likely to produce tachycardia but as a pure vasopressor would be expected to decrease stroke volume.”
    - 2012: Recommendation: “Phenylephrine is not recommended in the treatment of septic shock except in circumstances where (a) norepinephrine is associated with serious arrhythmias, (b) cardiac output is known to be high and blood pressure persistently low, or (c) as salvage therapy when combined inotrope/vasopressor drugs and low-dose vasopressin have failed to achieve MAP target”
    - 2016: No recommendation. Text body: “A network meta-analysis resulted in imprecise estimates (wide confidence intervals) when phenylephrine was compared to other vasopressors (281). Therefore, the impact on clinical outcomes is uncertain, and phenylephrine use should be limited until more research is available.”
    - 2021: No recommendation. Not mentioned in text body.

In the call below, adjudicate and tabulate eligibility criteria.

▼ Code

```
ne_incl <- ne_incl |>
  left_join(adj |> select(encounter_id, adj_baseline_neq),
            by = join_by(encounter_id),
            relationship = "one-to-one") |>
  mutate(elig = case_when(
    event_3_level_type != "1-adjunct" ~ "exclude-1_no_adjunct",
    first_adj %notin% c("vasopressin", "epinephrine", "phenylephrine") ~
      "exclude-2_adjunct_regimen_not_of_interest",
    adj_baseline_neq == 0 ~ "exclude-3_not_adjunctive;baseline_neq_zero",
    .default = "include"
    )) |>
  select(-adj_baseline_neq)

ne_incl |>
  tabyl(elig, shock_onset_hosp_state) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
```

```
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  kable(align = c('lcc'),
        caption = "Eligibility Adjudication"
        ) |>
  kable_paper(full_width = FALSE)
```

Eligibility Adjudication

ELIG/SHOCK_ONSET_HOSP_STATE	OHIO	FLORIDA	TOTAL
exclude-1_no_adjunct	15,382 (67.2%)	1,873 (60.8%)	17,255 (66.4%)
exclude-2_adjunct_regimen_not_of_interest	333 (1.5%)	48 (1.6%)	381 (1.5%)
exclude-3_not_adjunctive;baseline_neq_zero	254 (1.1%)	30 (1.0%)	284 (1.1%)
include	6,934 (30.3%)	1,128 (36.6%)	8,062 (31.0%)
Total	22,903 (100.0%)	3,079 (100.0%)	25,982 (100.0%)

Variables of interest were transformed or joined to the `adj` tibble, which only includes patients who received adjunctive vasopressors. Use this tibble and apply the other eligibility criterion to create analytic tibble (`incl`). Then show the number of observations in the `incl` tibble to ensure the numbers align with above.

▼ Code

```
incl <- adj |>
  filter(first_adj %in% c("vasopressin", "epinephrine", "phenylephrine") &
           adj_baseline_neq != 0
           ) |>
  droplevels()

count(incl, .by = shock_onset_hosp_state) |>
  rename("State" = .by) |>
  kable(align = c('lc'),
        caption = "Included Count"
        ) |>
  kable_paper(full_width = FALSE)
```

Included Count

STATE	N
Ohio	6934
Florida	1128

The counts match, therefore continue with the analysis.

Show the first adjunctive vasopressor received in included patients by cohort.

▼ Code

```
incl |>
  tabyl(first_adj, shock_onset_hosp_state) |>
  adorn_totals() |>
  rename("First Adjunct" = first_adj) |>
  kable(align = c('lcc'),
        caption = "First Adjunctive Vasopressor"
        ) |>
  kable_paper(full_width = FALSE)
```

First Adjunctive Vasopressor

FIRST ADJUNCT	OHIO	FLORIDA
vasopressin	5289	976
epinephrine	775	95
phenylephrine	870	57
Total	6934	1128

8 Setting Characteristics

Count the number of distinct healthcare locations (settings) from which the data were derived, by state.

▼ Code

```
incl |>
  summarise(
    n_settings = n_distinct(first_adj_hospital),
    .by = first_adj_hosp_state
  ) |>
  rename("State" = first_adj_hosp_state,
         "Treatment Locations (n)" = n_settings) |>
  kable(align = c('lc'),
        caption = "Count of Locations by State"
        ) |>
  kable_paper(full_width = FALSE)
```

Count of Locations by State

STATE	TREATMENT LOCATIONS (N)

STATE	TREATMENT LOCATIONS (N)
Florida	5
Ohio	18

Included patients were treated in 18 distinct locations in Ohio (comprising the model derivation cohorts) and 5 distinct locations in Florida (comprising the validation cohorts).

9 Outcome Measure Assessment

Describe outcome measure components by first adjunctive vasopressor by state.

▼ Code

```
incl |>
  tabyl(response_type, first_adj, first_adj_hosp_state) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front")
```

```
$Ohio
                response_type    vasopressin  epinephrine
                   1-response 1,833  (34.7%) 222  (28.6%)
 0-no_resp_neq_same_or_higher 2,930  (55.4%) 452  (58.3%)
       0-no_resp_map_below_65   356   (6.7%)  52   (6.7%)
              0-no_resp_death   144   (2.7%)  36   (4.6%)
   2-resp_unclear_map_missing    26   (0.5%)  13   (1.7%)
                        Total 5,289 (100.0%) 775 (100.0%)
 phenylephrine          Total
  319  (36.7%) 2,374  (34.2%)
  445  (51.1%) 3,827  (55.2%)
   75   (8.6%)   483   (7.0%)
   23   (2.6%)   203   (2.9%)
    8   (0.9%)    47   (0.7%)
  870 (100.0%) 6,934 (100.0%)

$Florida
                response_type  vasopressin epinephrine phenylephrine
                   1-response 274  (28.1%) 21  (22.1%)   20  (35.1%)
 0-no_resp_neq_same_or_higher 648  (66.4%) 71  (74.7%)   37  (64.9%)
       0-no_resp_map_below_65  35   (3.6%)  1   (1.1%)    0   (0.0%)
              0-no_resp_death  17   (1.7%)  2   (2.1%)    0   (0.0%)
   2-resp_unclear_map_missing   2   (0.2%)  0   (0.0%)    0   (0.0%)
                        Total 976 (100.0%) 95 (100.0%)   57 (100.0%)
          Total
   315  (27.9%)
   756  (67.0%)
```

```
    36   (3.2%)
    19   (1.7%)
     2   (0.2%)
 1,128 (100.0%)
```

As noted in Section 5.2, the 49 observations (47 and 2 observations in the Ohio and Florida cohorts, respectively) with missing MAP data were considered as not meeting criteria for response. Show the outcomes with this response type collapsed with the "0-no_resp_map_below_65" category.

▼ Code

```
(out_by_cohort <- incl |>
   select(response_type, first_adj, first_adj_hosp_state) |>
   mutate(response_type = case_when(
     response_type == "2-resp_unclear_map_missing" ~
       "0-no_resp_map_below_65_or_missing",
     response_type == "0-no_resp_map_below_65" ~
       "0-no_resp_map_below_65_or_missing",
     .default = response_type
     ),
     response_type = fct_relevel(response_type, "1-response",
                                 "0-no_resp_neq_same_or_higher",
                                 "0-no_resp_map_below_65_or_missing",
                                 "0-no_resp_death"
                                 )
     ) |>
   tbl_strata(
     strata = first_adj_hosp_state,
     .tbl_fun =
       ~ .x |>
       tbl_summary(by = first_adj, missing = "no",
                   statistic = all_categorical() ~ "{n} ({p}%)",
                   digits = all_categorical() ~ c(0, 1)
                   ) |>
       add_overall()
    ) |>
    modify_fmt_fun(
     all_stat_cols() ~ function(x) gsub("%", "", x)
     ))
```

		Ohio			
Characteristic	**Overall** N = 6,934[1]	**vasopressin** N = 5,289[1]	**epinephrine** N = 775[1]	**phenylephrine** N = 870[1]	**Overall** N = 1,128[1]
response_type					

[1] n (%)

Characteristic	Ohio				
	Overall N = 6,934[1]	**vasopressin** N = 5,289[1]	**epinephrine** N = 775[1]	**phenylephrine** N = 870[1]	**Overall** N = 1,128[1]
1-response	2,374 (34.2)	1,833 (34.7)	222 (28.6)	319 (36.7)	315 (27.9)
0-no_resp_neq_same_or_higher	3,827 (55.2)	2,930 (55.4)	452 (58.3)	445 (51.1)	756 (67.0)
0-no_resp_map_below_65_or_missing	530 (7.6)	382 (7.2)	65 (8.4)	83 (9.5)	38 (3.4)
0-no_resp_death	203 (2.9)	144 (2.7)	36 (4.6)	23 (2.6)	19 (1.7)

[1] n (%)

As noted in Section 5.2, the incidence of response after vasopressin initiation (34% and 28%, respectively, in the Derivation and Validation cohorts) is lower than in previous work from our research team with vasopressin (40%), but is reasonably close. The difference is explainable because patients and criteria in the previous work were slightly different - in this work all patients received initial norepinephrine (previously all initial vasopressors were included), death between first adjunct start and 6 hours later was considered non-response (not considered in prior work), the vasopressor contribution of vasopressin (equated through the Kotani norepinephrine-equivalent dose [NEQ]) was considered in the response criteria (previously it was ignored), and the NEQ was required to be lower than baseline (not lower or equal to baseline since the vasopressin NEQ contribution was considered). Inclusion of the vasopressor contribution of vasopressin as part of the NEQ is likely the largest driver of the difference in response incidence between studies.

Export to Word

▼ Code

```
as_flex_table(out_by_cohort) |>
  autofit() |>
  set_table_properties(layout = "autofit") |>
  font(fontname = "Arial", part = "all") |>
  fontsize(size = 10, part = "all") |>
  line_spacing(space = 1, part = "all") |>
  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
  save_as_docx(
    path = "outcome_components_by_cohort.docx",
    pr_section = prop_section(
      page_size = page_size(orient = "landscape", width = 11, height = 8.5),
```

```
      page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
      )
    )
```

Compare response frequency by first adjunctive vasopressor in the derivation (Ohio) cohort.

▼ Code

```
incl |>
  filter(first_adj_hosp_state == "Ohio") |>
  tabyl(first_adj, response) |>
  janitor::chisq.test()
```

```
    Pearson's Chi-squared test

data:  tabyl(filter(incl, first_adj_hosp_state == "Ohio"), first_adj,     response)
X-squared = 13.458, df = 2, p-value = 0.001196
```

▼ Code

```
glm(response ~ first_adj,
    data = incl |> filter(first_adj_hosp_state == "Ohio"),
    family = binomial(link = "logit")
    ) |>
  tidy(exponentiate = TRUE, conf.int = TRUE) |>
  select(term, estimate, conf.low, conf.high, p.value) |>
  mutate(term = sub("^first_adj", "", term)) |>
  kable(align = c('lcccc'), digits = 2) |>
  kable_paper(full_width = FALSE)
```

TERM	ESTIMATE	CONF.LOW	CONF.HIGH	P.VALUE
(Intercept)	0.53	0.50	0.56	0.00
epinephrine	0.76	0.64	0.89	0.00
phenylephrine	1.09	0.94	1.27	0.25

Compared with vasopressin, receipt of epinephrine as the first adjunctive vasopressor was associated with 34% lower odds of hemodynamic response at 6 hours (OR 0.76, 95% CI; 0.64, 0.89). There was no substantial difference in response between phenylephrine and vasopressin (OR 1.09, 95% CI; 0.94, 1.27).

Determine the association of adjunctive vasopressor response with hospital mortality in the derivation and validation cohorts.

▼ Code

```
incl |>
  filter(first_adj_hosp_state == "Ohio") |>
  mutate(response = fct_relevel(response, "Response")) |>
  with(table(response, mort_hosp)) |>
  Epi::twoby2()
```

```
2 by 2 table analysis:
------------------------------------------------------
Outcome   : Died
Comparing : Response vs. No Response

            Died Survived    P(Died) 95% conf. interval
Response    1051     1318     0.4436    0.4237   0.4637
No Response 2555     1999     0.5610    0.5466   0.5754

                                    95% conf. interval
             Relative Risk:  0.7908    0.7508   0.8329
         Sample Odds Ratio:  0.6239    0.5645   0.6895
Conditional MLE Odds Ratio:  0.6240    0.5638   0.6904
    Probability difference: -0.1174   -0.1419  -0.0927

             Exact P-value: 0.0000
        Asymptotic P-value: 0.0000
------------------------------------------------------
```

▼ Code

```
incl |>
  filter(first_adj_hosp_state == "Florida") |>
  mutate(response = fct_relevel(response, "Response")) |>
  with(table(response, mort_hosp)) |>
  Epi::twoby2()
```

```
2 by 2 table analysis:
------------------------------------------------------
Outcome   : Died
Comparing : Response vs. No Response

            Died Survived    P(Died) 95% conf. interval
Response     117      197     0.3726    0.3209   0.4274
No Response  371      437     0.4592    0.4250   0.4937

                                    95% conf. interval
             Relative Risk:  0.8115    0.6902   0.9541
         Sample Odds Ratio:  0.6996    0.5354   0.9140
Conditional MLE Odds Ratio:  0.6998    0.5301   0.9214
    Probability difference: -0.0865   -0.1486  -0.0221

             Exact P-value: 0.0089
```

```
        Asymptotic P-value: 0.0088
------------------------------------------------------
```

In the derivation cohort, in the 2,369 patients who had hemodynamic response at 6 hours, hospital mortality was lower (n = 1051 [42.4%]) compared with the 4,554 patients without response (n = 2,555 [56.1%]; unadjusted OR 0.62 [95% CI 0.56, 0.69]). Findings were similar in the validation cohort, where hospital mortality developed in 117 of 314 patients with hemodynamic response (37.3%) versus 371 of 808 (45.9%) patients without hemodynamic response (unadjusted OR 0.70 [95% CI 0.54, 0.91]).

Use a simple logistic regression model to determine risk-adjusted odds ratio for mortality with adjunctive vasopressor response. In these models, use the complete SOFA-2 score for risk adjustment.

▼ Code

```
glm(mort_hosp == "Died" ~ response + age + elixhauser_index + first_adj_sofa_2,
    data = incl |> filter(first_adj_hosp_state == "Ohio"),
    family = binomial(link = "logit")
    ) |>
  tidy(exponentiate = TRUE, conf.int = TRUE) |>
  select(term, estimate, conf.low, conf.high) |>
  rename(OR = estimate) |>
  mutate(term = sub("^response", "", term))
```

```
# A tibble: 5 × 4
  term                OR conf.low conf.high
  <chr>            <dbl>    <dbl>     <dbl>
1 (Intercept)      0.0661   0.0494    0.0880
2 Response         0.626    0.563     0.696
3 age              1.02     1.01      1.02
4 elixhauser_index 1.00     1.00      1.01
5 first_adj_sofa_2 1.24     1.22      1.26
```

▼ Code

```
glm(mort_hosp == "Died" ~ response + age + elixhauser_index + first_adj_sofa_2,
    data = incl |> filter(first_adj_hosp_state == "Florida"),
    family = binomial(link = "logit")
    ) |>
  tidy(exponentiate = TRUE, conf.int = TRUE) |>
  select(term, estimate, conf.low, conf.high) |>
  rename(OR = estimate) |>
  mutate(term = sub("^response", "", term))
```

```
# A tibble: 5 × 4
  term                OR conf.low conf.high
  <chr>            <dbl>    <dbl>     <dbl>
1 (Intercept)      0.0721   0.0350     0.146
```

```
2 Response          0.701    0.529      0.927
3 age               1.01     1.01       1.02
4 elixhauser_index  1.00     0.992      1.01
5 first_adj_sofa_2  1.20     1.16       1.26
```

10 Patient Characteristics

Describe the characteristics of included patients in the derivation (Ohio) and external validation (Florida) cohorts by cohort (by state).

▼ Code

```
(char_by_cohort <- incl |>
   droplevels() |>
   mutate(first_adj_hosp_beds_cat =
            fct_relevel(first_adj_hosp_beds_cat, "<200", "200-499"),
          ssc_era = fct_relevel(ssc_era, "SSC 2008", "SSC 2012", "SSC 2016"),
          adv_resp_before_adj =
            fct_relevel(adv_resp_before_adj, "Adv Resp Before Adjunct"),
          mv_before_adj = fct_relevel(mv_before_adj, "MV Before Adjunct"),
          response = fct_relevel(response, "Response")
          ) |>
  select(age, sex, first_adj_wt_kg, race_eth, elixhauser_index,
         cerebrovascular_disease, hypertension, heart_failure,
         valvular_disease, chronic_pulmonary_disease,
         pulmonary_circulation_disease, liver_disease, kidney_disease,
         malignancy, diabetes, obesity, alcohol_abuse, first_adj_lvef,
         first_adj_hosp_beds_cat, first_adj_hosp_teaching_status,
         first_adj_tx_unit, ssc_era,  criteria_met, comm_onset, prior_surg,
         first_adj_abx, first_adj_sofa_2, first_adj_gcs_no_sed,
         adv_resp_before_adj, mv_before_adj, first_adj_pf, first_adj_pH,
         first_adj_scr, first_adj_aki, rrt_before_adj, first_adj_platelets,
         first_adj_bili, first_adj_temp_c, first_adj_heart_rate,
         first_adj_resp_rate, first_adj_map, first_adj_lactate,
         first_adj_fluid_mL, first_adj_fluid_mL_kg, adj_baseline_neq,
         adj_baseline_neq_mcg_min, hours_to_first_adj, dbt_before_adj,
         first_adj_steroid, first_adj, pressors_dur_hours, abx_days, mort_hosp,
         first_adj_hosp_state
         ) |>
  tbl_summary(
    by = first_adj_hosp_state,
    digits = list(
      all_categorical() ~ c(0, 1),
      first_adj_gcs_no_sed ~ 0,
      first_adj_sofa_2 ~ 0,
      first_adj_temp_c ~ 1
      ),
    label = list(response = "Adjunct Response",
                 age = "Age",
```

```
                sex = "Sex",
                race_eth = "Race/Ethnicity",
                prior_surg = "Prior surgery",
                elixhauser_index = "Elixhauser comorbidity index",
                ssc_era = "Surviving Sepsis Campaign guideline treatment era",
                wt_kg = "Weight (kg)",
                first_adj_hosp_beds_cat = "Hospital beds",
                first_adj_hosp_teaching_status = "Teaching status",
                first_adj_tx_unit = "Treatment unit",
                criteria_met = "Clinical criteria met",
                first_adj_lvef = "LVEF (%)",
                first_adj_sofa_2 = "SOFA-2",
                adv_resp_before_adj = "Advanced respiratory support",
                mv_before_adj = "Mechanical ventilation",
                rrt_before_adj = "Kidney replacement therapy",
                first_adj_fluid_mL = "Fluid bolus volume administered (mL)",
                first_adj_fluid_mL_kg = "Fluid bolus volume administered (mL/kg)",
                first_adj_steroid = "Corticosteroid administration",
                adj_baseline_neq = "Norepinephrine dosage (mcg/kg/min)",
                adj_baseline_neq_mcg_min = "Norepinephrine dosage (mcg/min)",
                pressors_dur_hours = "Vasopressor duration (hours)",
                abx_days = "Antibiotic duration (days)",
                mort_hosp = "In-Hospital mortality",
                first_adj = "First Adjunctive Vasopressor"
                ),
  missing = "no"
   ) |>
  modify_fmt_fun(
    all_stat_cols() ~ function(x) gsub("%", "", x)
 ) |>
 modify_caption("**Patient Characteristics by Cohort**")
)
```

Patient Characteristics by Cohort

Characteristic	Ohio N = 6,934[1]	Florida N = 1,128[1]
Age	67 (57, 76)	69 (59, 77)
Sex		
Female	3,042 (43.9)	471 (41.8)
Male	3,891 (56.1)	657 (58.2)
first_adj_wt_kg	84 (69, 102)	80 (67, 96)

[1] Median (Q1, Q3); n (%)

Characteristic	Ohio N = 6,934[1]	Florida N = 1,128[1]
Race/Ethnicity		
Black, non-Hispanic	1,371 (21.1)	150 (13.5)
Hispanic or Latino	114 (1.8)	200 (18.0)
Multiracial, non-Hispanic	59 (0.9)	19 (1.7)
Other, non-Hispanic	69 (1.1)	21 (1.9)
White, non-Hispanic	4,883 (75.2)	721 (64.9)
Elixhauser comorbidity index	8 (0, 21)	5 (0, 17)
cerebrovascular_disease	696 (10.0)	71 (6.3)
hypertension	2,966 (42.8)	355 (31.5)
heart_failure	1,309 (18.9)	175 (15.5)
valvular_disease	770 (11.1)	104 (9.2)
chronic_pulmonary_disease	1,136 (16.4)	111 (9.8)
pulmonary_circulation_disease	414 (6.0)	66 (5.9)
liver_disease	817 (11.8)	117 (10.4)
kidney_disease	1,073 (15.5)	160 (14.2)
malignancy	1,495 (21.6)	231 (20.5)
diabetes	1,652 (23.8)	166 (14.7)
obesity	1,437 (20.7)	162 (14.4)
alcohol_abuse	344 (5.0)	35 (3.1)
LVEF (%)	56 (45, 65)	56 (40, 64)
Hospital beds		
<200	622 (9.0)	92 (8.2)
200-499	1,702 (24.5)	1,036 (91.8)

[1] Median (Q1, Q3); n (%)

Characteristic	Ohio N = 6,934[1]	Florida N = 1,128[1]
≥500	4,610 (66.5)	0 (0.0)
Teaching status		
Teaching	6,559 (94.6)	783 (69.4)
Non-teaching	375 (5.4)	345 (30.6)
Treatment unit		
Cardiac ICU	701 (10.1)	0 (0.0)
ED/Rapid Response	416 (6.0)	125 (11.1)
Medical ICU	2,265 (32.7)	529 (46.9)
Mixed Med/Surg ICU	1,907 (27.5)	202 (17.9)
Surgical ICU	1,645 (23.7)	272 (24.1)
Surviving Sepsis Campaign guideline treatment era		
SSC 2008	395 (5.7)	19 (1.7)
SSC 2012	1,401 (20.2)	162 (14.4)
SSC 2016	2,587 (37.3)	340 (30.1)
SSC 2021	2,551 (36.8)	607 (53.8)
Clinical criteria met		
Both	4,047 (58.4)	642 (56.9)
ASE only	2,385 (34.4)	389 (34.5)
Sep3 only	502 (7.2)	97 (8.6)
comm_onset		
Community	4,955 (71.5)	842 (74.6)
Hospital	1,979 (28.5)	286 (25.4)

[1] Median (Q1, Q3); n (%)

Characteristic	Ohio N = 6,934[1]	Florida N = 1,128[1]
Prior surgery		
No Surgery	5,474 (78.9)	921 (81.6)
Surgery	1,460 (21.1)	207 (18.4)
first_adj_abx		
Antibiotics	5,415 (78.1)	889 (78.8)
No Antibiotics	1,519 (21.9)	239 (21.2)
SOFA-2	8 (6, 11)	8 (6, 10)
first_adj_gcs_no_sed	11 (6, 15)	14 (7, 15)
Advanced respiratory support		
Adv Resp Before Adjunct	5,119 (73.8)	845 (74.9)
No Adv Resp Before Adjunct	1,815 (26.2)	283 (25.1)
Mechanical ventilation		
MV Before Adjunct	4,746 (68.4)	778 (69.0)
No MV Before Adjunct	2,188 (31.6)	350 (31.0)
first_adj_pf	197 (116, 315)	169 (97, 280)
first_adj_pH	7.31 (7.22, 7.39)	7.31 (7.21, 7.38)
first_adj_scr	1.75 (1.13, 2.89)	1.63 (1.06, 2.66)
first_adj_aki		
No AKI	2,985 (43.6)	523 (46.9)
AKI Stage 1	1,402 (20.5)	255 (22.9)
AKI Stage 2	659 (9.6)	104 (9.3)
AKI Stage 3	1,360 (19.9)	169 (15.2)
ESKD	437 (6.4)	63 (5.7)

[1] Median (Q1, Q3); n (%)

Characteristic	Ohio N = 6,934[1]	Florida N = 1,128[1]
Kidney replacement therapy		
No RRT Before Adjunct	6,164 (88.9)	1,055 (93.5)
RRT Before Adjunct	770 (11.1)	73 (6.5)
first_adj_platelets	173 (107, 261)	170 (103, 254)
first_adj_bili	0.9 (0.5, 2.0)	0.9 (0.5, 1.9)
first_adj_temp_c	36.8 (36.3, 37.4)	36.7 (36.3, 37.2)
first_adj_heart_rate	101 (84, 118)	97 (80, 114)
first_adj_resp_rate	22 (18, 28)	22 (18, 27)
first_adj_map	68 (61, 76)	69 (62, 78)
first_adj_lactate	3.3 (1.9, 6.3)	3.1 (1.7, 6.0)
Fluid bolus volume administered (mL)	600 (0, 2,000)	1,000 (0, 2,000)
Fluid bolus volume administered (mL/kg)	8 (0, 25)	11 (0, 30)
Norepinephrine dosage (mcg/kg/min)	0.27 (0.15, 0.43)	0.20 (0.12, 0.32)
Norepinephrine dosage (mcg/min)	22 (14, 35)	16 (10, 25)
hours_to_first_adj	6 (2, 16)	4 (2, 11)
dbt_before_adj		
Dobutamine Before Adjunct	179 (2.6)	28 (2.5)
No Dobutamine Before Adjunct	6,755 (97.4)	1,100 (97.5)
Corticosteroid administration		
No Steroids	5,129 (74.0)	750 (66.5)

[1] Median (Q1, Q3); n (%)

Characteristic	Ohio N = 6,934[1]	Florida N = 1,128[1]
Steroids	1,805 (26.0)	378 (33.5)
First Adjunctive Vasopressor		
vasopressin	5,289 (76.3)	976 (86.5)
epinephrine	775 (11.2)	95 (8.4)
phenylephrine	870 (12.5)	57 (5.1)
Vasopressor duration (hours)	56 (28, 102)	57 (33, 99)
Antibiotic duration (days)	6 (2, 11)	6 (3, 10)
In-Hospital mortality		
Died	3,606 (52.1)	488 (43.5)
Survived	3,317 (47.9)	634 (56.5)

[1] Median (Q1, Q3); n (%)

Export to Word

▼ Code

```
as_flex_table(char_by_cohort) |>
  autofit() |>
  set_table_properties(layout = "autofit") |>
  font(fontname = "Arial", part = "all") |>
  fontsize(size = 10, part = "all") |>
  line_spacing(space = 1, part = "all") |>
  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
  save_as_docx(
    path = "characteristics_by_cohort.docx",
    pr_section = prop_section(
      page_size = page_size(orient = "landscape", width = 11, height = 8.5),
      page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
      )
    )
```

Develop and export missingness count in the above characteristics table.

▼ Code

```
(miss_char_by_cohort <- incl |>
  select(
```

```
    age, sex, first_adj_wt_kg, race_eth, elixhauser_index,
    cerebrovascular_disease, hypertension, heart_failure,
    valvular_disease, chronic_pulmonary_disease,
    pulmonary_circulation_disease, liver_disease, kidney_disease,
    malignancy, diabetes, obesity, alcohol_abuse, first_adj_lvef,
    first_adj_hosp_beds_cat, first_adj_hosp_teaching_status,
    first_adj_tx_unit, ssc_era, criteria_met, comm_onset, prior_surg,
    first_adj_abx, first_adj_sofa_2, first_adj_gcs_no_sed,
    adv_resp_before_adj, mv_before_adj, first_adj_pf, first_adj_pH,
    first_adj_scr, first_adj_aki, rrt_before_adj, first_adj_platelets,
    first_adj_bili, first_adj_temp_c, first_adj_heart_rate,
    first_adj_resp_rate, first_adj_map, first_adj_lactate,
    first_adj_fluid_mL, first_adj_fluid_mL_kg, adj_baseline_neq,
    adj_baseline_neq_mcg_min, hours_to_first_adj, dbt_before_adj,
    first_adj_steroid, first_adj, pressors_dur_hours, abx_days,
    mort_hosp, first_adj_hosp_state
  ) |>
  group_by(first_adj_hosp_state) |>
  summarise(
    across(
      everything(),
      ~ sprintf(
        "%d (%.1f)",
        sum(is.na(.)),
        100 * mean(is.na(.))
      )
    ),
    .groups = "drop"
  ) |>
  pivot_longer(
    -first_adj_hosp_state,
    names_to = "variable",
    values_to = "missing"
  ) |>
  pivot_wider(
    names_from = first_adj_hosp_state,
    values_from = missing
  ))
```

```
# A tibble: 53 × 3
   variable                Ohio      Florida
   <chr>                   <chr>     <chr>
 1 age                     0 (0.0)   0 (0.0)
 2 sex                     1 (0.0)   0 (0.0)
 3 first_adj_wt_kg         177 (2.6) 13 (1.2)
 4 race_eth                438 (6.3) 17 (1.5)
 5 elixhauser_index        0 (0.0)   0 (0.0)
 6 cerebrovascular_disease 0 (0.0)   0 (0.0)
 7 hypertension            0 (0.0)   0 (0.0)
 8 heart_failure           0 (0.0)   0 (0.0)
```

```
 9 valvular_disease          0 (0.0)   0 (0.0)
10 chronic_pulmonary_disease 0 (0.0)   0 (0.0)
# ℹ 43 more rows
```

▼ Code

```
#Commented out export for rendering
#miss_char_by_cohort |>
#  flextable() |>
#  autofit() |>
#  set_table_properties(layout = "autofit") |>
#  font(fontname = "Arial", part = "all") |>
#  fontsize(size = 10, part = "all") |>
#  line_spacing(space = 1, part = "all") |>
#  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
#  save_as_docx(
#    path = "characteristics_by_cohort_missingness.docx",
#    pr_section = prop_section(
#      page_size = page_size(orient = "portrait", width = 8.5, height = 11),
#      page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
#      )
#    )
```

Describe the characteristics of included patients in the derivation (Ohio) cohort by adjunctive vasopressor agent. Notably, the below table includes characteristics that will not be used as model variables (predictors), such as race/ethnicity. Further, because antimicrobials are components of the clinical criteria (CDC ASE and Sepsis-3) used to identify patients with septic shock, each included patient received antimicrobials as part of their sepsis treatment course.

▼ Code

```
(der_char_by_adj <- incl |>
   filter(first_adj_hosp_state == "Ohio") |>
   droplevels() |>
   mutate(first_adj_hosp_beds_cat =
            fct_relevel(first_adj_hosp_beds_cat, "<200", "200-499"),
          ssc_era = fct_relevel(ssc_era, "SSC 2008", "SSC 2012", "SSC 2016"),
          adv_resp_before_adj =
            fct_relevel(adv_resp_before_adj, "Adv Resp Before Adjunct"),
          mv_before_adj = fct_relevel(mv_before_adj, "MV Before Adjunct"),
          response = fct_relevel(response, "Response"),
          first_adj = fct_recode(first_adj, "Vasopressin" = "vasopressin",
                                 "Epinephrine" = "epinephrine",
                                 "Phenylephrine" = "phenylephrine"
                                 )
          ) |>
  select(age, sex, first_adj_wt_kg, race_eth, elixhauser_index,
         cerebrovascular_disease, hypertension, heart_failure,
         valvular_disease, chronic_pulmonary_disease,
```

```
        pulmonary_circulation_disease, liver_disease, kidney_disease,
        malignancy, diabetes, obesity, alcohol_abuse, first_adj_lvef,
        first_adj_hosp_beds_cat, first_adj_hosp_teaching_status,
        first_adj_tx_unit, ssc_era,  criteria_met, comm_onset, prior_surg,
        first_adj_abx, first_adj_sofa_2, first_adj_gcs_no_sed,
        adv_resp_before_adj, mv_before_adj, first_adj_pf, first_adj_pH,
        first_adj_scr, first_adj_aki, rrt_before_adj, first_adj_platelets,
        first_adj_bili, first_adj_temp_c, first_adj_heart_rate,
        first_adj_resp_rate, first_adj_map, first_adj_lactate,
        first_adj_fluid_mL, first_adj_fluid_mL_kg, adj_baseline_neq,
        adj_baseline_neq_mcg_min, hours_to_first_adj, dbt_before_adj,
        first_adj_steroid, pressors_dur_hours, abx_days, mort_hosp, first_adj
        ) |>
tbl_summary(
  by = first_adj,
  digits = list(
    all_categorical() ~ c(0, 1),
    first_adj_gcs_no_sed ~ 0,
    first_adj_sofa_2 ~ 0,
    first_adj_temp_c ~ 1
    ),
  label = list(response = "Adjunct Response",
               age = "Age",
               sex = "Sex",
               race_eth = "Race/Ethnicity",
               prior_surg = "Prior surgery",
               elixhauser_index = "Elixhauser comorbidity index",
               ssc_era = "Surviving Sepsis Campaign guideline treatment era",
               wt_kg = "Weight (kg)",
               first_adj_hosp_beds_cat = "Hospital beds",
               first_adj_hosp_teaching_status = "Teaching status",
               first_adj_tx_unit = "Treatment unit",
               criteria_met = "Clinical criteria met",
               first_adj_lvef = "LVEF (%)",
               first_adj_sofa_2 = "SOFA-2",
               adv_resp_before_adj = "Advanced respiratory support",
               mv_before_adj = "Mechanical ventilation",
               rrt_before_adj = "Kidney replacement therapy",
               first_adj_fluid_mL = "Fluid bolus volume administered (mL)",
               first_adj_fluid_mL_kg = "Fluid bolus volume administered (mL/kg)",
               first_adj_steroid = "Corticosteroid administration",
               adj_baseline_neq = "NE dosage (mcg/kg/min)",
               adj_baseline_neq_mcg_min = "NE dosage (mcg/min)",
               pressors_dur_hours = "Vasopressor duration (hours)",
               abx_days = "Antibiotic duration (days)",
               mort_hosp = "In-Hospital mortality",
               first_adj = "First Adjunctive Vasopressor",
               dbt_before_adj = "Dobutamine administration"
               ),
  missing = "ifany"
  ) |>
```

```
  add_overall() |>
  modify_fmt_fun(
    all_stat_cols() ~ function(x) gsub("%", "", x)
  ) |>
  modify_caption("**Patient Characteristics in Derivation (Ohio) Dataset by Adjunct**")
)
```

Patient Characteristics in Derivation (Ohio) Dataset by Adjunct

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
Age	67 (57, 76)	67 (56, 76)	65 (56, 74)	69 (58, 77)
Sex				
Female	3,042 (43.9)	2,320 (43.9)	298 (38.5)	424 (48.7)
Male	3,891 (56.1)	2,968 (56.1)	477 (61.5)	446 (51.3)
Unknown	1	1	0	0
first_adj_wt_kg	84 (69, 102)	84 (69, 103)	83 (70, 100)	82 (68, 100)
Unknown	177	128	41	8
Race/Ethnicity				
Black, non-Hispanic	1,371 (21.1)	1,039 (21.0)	161 (22.2)	171 (20.9)
Hispanic or Latino	114 (1.8)	86 (1.7)	16 (2.2)	12 (1.5)
Multiracial, non-Hispanic	59 (0.9)	47 (0.9)	7 (1.0)	5 (0.6)
Other, non-Hispanic	69 (1.1)	54 (1.1)	10 (1.4)	5 (0.6)
White, non-Hispanic	4,883 (75.2)	3,726 (75.2)	530 (73.2)	627 (76.5)
Unknown	438	337	51	50
Elixhauser comorbidity index	8 (0, 21)	7 (0, 22)	8 (0, 20)	9 (0, 21)
cerebrovascular_disease	696 (10.0)	513 (9.7)	77 (9.9)	106 (12.2)
hypertension	2,966 (42.8)	2,212 (41.8)	356 (45.9)	398 (45.7)
heart_failure	1,309 (18.9)	879 (16.6)	262 (33.8)	168 (19.3)

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
valvular_disease	770 (11.1)	482 (9.1)	175 (22.6)	113 (13.0)
chronic_pulmonary_disease	1,136 (16.4)	849 (16.1)	140 (18.1)	147 (16.9)
pulmonary_circulation_disease	414 (6.0)	315 (6.0)	58 (7.5)	41 (4.7)
liver_disease	817 (11.8)	690 (13.0)	58 (7.5)	69 (7.9)
kidney_disease	1,073 (15.5)	778 (14.7)	146 (18.8)	149 (17.1)
malignancy	1,495 (21.6)	1,169 (22.1)	117 (15.1)	209 (24.0)
diabetes	1,652 (23.8)	1,248 (23.6)	187 (24.1)	217 (24.9)
obesity	1,437 (20.7)	1,125 (21.3)	154 (19.9)	158 (18.2)
alcohol_abuse	344 (5.0)	275 (5.2)	34 (4.4)	35 (4.0)
LVEF (%)	56 (45, 65)	57 (46, 65)	52 (30, 60)	59 (50, 65)
Unknown	4,220	3,369	378	473
Hospital beds				
<200	622 (9.0)	477 (9.0)	68 (8.8)	77 (8.9)
200-499	1,702 (24.5)	1,255 (23.7)	138 (17.8)	309 (35.5)
≥500	4,610 (66.5)	3,557 (67.3)	569 (73.4)	484 (55.6)
Teaching status				
Teaching	6,559 (94.6)	4,995 (94.4)	738 (95.2)	826 (94.9)
Non-teaching	375 (5.4)	294 (5.6)	37 (4.8)	44 (5.1)
Treatment unit				
Cardiac ICU	701 (10.1)	507 (9.6)	129 (16.6)	65 (7.5)
ED/Rapid Response	416 (6.0)	281 (5.3)	105 (13.5)	30 (3.4)
Medical ICU	2,265 (32.7)	1,849 (35.0)	140 (18.1)	276 (31.7)
Mixed Med/Surg ICU	1,907 (27.5)	1,448 (27.4)	106 (13.7)	353 (40.6)

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
Surgical ICU	1,645 (23.7)	1,204 (22.8)	295 (38.1)	146 (16.8)
Surviving Sepsis Campaign guideline treatment era				
SSC 2008	395 (5.7)	311 (5.9)	23 (3.0)	61 (7.0)
SSC 2012	1,401 (20.2)	1,090 (20.6)	109 (14.1)	202 (23.2)
SSC 2016	2,587 (37.3)	1,784 (33.7)	358 (46.2)	445 (51.1)
SSC 2021	2,551 (36.8)	2,104 (39.8)	285 (36.8)	162 (18.6)
Clinical criteria met				
Both	4,047 (58.4)	3,233 (61.1)	379 (48.9)	435 (50.0)
ASE only	2,385 (34.4)	1,748 (33.0)	252 (32.5)	385 (44.3)
Sep3 only	502 (7.2)	308 (5.8)	144 (18.6)	50 (5.7)
comm_onset				
Community	4,955 (71.5)	3,837 (72.5)	519 (67.0)	599 (68.9)
Hospital	1,979 (28.5)	1,452 (27.5)	256 (33.0)	271 (31.1)
Prior surgery				
No Surgery	5,474 (78.9)	4,285 (81.0)	480 (61.9)	709 (81.5)
Surgery	1,460 (21.1)	1,004 (19.0)	295 (38.1)	161 (18.5)
first_adj_abx				
Antibiotics	5,415 (78.1)	4,308 (81.5)	418 (53.9)	689 (79.2)
No Antibiotics	1,519 (21.9)	981 (18.5)	357 (46.1)	181 (20.8)
SOFA-2	8 (6, 11)	8 (6, 11)	7 (6, 10)	7 (5, 10)
first_adj_gcs_no_sed	11 (6, 15)	11 (6, 15)	7 (3, 14)	13 (9, 15)
Unknown	3,549	2,714	403	432

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
Advanced respiratory support				
Adv Resp Before Adjunct	5,119 (73.8)	3,903 (73.8)	611 (78.8)	605 (69.5)
No Adv Resp Before Adjunct	1,815 (26.2)	1,386 (26.2)	164 (21.2)	265 (30.5)
Mechanical ventilation				
MV Before Adjunct	4,746 (68.4)	3,617 (68.4)	586 (75.6)	543 (62.4)
No MV Before Adjunct	2,188 (31.6)	1,672 (31.6)	189 (24.4)	327 (37.6)
first_adj_pf	197 (116, 315)	194 (114, 308)	233 (123, 341)	195 (119, 298)
Unknown	2,629	1,983	203	443
first_adj_pH	7.31 (7.22, 7.39)	7.31 (7.22, 7.38)	7.31 (7.21, 7.38)	7.33 (7.25, 7.41)
Unknown	1,724	1,265	118	341
first_adj_scr	1.75 (1.13, 2.89)	1.82 (1.17, 2.98)	1.51 (0.99, 2.41)	1.59 (1.00, 2.65)
Unknown	173	110	40	23
first_adj_aki				
No AKI	2,985 (43.6)	2,163 (41.3)	401 (53.6)	421 (48.7)
AKI Stage 1	1,402 (20.5)	1,089 (20.8)	156 (20.9)	157 (18.2)
AKI Stage 2	659 (9.6)	536 (10.2)	49 (6.6)	74 (8.6)
AKI Stage 3	1,360 (19.9)	1,121 (21.4)	96 (12.8)	143 (16.6)
ESKD	437 (6.4)	322 (6.2)	46 (6.1)	69 (8.0)
Unknown	91	58	27	6
Kidney replacement therapy				
No RRT Before Adjunct	6,164 (88.9)	4,698 (88.8)	696 (89.8)	770 (88.5)

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
RRT Before Adjunct	770 (11.1)	591 (11.2)	79 (10.2)	100 (11.5)
first_adj_platelets	173 (107, 261)	173 (106, 262)	175 (110, 246)	177 (115, 263)
Unknown	175	120	28	27
first_adj_bili	0.9 (0.5, 2.0)	0.9 (0.5, 2.2)	0.9 (0.5, 1.6)	0.8 (0.4, 1.6)
Unknown	989	697	130	162
first_adj_temp_c	36.8 (36.3, 37.4)	36.8 (36.3, 37.4)	36.5 (35.9, 37.0)	36.8 (36.4, 37.4)
Unknown	963	612	265	86
first_adj_heart_rate	101 (84, 118)	101 (85, 117)	90 (75, 107)	113 (92, 130)
Unknown	159	112	34	13
first_adj_resp_rate	22 (18, 28)	23 (18, 28)	20 (16, 26)	22 (18, 27)
Unknown	256	179	42	35
first_adj_map	68 (61, 76)	67 (61, 76)	69 (62, 78)	69 (62, 78)
Unknown	251	170	55	26
first_adj_lactate	3.3 (1.9, 6.3)	3.3 (2.0, 6.3)	4.4 (2.3, 8.0)	2.4 (1.4, 5.0)
Unknown	843	568	61	214
Fluid bolus volume administered (mL)	600 (0, 2,000)	1,000 (0, 2,000)	100 (0, 1,000)	500 (0, 1,500)
Fluid bolus volume administered (mL/kg)	8 (0, 25)	9 (0, 27)	1 (0, 13)	6 (0, 19)
Unknown	177	128	41	8
NE dosage (mcg/kg/min)	0.27 (0.15, 0.43)	0.29 (0.18, 0.45)	0.16 (0.07, 0.36)	0.17 (0.08, 0.33)
NE dosage (mcg/min)	22 (14, 35)	25 (16, 39)	13 (6, 30)	15 (7, 28)

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
Unknown	2	2	0	0
hours_to_first_adj	6 (2, 16)	6 (2, 15)	3 (1, 10)	8 (3, 26)
Dobutamine administration				
Dobutamine Before Adjunct	179 (2.6)	114 (2.2)	54 (7.0)	11 (1.3)
No Dobutamine Before Adjunct	6,755 (97.4)	5,175 (97.8)	721 (93.0)	859 (98.7)
Corticosteroid administration				
No Steroids	5,129 (74.0)	3,844 (72.7)	626 (80.8)	659 (75.7)
Steroids	1,805 (26.0)	1,445 (27.3)	149 (19.2)	211 (24.3)
Vasopressor duration (hours)	56 (28, 102)	56 (28, 101)	48 (21, 102)	63 (31, 114)
Antibiotic duration (days)	6 (2, 11)	6 (2, 11)	5 (1, 10)	6 (2, 11)
In-Hospital mortality				
Died	3,606 (52.1)	2,777 (52.6)	379 (49.0)	450 (51.8)
Survived	3,317 (47.9)	2,503 (47.4)	395 (51.0)	419 (48.2)
Unknown	11	9	1	1

[1] Median (Q1, Q3); n (%)

Export to Word

▼ Code

```
as_flex_table(der_char_by_adj) |>
  autofit() |>
  set_table_properties(layout = "autofit") |>
  font(fontname = "Arial", part = "all") |>
  fontsize(size = 10, part = "all") |>
  line_spacing(space = 1, part = "all") |>
  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
  save_as_docx(
    path = "der_characteristics_by_adjunct.docx",
    pr_section = prop_section(
      page_size = page_size(orient = "landscape", width = 11, height = 8.5),
```

```
    page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
    )
  )
```

11 Codebook and Data Description

11.1 Create Derivation and Validation Datasets

Included cases will be divided into a derivation cohort from Ohio (`oh`) and an external validation cohort from Florida (`fl`).

▼ Code

```
oh <- incl |> filter(first_adj_hosp_state == "Ohio")

fl <- incl |> filter(first_adj_hosp_state == "Florida")
```

Verify number of observations in each tibble matches expectations.

▼ Code

```
nrow(oh)
```

```
[1] 6934
```

▼ Code

```
nrow(fl)
```

```
[1] 1128
```

The number of observations is correct (6,934 in the derivation [`oh`] dataset and 1,128 in the validation [`fl`] dataset).

11.2 Selecting Model Variables

Here, create an analytic tibble with only the variables that will be used for model derivation. In this process, vector names will also be adjusted for ease moving forward. The `race_eth` vector will be included for future use as a part of fairness checks for the final model, but the variable **will not** be included as a predictor in the models.

Each candidate predictor was selected based on clinical knowledge and biological plausibility for influencing the outcome (adjunctive vasopressor response) based on input from investigators, who are domain experts. Further, predictor variables were assimilated based on a review of the literature

(Sacha 2018, Bauer 2022, Dugar 2023, Sacha 2023). Left ventricular ejection fraction (LVEF; `lvef`) was considered as a candidate predictor based on Dugar 2023. Considering the limitations of the variable outlined in Section 5.3.2.14 (timing of within 30 days prior to adjunct instead of after norepinephrine initiation and relatively high missingness), this variable was omitted. As surrogates for LVEF, history of heart failure (`heart_failure`) and treatment unit (`tx_unit`) with preservation of "Cardiac ICU" as a category level (due to frequent treatment of patients with low LVEF in that unit type) were utilized.

The candidate predictors and number of candidate predictor parameters were constrained to those that could be supported across all adjunctive vasopressor treatment cohorts. **In a sensitivity analysis, expansion of the vasopressin model from 15 to 29 degrees of freedom produced negligible improvement in discrimination and overall performance, supporting use of the common predictor set for all treatment-specific models.**

▼ Code

```
incl_mod_var <- oh |>
  select(response, first_adj, age, sex, race_eth, heart_failure,
         tx_unit = first_adj_tx_unit, ssc_era, criteria_met,
         sofa2_no_cv = first_adj_sofa_2_no_cv,
         pH = first_adj_pH, neq = adj_baseline_neq,
         shock_dur_h = hours_to_first_adj, map = first_adj_map,
         lactate = first_adj_lactate
         )
```

Next, verify variable structure.

▼ Code

```
str(incl_mod_var, vec.len = 0)
```

```
tibble [6,934 × 15] (S3: tbl_df/tbl/data.frame)
 $ response     : Factor w/ 2 levels "No Response",..: NULL ...
 $ first_adj    : Factor w/ 3 levels "vasopressin",..: NULL ...
 $ age          : num [1:6934] NULL ...
 $ sex          : Factor w/ 2 levels "Female","Male": NULL ...
 $ race_eth     : Factor w/ 5 levels "Black, non-Hispanic",..: NULL ...
 $ heart_failure: Factor w/ 2 levels "No","Yes": NULL ...
 $ tx_unit      : Factor w/ 5 levels "Cardiac ICU",..: NULL ...
 $ ssc_era      : Factor w/ 4 levels "SSC 2021","SSC 2008",..: NULL ...
 $ criteria_met : Factor w/ 3 levels "Both","ASE only",..: NULL ...
 $ sofa2_no_cv  : num [1:6934] NULL ...
 $ pH           : num [1:6934] NULL ...
 $ neq          : num [1:6934] NULL ...
 $ shock_dur_h  : num [1:6934] NULL ...
 $ map          : num [1:6934] NULL ...
 $ lactate      : num [1:6934] NULL ...
```

The variables each have the intended structure, however some factor variables need re-leveled to place the intended baseline (reference) category first. Also, collapse the levels for some categorical variables.

▼ Code

```
incl_mod_var <- incl_mod_var |>
  mutate(response = fct_relevel(response, "Response", "No Response"),
         tx_unit = factor(if_else(tx_unit == "Surgical ICU" |
                                    tx_unit == "Mixed Med/Surg ICU",
                                  "Surgical or Mixed Med/Surg ICU",
                                  tx_unit
                                  ),
                          levels = c("Medical ICU", "Cardiac ICU",
                                     "ED/Rapid Response",
                                     "Surgical or Mixed Med/Surg ICU"
                                     )),
         ssc_era = factor(if_else(ssc_era == "SSC 2021",
                                  "SSC 2021",
                                  "Before SSC 2021"
                                  ),
                          levels = c("Before SSC 2021", "SSC 2021")
                          ),
         criteria_met = factor(if_else(criteria_met == "ASE only",
                                       "ASE only",
                                       "Both or Sepsis-3 only"
                                       ),
                               levels = c("ASE only", "Both or Sepsis-3 only")
                               )
         )
```

Verify variable structure after changes.

▼ Code

```
str(incl_mod_var, vec.len = 0)
```

```
tibble [6,934 × 15] (S3: tbl_df/tbl/data.frame)
 $ response     : Factor w/ 2 levels "Response","No Response": NULL ...
 $ first_adj    : Factor w/ 3 levels "vasopressin",..: NULL ...
 $ age          : num [1:6934] NULL ...
 $ sex          : Factor w/ 2 levels "Female","Male": NULL ...
 $ race_eth     : Factor w/ 5 levels "Black, non-Hispanic",..: NULL ...
 $ heart_failure: Factor w/ 2 levels "No","Yes": NULL ...
 $ tx_unit      : Factor w/ 4 levels "Medical ICU",..: NULL ...
 $ ssc_era      : Factor w/ 2 levels "Before SSC 2021",..: NULL ...
 $ criteria_met : Factor w/ 2 levels "ASE only","Both or Sepsis-3 only": NULL ...
 $ sofa2_no_cv  : num [1:6934] NULL ...
 $ pH           : num [1:6934] NULL ...
```

```
$ neq          : num [1:6934] NULL ...
$ shock_dur_h  : num [1:6934] NULL ...
$ map          : num [1:6934] NULL ...
$ lactate      : num [1:6934] NULL ...
```

11.3 Codebook

The codebook below outlines variables that will be used for prediction model derivation and validation. The baseline (reference) category for categorical predictors is listed first. All predictor (independent) variables were available at the time of first adjunctive vasopressor initiation. Missing values are outlined in Section 11.5 and Section 14.2 and managed within the model building steps. Note that `age`, `sex`, and `race_eth` are included in the `incl_mod_var` tibble for subsequent fairness checks, but will **not** be utilized as a predictor (rather, they will be in a "bystander" role in the models).

VARIABLE	ROLE	TYPE	DESCRIPTION
`response`	outcome	2-cat	Response, No Response: Response or not (No Response) to first adjunctive vasopressor initiation. Response is defined as achieving mean arterial pressure (MAP) ≥65 mm Hg with a decrease from baseline in norepinephrine-equivalent dosage at 6 hours after first adjunctive vasopressor initiation.
`first_adj`	identifier	3-cat	vasopressin, epinephrine, phenylephrine: First adjunctive vasopressor initiated. Used as a cohort identifier for data splitting.

VARIABLE	ROLE	TYPE	DESCRIPTION
heart_failure	predictor	2-cat	No, Yes: Absence (No) or presence (Yes) of chronic heart failure (per Elixhauser criteria) at admission.
tx_unit	predictor	4-cat	Medical ICU, Cardiac ICU, ED/Rapid Response, Surgical or Mixed Med/Surg ICU: Treatment unit at first adjunctive vasopressor initiation, whether medical intensive care unit (ICU; Medical ICU), cardiac ICU (Cardiac ICU), emergency department/rapid response team (ED/Rapid Response), or surgical/mixed medical-surgical ICU (Mixed Med/Surg ICU).
ssc_era	predictor	2-cat	Before SSC 2021, SSC 2021: Surviving Sepsis Campaign (SSC) Guideline treatment “era” based on the patient admission date and guideline publication date, whether before publication of the 2021 SSC guidelines (Before SSC 2021) or after publication of the 2021 SSC guidelines (SSC 2021).
criteria_met	predictor	2-cat	ASE only, Both or Sepsis-3 only: Clinical criteria for septic shock met, whether CDC Adult Sepsis Event

VARIABLE	ROLE	TYPE	DESCRIPTION
			(ASE) only (ASE only) or both CDC ASE and Sepsis-3 or Sepsis-3 only (Both or Sepsis-3 only).
sofa2_no_cv	predictor	quant	Sequential Organ Failure Assessment(SOFA)-2 score without the cardiovascular sub-score component at first adjunctive vasopressor initiation.
pH	predictor	quant	Arterial pH, value at the time of first adjunctive vasopressor initiation.
neq	predictor	quant	Norepinephrine dosage (mcg/kg/min), value at the time of first adjunctive vasopressor initiation.
shock_dur_h	predictor	quant	Duration (hours) of norepinephrine monotherapy administration at the time of first adjunctive vasopressor initiation.
map	predictor	quant	Mean arterial pressure (mm Hg), closest value (within 6 hours prior) at the time of first adjunctive vasopressor initiation.
lactate	predictor	quant	Lactate concentration (mg/dL), closest value (within 12 hours prior) at the time of first

VARIABLE	ROLE	TYPE	DESCRIPTION
			adjunctive vasopressor initiation.

11.4 Numerical Description

11.4.1 Derivation Dataset Overall and Outcome

Show date of final follow-up (maximum hospital discharge date).

▼ Code

```
max(incl$discharge_time[incl$first_adj_hosp_state == "Ohio"])
```

```
[1] "2025-01-12 06:10:00 UTC"
```

Show response frequency by first adjunctive vasopressor.

▼ Code

```
incl_mod_var |>
  tabyl(response, first_adj) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  kable(align = c('lcc'),
        caption = "Response by First Adjunctive Vasopressor (Derivation Cohort)"
        ) |>
  kable_paper(full_width = FALSE)
```

Response by First Adjunctive Vasopressor (Derivation Cohort)

RESPONSE/FIRST_ADJ	VASOPRESSIN	EPINEPHRINE	PHENYLEPHRINE	TOTAL
Response	1,833 (34.7%)	222 (28.6%)	319 (36.7%)	2,374 (34.2%)
No Response	3,456 (65.3%)	553 (71.4%)	551 (63.3%)	4,560 (65.8%)
Total	5,289 (100.0%)	775 (100.0%)	870 (100.0%)	6,934 (100.0%)

There are 5,289 included patients who received vasopressin as the first adjunctive vasopressor, 775 who received epinephrine, and 870 who received phenylephrine. Adjunctive vasopressor response was observed in 1,833 (34.7%) patients receiving vasopressin, 222 (28.6%) receiving epinephrine, and 319 (36.7%) receiving phenylephrine.

11.4.2 Predictors

Show a numerical description of all predictors in the model derivation tibble (dataset).

▼ Code

```
(der_predictors_by_adjunct <- incl_mod_var |>
  select(-c(response, age, sex, race_eth)) |>
  droplevels() |>
  mutate(first_adj = fct_recode(first_adj, "Vasopressin" = "vasopressin",
                                "Epinephrine" = "epinephrine",
                                "Phenylephrine" = "phenylephrine"
                                )
         ) |>
  tbl_summary(
    by = first_adj,
    digits = list(
      all_categorical() ~ c(0, 1),
      sofa2_no_cv ~ 0
      ),
    label = list(heart_failure = "History of heart failure",
                 tx_unit = "Treatment unit",
                 sex = "Sex",
                 ssc_era = "Surviving Sepsis Campaign guideline treatment era",
                 criteria_met = "Clinical criteria met",
                 sofa2_no_cv = "SOFA-2 (without cardiovascular score)",
                 pH = "Arterial pH",
                 neq = "Norepinephrine dosage (mcg/kg/min)",
                 shock_dur_h = "Shock duration (hours)",
                 map = "MAP (mm Hg)",
                 lactate = "Lactate (mmol/L)"
                 ),
   missing = "no"
    ) |>
   add_overall() |>
   modify_fmt_fun(
     all_stat_cols() ~ function(x) gsub("%", "", x)
  ) |>
  modify_caption("**Predictor Values in Derivation (Ohio) Dataset**"))
```

Predictor Values in Derivation (Ohio) Dataset

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
History of heart failure	1,309 (18.9)	879 (16.6)	262 (33.8)	168 (19.3)

[1] n (%); Median (Q1, Q3)

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
Treatment unit				
Medical ICU	2,265 (32.7)	1,849 (35.0)	140 (18.1)	276 (31.7)
Cardiac ICU	701 (10.1)	507 (9.6)	129 (16.6)	65 (7.5)
ED/Rapid Response	416 (6.0)	281 (5.3)	105 (13.5)	30 (3.4)
Surgical or Mixed Med/Surg ICU	3,552 (51.2)	2,652 (50.1)	401 (51.7)	499 (57.4)
Surviving Sepsis Campaign guideline treatment era				
Before SSC 2021	4,383 (63.2)	3,185 (60.2)	490 (63.2)	708 (81.4)
SSC 2021	2,551 (36.8)	2,104 (39.8)	285 (36.8)	162 (18.6)
Clinical criteria met				
ASE only	2,385 (34.4)	1,748 (33.0)	252 (32.5)	385 (44.3)
Both or Sepsis-3 only	4,549 (65.6)	3,541 (67.0)	523 (67.5)	485 (55.7)
SOFA-2 (without cardiovascular score)	5 (3, 7)	5 (3, 8)	5 (3, 7)	5 (3, 7)
Arterial pH	7.31 (7.22, 7.39)	7.31 (7.22, 7.38)	7.31 (7.21, 7.38)	7.33 (7.25, 7.41)
Norepinephrine dosage (mcg/kg/min)	0.27 (0.15, 0.43)	0.29 (0.18, 0.45)	0.16 (0.07, 0.36)	0.17 (0.08, 0.33)
Shock duration (hours)	6 (2, 16)	6 (2, 15)	3 (1, 10)	8 (3, 26)

[1] n (%); Median (Q1, Q3)

Characteristic	Overall N = 6,934[1]	Vasopressin N = 5,289[1]	Epinephrine N = 775[1]	Phenylephrine N = 870[1]
MAP (mm Hg)	68 (61, 76)	67 (61, 76)	69 (62, 78)	69 (62, 78)
Lactate (mmol/L)	3.3 (1.9, 6.3)	3.3 (2.0, 6.3)	4.4 (2.3, 8.0)	2.4 (1.4, 5.0)

[1] n (%); Median (Q1, Q3)

Export to Word

▼ Code

```
as_flex_table(der_predictors_by_adjunct) |>
  autofit() |>
  set_table_properties(layout = "autofit") |>
  font(fontname = "Arial", part = "all") |>
  fontsize(size = 10, part = "all") |>
  line_spacing(space = 1, part = "all") |>
  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
  save_as_docx(
    path = "der_predictors_by_adjunct.docx",
    pr_section = prop_section(
      page_size = page_size(orient = "landscape", width = 11, height = 8.5),
      page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
      )
    )
```

11.5 Missing Values

Missing values will be outlined here. An *a priori* threshold for allowable missingness will not be utilized. Based on the TRIPOD+AI guidelines Collins 2024, the number of missing values per variable will be described and the details of the imputation method reported.

First, evaluate for missing data in the outcome variable.

▼ Code

```
incl_mod_var |> select(response) |>
  miss_var_summary() |> mutate(pct_miss = as.numeric(pct_miss)) |>
  kable(digits = 3,
        caption = "Missingness in Outcome in Derivation (Ohio) Data"
        ) |>
  kable_paper(full_width = FALSE)
```

Missingness in Outcome in Derivation (Ohio) Data

VARIABLE	N_MISS	PCT_MISS
response	0	0

There are no missing values in the outcome variable.

Next, evaluate for missing data in the predictors of interest by first adjunctive vasopressor medication. Notably, since race/ethnicity (`race_eth`) will not be a predictor in models it will be omitted from this assessment.

▼ Code

```
incl_mod_var |>
  select(-c(response, age, sex, race_eth)) |>
  group_by(first_adj) |>
  miss_var_summary() |>
  mutate(pct_miss = as.numeric(pct_miss)) |>
  kable(digits = 1,
        caption = "Missingness in Predictors in Derivation (Ohio) Data"
        ) |>
  kable_paper(full_width = FALSE)
```

Missingness in Predictors in Derivation (Ohio) Data

FIRST_ADJ	VARIABLE	N_MISS	PCT_MISS
vasopressin	pH	1265	23.9
vasopressin	lactate	568	10.7
vasopressin	map	170	3.2
vasopressin	heart_failure	0	0.0
vasopressin	tx_unit	0	0.0
vasopressin	ssc_era	0	0.0
vasopressin	criteria_met	0	0.0
vasopressin	sofa2_no_cv	0	0.0
vasopressin	neq	0	0.0
vasopressin	shock_dur_h	0	0.0
epinephrine	pH	118	15.2
epinephrine	lactate	61	7.9
epinephrine	map	55	7.1
epinephrine	heart_failure	0	0.0
epinephrine	tx_unit	0	0.0

FIRST_ADJ	VARIABLE	N_MISS	PCT_MISS
epinephrine	ssc_era	0	0.0
epinephrine	criteria_met	0	0.0
epinephrine	sofa2_no_cv	0	0.0
epinephrine	neq	0	0.0
epinephrine	shock_dur_h	0	0.0
phenylephrine	pH	341	39.2
phenylephrine	lactate	214	24.6
phenylephrine	map	26	3.0
phenylephrine	heart_failure	0	0.0
phenylephrine	tx_unit	0	0.0
phenylephrine	ssc_era	0	0.0
phenylephrine	criteria_met	0	0.0
phenylephrine	sofa2_no_cv	0	0.0
phenylephrine	neq	0	0.0
phenylephrine	shock_dur_h	0	0.0

There are missing data in the `pH`, `lactate`, and `map` variables for each first adjunctive vasopressor, which need to be managed. Missing data will be imputed with regression (single) imputation instead of multiple imputation to mimic model deployment in an electronic health record where model derivation data are inaccessible. Sisk 2023. Missing data will be imputed under a missing-at-random assumption and the outcome will not be utilized as a variable in the imputation model. Since each of the variables with missing data is continuous (numeric), these predictors will be imputed with linear regression. Because the measurement of predictors is likely to vary across settings (locations) and over time (and therefore the missingness mechanism is not stable), a missing indicator will not be incorporated, as recommended by van Smeden 2020 and Sisk 2023. Missing data will be imputed sing the {tidymodels} `recipe::step_impute_linear()` function within each first adjunctive vasopressor cohort in order to avoid data leakage between cohorts.

Assess missingness patterns.

▼ Code

```
incl_mod_var |>
  count(
    pH_missing = is.na(pH),
    lactate_missing = is.na(lactate),
    map_missing = is.na(map)
    )
```

```
# A tibble: 8 × 4
  pH_missing lactate_missing map_missing     n
  <lgl>      <lgl>           <lgl>       <int>
1 FALSE      FALSE           FALSE        4786
2 FALSE      FALSE           TRUE          207
3 FALSE      TRUE            FALSE         206
4 FALSE      TRUE            TRUE           11
5 TRUE       FALSE           FALSE        1081
6 TRUE       FALSE           TRUE           17
7 TRUE       TRUE            FALSE         610
8 TRUE       TRUE            TRUE           16
```

Since there is some overlap in missingness (particularly for `pH` and `lactate`), imputation will be done sequentially to maximize available information.

12 Prediction Model Building and Evaluation

Clinical prediction models will be developed and evaluated using the `{tidymodels}` meta-package (with the `{bonsai}` extension package, as needed).

12.1 Split Data

Within the derivation (Ohio) dataset, create separate cohorts for building models based on the first adjunctive vasopressor (`first_adj`), whether vasopressin (`avp`), epinephrine (`epi`), or phenylephrine (`pe`). Then confirm that the observation count in the `oh` tibble matches the sum of the count in the individual tibbles.

▼ Code

```
avp <- incl_mod_var |> filter(first_adj == "vasopressin") |> select(-first_adj)
epi <- incl_mod_var |> filter(first_adj == "epinephrine") |> select(-first_adj)
pe <- incl_mod_var |> filter(first_adj == "phenylephrine") |> select(-first_adj)

nrow(incl_mod_var) == (nrow(avp) + nrow(epi) + nrow(pe))
```

```
[1] TRUE
```

12.2 Vasopressin

This section will build and evaluate prediction models for response after initiation of vasopressin as the first adjunctive vasopressor.

12.2.1 Logistic Regression

Here, use logistic regression to generate prediction models using two approaches: 1) main effects (additive) only, and 2) non-linear effects of predictors.

12.2.1.1 Main Effects

Create the model specifications using the {parsnip} package here.

▼ Code

```
lr_spec <-
  logistic_reg() |>
  set_engine("glm")
```

Create a basic (main effects) {parsnip} `recipe()` object for the model. Include a step for imputation of missing predictors (`map`, `lactate`, and `pH` in that sequence due to their missingness levels and patterns).

▼ Code

```
avp_m1_lr_rec_main <-
  recipe(response ~ ., data = avp) |>
  # Declare age, sex, and race_eth in a bystander role to keep them in the
  # resampling dataset for future use in fairness checks
  update_role(age, sex, race_eth, new_role = "bystander") |>
  # Single (linear regression) imputation for missing quantitatitve predictors
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate
                       )
    )
```

Now set workflow by adding the model specification and formula.

▼ Code

```
avp_m1_lr_main_wflow <- workflow() |>
  add_recipe(avp_m1_lr_rec_main) |>
  add_model(lr_spec)
```

Fit model.

▼ Code

```
avp_m1_lr_main_fit <-
  fit(avp_m1_lr_main_wflow, data = avp)
```

Check the number of parameters in this fitted model.

▼ Code

```
avp_m1_fit_obj <- extract_fit_engine(avp_m1_lr_main_fit)

avp_m1_fit_obj$rank - 1 # Subtract 1 to remove the intercept from the count
```

```
[1] 12
```

▼ Code

```
rm(avp_m1_fit_obj)
```

There are 12 predictor parameters (degrees of freedom) in this model with only main effects, therefore non-linear terms can be explored.

The metrics for this model will be generated and assessed in the next section.

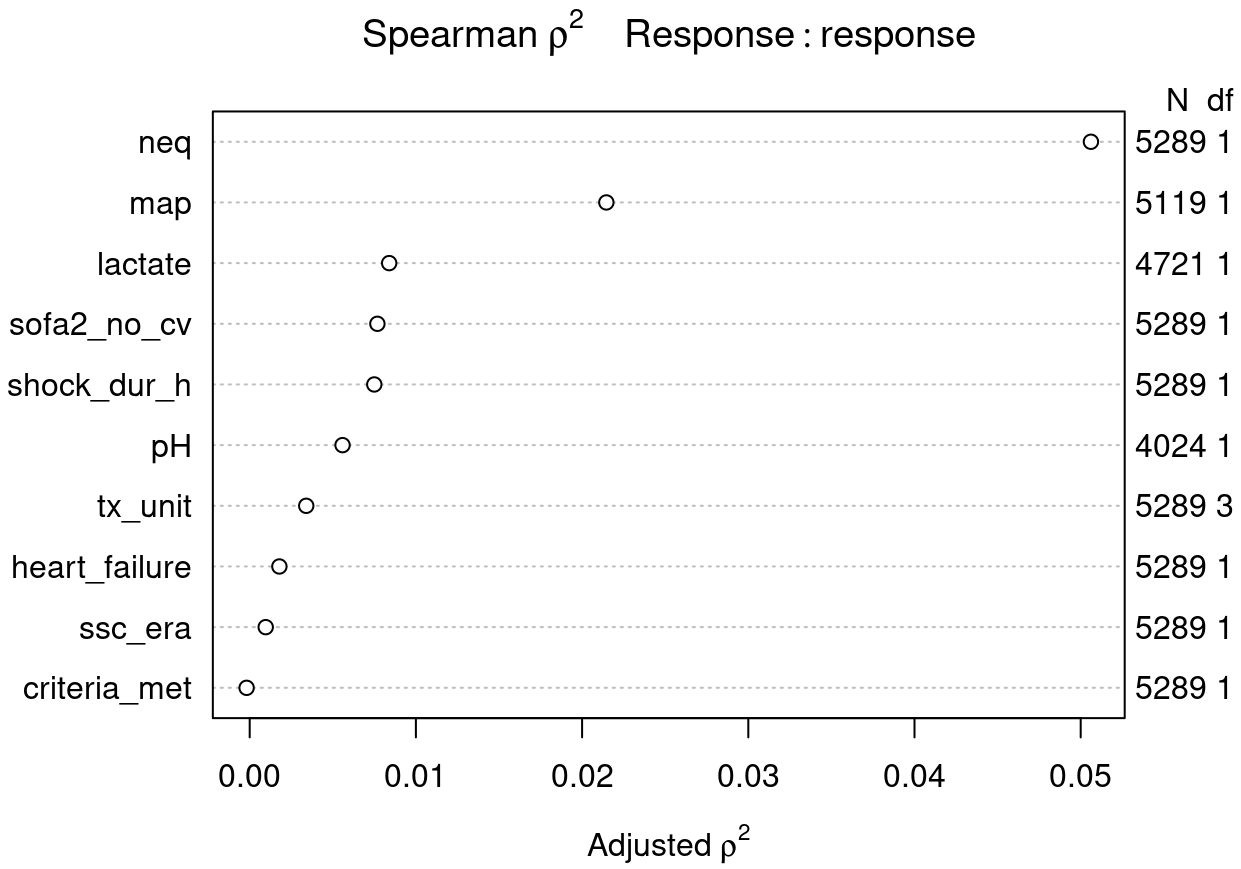
12.2.1.2 Transformations of Predictors

Here consider transformations of the predictors to discern if non-linear terms for the predictors will improve the fit of the model.

Generate a plot of Spearman's ρ^2 rank correlation to show variables with highest potential ability to improve the model predictive performance through introduction of a non-linear term as described by Harrell in *Regression Modeling Strategies, 2nd edition*.

▼ Code

```
plot(spearman2(response ~ ., data = avp |> select(-c(age, sex, race_eth))))
```



▼ Code

```
invisible(pdf("avp_spearman.pdf"))
plot(spearman2(response ~ ., data = avp |> select(-c(age, sex, race_eth))),
     main = ""
     )
title(main = expression("Vasopressin Spearman " * rho^2))
invisible(dev.off())
```

The Spearman's ρ^2 rank correlation plot shows that `neq` has the highest ρ^2, indicating `neq` has the highest potential ability to improve the model predictive performance through introduction of a non-linear term. Since `neq` is quantitative variable, introduce non-linearity with a natural spline with 3 knots (4 degrees of freedom). This approach will add three degrees of freedom to the main effects model (which had 12 parameter degrees of freedom) for a total of 15 parameter degrees of freedom (our budget amount based on the sample size calculation).

In the call below add the natural spline term to determine if it improve model fit. For this work, use a `workflow_set()` for the within-model comparisons.

▼ Code

```
avp_m1_lr_rec_ns <-
  recipe(response ~ ., data = avp) |>
```

```
  # Declare age, sex, and race_eth in a bystander role to keep them in the
  # resampling dataset for future use in fairness checks
  update_role(age, sex, race_eth, new_role = "bystander") |>
  # Single (linear regression) imputation for missing quantitatitve predictors
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h,
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map,
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate,
                       )
    ) |>
  # Add natural spline with 3 knots (4 df) for neq
  step_spline_natural(neq, deg_free = 4)

avp_m1_lr_preproc <- list(
  avp_m1_main = avp_m1_lr_rec_main,
  avp_m1_ns = avp_m1_lr_rec_ns
  )

avp_m1_lr_models <-
  workflow_set(avp_m1_lr_preproc,
               list(logistic = logistic_reg(engine = "glm")),
               cross = FALSE)
```

Determine performance metrics (AUCROC and Brier score) using cross-validation resampling. These metrics were selected based on recommendations in Van Calster 2025, First, declare the survival metrics. Second, establish the cross-validation folds. Will use 10-fold cross-validation with 5 repeats, stratified by event. Third, save the predictions in order to visualize the model fit and residuals. Fourth, fit the model to the resampled dataset. Lastly, collect the predictions.

Resample each model in the workflow set (`avp_m1_lr_models`).

▼ Code

```
set.seed(2026)
train_cv <- vfold_cv(avp, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)
```

```
avp_m1_lr_models_fit <-
  avp_m1_lr_models |>
  workflow_map("fit_resamples", seed = 2026, verbose = TRUE,
               resamples = train_cv, metrics = class_metrics,
               control = keep_pred
               )
```

```
i 1 of 2 resampling: avp_m1_main_logistic

✓ 1 of 2 resampling: avp_m1_main_logistic (19.7s)

i 2 of 2 resampling: avp_m1_ns_logistic

✓ 2 of 2 resampling: avp_m1_ns_logistic (25.1s)
```

▼ Code

```
avp_m1_lr_preds <-
  collect_predictions(avp_m1_lr_models_fit, summarize = FALSE)
```

Collect then show the metrics (summarized over all cross-validation folds).

▼ Code

```
avp_m1_lr_mets <-
  collect_metrics(avp_m1_lr_models_fit) |>
  # Compute 95% Wald CIs across the 50 resamples
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

avp_m1_lr_mets |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Vasopressin m1 (Logistic Regression) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
```

```
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

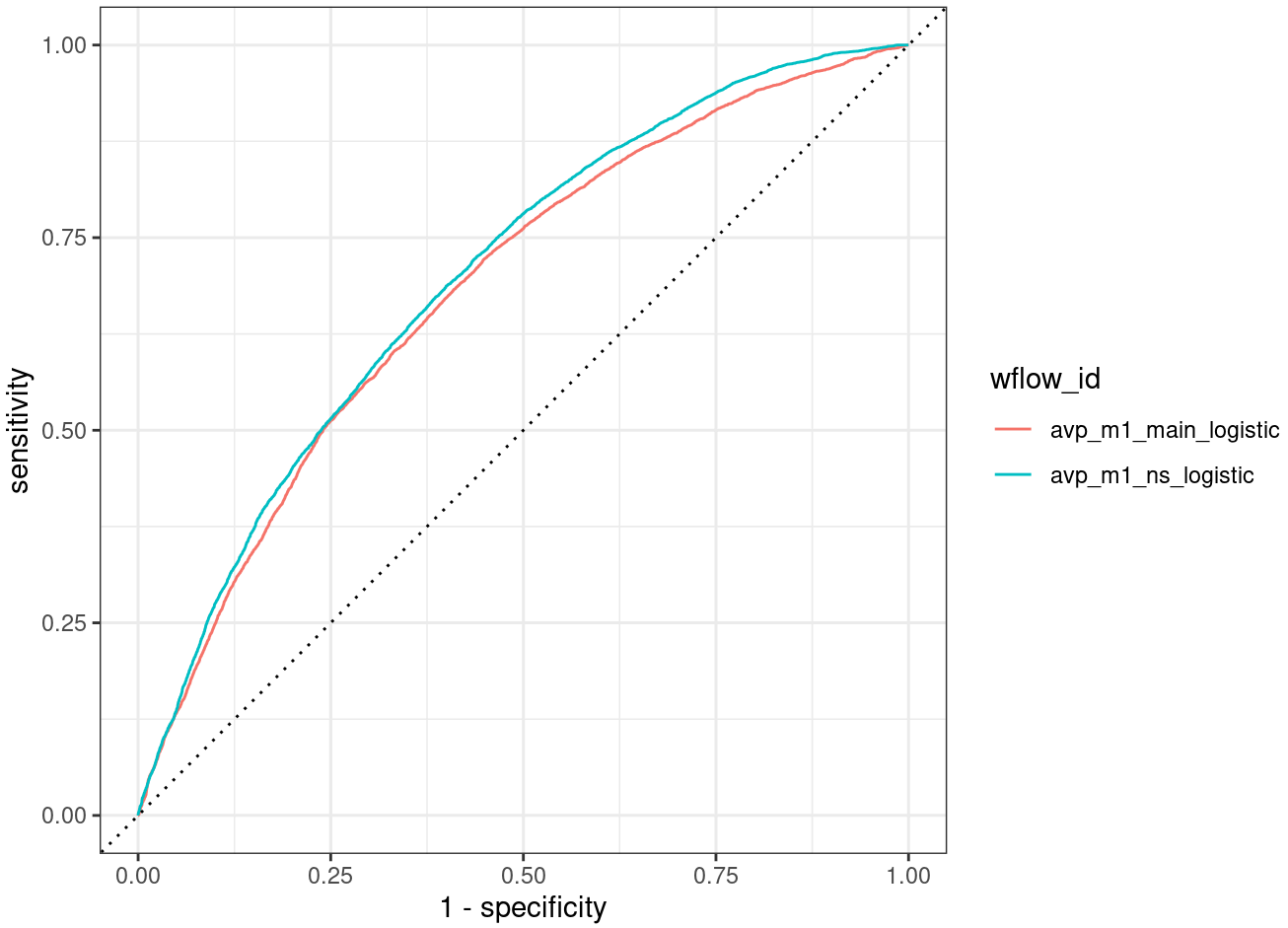
Vasopressin m1 (Logistic Regression) Metric Comparison		
Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
avp_m1_main_logistic	0.684 (0.678, 0.690)	0.207 (0.206, 0.209)
avp_m1_ns_logistic	0.700 (0.695, 0.706)	0.202 (0.201, 0.203)

The model with non-linear terms (`avp_m1_ns_logistic`) had better than the main effects model (`avp_m1_main_logistic`).

Show a non-stylized plot (autoplot) of the AUROC comparison.

▼ Code

```
avp_m1_lr_preds |>
  group_by(wflow_id) |>
  roc_curve(response, .pred_Response) |>
  autoplot()
```



The plot shows the ROC curve is consistently higher with the model incorporating the spline term compared with the model with main effects only.

12.2.2 Machine Learning

12.2.2.1 Regularized (Elastic Net)

Fit regularized (elastic net) models. Use essentially the same recipes as for the logistic regression models (main effects and spline term for `neq`) except dummy code nominal predictors and normalize (center and scale) continuous predictors (normalization is done automatically by `glmnet` but this makes the step explicit).

▼ Code

```
avp_m2_rec_main <- avp_m1_lr_rec_main |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

avp_m2_rec_ns <- avp_m1_lr_rec_ns |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

avp_m2_glmnet_preproc <- list(
```

```
  avp_m2_main = avp_m2_rec_main,
  avp_m2_ns = avp_m2_rec_ns
  )

avp_m2_glmnet_models <-
  workflow_set(avp_m2_glmnet_preproc,
               list(glmnet = logistic_reg(
                 engine = "glmnet",
                 mode = "classification",
                 penalty = tune(),     # lambda
                 mixture = tune()      # alpha
                 )),
               cross = TRUE
               )

set.seed(2026)
train_cv <- vfold_cv(avp, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

pen_grid <- grid_space_filling(
  penalty(range = c(-5, 0)), # log10 scale, reasonable range for elastic net
  mixture(range = c(0, 1)),  # default
  size = 30
  )

avp_m2_glmnet_models_fit <-
  avp_m2_glmnet_models |>
  workflow_map("tune_grid", seed = 2026, verbose = TRUE,
               resamples = train_cv, grid = pen_grid,
               metrics = class_metrics, control = keep_pred
               )
```

```
i 1 of 2 tuning:     avp_m2_main_glmnet

✓ 1 of 2 tuning:     avp_m2_main_glmnet (2m 19.7s)

i 2 of 2 tuning:     avp_m2_ns_glmnet

✓ 2 of 2 tuning:     avp_m2_ns_glmnet (3m 7.8s)
```

▼ Code

```
workflow_ids <- avp_m2_glmnet_models_fit$wflow_id

# Loop over workflows to finalize, fit, and collect results
avp_m2_glmnet_final_fit <- map(workflow_ids, function(wf_id) {

  # Extract tuning results for this workflow
  wf_result <- extract_workflow_set_result(avp_m2_glmnet_models_fit, id = wf_id)
```

```
  # Get best parameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")

  # Extract the workflow object itself
  wf <- extract_workflow(avp_m2_glmnet_models_fit, id = wf_id)

  # Finalize the workflow with best hyperparameters
  final_wf <- finalize_workflow(
    wf,
    best_params
  )

  # Fit the final workflow to the training set
  final_fit <- fit(final_wf, data = avp)

  # Collect resampling metrics for this workflow from tuning
  cv_metrics <- collect_metrics(wf_result) |>
    filter(penalty == best_params$penalty,
           mixture == best_params$mixture)

  # Collect predictions for the best parameters
  cv_preds <- collect_predictions(wf_result, summarize = TRUE) |>
    filter(penalty == best_params$penalty,
           mixture == best_params$mixture)

  # Return as a list
  list(
    workflow_id = wf_id,
    best_params = best_params,
    final_workflow = final_wf,
    final_fit = final_fit,
    cv_metrics = cv_metrics,
    cv_predictions = cv_preds
  )
})

# Name the list elements by workflow
names(avp_m2_glmnet_final_fit) <- workflow_ids
rm(workflow_ids)
```

Show elastic net hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(avp_m2_glmnet_final_fit,
         ~ .x$best_params |> mutate(wflow_id = .y)
         ) |>
  select(wflow_id, penalty, mixture) |>
  gt() |>
```

```
  tab_header(
    title = "Vasopressin m2 Elastic Net Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    wflow_id = "Model",
    penalty = "Penalty (lambda [λ])",
    mixture = "Mixture (alpha [α])"
    ) |>
  fmt_number(decimals = 4) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(penalty, mixture)
    )
```

Vasopressin m2 Elastic Net Best Hyperparameters		
Using 10-fold Cross-Validation (5 repeats)		
Model	Penalty (lambda [λ])	Mixture (alpha [α])
avp_m2_main_glmnet	0.0085	0.8966
avp_m2_ns_glmnet	0.0004	0.0345

These parameters suggest that applying a small penalty term (lambda) to the main effects model helps a little and it is essentially a LASSO situation with mixture 0.9. For the spline model there is no meaningful penalty and the elastic net model is very close to the unpenalized logistic regression model.

Collect metrics and display table (comparing to the logistic regression model fits).

▼ Code

```
avp_m2_glmnet_mets <-
 avp_m2_glmnet_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
```

```
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(avp_m1_lr_mets, avp_m2_glmnet_mets)  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Vasopressin m2 (Elastic Net) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Vasopressin m2 (Elastic Net) Metric Comparison		
Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
avp_m1_main_logistic	0.684 (0.678, 0.690)	0.207 (0.206, 0.209)
avp_m1_ns_logistic	0.700 (0.695, 0.706)	0.202 (0.201, 0.203)
avp_m2_main_glmnet	0.685 (0.680, 0.691)	0.208 (0.207, 0.209)
avp_m2_ns_glmnet	0.700 (0.695, 0.706)	0.202 (0.200, 0.203)

The metrics are essentially the same without and with regularization. This suggests that the model is not materially overfit. And in both models the spline term for `neq` improves fit.

12.2.2.2 Random Forest

Fit a random forest model with the “ranger” engine.

▼ Code

```r
avp_m3_rf_rec <-
  recipe(response ~ ., data = avp) |>
  # Declare age, sex, and race_eth in a bystander role to keep them in the
  # resampling dataset for future use in fairness checks
  update_role(age, sex, race_eth, new_role = "bystander") |>
  # Single (linear regression) imputation for missing quantitative predictors
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate
                       )
    )

avp_m3_rf_preproc <- list(avp_m3_rf = avp_m3_rf_rec)

avp_m3_rf_models <-
  workflow_set(avp_m3_rf_preproc,
               list(ranger = rand_forest(
                 engine = "ranger",
                 mode = "classification",
                 trees = 1000,      # Default is 500, 1000 is better
                 mtry = tune(),     # Number of randomly selected predictors
                 min_n = tune()     # Minimum node size
                 )),
               cross = TRUE
               )

set.seed(2026)
train_cv <- vfold_cv(avp, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

# Prep recipe and determine predictor count
avp_m3_rf_rec_prep <- prep(avp_m3_rf_rec)

x_train <- bake(avp_m3_rf_rec_prep, new_data = NULL) |>
  select(-c(response, age, sex, race_eth))
```

```r
p <- ncol(x_train) # p == 10

rf_param_set <- parameters(
  mtry(range = c(1L, 10L)), #sqrt(p) ~= 3, (p / 2) == 5, which is too narrow
  min_n(range = c(2L, 50L))
)

initial_candidates <- grid_space_filling(
  rf_param_set,
  size = 30
)

race_ctrl <- control_race(
  save_pred = TRUE,
  save_workflow = TRUE,
  parallel_over = "resamples",
  verbose = FALSE
)

avp_m3_rf_model_fit <-
  avp_m3_rf_models |>
  workflow_map(
    "tune_race_anova", seed = 2026, resamples = train_cv,
    grid = initial_candidates, metrics = class_metrics, control = race_ctrl
  )

# Get workflow IDs
workflow_ids <- avp_m3_rf_model_fit$wflow_id

# Finalize, fit, and collect results for the workflow
avp_m3_rf_final_fit <- map(workflow_ids, function(wf_id) {
  # Extract tuning (race) results for this workflow
  wf_result <- extract_workflow_set_result(
    avp_m3_rf_model_fit,
    id = wf_id
    )
  # Select best hyperparameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")
  # Extract the workflow itself
  wf <- extract_workflow(avp_m3_rf_model_fit, id = wf_id)
  # Finalize workflow with best hyperparameters
  final_wf <- finalize_workflow(wf, best_params)
  # Fit finalized workflow to derivation data
  final_fit <- fit(final_wf, data = avp)
  # Collect CV metrics for the selected configuration
  cv_metrics <- collect_metrics(wf_result) |>
    filter(.config == best_params$.config)
  # Collect CV predictions for the selected configuration
  cv_predictions <- collect_predictions(wf_result) |>
    filter(.config == best_params$.config)
  # Return results
```

```
  list(
    workflow_id     = wf_id,
    best_params     = best_params,
    final_workflow  = final_wf,
    final_fit       = final_fit,
    cv_metrics      = cv_metrics,
    cv_predictions  = cv_predictions
    )
})

names(avp_m3_rf_final_fit) <- map_chr(avp_m3_rf_final_fit, "workflow_id")
```

Show hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(avp_m3_rf_final_fit,
         ~ .x$best_params |> mutate(workflow_id = .y)
         ) |>
  select(workflow_id, mtry, min_n) |>
  gt() |>
  tab_header(
    title = "Random Forest Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    workflow_id = "Model",
    mtry = "Random Variables at Split (mtry)",
    min_n = "Min Terminal Node Size (min_n)"
    ) |>
  fmt_number(decimals = 0) |>
  cols_align(
    align = "left",
    columns = c(workflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mtry, min_n)
    )
```

Random Forest Best Hyperparameters

Using 10-fold Cross-Validation (5 repeats)

Model	Random Variables at Split (mtry)	Min Terminal Node Size (min_n)
avp_m3_rf_ranger	1	25

These parameters show that in the best performing model at each split the forest considers only one randomly chosen predictor. This is “maximum decorrelation” among trees. Because CV chose mtry == 1, it suggest no single predictor is substantially dominant and that the ensemble benefits from forcing trees to explore different predictors. The selection of min_n = 25 indicates that the forest prefers a moderate amount of smoothing to represent non-linear effects, but following small local patterns (with lower min_n) is not useful.

Collect and display metrics for model.

▼ Code

```
avp_m3_rf_mets <-
 avp_m3_rf_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(avp_m1_lr_mets,
          avp_m2_glmnet_mets,
          avp_m3_rf_mets)  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Vasopressin m3 (Random Forest) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
```

```
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Vasopressin m3 (Random Forest) Metric Comparison Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
avp_m1_main_logistic	0.684 (0.678, 0.690)	0.207 (0.206, 0.209)
avp_m1_ns_logistic	0.700 (0.695, 0.706)	0.202 (0.201, 0.203)
avp_m2_main_glmnet	0.685 (0.680, 0.691)	0.208 (0.207, 0.209)
avp_m2_ns_glmnet	0.700 (0.695, 0.706)	0.202 (0.200, 0.203)
avp_m3_rf_ranger	0.703 (0.698, 0.708)	0.207 (0.207, 0.208)

The random forest model marginally improves discrimination (higher AUROC) at the expense of slightly worse overall performance (higher Brier).

12.2.2.3 Boosted Trees

Generate a boosted tree model with the “xgboost” engine in {tidymodels}.

▼ Code

```
avp_m4_xgb_rec <-
  recipe(response ~ ., data = avp) |>
  update_role(age, sex, race_eth, new_role = "bystander") |>
  step_impute_linear(
    map,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h
    )
  ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h, map
    )
  ) |>
  step_impute_linear(
    pH,
```

```
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h, map, lactate
    )
  ) |>
  # Dummy code nominal predictors
  step_dummy(all_nominal_predictors())

avp_m4_xgb_preproc <- list(avp_m4_boost = avp_m4_xgb_rec)

avp_m4_xgb_model <-
  workflow_set(avp_m4_xgb_preproc,
               list(xgboost = boost_tree(
                 trees = 1000,              # Number of trees
                 tree_depth = tune(),       # Tree depth: # of "interactions"
                 min_n = tune(),            # Minimum node size
                 loss_reduction = tune(),   # Reduction in the loss function required to split
         further
                 sample_size = tune(),      # Proportion of data exposed to fitting routine
                 mtry = tune(),             # Random predictors at each split
                 learn_rate = tune()        # Rate at which the boosting algorithm adapts
                 ) |>
                   set_engine("xgboost") |>
                   set_mode("classification")
                 ),
               cross = FALSE
               )

set.seed(2026)
train_cv <- vfold_cv(avp, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

# Prep recipe and determine post-dummy predictor count
avp_m4_xgb_rec_prep <- prep(avp_m4_xgb_rec)

x_train <- bake(avp_m4_xgb_rec_prep, new_data = NULL) |>
  select(-c(response, age, sex, race_eth))

p <- ncol(x_train) # p == 10

# Select parameters (particularly depth) based p==10 and RF optimal parameters
xgb_params <- parameters(
  tree_depth(range = c(1L, 6L)),
  min_n(range = c(2L, 50L)),
  learn_rate(range = c(-3, -0.5)),
  loss_reduction(),
  sample_prop(range = c(0.5, 1.0)),
  finalize(mtry(), x_train)
  )
```

```r
initial_candidates <- grid_space_filling(
  xgb_params,
  size = 30
  )

race_ctrl <- control_race(
  save_pred = TRUE,
  save_workflow = TRUE,
  parallel_over = "resamples",
  verbose = FALSE
  )

avp_m4_xgb_model_fit <-
  avp_m4_xgb_model |>
  workflow_map(
    "tune_race_anova",
    seed = 2026,
    resamples = train_cv,
    grid = initial_candidates,
    metrics = class_metrics,
    control = race_ctrl
    )

workflow_ids <- avp_m4_xgb_model_fit$wflow_id

# Finalize, fit, and collect results the workflow
avp_m4_xgb_final_fit <- map(workflow_ids, function(wf_id) {
  # Extract tuning (race) results for this workflow
  wf_result <- extract_workflow_set_result(
    avp_m4_xgb_model_fit,
    id = wf_id
    )
  # Select best hyperparameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")
  # Extract the workflow itself
  wf <- extract_workflow(avp_m4_xgb_model_fit, id = wf_id)
  # Finalize workflow with best hyperparameters
  final_wf <- finalize_workflow(wf, best_params)
  # Fit finalized workflow to derivation data
  final_fit <- fit(final_wf, data = avp)
  # Collect CV metrics for the selected configuration
  cv_metrics <- collect_metrics(wf_result) |>
    filter(.config == best_params$.config)
  # Collect CV predictions for the selected configuration
  cv_predictions <- collect_predictions(wf_result) |>
    filter(.config == best_params$.config)
  # Return results
  list(
    workflow_id     = wf_id,
    best_params     = best_params,
```

```
    final_workflow  = final_wf,
    final_fit       = final_fit,
    cv_metrics      = cv_metrics,
    cv_predictions  = cv_predictions
    )
})

names(avp_m4_xgb_final_fit) <- map_chr(avp_m4_xgb_final_fit, "workflow_id")
```

Show hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(avp_m4_xgb_final_fit,
         ~ .x$best_params |> mutate(wflow_id = .y)
         ) |>
  mutate(trees = 1000) |>
  select(wflow_id, trees, tree_depth, min_n, learn_rate, loss_reduction,
         sample_size, mtry
         ) |>
  gt() |>
  fmt_number(columns = learn_rate, decimals = 4) |>
  fmt_number(columns = loss_reduction, decimals = 3) |>
  tab_header(
    title = "Gradient Boosted Model Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    wflow_id = "Model",
    trees = "Trees",
    tree_depth = "Tree Depth",
    min_n = "Node Size",
    learn_rate = "Learn Rate",
    loss_reduction = "Min Loss Reduction",
    sample_size = "Proportion of data exposed",
    mtry = "Predictors at Splits",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(trees, tree_depth, min_n, learn_rate, loss_reduction,
                sample_size, mtry
                )
    )
```

Gradient Boosted Model Best Hyperparameters

Using 10-fold Cross-Validation (5 repeats)

Model	Trees	Tree Depth	Node Size	Learn Rate	Min Loss Reduction	Proportion of data exposed	Predictors at Splits
avp_m4_boost_xgboost	1000	2	18	0.0196	0.329	0.5	2

Optimal parameters are not at tuning boundaries. The algorithm preferred shallow trees with iterative small improvements (over aggressive boosting), which is consistent with the random forest findings.

Collect and display metrics for model.

▼ Code

```
avp_m4_xgb_mets <-
 avp_m4_xgb_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(avp_m1_lr_mets,
          avp_m2_glmnet_mets,
          avp_m3_rf_mets,
          avp_m4_xgb_mets
          )  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Vasopressin m4 (XGBoost) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
```

```
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Vasopressin m4 (XGBoost) Metric Comparison Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
avp_m1_main_logistic	0.684 (0.678, 0.690)	0.207 (0.206, 0.209)
avp_m1_ns_logistic	0.700 (0.695, 0.706)	0.202 (0.201, 0.203)
avp_m2_main_glmnet	0.685 (0.680, 0.691)	0.208 (0.207, 0.209)
avp_m2_ns_glmnet	0.700 (0.695, 0.706)	0.202 (0.200, 0.203)
avp_m3_rf_ranger	0.703 (0.698, 0.708)	0.207 (0.207, 0.208)
avp_m4_boost_xgboost	0.703 (0.697, 0.708)	0.201 (0.200, 0.202)

The boosted model has the best discrimination and accuracy metrics among all models evaluated, but the difference is marginal.

12.2.3 Discrimination/Accuracy Comparison

Now, show the discrimination and accuracy metrics for all models.

▼ Code

```
(avp_disc_perf_mets <- bind_rows(avp_m1_lr_mets,
          avp_m2_glmnet_mets,
          avp_m3_rf_mets,
          avp_m4_xgb_mets
          )  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Vasopressin Models Discrimination/Overall Performance Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
```

```
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    ))
```

Vasopressin Models Discrimination/Overall Performance Metric Comparison		
Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
avp_m1_main_logistic	0.684 (0.678, 0.690)	0.207 (0.206, 0.209)
avp_m1_ns_logistic	0.700 (0.695, 0.706)	0.202 (0.201, 0.203)
avp_m2_main_glmnet	0.685 (0.680, 0.691)	0.208 (0.207, 0.209)
avp_m2_ns_glmnet	0.700 (0.695, 0.706)	0.202 (0.200, 0.203)
avp_m3_rf_ranger	0.703 (0.698, 0.708)	0.207 (0.207, 0.208)
avp_m4_boost_xgboost	0.703 (0.697, 0.708)	0.201 (0.200, 0.202)

The largest performance gain came from modeling `neq` with a natural spline (`avp_m1_ns_logistic`) instead of main effects alone (`avp_m1_main_logistic`). The differences between the spline logistic regression model and machine learning models are very minor (<=0.01 difference for each metric point estimate) and they generally have similarly modest performance.

Considering the goal of future model implementation in the electronic medical record and explainability to clinicians, a logistic regression model (`avp_m1_ns_logistic`) is preferred. The XGBoost model (`avp_m4_boost_xgboost`) had marginally better performance, though. These two models will be carried forward for calibration assessment prior to final model selection.

Export comparison table to Word (commented out for rendering)

▼ Code

```
#gtsave(
#  data = avp_disc_perf_mets,
#  filename = "avp_disc_perf_metrics.docx"
#)
```

12.2.4 Calibration

Use the `pmcalibration()` function from the `{pmcalibration}` package to assess calibration (vignette here). Calibration will be assessed using pooled out-of-fold predictions from repeated 10-fold cross-validation. For each participant, predicted probabilities will be averaged across the five cross-validation repeats and compared with observed outcomes using flexible calibration curves from `pmcalibration()`. The `pmcalibration()` function estimates group-wise calibration using restricted cubic splines, with explicit handling of censoring via Kaplan–Meier–type estimators, and adds bootstrap uncertainty (1000 replications) around the calibration curve. Curves will be generated using a restricted cubic splines smoother with 3 knots; the `{pmcalibration}` package default is 5 knots for the restricted cubic splines smoother, but Austin 2020 found that 3 knots led to less bias. Notably, the `{pmcalibration}` package assesses "moderate" calibration via non-linear calibration curves as described in Van Calster 2016.

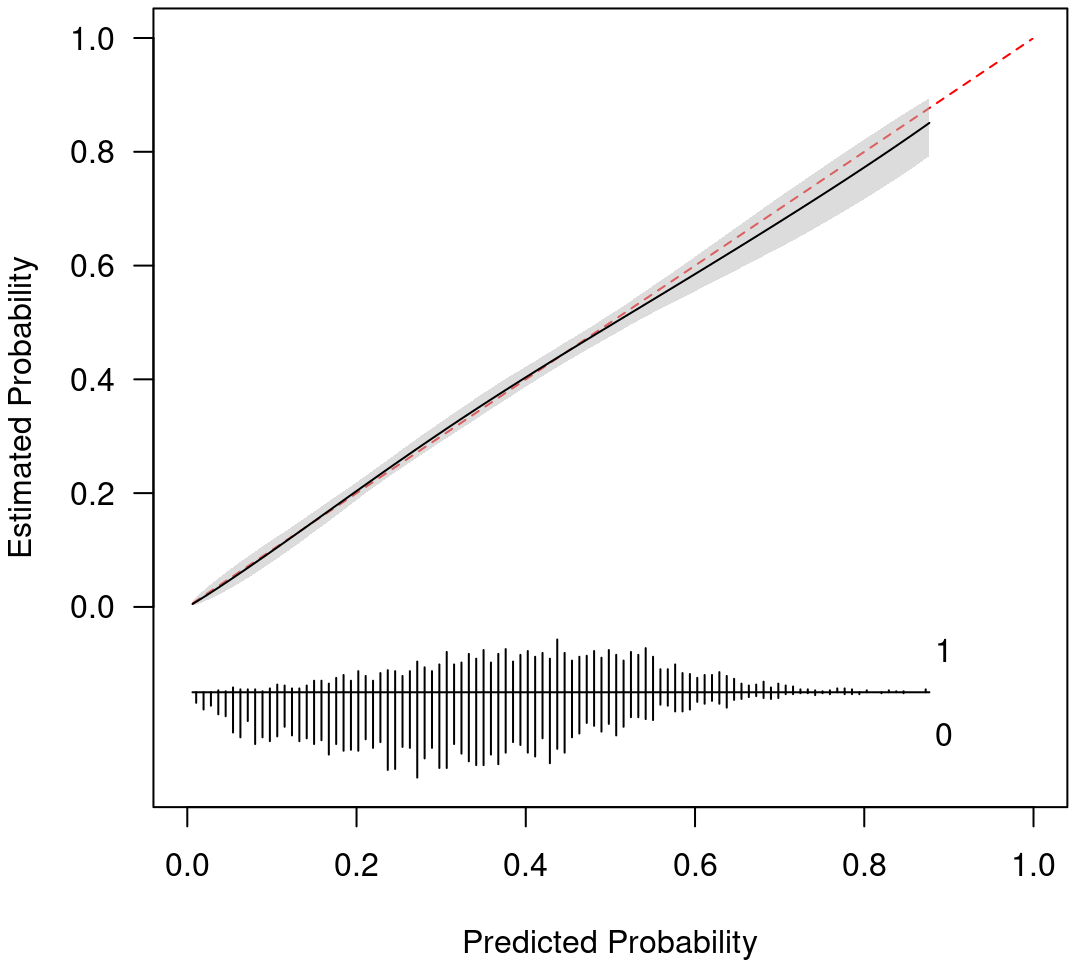
12.2.4.1 Logistic Regression Model

Compare the observed outcomes from the original derivation dataset with the predicted probabilities for the outcome generated in the out-of-fold predictions from the repeated cross-validation process (averaging the out-of-fold predictions for each patient).

▼ Code

```
avp_m1_lr_preds_cv <- avp_m1_lr_preds |>
  filter(wflow_id == "avp_m1_ns_logistic") |>
  group_by(.row) |>
  summarise(
    response = first(response),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
  ) |>
  mutate(
    response_num = if_else(response == "Response", 1, 0)
  )

set.seed(2026)
avp_lr_cal <-
  pmcalibration(
    y = avp_m1_lr_preds_cv$response_num,
    p = avp_m1_lr_preds_cv$.pred_Response,
    ci = "boot", # n = 1000 replicates is the default
    smooth = "rcs",
    nk = 3 # 5 knots is default, 3 has less bias
    )
```



▼ Code

```
invisible(pdf("avp_logistic_calibration.pdf"))
par(mfrow = c(1, 1),
    cex.axis = 1.4, cex.lab = 1.8, cex.main = 2,
    mar = c(5.1, 5.5, 1.4, 1.6), # bottom, left, top, right
    mgp = c(3.5, 1, 0)
    )
plot(avp_lr_cal)
invisible(dev.off())
```

The cross-validated predictions from the logistic regression model with a natural spline term for `neq` are very well calibrated relative to the observed outcomes in the derivation cohort.

Although the discrimination (AUROC) for the logistic regression model (with a spline term) was only modest, the calibration is very good. This means the probabilities the model produces are very trustworthy.

12.2.4.2 XGBoost Model

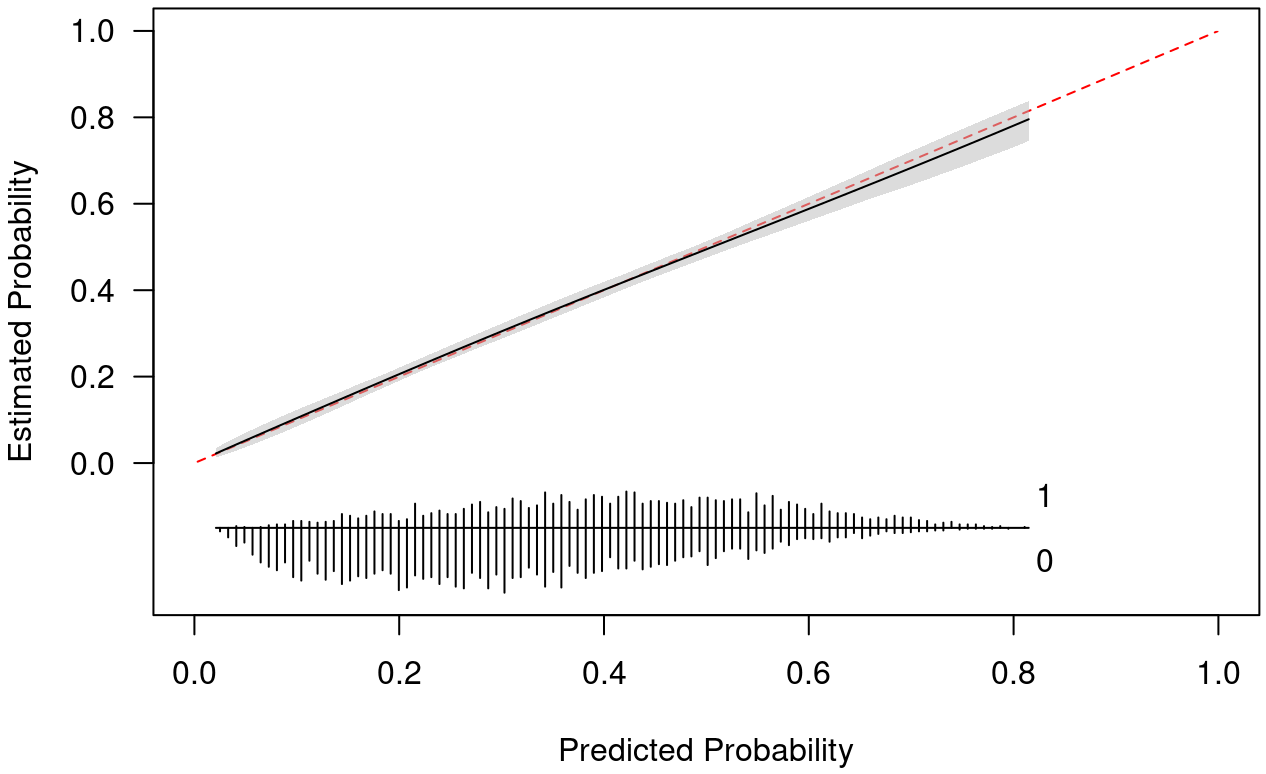
Use the same calibration assessment workflow for the boosted trees (XGBoost) model.

▼ Code

```
avp_m4_xgb_preds <- avp_m4_xgb_final_fit[[1]]$cv_predictions

avp_m4_xgb_preds_cv <- avp_m4_xgb_preds |>
  group_by(.row) |>
  summarise(
    response = first(response),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
  ) |>
  mutate(
    response_num = if_else(response == "Response", 1, 0)
  )

set.seed(2026)
avp_xgb_cal <- pmcalibration(
  y = avp_m4_xgb_preds_cv$response_num,
  p = avp_m4_xgb_preds_cv$.pred_Response,
  ci = "boot", # n = 1000 replicates is the default
  smooth = "rcs",
  nk = 3 # 5 knots is default, 3 has less bias
  )
plot(avp_xgb_cal)
title("Calibration Plot: Vasopressin XGBoost")
```



▼ Code

```
invisible(dev.copy(pdf, "avp_xgboost_calibration.pdf"))
invisible(dev.off())
```

Save the two vasopressin calibration plots as a panel figure.

▼ Code

```
invisible(ragg::agg_tiff("avp_calibration_panel.tiff",
                         width = 11, height = 5, units = "in",
                         res = 600, compression = "lzw"
                         ))

par(mfrow = c(1, 2), pty = "s")

plot(avp_lr_cal)
title("Vasopressin Logistic Regression (Non-Linear)")

plot(avp_xgb_cal)
title("Vasopressin XGBoost")
```

```
par(mfrow = c(1, 1))
invisible(dev.off())
```

12.2.4.3 Quantitative metrics

Generate and display metrics for the two models. The “Eavg” is the same as “integrated calibration index” (ICI), therefore will make this clarification in the output. The “ECI” is the average squared difference between predicted and observed probabilities (multiplied by 100). For each of the calibration metrics outlined, lower values are better.

▼ Code

```
# Display the table
(avp_cal_metrics_tbl <-
  bind_rows(
    as_tibble(avp_lr_cal$metrics, rownames = "metric") |>
      mutate(model = "Logistic (spline)"),
    as_tibble(avp_xgb_cal$metrics, rownames = "metric") |>
      mutate(model = "XGBoost")
  ) |>
  select(model, metric, value) |>
  pivot_wider(
    names_from = metric,
    values_from = value
  ) |>
  rename(
    Model = model,
    `Eavg (ICI)` = Eavg
  ) |>
  gt() |>
  tab_header(
    title = "Vasopressin Model Calibration Metrics"
  ) |>
  fmt_number(
    columns = c(`Eavg (ICI)`, E50, E90, Emax, ECI),
    decimals = 4
  ) |>
  cols_align(
    align = "center"
  ))
```

Vasopressin Model Calibration Metrics					
Model	Eavg (ICI)	E50	E90	Emax	ECI
Logistic (spline)	0.0056	0.0051	0.0104	0.0276	0.0053

Vasopressin Model Calibration Metrics					
Model	Eavg (ICI)	E50	E90	Emax	ECI
XGBoost	0.0053	0.0049	0.0101	0.0192	0.0040

Both models are extremely well calibrated. The XGBoost model has consistently better quantitative calibration metrics, but the differences between models are very small.

Export comparison table to Word (commented out for rendering)

▼ Code

```
#gtsave(
#  data = avp_cal_metrics_tbl,
#  filename = "avp_cal_metric_comparison.docx"
#)
```

12.2.5 Decision Curve Analysis

To assess if there may be clinical utility with the two vasopressin response prediction models under consideration, perform a decision curve analysis (DCA). To correct for overfitting, use the out-of-fold predictions for each model.

First, extract predictions from the logistic regression and XGBoost models, and combine.

▼ Code

```
# Extract the out-of-fold predictions for each model
# Then average predictions across the 5 CV repeats using .row
# Logistic model
avp_lr_dca <- avp_m1_lr_preds |>
  filter(wflow_id == "avp_m1_ns_logistic") |>
  group_by(.row) |>
  summarise(
    response = first(response),
    p_logistic = mean(.pred_Response),
    .groups = "drop"
  )

# XGBoost model
avp_xgb_dca <- avp_m4_xgb_final_fit[[1]]$cv_predictions |>
  group_by(.row) |>
  summarise(
    p_xgb = mean(.pred_Response),
    .groups = "drop"
  )

# Combine
```

```r
avp_dca_data <- avp_lr_dca |>
  left_join(avp_xgb_dca, by = ".row")
```

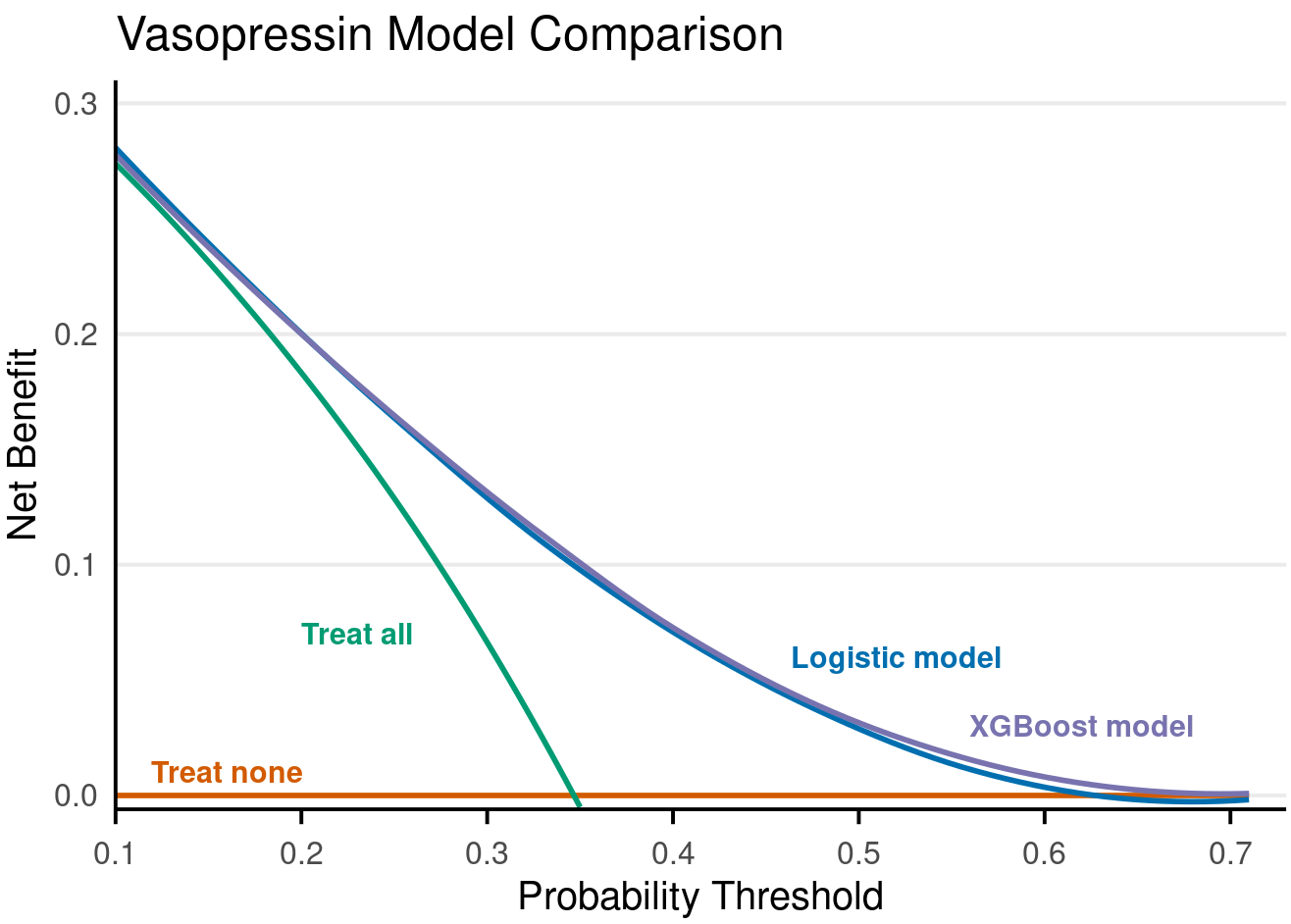
Then run DCA and plot.

▼ Code

```r
avp_dca <- dca(
  response == "Response" ~ p_logistic + p_xgb,
  data = avp_dca_data
  )

(avp_dca_plot <-
  ggplot(as_tibble(avp_dca) |>
           filter(net_benefit >= -0.006 &
                    net_benefit <= 0.31 &
                    threshold <=0.71
                  ) |>
           mutate(label = fct_relevel(label, "Treat None", "Treat All",
                                      "p_logistic", "p_xgb"
                                      )),
         aes(x = threshold,
             y = net_benefit,
             color = label,
             group = label)) +
    geom_smooth(method = loess, formula = y ~ x,
                linewidth = 1, se = FALSE) +
    scale_color_manual(values = c(
      "p_logistic" = "#0072B2",
      "p_xgb" = "#7876B1FF",
      "Treat All" = "#009E73",
      "Treat None" = "#D55E00"
      ),
      labels = c(
        "p_logistic" = "Logistic regression",
        "p_xgb" = "XGBoost",
        "Treat All" = "Treat all",
        "Treat None" = "Treat none"
      )) +
    scale_y_continuous(
      limits = c(-0.006, 0.31),
      expand = expansion(add = c(0, 0))
      ) +
    scale_x_continuous(
      limits = c(0.1, 0.71),
      breaks = seq(0, 0.7, by = 0.1),
      expand = expansion(add = c(0, 0.02))
      ) +
    labs(
      title = "Vasopressin Model Comparison",
```

```
  x = "Probability Threshold",
  y = "Net Benefit",
  color = NULL
  ) +
annotate("text", x = 0.52, y = 0.06, label = "Logistic model",
         color = "#0072B2", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.62, y = 0.03, label = "XGBoost model",
         color = "#7876B1FF", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.23, y = 0.07, label = "Treat all",
         color = "#009E73", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.16, y = 0.01, label = "Treat none",
         color = "#D55E00", family = "sans", fontface = "bold",
         size = 4
         ) +
theme_minimal(base_size = 15) +
theme(
  text = element_text(family = "sans"),
  legend.position = "none",
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x = element_line(),
  axis.line.y = element_line(),
  axis.ticks.x = element_line(),
  axis.ticks.length = unit(0.2, "cm")
))
```



Both models are better than “Treat none” and “Treat all” from probability thresholds about 0.1 to about 0.6. The two models have essentially the same net benefit at probability thresholds below 0.5, with the XGBoost model having slightly higher net benefit at probability thresholds from 0.5 to 0.65.

Obtain exact threshold values for positive net benefit with logistic regression and XGBoost models. To improve matching with visualization in the figure, use a practical tolerance value of 0.001 for the thresholds.

▼ Code

```
as_tibble(avp_dca) |>
  select(threshold, label, net_benefit) |>
  pivot_wider(names_from = label, values_from = net_benefit) |>
  filter(!is.na(p_logistic), !is.na(p_xgb),
         !is.na(`Treat All`), !is.na(`Treat None`)
         ) |>
  summarise(
    min_prob_thresh_lr = min(threshold[p_logistic > `Treat All` + 0.001 &
                                       p_logistic > `Treat None` + 0.001]
                             ),
    max_prob_thresh_lr = max(threshold[p_logistic > `Treat All` + 0.001 &
                                       p_logistic > `Treat None` + 0.001]
                             ),
    min_prob_thresh_xgb = min(threshold[p_xgb > `Treat All` + 0.001 &
```

```
                                        p_xgb > `Treat None` + 0.001]
                            ),
    max_prob_thresh_xgb = max(threshold[p_xgb > `Treat All` + 0.001 &
                                        p_xgb > `Treat None` + 0.001]
                            )
  )
```

```
# A tibble: 1 × 4
  min_prob_thresh_lr max_prob_thresh_lr min_prob_thresh_xgb
               <dbl>              <dbl>               <dbl>
1               0.08               0.61                0.08
# ℹ 1 more variable: max_prob_thresh_xgb <dbl>
```

The vasopressin response logistic regression model had higher net benefit than the “Treat all” and “Treat none” curves at threshold probabilities from 0.08 to 0.61. The vasopressin response XGBoost model had higher net benefit than the “Treat all” and “Treat none” curves at threshold probabilities from 0.08 to 0.68.

Save DCA plot

▼ Code

```
ggsave(
  "avp_dca_compare.tiff",
  plot = avp_dca_plot,
  device = ragg::agg_tiff,
  width = dev.size("in")[1],
  height = dev.size("in")[2],
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

12.2.6 Model Selection

Overall, compared with the logistic regression model with a spline term for `neq`, the XGBoost model demonstrated slightly better discrimination, overall performance, calibration, and net benefit on decision curve analysis. However, the magnitude of improvement was very small. Considering overall performance similarity but higher interpretability and implementation readiness for a logistic regression model, the logistic regression model with a spline term for norepinephrine dose (`avp_m1_ns_logistic`) was selected as the final model for vasopressin.

12.2.6.1 Refit to Whole Derivation Dataset

Refit the final model (`avp_m1_ns_logistic`) in the whole vasopressin derivation dataset (`avp`).

▼ Code

```r
set.seed(2026)
avp_mod_final_wf <- extract_workflow(
  avp_m1_lr_models,
  id = "avp_m1_ns_logistic"
  )
avp_mod_final_fit_dev <- fit(avp_mod_final_wf, avp)
```

Extract engine data and estimate apparent performance on derivation data.

▼ Code

```r
avp_lr_fit <- extract_fit_engine(avp_mod_final_fit_dev)
avp_apparent_performance <- glance(avp_lr_fit) |>
  transmute(
    nobs,
    n_parameters = length(coef(avp_lr_fit)) - 1,
    r2_nag = performance::r2_nagelkerke(avp_lr_fit)
  )

## Prepare data
avp_rec_trained <- extract_recipe(avp_mod_final_fit_dev)

avp_baked_dev <- bake(
  avp_rec_trained,
  new_data = avp,
  all_predictors()
)

avp_dev_score_data <- bind_cols(
  avp |> select(response),
  avp_baked_dev
)

# Generate metrics
avp_dev_score <-
  Score(
    object = list(avp_lr_fit),
    formula = response ~ 1,
    data = avp_dev_score_data,
    metrics = c("auc", "brier")
    )

# Extract and format results
auc_ci <-
  avp_dev_score$AUC$score |>
  as_tibble() |>
  transmute(
    auc = as.numeric(AUC),
    auc_lower = as.numeric(lower),
    auc_upper = as.numeric(upper)
```

```
  )

brier_ci <-
  avp_dev_score$Brier$score |>
  as_tibble() |>
  filter(model == "glm") |>
  transmute(
    brier = as.numeric(Brier),
    brier_lower = as.numeric(lower),
    brier_upper = as.numeric(upper)
  )

avp_dev_mets <- cbind(auc_ci, brier_ci) |>
  mutate(
    AUROC = sprintf("%.3f (%.3f, %.3f)", auc, auc_lower, auc_upper),
    Brier = sprintf("%.3f (%.3f, %.3f)", brier, brier_lower, brier_upper)
  )
rm(auc_ci, brier_ci)

cbind(avp_apparent_performance,
      avp_dev_mets |> select(AUROC, Brier)) |>
  gt() |>
  fmt_number(columns = c(r2_nag),
             decimals = 3
             ) |>
  tab_header(title = "Vasopressin Logistic Regression Model Fit and Apparent Performance
        Parameters",
             subtitle = "In Derivation (Ohio) Dataset") |>
  cols_label(
    nobs = "N",
    n_parameters =  "Parameters (n)",
    r2_nag = "R2 (Nagelkerke)",
    Brier = "Brier Score"
    ) |>
  cols_align(align = "center")
```

Vasopressin Logistic Regression Model Fit and Apparent Performance Parameters

In Derivation (Ohio) Dataset

N	Parameters (n)	R2 (Nagelkerke)	AUROC	Brier Score
5289	15	0.160	0.704 (0.690, 0.719)	0.201 (0.196, 0.205)

Show model parameters.

▼ Code

```
(avp_lr_param_estimates <-
   tidy(avp_lr_fit,
        conf.int = TRUE
        ) |>
   # Since the model predicts the second factor level ("No Response") for
   # yardstick() ease, invert the coefficients to express ORs for "Response"
   # Use a higher number of reported digits for some parameters
   mutate(estimate_ci = case_when(
     term == "(Intercept)" ~ sprintf("%.9f (%.9f, %.9f)",
                                     exp(-estimate),
                                     exp(-conf.high),
                                     exp(-conf.low)
                                     ),
     term == "shock_dur_h" ~ sprintf("%.4f (%.4f, %.4f)",
                                     exp(-estimate),
                                     exp(-conf.high),
                                     exp(-conf.low)
                                     ),
     term == "map" ~ sprintf("%.3f (%.3f, %.3f)",
                                     exp(-estimate),
                                     exp(-conf.high),
                                     exp(-conf.low)
                                     ),
     .default = sprintf("%.2f (%.2f, %.2f)",
                        exp(-estimate),
                        exp(-conf.high),
                        exp(-conf.low)
                        )
     )) |>
   select(term, estimate_ci)  |>
   gt() |>
   tab_header(title = "Vasopressin Logistic Regression Model Parameter Estimates") |>
   cols_label(
     term = "Variable",
     estimate_ci = "Response Odds Ratio (95% CI)"
     ) |>
   cols_align(align = "left", columns = term) |>
   cols_align(align = "center", columns = estimate_ci))
```

Vasopressin Logistic Regression Model Parameter Estimates	
Variable	Response Odds Ratio (95% CI)
(Intercept)	0.000000731 (0.000000007, 0.000069388)
heart_failureYes	0.78 (0.66, 0.93)
tx_unitCardiac ICU	0.73 (0.58, 0.93)

Vasopressin Logistic Regression Model Parameter Estimates	
Variable	Response Odds Ratio (95% CI)
tx_unitED/Rapid Response	0.78 (0.58, 1.05)
tx_unitSurgical or Mixed Med/Surg ICU	0.99 (0.87, 1.14)
ssc_eraSSC 2021	1.15 (1.02, 1.31)
criteria_metBoth or Sepsis-3 only	0.93 (0.82, 1.06)
sofa2_no_cv	0.94 (0.92, 0.96)
pH	3.66 (1.99, 6.76)
shock_dur_h	1.0021 (1.0002, 1.0040)
map	1.028 (1.022, 1.033)
lactate	0.95 (0.93, 0.97)
neq_1	32.20 (13.16, 82.46)
neq_2	19.28 (12.03, 31.85)
neq_3	26.38 (0.50, 3255.22)
neq_4	2051.41 (0.45, 2146530.76)

Export table to Word (commented out for rendering)

▼ Code

```
#gtsave(
#  data = avp_lr_param_estimates,
#  filename = "avp_lr_parameter_estimates.docx"
#)
```

12.2.6.2 Compare to Simple Model

Compare the selected multivariable model (`avp_m1_ns_logistic`) with a simple model that only includes `neq` with a spline term.

▼ Code

```
lr_spec <-
  logistic_reg() |>
  set_engine("glm")
```

```
avp_m0_lr_rec_ns <-
  recipe(response ~ neq, data = avp) |>
  step_spline_natural(neq, deg_free = 5)

avp_m0_lr_preproc <- list(avp_m0_neq_only = avp_m0_lr_rec_ns,
                          avp_m1_multivar = avp_m1_lr_rec_ns
                          )

avp_m0_lr_model <-
  workflow_set(avp_m0_lr_preproc,
               list(logistic = logistic_reg(engine = "glm")),
               cross = FALSE
               )

set.seed(2026)
train_cv <- vfold_cv(avp, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

avp_m0_lr_model_fit <-
  avp_m0_lr_model |>
  workflow_map("fit_resamples", seed = 2026, verbose = TRUE,
               resamples = train_cv, metrics = class_metrics,
               control = keep_pred
               )
```

```
i 1 of 2 resampling: avp_m0_neq_only_logistic

✓ 1 of 2 resampling: avp_m0_neq_only_logistic (10.9s)

i 2 of 2 resampling: avp_m1_multivar_logistic

✓ 2 of 2 resampling: avp_m1_multivar_logistic (26.2s)
```

▼ Code

```
avp_m0_lr_preds <-
  collect_predictions(avp_m0_lr_model_fit, summarize = FALSE)

avp_m0_lr_mets <-
  collect_metrics(avp_m0_lr_model_fit) |>
  # Compute 95% Wald CIs across the 50 resamples
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
```

```
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

avp_m0_lr_mets |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Vasopressin Final Model Performance vs. NE Alone",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Vasopressin Final Model Performance vs. NE Alone

Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).

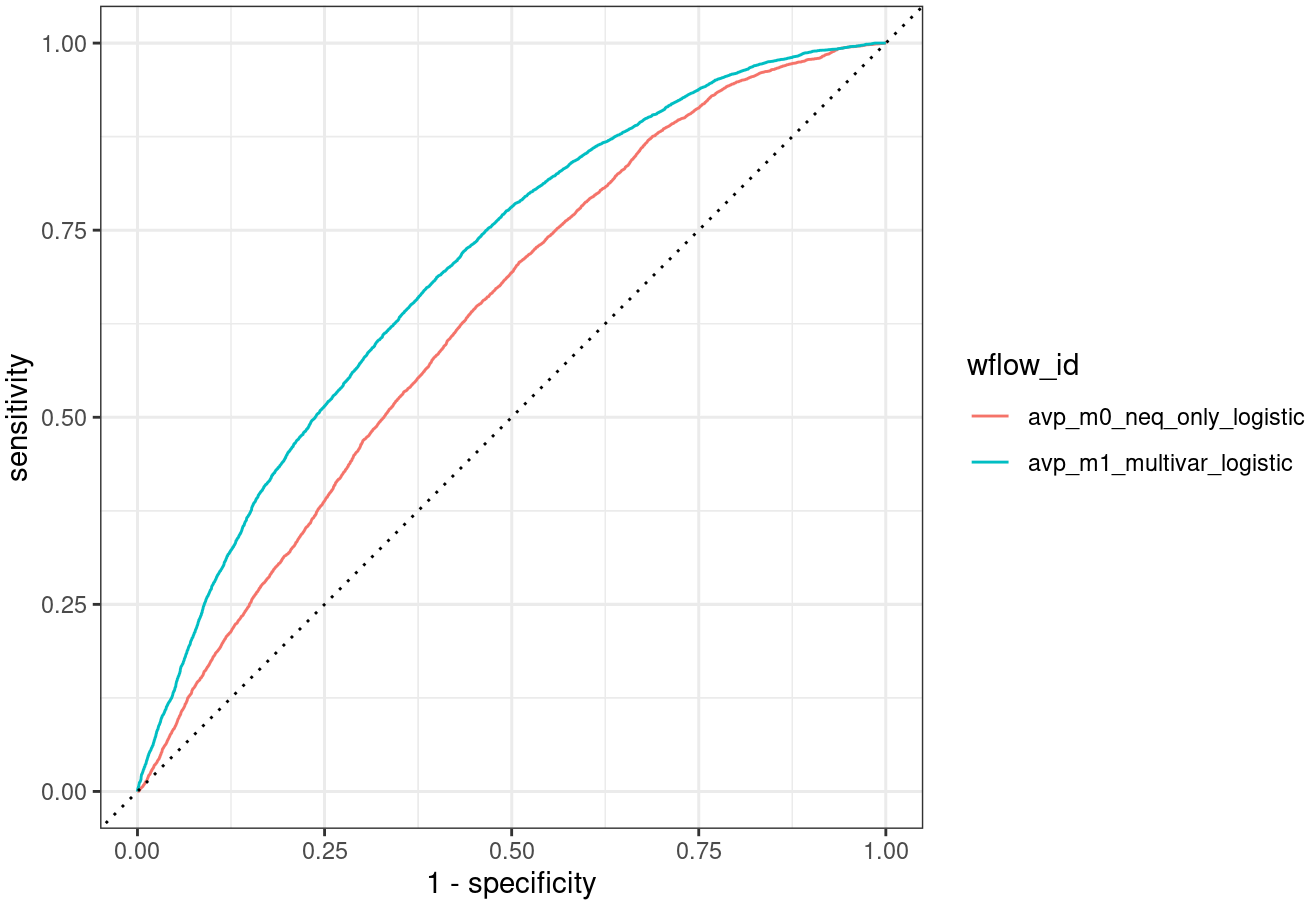
Model	AUROC	Brier Score
avp_m0_neq_only_logistic	0.636 (0.629, 0.643)	0.214 (0.213, 0.215)
avp_m1_multivar_logistic	0.700 (0.695, 0.706)	0.202 (0.201, 0.203)

The multivariable model has substantially better metrics than the simple model using `neq` alone. Using additional patient information beyond relying on norepinephrine dose alone provides meaningful improvement in the prediction of vasopressin response.

Show a non-stylized plot (autoplot) of the AUROC comparison.

▼ Code

```
avp_m0_lr_preds |>
  group_by(wflow_id) |>
  roc_curve(response, .pred_Response) |>
  autoplot()
```



The ROC curves are separated across essentially the entire plot range, with the multivariable model showing consistently better predictive discrimination than the univariable model only including norepinephrine dose (with a spline term).

Potential interpretation for manuscript: “Although norepinephrine dose is commonly used to estimate the likelihood of vasopressin response, a model based on norepinephrine dose alone demonstrated substantially lower discrimination (AUROC 0.636) and worse overall predictive performance (Brier score 0.214) than the multivariable model (AUROC 0.700, Brier score 0.202).”

12.3 Epinephrine

This section will build and evaluate prediction models for response after initiation of epinephrine as the first adjunctive vasopressor. The same approaches as for vasopressin models will be used here with less annotation. For more annotation regarding the rationale for model building and evaluation steps see Section 12.2.

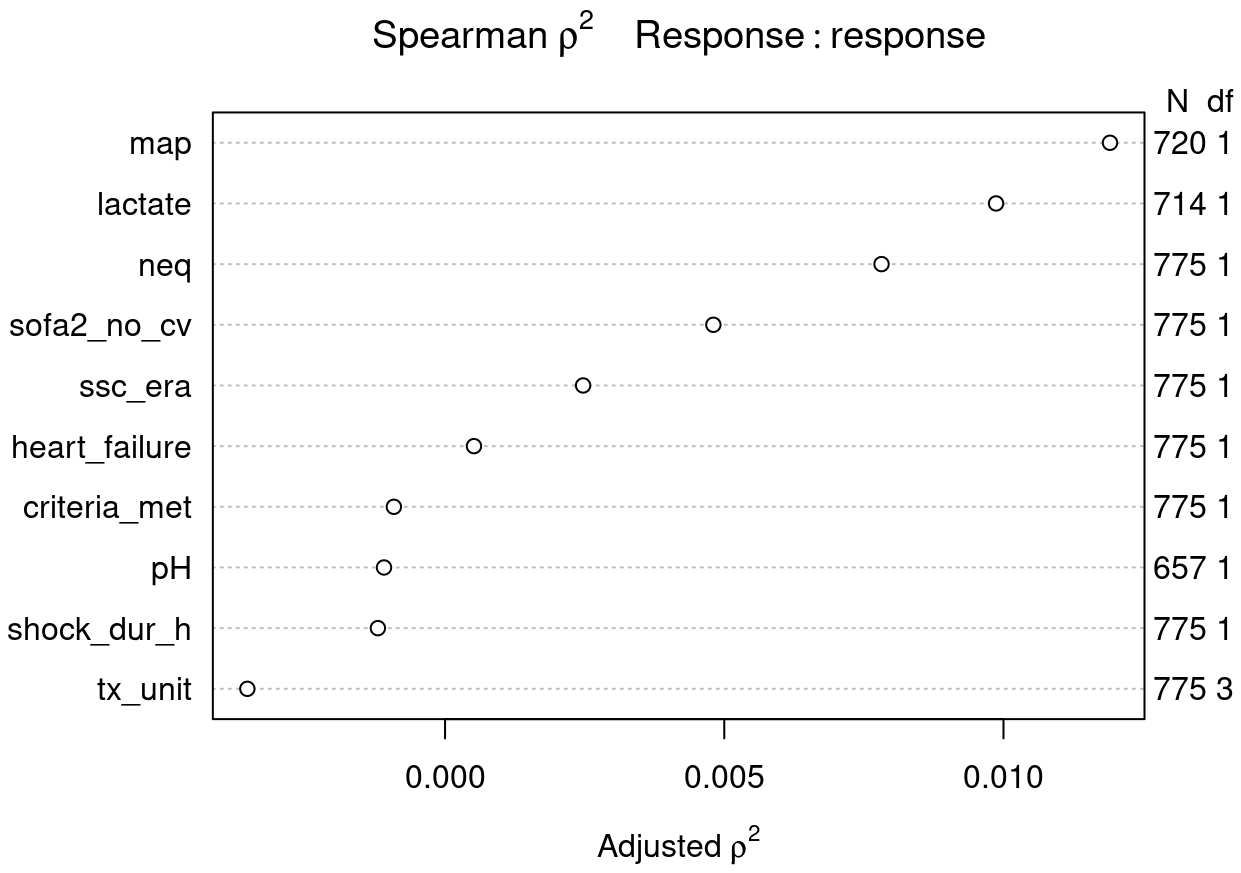
12.3.1 Logistic Regression

Here, use logistic regression to generate prediction models using two approaches: 1) main effects (additive) only, and 2) non-linear effects of predictors.

The two models will be built in one `workflow_set()`. First, generate a Spearman's ρ^2 rank correlation plot to assess for potential predictor transformation (non-linear effects).

▼ Code

```
plot(spearman2(response ~ ., data = epi |> select(-c(age, sex, race_eth))))
```



▼ Code

```
invisible(pdf("epi_spearman.pdf"))
plot(spearman2(response ~ ., data = epi |> select(-c(age, sex, race_eth))),
     main = ""
     )
title(main = expression("Epinephrine Spearman " * rho^2))
invisible(dev.off())
```

The Spearman's ρ^2 rank correlation plot shows that `map` has the highest potential to improve model fit with a non-linear transformation. Because this is a quantitative variable, introduce a natural spline with 3 knots (4 df) to spend all of the available degrees of freedom (12 df in the main effects model and 15 total available). Notably, all of the predictors have a relatively low ρ^2 value, therefore the non-linear model is not expected to substantially improve fit.

As noted in Section 11.5, there are missing data in the same three variables as for vasopressin. Therefore, the same imputation approach will be used.

▼ Code

```
# Main effects recipe
epi_m1_lr_rec_main <-
  recipe(response ~ ., data = epi) |>
  # Declare age, sex, and race_eth in a bystander role to keep them in the
  # resampling dataset for future use in fairness checks
  update_role(age, sex, race_eth, new_role = "bystander") |>
  # Single (linear regression) imputation for missing quantitatitve predictors
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate
                       )
    )

# Non-linear effects recipe
epi_m1_lr_rec_ns <-
  recipe(response ~ ., data = epi) |>
  update_role(age, sex, race_eth, new_role = "bystander") |>
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h,
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map,
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
```

```
                      sofa2_no_cv, neq, shock_dur_h, map, lactate,
                      )
    ) |>
  # Add natural spline with 3 knots (4 df) for map
  step_spline_natural(map, deg_free = 4)

# Set up pre-processing and model workflow_set()
epi_m1_lr_preproc <- list(
  epi_m1_main = epi_m1_lr_rec_main,
  epi_m1_ns = epi_m1_lr_rec_ns
  )

epi_m1_lr_models <-
  workflow_set(epi_m1_lr_preproc,
               list(logistic = logistic_reg(engine = "glm")),
               cross = FALSE)

# Resample each model in the set with 10-fold CV (5 repeats)
set.seed(2026)
train_cv <- vfold_cv(epi, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

epi_m1_lr_models_fit <-
  epi_m1_lr_models |>
  workflow_map("fit_resamples", seed = 2026, verbose = TRUE,
               resamples = train_cv, metrics = class_metrics,
               control = keep_pred
               )
```

```
i 1 of 2 resampling: epi_m1_main_logistic

✓ 1 of 2 resampling: epi_m1_main_logistic (18.1s)

i 2 of 2 resampling: epi_m1_ns_logistic

✓ 2 of 2 resampling: epi_m1_ns_logistic (19.2s)
```

▼ Code

```
# Collect predictions
epi_m1_lr_preds <-
  collect_predictions(epi_m1_lr_models_fit, summarize = FALSE)

# Collect then show the metrics
epi_m1_lr_mets <-
  collect_metrics(epi_m1_lr_models_fit) |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
```

```
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

epi_m1_lr_mets |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Epinephrine m1 (Logistic Regression) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

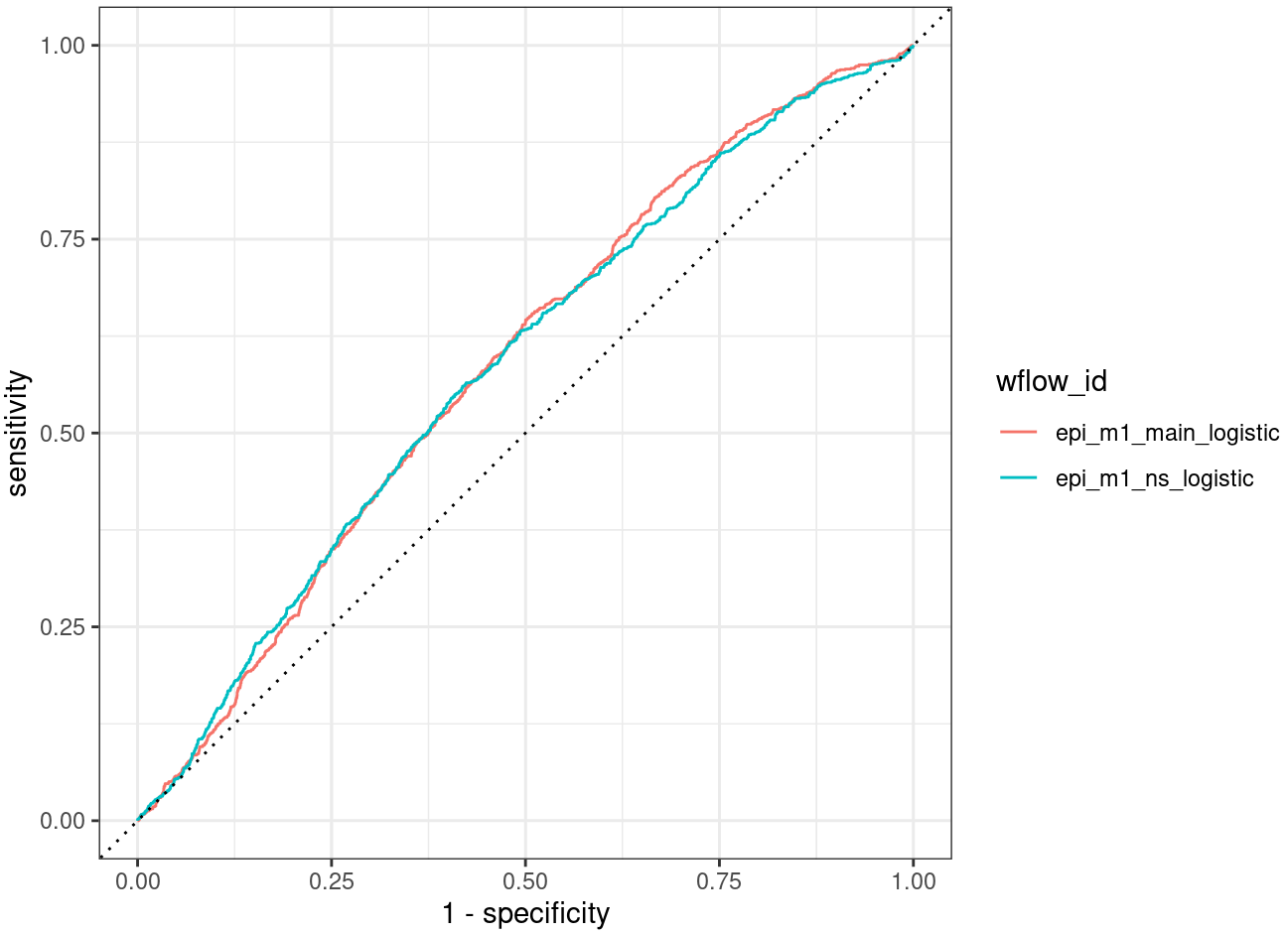
Epinephrine m1 (Logistic Regression) Metric Comparison		
Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
epi_m1_main_logistic	0.589 (0.571, 0.607)	0.203 (0.201, 0.206)
epi_m1_ns_logistic	0.589 (0.569, 0.609)	0.204 (0.201, 0.207)

The two logistic regression modeling approaches had essentially the same discrimination (AUROC) and overall performance (Brier score). The performance of both models was better than chance but not impressive.

▼ Code

```
epi_m1_lr_preds |>
  group_by(wflow_id) |>
```

```
  roc_curve(response, .pred_Response) |>
  autoplot()
```



The two curves have a lot of overlap. The non-linear (spline) model is slightly better at lower values and the main effects model is slightly better at higher values.

12.3.2 Machine Learning

12.3.2.1 Regularized (Elastic Net)

Fit regularized (elastic net) models.

▼ Code

```
epi_m2_rec_main <- epi_m1_lr_rec_main |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

epi_m2_rec_ns <- epi_m1_lr_rec_ns |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

epi_m2_glmnet_preproc <- list(
  epi_m2_main = epi_m2_rec_main,
```

```
  epi_m2_ns = epi_m2_rec_ns
  )

epi_m2_glmnet_models <-
  workflow_set(epi_m2_glmnet_preproc,
               list(glmnet = logistic_reg(
                 engine = "glmnet",
                 mode = "classification",
                 penalty = tune(),     # lambda
                 mixture = tune()      # alpha
                 )),
               cross = TRUE
               )

set.seed(2026)
train_cv <- vfold_cv(epi, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

pen_grid <- grid_space_filling(
  penalty(range = c(-5, 0)), # log10 scale, reasonable range for elastic net
  mixture(range = c(0, 1)),  # default
  size = 30
  )

epi_m2_glmnet_models_fit <-
  epi_m2_glmnet_models |>
  workflow_map("tune_grid", seed = 2026, verbose = TRUE,
               resamples = train_cv, grid = pen_grid,
               metrics = class_metrics, control = keep_pred
               )
```

```
i 1 of 2 tuning:     epi_m2_main_glmnet

✓ 1 of 2 tuning:     epi_m2_main_glmnet (1m 39.6s)

i 2 of 2 tuning:     epi_m2_ns_glmnet

✓ 2 of 2 tuning:     epi_m2_ns_glmnet (1m 45s)
```

▼ Code

```
workflow_ids <- epi_m2_glmnet_models_fit$wflow_id

# Loop over workflows to finalize, fit, and collect results
epi_m2_glmnet_final_fit <- map(workflow_ids, function(wf_id) {

  # Extract tuning results for this workflow
  wf_result <- extract_workflow_set_result(epi_m2_glmnet_models_fit, id = wf_id)
```

```r
  # Get best parameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")

  # Extract the workflow object itself
  wf <- extract_workflow(epi_m2_glmnet_models_fit, id = wf_id)

  # Finalize the workflow with best hyperparameters
  final_wf <- finalize_workflow(
    wf,
    best_params
  )

  # Fit the final workflow to the training set
  final_fit <- fit(final_wf, data = epi)

  # Collect resampling metrics for this workflow from tuning
  cv_metrics <- collect_metrics(wf_result) |>
    filter(penalty == best_params$penalty,
           mixture == best_params$mixture)

  # Collect predictions for the best parameters
  cv_preds <- collect_predictions(wf_result, summarize = TRUE) |>
    filter(penalty == best_params$penalty,
           mixture == best_params$mixture)

  # Return as a list
  list(
    workflow_id = wf_id,
    best_params = best_params,
    final_workflow = final_wf,
    final_fit = final_fit,
    cv_metrics = cv_metrics,
    cv_predictions = cv_preds
  )
})

# Name the list elements by workflow
names(epi_m2_glmnet_final_fit) <- workflow_ids
rm(workflow_ids)
```

Show elastic net hyperparameters resulting in the best fit.

▼ Code

```r
imap_dfr(epi_m2_glmnet_final_fit,
         ~ .x$best_params |> mutate(wflow_id = .y)
         ) |>
  select(wflow_id, penalty, mixture) |>
  gt() |>
  tab_header(
```

```
    title = "Epinephrine m2 Elastic Net Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    wflow_id = "Model",
    penalty = "Penalty (lambda [λ])",
    mixture = "Mixture (alpha [α])"
    ) |>
  fmt_number(decimals = 4) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(penalty, mixture)
    )
```

Epinephrine m2 Elastic Net Best Hyperparameters		
Using 10-fold Cross-Validation (5 repeats)		
Model	Penalty (lambda [λ])	Mixture (alpha [α])
epi_m2_main_glmnet	0.0085	0.8966
epi_m2_ns_glmnet	0.0189	0.7241

Collect metrics and display table (comparing to the logistic regression model fits).

▼ Code

```
epi_m2_glmnet_mets <-
 epi_m2_glmnet_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )
```

```
bind_rows(epi_m1_lr_mets, epi_m2_glmnet_mets)  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Epinephrine m2 (Elastic Net) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Epinephrine m2 (Elastic Net) Metric Comparison

Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).

Model	AUROC	Brier Score
epi_m1_main_logistic	0.589 (0.571, 0.607)	0.203 (0.201, 0.206)
epi_m1_ns_logistic	0.589 (0.569, 0.609)	0.204 (0.201, 0.207)
epi_m2_main_glmnet	0.595 (0.578, 0.611)	0.201 (0.199, 0.203)
epi_m2_ns_glmnet	0.597 (0.579, 0.615)	0.201 (0.199, 0.202)

The regularized (elastic net) models performed slightly better than the logistic regression models.

12.3.2.2 Random Forest

Fit a random forest model.

▼ Code

```
epi_m3_rf_rec <-
  recipe(response ~ ., data = epi) |>
  # Declare age, sex, and race_eth in a bystander role to keep them in the
  # resampling dataset for future use in fairness checks
  update_role(age, sex, race_eth, new_role = "bystander") |>
  # Single (linear regression) imputation for missing quantitative predictors
```

```
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate
                       )
    )

epi_m3_rf_preproc <- list(epi_m3_rf = epi_m3_rf_rec)

epi_m3_rf_models <-
  workflow_set(epi_m3_rf_preproc,
               list(ranger = rand_forest(
                 engine = "ranger",
                 mode = "classification",
                 trees = 1000,      # Default is 500, 1000 is better
                 mtry = tune(),     # Number of randomly selected predictors
                 min_n = tune()     # Minimum node size
                 )),
               cross = TRUE
               )

set.seed(2026)
train_cv <- vfold_cv(epi, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

# Prep recipe and determine predictor count
epi_m3_rf_rec_prep <- prep(epi_m3_rf_rec)

x_train <- bake(epi_m3_rf_rec_prep, new_data = NULL) |>
  select(-c(response, age, sex, race_eth))

p <- ncol(x_train) # p == 10

rf_param_set <- parameters(
  mtry(range = c(1L, 10L)), #sqrt(p) ~= 3, (p / 2) == 5, which is too narrow
  min_n(range = c(2L, 50L))
)
```

```r
initial_candidates <- grid_space_filling(
  rf_param_set,
  size = 30
)

race_ctrl <- control_race(
  save_pred = TRUE,
  save_workflow = TRUE,
  parallel_over = "resamples",
  verbose = FALSE
)

epi_m3_rf_model_fit <-
  epi_m3_rf_models |>
  workflow_map(
    "tune_race_anova", seed = 2026, resamples = train_cv,
    grid = initial_candidates, metrics = class_metrics, control = race_ctrl
  )

# Get workflow IDs
workflow_ids <- epi_m3_rf_model_fit$wflow_id

# Finalize, fit, and collect results for the workflow
epi_m3_rf_final_fit <- map(workflow_ids, function(wf_id) {
  # Extract tuning (race) results for this workflow
  wf_result <- extract_workflow_set_result(
    epi_m3_rf_model_fit,
    id = wf_id
    )
  # Select best hyperparameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")
  # Extract the workflow itself
  wf <- extract_workflow(epi_m3_rf_model_fit, id = wf_id)
  # Finalize workflow with best hyperparameters
  final_wf <- finalize_workflow(wf, best_params)
  # Fit finalized workflow to derivation data
  final_fit <- fit(final_wf, data = epi)
  # Collect CV metrics for the selected configuration
  cv_metrics <- collect_metrics(wf_result) |>
    filter(.config == best_params$.config)
  # Collect CV predictions for the selected configuration
  cv_predictions <- collect_predictions(wf_result) |>
    filter(.config == best_params$.config)
  # Return results
  list(
    workflow_id     = wf_id,
    best_params     = best_params,
    final_workflow  = final_wf,
    final_fit       = final_fit,
    cv_metrics      = cv_metrics,
    cv_predictions  = cv_predictions
```

```
    )
})

names(epi_m3_rf_final_fit) <- map_chr(epi_m3_rf_final_fit, "workflow_id")
```

Show hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(epi_m3_rf_final_fit,
         ~ .x$best_params |> mutate(workflow_id = .y)
         ) |>
  select(workflow_id, mtry, min_n) |>
  gt() |>
  tab_header(
    title = "Random Forest Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    workflow_id = "Model",
    mtry = "Random Variables at Split (mtry)",
    min_n = "Min Terminal Node Size (min_n)"
    ) |>
  fmt_number(decimals = 0) |>
  cols_align(
    align = "left",
    columns = c(workflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mtry, min_n)
    )
```

Random Forest Best Hyperparameters

Using 10-fold Cross-Validation (5 repeats)

Model	Random Variables at Split (mtry)	Min Terminal Node Size (min_n)
epi_m3_rf_ranger	2	48

Collect and display metrics for model.

▼ Code

```
epi_m3_rf_mets <-
 epi_m3_rf_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
```

```
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(epi_m1_lr_mets,
          epi_m2_glmnet_mets,
          epi_m3_rf_mets)  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Epinephrine m3 (Random Forest) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Epinephrine m3 (Random Forest) Metric Comparison

Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).

Model	AUROC	Brier Score
epi_m1_main_logistic	0.589 (0.571, 0.607)	0.203 (0.201, 0.206)
epi_m1_ns_logistic	0.589 (0.569, 0.609)	0.204 (0.201, 0.207)
epi_m2_main_glmnet	0.595 (0.578, 0.611)	0.201 (0.199, 0.203)
epi_m2_ns_glmnet	0.597 (0.579, 0.615)	0.201 (0.199, 0.202)

Epinephrine m3 (Random Forest) Metric Comparison Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
epi_m3_rf_ranger	0.590 (0.574, 0.605)	0.201 (0.199, 0.203)

The random forest model performed similarly to the logistic regression models and slightly worse than the elastic net model.

12.3.2.3 Boosted Trees

Generate a boosted tree model.

▼ Code

```
epi_m4_xgb_rec <-
  recipe(response ~ ., data = epi) |>
  update_role(age, sex, race_eth, new_role = "bystander") |>
  step_impute_linear(
    map,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h
    )
  ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h, map
    )
  ) |>
  step_impute_linear(
    pH,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h, map, lactate
    )
  ) |>
  # Dummy code nominal predictors
  step_dummy(all_nominal_predictors())

epi_m4_xgb_preproc <- list(epi_m4_boost = epi_m4_xgb_rec)

epi_m4_xgb_model <-
```

```
  workflow_set(epi_m4_xgb_preproc,
               list(xgboost = boost_tree(
                 trees = 1000,              # Number of trees
                 tree_depth = tune(),       # Tree depth: # of "interactions"
                 min_n = tune(),            # Minimum node size
                 loss_reduction = tune(),   # Reduction in the loss function required to split
         further
                 sample_size = tune(),      # Proportion of data exposed to fitting routine
                 mtry = tune(),             # Random predictors at each split
                 learn_rate = tune()        # Rate at which the boosting algorithm adapts
                 ) |>
                   set_engine("xgboost") |>
                   set_mode("classification")
                 ),
               cross = FALSE
               )

set.seed(2026)
train_cv <- vfold_cv(epi, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

# Prep recipe and determine post-dummy predictor count
epi_m4_xgb_rec_prep <- prep(epi_m4_xgb_rec)

x_train <- bake(epi_m4_xgb_rec_prep, new_data = NULL) |>
  select(-c(response, age, sex, race_eth))

p <- ncol(x_train) # p == 10

# Select parameters (particularly depth) based p==10 and RF optimal parameters
xgb_params <- parameters(
  tree_depth(range = c(1L, 6L)),
  min_n(range = c(2L, 50L)),
  learn_rate(range = c(-3, -0.5)),
  loss_reduction(),
  sample_prop(range = c(0.5, 1.0)),
  finalize(mtry(), x_train)
  )

initial_candidates <- grid_space_filling(
  xgb_params,
  size = 30
  )

race_ctrl <- control_race(
  save_pred = TRUE,
  save_workflow = TRUE,
  parallel_over = "resamples",
  verbose = FALSE
  )
```

```r
epi_m4_xgb_model_fit <-
  epi_m4_xgb_model |>
  workflow_map(
    "tune_race_anova",
    seed = 2026,
    resamples = train_cv,
    grid = initial_candidates,
    metrics = class_metrics,
    control = race_ctrl
    )

workflow_ids <- epi_m4_xgb_model_fit$wflow_id

# Finalize, fit, and collect results the workflow
epi_m4_xgb_final_fit <- map(workflow_ids, function(wf_id) {
  # Extract tuning (race) results for this workflow
  wf_result <- extract_workflow_set_result(
    epi_m4_xgb_model_fit,
    id = wf_id
    )
  # Select best hyperparameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")
  # Extract the workflow itself
  wf <- extract_workflow(epi_m4_xgb_model_fit, id = wf_id)
  # Finalize workflow with best hyperparameters
  final_wf <- finalize_workflow(wf, best_params)
  # Fit finalized workflow to derivation data
  final_fit <- fit(final_wf, data = epi)
  # Collect CV metrics for the selected configuration
  cv_metrics <- collect_metrics(wf_result) |>
    filter(.config == best_params$.config)
  # Collect CV predictions for the selected configuration
  cv_predictions <- collect_predictions(wf_result) |>
    filter(.config == best_params$.config)
  # Return results
  list(
    workflow_id     = wf_id,
    best_params     = best_params,
    final_workflow  = final_wf,
    final_fit       = final_fit,
    cv_metrics      = cv_metrics,
    cv_predictions  = cv_predictions
    )
})

names(epi_m4_xgb_final_fit) <- map_chr(epi_m4_xgb_final_fit, "workflow_id")
```

Show hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(epi_m4_xgb_final_fit,
         ~ .x$best_params |> mutate(wflow_id = .y)
         ) |>
  mutate(trees = 1000) |>
  select(wflow_id, trees, tree_depth, min_n, learn_rate, loss_reduction,
         sample_size, mtry
         ) |>
  gt() |>
  fmt_number(columns = c(learn_rate, loss_reduction, sample_size),
             decimals = 3
             ) |>
  tab_header(
    title = "Gradient Boosted Model Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    wflow_id = "Model",
    trees = "Trees",
    tree_depth = "Tree Depth",
    min_n = "Node Size",
    learn_rate = "Learn Rate",
    loss_reduction = "Min Loss Reduction",
    sample_size = "Proportion of data exposed",
    mtry = "Predictors at Splits",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(trees, tree_depth, min_n, learn_rate, loss_reduction,
                sample_size, mtry
                )
    )
```

Gradient Boosted Model Best Hyperparameters

Using 10-fold Cross-Validation (5 repeats)

Model	Trees	Tree Depth	Node Size	Learn Rate	Min Loss Reduction	Proportion of data exposed	Predictors at Splits
epi_m4_boost_xgboost	1000	2	13	0.006	0.000	0.638	3

Optimal parameters are not at tuning boundaries. The algorithm preferred shallow trees with iterative small improvements (over aggressive boosting), which is consistent with the random forest findings.

Collect and display metrics for model.

▼ Code

```
epi_m4_xgb_mets <-
 epi_m4_xgb_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(epi_m1_lr_mets,
          epi_m2_glmnet_mets,
          epi_m3_rf_mets,
          epi_m4_xgb_mets
          )  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Epinephrine m4 (XGBoost) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Epinephrine m4 (XGBoost) Metric Comparison Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
epi_m1_main_logistic	0.589 (0.571, 0.607)	0.203 (0.201, 0.206)
epi_m1_ns_logistic	0.589 (0.569, 0.609)	0.204 (0.201, 0.207)
epi_m2_main_glmnet	0.595 (0.578, 0.611)	0.201 (0.199, 0.203)
epi_m2_ns_glmnet	0.597 (0.579, 0.615)	0.201 (0.199, 0.202)
epi_m3_rf_ranger	0.590 (0.574, 0.605)	0.201 (0.199, 0.203)
epi_m4_boost_xgboost	0.619 (0.605, 0.633)	0.198 (0.196, 0.200)

The XGBoost performed better than each of the previous models.

12.3.3 Discrimination/Accuracy Comparison

Now, show the discrimination and accuracy metrics for all models.

▼ Code

```
(epi_disc_perf_mets <- bind_rows(epi_m1_lr_mets,
          epi_m2_glmnet_mets,
          epi_m3_rf_mets,
          epi_m4_xgb_mets
          )  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Epinephrine Models Discrimination/Overall Performance Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
```

```
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    ))
```

Epinephrine Models Discrimination/Overall Performance Metric Comparison Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
epi_m1_main_logistic	0.589 (0.571, 0.607)	0.203 (0.201, 0.206)
epi_m1_ns_logistic	0.589 (0.569, 0.609)	0.204 (0.201, 0.207)
epi_m2_main_glmnet	0.595 (0.578, 0.611)	0.201 (0.199, 0.203)
epi_m2_ns_glmnet	0.597 (0.579, 0.615)	0.201 (0.199, 0.202)
epi_m3_rf_ranger	0.590 (0.574, 0.605)	0.201 (0.199, 0.203)
epi_m4_boost_xgboost	0.619 (0.605, 0.633)	0.198 (0.196, 0.200)

The XGBoost model (`epi_m4_boost_xgboost`) had the best performance and the logistic regression models had the worst performance. Will carry forward the XGBoost model for calibration assessment prior to final model selection.

Export comparison table to Word (commented out for rendering)

▼ Code

```
#gtsave(
#  data = epi_disc_perf_mets,
#  filename = "epi_disc_perf_metrics.docx"
#)
```

12.3.4 Calibration

Use the same calibration approach as in Section 12.2.4 that was used for vasopressin.

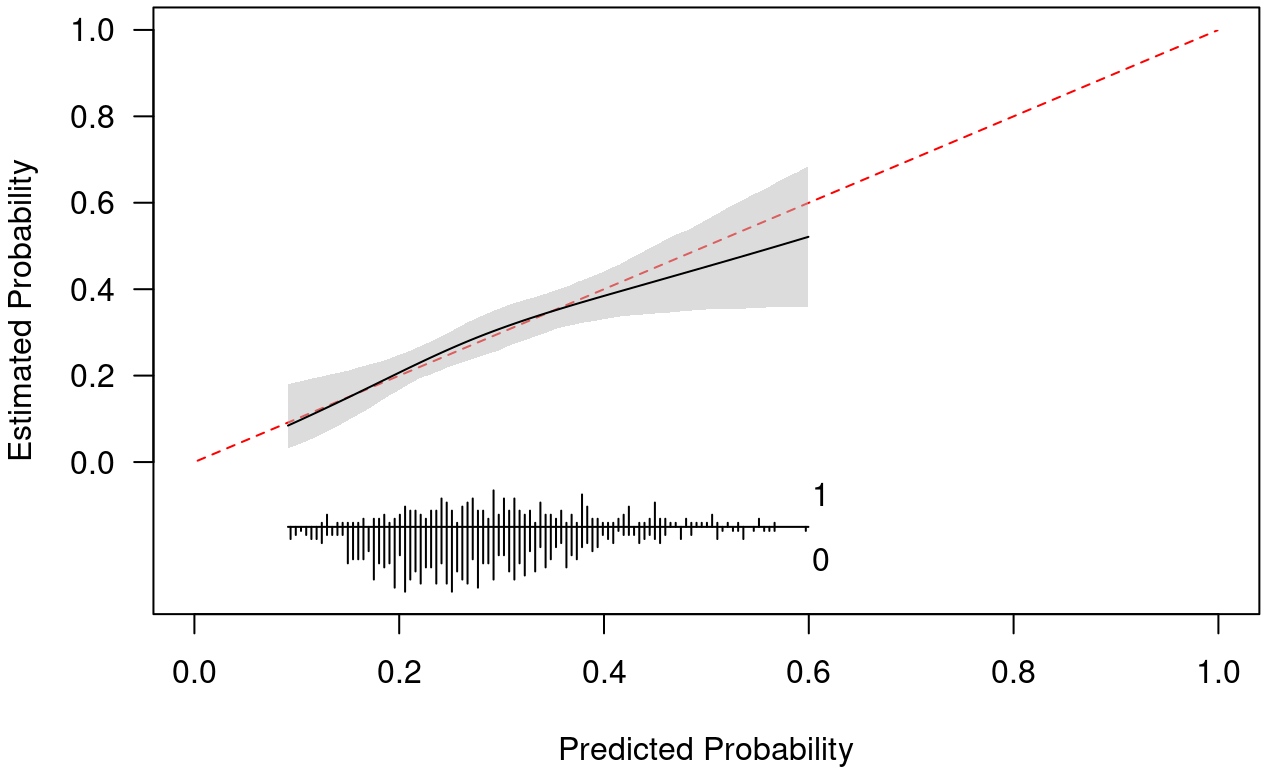
▼ Code

```
epi_m4_xgb_preds <- epi_m4_xgb_final_fit[[1]]$cv_predictions

epi_m4_xgb_preds_cv <- epi_m4_xgb_preds |>
  group_by(.row) |>
  summarise(
    response = first(response),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
  ) |>
```

```
  mutate(
    response_num = if_else(response == "Response", 1, 0)
  )

set.seed(2026)
epi_xgb_cal <- pmcalibration(
  y = epi_m4_xgb_preds_cv$response_num,
  p = epi_m4_xgb_preds_cv$.pred_Response,
  ci = "boot", # n = 1000 replicates is the default
  smooth = "rcs",
  nk = 3 # 5 knots is default, 3 has less bias
  )
```



▼ Code

```
invisible(pdf("epi_xgboost_calibration.pdf"))
par(mfrow = c(1, 1),
    cex.axis = 1.4, cex.lab = 1.8, cex.main = 2,
    mar = c(5.1, 5.5, 1.4, 1.6), # bottom, left, top, right
    mgp = c(3.5, 1, 0)
    )
plot(epi_xgb_cal)
invisible(dev.off())
```

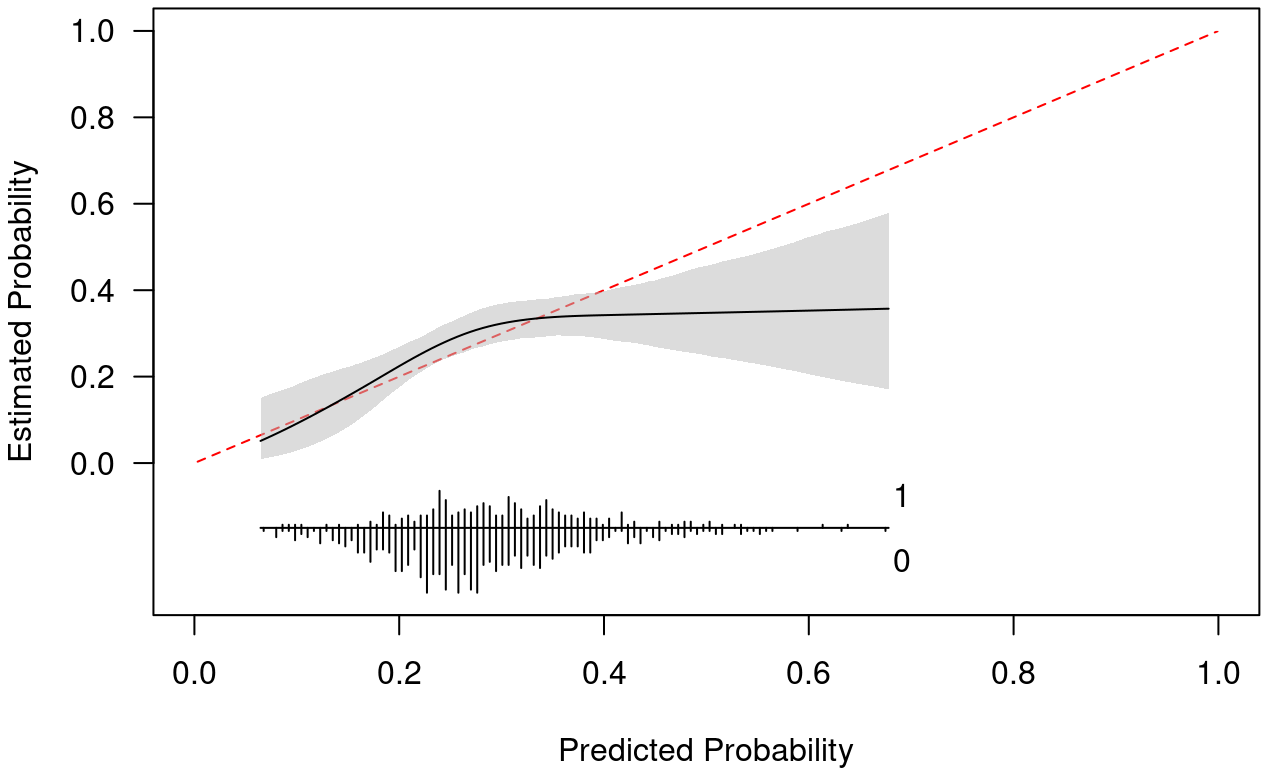
The calibration of the epinephrine XGBoost model is good, with some over-prediction at higher probabilities (above ~0.4) where data are sparse.

Since the logistic regression models are generally preferred to machine learning models for this modeling goal (as noted in Section 12.2.4) and the discrimination between models was not substantially different (AUROC for logistic regression 0.59 and for XGBoost 0.62), assess calibration for the main effects logistic regression model for comparison.

▼ Code

```
epi_m1_lr_preds_cv <- epi_m1_lr_preds |>
  filter(wflow_id == "epi_m1_main_logistic") |>
  group_by(.row) |>
  summarise(
    response = first(response),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
  ) |>
  mutate(
    response_num = if_else(response == "Response", 1, 0)
  )

set.seed(2026)
epi_lr_cal <- pmcalibration(
  y = epi_m1_lr_preds_cv$response_num,
  p = epi_m1_lr_preds_cv$.pred_Response,
  ci = "boot", # n = 1000 replicates is the default
  smooth = "rcs",
  nk = 3 # 5 knots is default, 3 has less bias
  )
plot(epi_lr_cal)
title("Calibration Plot: Epinephrine Logistic Regression")
```



▼ Code

```
par(mfrow = c(1, 1))
invisible(dev.copy(pdf, "epi_logistic_calibration.pdf"))
invisible(dev.off())
```

Calibration with the main effects logistic regression model appears worse than the XGBoost model, but it is challenging to assess the upper range because of few observations.

Save the two epinephrine calibration plots as a panel figure.

▼ Code

```
invisible(ragg::agg_tiff("epi_calibration_panel.tiff",
                         width = 11, height = 5, units = "in",
                         res = 600, compression = "lzw"
                         ))

par(mfrow = c(1, 2), pty = "s")

plot(epi_lr_cal)
title("Epinephrine Logistic Regression (Main)")

plot(epi_xgb_cal)
```

```
title("Epinephrine XGBoost")

par(mfrow = c(1, 1))
invisible(dev.off())
```

Show the quantitative calibration metrics for comparison.

▼ Code

```
(epi_cal_metrics_tbl <-
   bind_rows(
    as_tibble(epi_lr_cal$metrics, rownames = "metric") |>
      mutate(model = "Logistic (main)"),
    as_tibble(epi_xgb_cal$metrics, rownames = "metric") |>
      mutate(model = "XGBoost")
  ) |>
  select(model, metric, value) |>
  pivot_wider(
    names_from = metric,
    values_from = value
  ) |>
  rename(
    Model = model,
    `Eavg (ICI)` = Eavg
  ) |>
  gt() |>
  tab_header(
    title = "Epinephrine Models Calibration Metrics"
  ) |>
  fmt_number(
    columns = c(`Eavg (ICI)`, E50, E90, Emax, ECI),
    decimals = 4
  ) |>
  cols_align(
    align = "center"
  ))
```

Epinephrine Models Calibration Metrics					
Model	Eavg (ICI)	E50	E90	Emax	ECI
Logistic (main)	0.0352	0.0300	0.0626	0.3208	0.2720
XGBoost	0.0114	0.0095	0.0221	0.0785	0.0257

Calibration is better for the epinephrine XGBoost model.

Export comparison table to Word (commented out for rendering)

▼ Code

```
#gtsave(
#  data = epi_cal_metrics_tbl,
#  filename = "epi_cal_metric_comparison.docx"
#)
```

12.3.5 Decision Curve Analysis

Extract predictions from the XGBoost and logistic regression models, then combine.

▼ Code

```
# Extract the out-of-fold predictions for each model
# Then average predictions across the 5 CV repeats using .row
# Logistic model
epi_lr_dca <- epi_m1_lr_preds |>
  filter(wflow_id == "epi_m1_main_logistic") |>
  group_by(.row) |>
  summarise(
    response = first(response),
    p_logistic = mean(.pred_Response),
    .groups = "drop"
  )

# XGBoost model
epi_xgb_dca <- epi_m4_xgb_final_fit[[1]]$cv_predictions |>
  group_by(.row) |>
  summarise(
    p_xgb = mean(.pred_Response),
    .groups = "drop"
  )

# Combine
epi_dca_data <- epi_lr_dca |>
  left_join(epi_xgb_dca, by = ".row")
```

Then run DCA and plot.

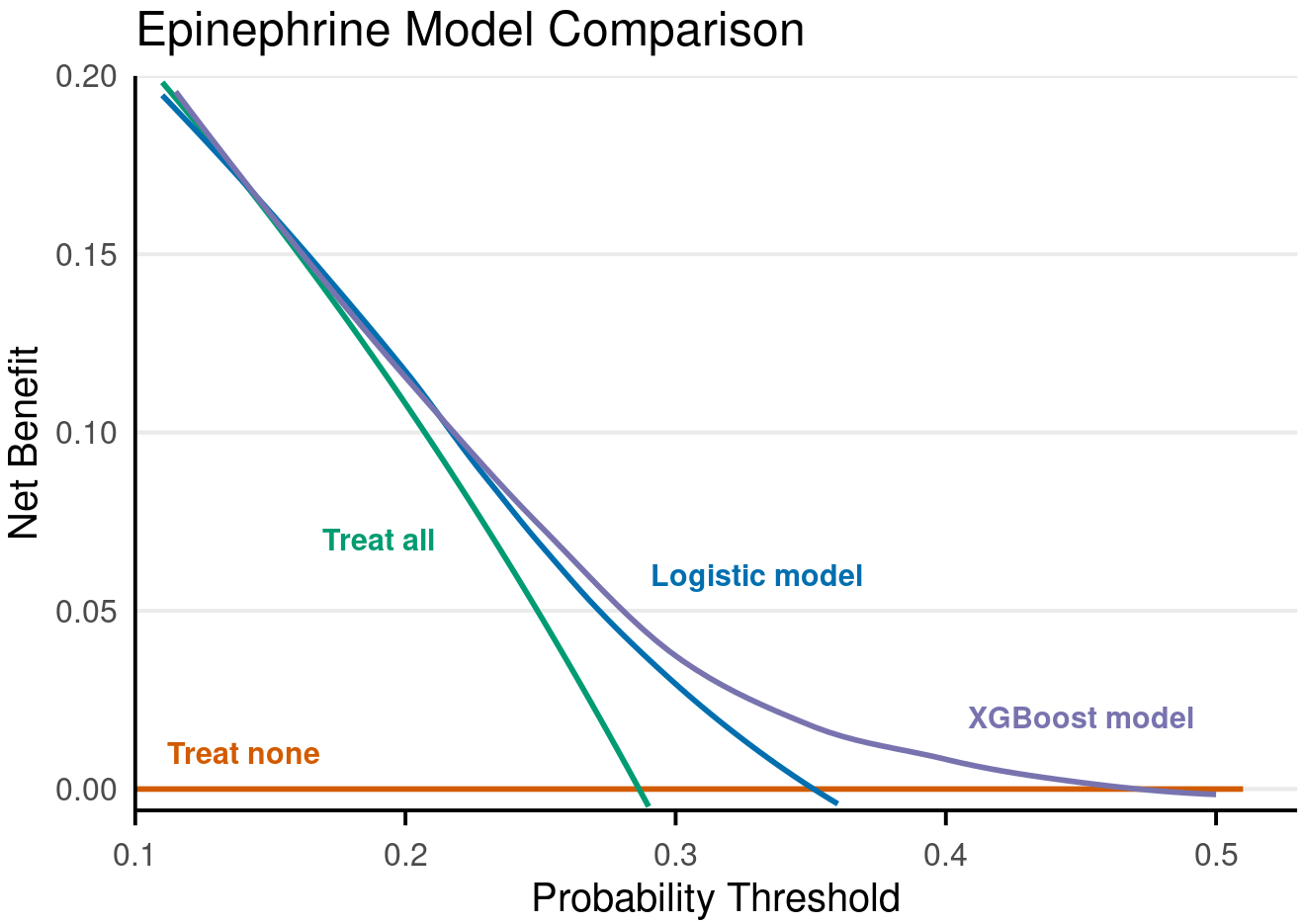
▼ Code

```
epi_dca <- dca(
  response == "Response" ~ p_logistic + p_xgb,
  data = epi_dca_data
  )

(epi_dca_plot <-
    ggplot(as_tibble(epi_dca) |>
             filter(net_benefit >= -0.006 &
```

```
                  net_benefit <= 0.2 &
                  threshold <=0.51
                ) |>
         mutate(label = fct_relevel(label, "Treat None", "Treat All",
                                    "p_logistic", "p_xgb"
                                    )),
       aes(x = threshold,
           y = net_benefit,
           color = label,
           group = label)) +
geom_smooth(method = loess, formula = y ~ x, se = FALSE, linewidth = 1) +
scale_color_manual(values = c(
  "p_logistic" = "#0072B2",
  "p_xgb" = "#7876B1FF",
  "Treat All" = "#009E73",
  "Treat None" = "#D55E00"
  ),
  labels = c(
    "p_logistic" = "Logistic regression",
    "p_xgb" = "XGBoost",
    "Treat All" = "Treat all",
    "Treat None" = "Treat none"
  )) +
scale_y_continuous(
  limits = c(-0.006, 0.2),
  expand = expansion(add = c(0, 0))
  ) +
scale_x_continuous(
  limits = c(0.1, 0.51),
  breaks = seq(0,0.5, by = 0.1),
  expand = expansion(add = c(0, 0.02))
  ) +
labs(
  title = "Epinephrine Model Comparison",
  x = "Probability Threshold",
  y = "Net Benefit",
  color = NULL
  ) +
annotate("text", x = 0.33, y = 0.06, label = "Logistic model",
         color = "#0072B2", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.45, y = 0.02, label = "XGBoost model",
         color = "#7876B1FF", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.19, y = 0.07, label = "Treat all",
         color = "#009E73", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.14, y = 0.01, label = "Treat none",
```

```
          color = "#D55E00", family = "sans", fontface = "bold",
          size = 4
          ) +
theme_minimal(base_size = 15) +
theme(
  legend.position = "none",
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x = element_line(),
  axis.line.y = element_line(),
  axis.ticks.x = element_line(),
  axis.ticks.length = unit(0.2, "cm")
))
```



Both models are better than “Treat none” and “Treat all” in the probability threshold range from about 0.2 to 0.35, the XGBoost model has higher net benefit than the logistic regression model. The XGBoost model has higher net benefit than the “Treat none” model from probability thresholds about 0.2 to about 0.45.

Obtain exact threshold values for positive net benefit with logistic regression and XGBoost models.

▼ Code

```r
as_tibble(epi_dca) |>
  select(threshold, label, net_benefit) |>
  pivot_wider(names_from = label, values_from = net_benefit) |>
  filter(!is.na(p_logistic), !is.na(p_xgb),
         !is.na(`Treat All`), !is.na(`Treat None`)
         ) |>
  summarise(
    min_prob_thresh_lr = min(threshold[p_logistic > `Treat All` + 0.001 &
                                       p_logistic > `Treat None` + 0.001]
                             ),
    max_prob_thresh_lr = max(threshold[p_logistic > `Treat All` + 0.001 &
                                       p_logistic > `Treat None` + 0.001]
                             ),
    min_prob_thresh_xgb = min(threshold[p_xgb > `Treat All` + 0.001 &
                                       p_xgb > `Treat None` + 0.001]
                             ),
    max_prob_thresh_xgb = max(threshold[p_xgb > `Treat All` + 0.001 &
                                       p_xgb > `Treat None` + 0.001]
                             )
  )
```

```
# A tibble: 1 × 4
  min_prob_thresh_lr max_prob_thresh_lr min_prob_thresh_xgb
               <dbl>              <dbl>               <dbl>
1               0.16               0.34                0.11
# ℹ 1 more variable: max_prob_thresh_xgb <dbl>
```

The epinephrine response logistic regression model had higher net benefit than the “Treat all” and “Treat none” curves at threshold probabilities from 0.16 to 0.34. The epinephrine response XGBoost model had higher net benefit than the “Treat all” and “Treat none” curves at threshold probabilities from 0.11 to 0.46 (nearly double the number of threshold probability values compared with the logistic regression model).

Save DCA plot

▼ Code

```r
ggsave(
  "epi_dca_compare.tiff",
  plot = epi_dca_plot,
  device = ragg::agg_tiff,
  width = dev.size("in")[1],
  height = dev.size("in")[2],
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

12.3.6 Model Selection

Among the models assessed, the boosted trees (XGBoost) machine learning model for epinephrine response demonstrated the best discrimination, overall performance, and calibration. It also had a higher net benefit in decision curve analysis than the logistic regression model. In light of these findings, the XBGoost model (`epi_m4_boost_xgboost`) was selected as the final model for epinephrine.

12.3.6.1 Refit to Whole Derivation Dataset

Refit the final model (`epi_m4_boost_xgboost`) in the whole epinephrine derivation dataset (`epi`).

▼ Code

```
set.seed(2026)
epi_mod_final_wf <-
  epi_m4_xgb_final_fit[[1]]$final_workflow

epi_mod_final_fit_dev <-
  fit(epi_mod_final_wf, data = epi)
```

Estimate apparent performance on derivation data.

▼ Code

```
# Generate predictions
epi_dev_preds <-
  predict(
    epi_mod_final_fit_dev,
    new_data = epi,
    type = "prob"
    ) |>
  bind_cols(
    epi |> select(response)
    )

epi_dev_score_data <-
  epi_dev_preds |>
  transmute(
    response_num = if_else(response == "Response", 1, 0),
    p_xgb = .pred_Response
  )

# Generate metrics
epi_dev_score <-
  Score(
    object = list(
      "XGBoost" = epi_dev_score_data$p_xgb
    ),
    formula = response_num ~ 1,
    data = epi_dev_score_data,
```

```
    metrics = c("auc", "brier")
  )

# Extract and format results
auc_ci <-
  epi_dev_score$AUC$score |>
  as_tibble() |>
  transmute(
    auc = as.numeric(AUC),
    auc_lower = as.numeric(lower),
    auc_upper = as.numeric(upper)
  )

brier_ci <-
  epi_dev_score$Brier$score |>
  as_tibble() |>
  filter(model == "XGBoost") |>
  transmute(
    brier = as.numeric(Brier),
    brier_lower = as.numeric(lower),
    brier_upper = as.numeric(upper)
  )

epi_dev_mets <- cbind(auc_ci, brier_ci) |>
  mutate(
    AUROC = sprintf("%.3f (%.3f, %.3f)", auc, auc_lower, auc_upper),
    Brier = sprintf("%.3f (%.3f, %.3f)", brier, brier_lower, brier_upper)
  ) |>
  select(AUROC, Brier)
rm(auc_ci, brier_ci)

epi_dev_mets |>
  gt() |>
  tab_header(title = "Epinephrine XGBoost Model Fit and Apparent Performance Parameters",
             subtitle = "In Derivation (Ohio) Dataset") |>
  cols_align(align = "center")
```

Epinephrine XGBoost Model Fit and Apparent Performance Parameters	
In Derivation (Ohio) Dataset	
AUROC	Brier
0.734 (0.695, 0.772)	0.181 (0.168, 0.194)

Notably, the apparent performance metrics for the epinephrine XGBoost model are very optimistic and substantially better than the cross-validated values (AUROC apparent 0.73 vs. CV 0.62, Brier apparent 0.18 vs. CV 0.20).

12.3.6.2 Compare to Simple Model

Compare the selected multivariable model (`epi_m4_boost_xgboost`) with a simple logistic regression model that only includes `neq` with a spline term.

▼ Code

```
epi_m0_lr_rec_ns <-
  recipe(response ~ neq, data = epi) |>
  step_spline_natural(neq, deg_free = 5)

epi_m0_lr_preproc <- list(epi_m0_neq_only = epi_m0_lr_rec_ns)

epi_m0_lr_model <-
  workflow_set(epi_m0_lr_preproc,
               list(logistic = logistic_reg(engine = "glm")),
               cross = FALSE
               )

set.seed(2026)
train_cv <- vfold_cv(epi, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

epi_m0_lr_model_fit <-
  epi_m0_lr_model |>
  workflow_map("fit_resamples", seed = 2026, verbose = TRUE,
               resamples = train_cv, metrics = class_metrics,
               control = keep_pred
               )
```

```
i 1 of 1 resampling: epi_m0_neq_only_logistic

✓ 1 of 1 resampling: epi_m0_neq_only_logistic (10.9s)
```

▼ Code

```
epi_m0_lr_preds <-
  collect_predictions(epi_m0_lr_model_fit, summarize = FALSE)

epi_m0_lr_mets <-
  collect_metrics(epi_m0_lr_model_fit) |>
  # Compute 95% Wald CIs across the 50 resamples
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
```

```
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(epi_m0_lr_mets, epi_m4_xgb_mets) |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Epinephrine Final Model Performance vs. NE Alone",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Epinephrine Final Model Performance vs. NE Alone

Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).

Model	AUROC	Brier Score
epi_m0_neq_only_logistic	0.546 (0.527, 0.565)	0.204 (0.202, 0.205)
epi_m4_boost_xgboost	0.619 (0.605, 0.633)	0.198 (0.196, 0.200)

The multivariable model has better metrics than the simple model using `neq` alone. Using additional patient information beyond relying on norepinephrine dose alone provides meaningful improvement in the prediction of epinephrine response.

12.4 Phenylephrine

This section will build and evaluate prediction models for response after initiation of phenylephrine as the first adjunctive vasopressor. The same approaches as for vasopressin models will be used here with less annotation. For more annotation regarding the rationale for model building and evaluation steps see Section 12.2.

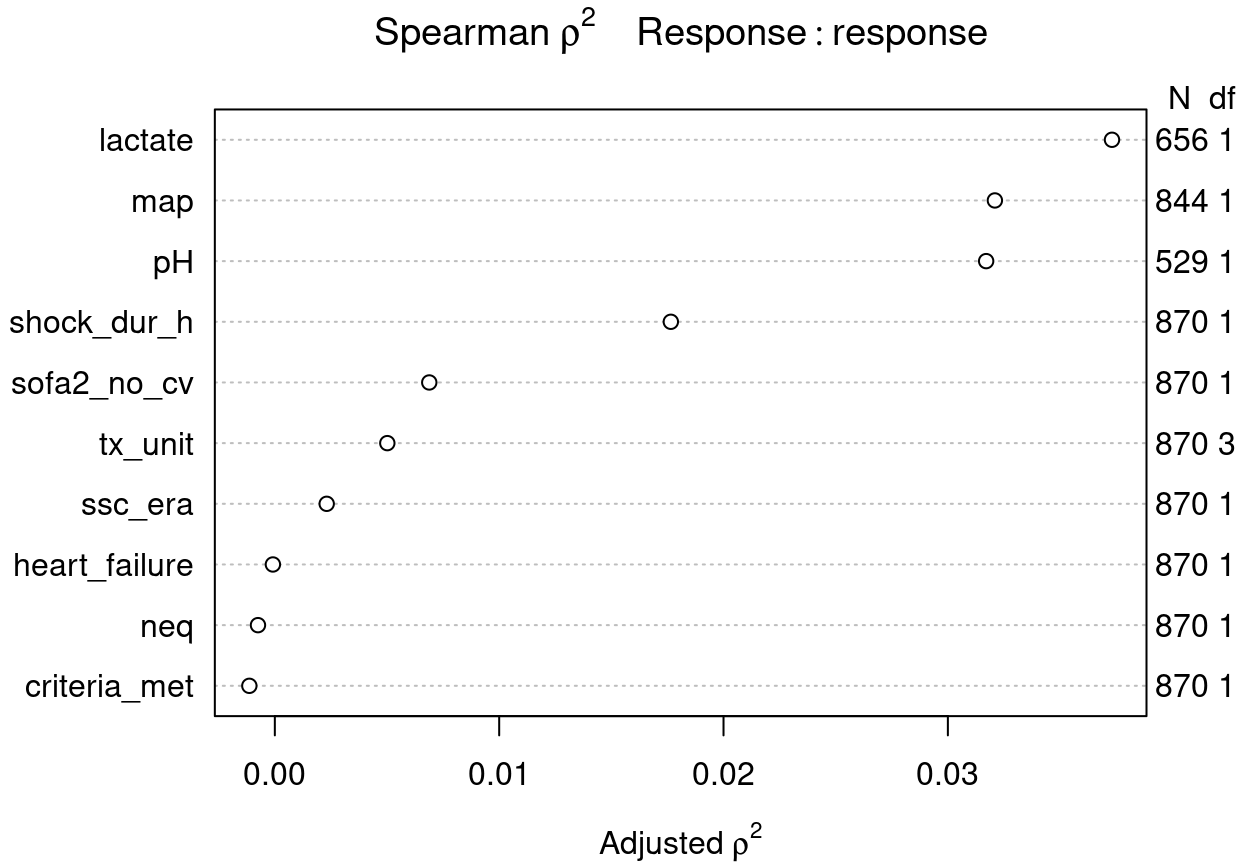
12.4.1 Logistic Regression

Here, use logistic regression to generate prediction models using two approaches: 1) main effects (additive) only, and 2) non-linear effects of predictors.

The two models will be built in one `workflow_set()`. First, generate a Spearman's ρ^2 rank correlation plot to assess for potential predictor transformation (non-linear effects).

▼ Code

```
plot(spearman2(response ~ ., data = pe |> select(-c(age, sex, race_eth))))
```



▼ Code

```
invisible(pdf("pe_spearman.pdf"))
plot(spearman2(response ~ ., data = pe |> select(-c(age, sex, race_eth))),
     main = ""
     )
title(main = expression("Phenylephrine Spearman " * rho^2))
invisible(dev.off())
```

The Spearman's ρ^2 rank correlation plot shows that `lactate`, `map`, and `pH` have the highest potential to improve model fit with a non-linear transformation. The ρ^2 values for these predictors is relatively close to each other. From a biological plausibility and outcome component perspective, non-linear terms for `lactate` and `pH` are most likely to provide value. Because these two predictors are

quantitative variables, introduce a natural spline with 2 knots (3 df) for `lactate` and a natural spline with 1 knot (2 df) for `pH`. This will add 3 total df to the model and spend all of the available degrees of freedom (12 df in the main effects model and 15 total available).

As noted in Section 11.5, there are missing data in the same three variables as for vasopressin and epinephrine. Therefore, the same imputation approach will be used.

▼ Code

```
# Main effects recipe
pe_m1_lr_rec_main <-
  recipe(response ~ ., data = pe) |>
  # Declare age, sex, and race_eth in a bystander role
  update_role(age, sex, race_eth, new_role = "bystander") |>
  # Single (linear regression) imputation for missing quantitatitve predictors
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate
                       )
    )

# Non-linear effects recipe
pe_m1_lr_rec_ns <-
  recipe(response ~ ., data = pe) |>
  update_role(age, sex, race_eth, new_role = "bystander") |>
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h,
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map,
                       )
    ) |>
```

```
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate,
                       )
    ) |>
  # Add natural spline with 2 knots (3 df) for lactate
  step_spline_natural(lactate, deg_free = 3) |>
  # Add natural spline with 1 knot (2 df) for pH
  step_spline_natural(pH, deg_free = 2)

# Set up pre-processing and model workflow_set()
pe_m1_lr_preproc <- list(
  pe_m1_main = pe_m1_lr_rec_main,
  pe_m1_ns = pe_m1_lr_rec_ns
  )

pe_m1_lr_models <-
  workflow_set(pe_m1_lr_preproc,
               list(logistic = logistic_reg(engine = "glm")),
               cross = FALSE)

# Resample each model in the set with 10-fold CV (5 repeats)
set.seed(2026)
train_cv <- vfold_cv(pe, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

pe_m1_lr_models_fit <-
  pe_m1_lr_models |>
  workflow_map("fit_resamples", seed = 2026, verbose = TRUE,
               resamples = train_cv, metrics = class_metrics,
               control = keep_pred
               )
```

```
i 1 of 2 resampling: pe_m1_main_logistic

✓ 1 of 2 resampling: pe_m1_main_logistic (18.4s)

i 2 of 2 resampling: pe_m1_ns_logistic

✓ 2 of 2 resampling: pe_m1_ns_logistic (31s)
```

▼ Code

```
# Collect predictions
pe_m1_lr_preds <-
  collect_predictions(pe_m1_lr_models_fit, summarize = FALSE)

# Collect then show the metrics
```

```r
pe_m1_lr_mets <-
  collect_metrics(pe_m1_lr_models_fit) |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

pe_m1_lr_mets |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Phenylephrine m1 (Logistic Regression) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Phenylephrine m1 (Logistic Regression) Metric Comparison

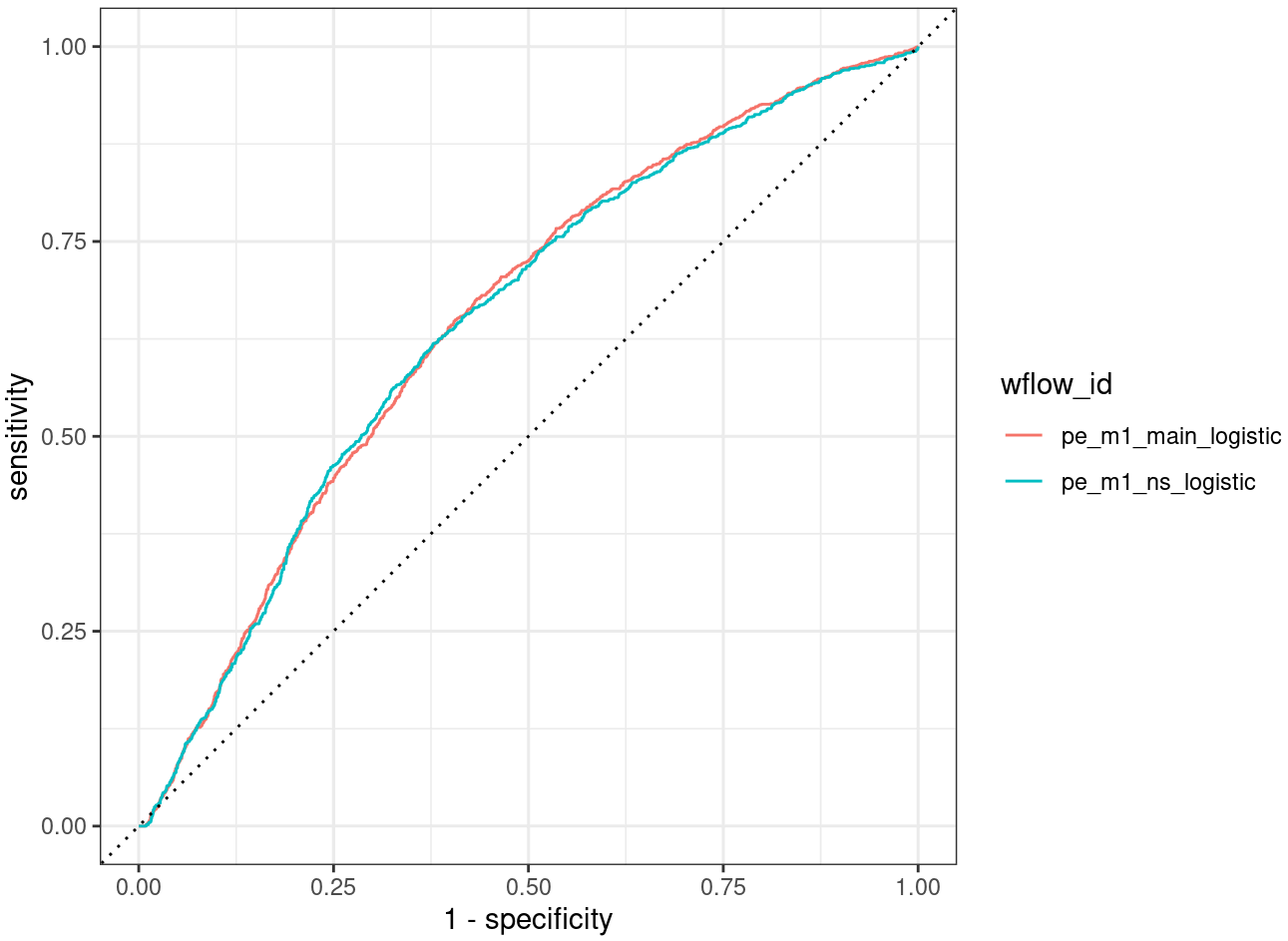
Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).

Model	AUROC	Brier Score
pe_m1_main_logistic	0.650 (0.634, 0.666)	0.221 (0.217, 0.224)
pe_m1_ns_logistic	0.647 (0.631, 0.664)	0.222 (0.218, 0.226)

The two logistic regression modeling approaches had essentially the same discrimination (AUROC) and overall performance (Brier score).

▼ Code

```
pe_m1_lr_preds |>
  group_by(wflow_id) |>
  roc_curve(response, .pred_Response) |>
  autoplot()
```



The two curves are essentially overlapping.

12.4.2 Machine Learning

12.4.2.1 Regularized (Elastic Net)

Fit regularized (elastic net) models.

▼ Code

```
pe_m2_rec_main <- pe_m1_lr_rec_main |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

pe_m2_rec_ns <- pe_m1_lr_rec_ns |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())
```

```
pe_m2_glmnet_preproc <- list(
  pe_m2_main = pe_m2_rec_main,
  pe_m2_ns = pe_m2_rec_ns
  )

pe_m2_glmnet_models <-
  workflow_set(pe_m2_glmnet_preproc,
               list(glmnet = logistic_reg(
                 engine = "glmnet",
                 mode = "classification",
                 penalty = tune(),     # lambda
                 mixture = tune()      # alpha
                 )),
               cross = TRUE
               )

set.seed(2026)
train_cv <- vfold_cv(pe, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

pen_grid <- grid_space_filling(
  penalty(range = c(-5, 0)), # log10 scale, reasonable range for elastic net
  mixture(range = c(0, 1)),  # default
  size = 30
  )

pe_m2_glmnet_models_fit <-
  pe_m2_glmnet_models |>
  workflow_map("tune_grid", seed = 2026, verbose = TRUE,
               resamples = train_cv, grid = pen_grid,
               metrics = class_metrics, control = keep_pred
               )
```

```
i 1 of 2 tuning:     pe_m2_main_glmnet

✓ 1 of 2 tuning:     pe_m2_main_glmnet (1m 42.3s)

i 2 of 2 tuning:     pe_m2_ns_glmnet

✓ 2 of 2 tuning:     pe_m2_ns_glmnet (1m 58.9s)
```

▼ Code

```
workflow_ids <- pe_m2_glmnet_models_fit$wflow_id

# Loop over workflows to finalize, fit, and collect results
pe_m2_glmnet_final_fit <- map(workflow_ids, function(wf_id) {

  # Extract tuning results for this workflow
```

```
  wf_result <- extract_workflow_set_result(pe_m2_glmnet_models_fit, id = wf_id)

  # Get best parameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")

  # Extract the workflow object itself
  wf <- extract_workflow(pe_m2_glmnet_models_fit, id = wf_id)

  # Finalize the workflow with best hyperparameters
  final_wf <- finalize_workflow(
    wf,
    best_params
  )

  # Fit the final workflow to the training set
  final_fit <- fit(final_wf, data = pe)

  # Collect resampling metrics for this workflow from tuning
  cv_metrics <- collect_metrics(wf_result) |>
    filter(penalty == best_params$penalty,
           mixture == best_params$mixture)

  # Collect predictions for the best parameters
  cv_preds <- collect_predictions(wf_result, summarize = TRUE) |>
    filter(penalty == best_params$penalty,
           mixture == best_params$mixture)

  # Return as a list
  list(
    workflow_id = wf_id,
    best_params = best_params,
    final_workflow = final_wf,
    final_fit = final_fit,
    cv_metrics = cv_metrics,
    cv_predictions = cv_preds
  )
})

# Name the list elements by workflow
names(pe_m2_glmnet_final_fit) <- workflow_ids
rm(workflow_ids)
```

Show elastic net hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(pe_m2_glmnet_final_fit,
         ~ .x$best_params |> mutate(wflow_id = .y)
         ) |>
  select(wflow_id, penalty, mixture) |>
```

```
  gt() |>
  tab_header(
    title = "Phenylephrine m2 Elastic Net Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    wflow_id = "Model",
    penalty = "Penalty (lambda [λ])",
    mixture = "Mixture (alpha [α])"
    ) |>
  fmt_number(decimals = 4) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(penalty, mixture)
    )
```

Phenylephrine m2 Elastic Net Best Hyperparameters Using 10-fold Cross-Validation (5 repeats)		
Model	Penalty (lambda [λ])	Mixture (alpha [α])
pe_m2_main_glmnet	0.0924	0.3103
pe_m2_ns_glmnet	0.0057	0.0000

Collect metrics and display table (comparing to the logistic regression model fits).

▼ Code

```
pe_m2_glmnet_mets <-
 pe_m2_glmnet_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
```

```
    )

bind_rows(pe_m1_lr_mets, pe_m2_glmnet_mets)  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Phenylephrine m2 (Elastic Net) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Phenylephrine m2 (Elastic Net) Metric Comparison		
Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
pe_m1_main_logistic	0.650 (0.634, 0.666)	0.221 (0.217, 0.224)
pe_m1_ns_logistic	0.647 (0.631, 0.664)	0.222 (0.218, 0.226)
pe_m2_main_glmnet	0.651 (0.635, 0.666)	0.222 (0.220, 0.224)
pe_m2_ns_glmnet	0.648 (0.632, 0.665)	0.221 (0.218, 0.225)

The regularized (elastic net) models performed slightly better than the logistic regression models.

12.4.2.2 Random Forest

Fit a random forest model.

▼ Code

```
pe_m3_rf_rec <-
  recipe(response ~ ., data = pe) |>
  # Declare age, sex, race_eth in a bystander role to keep them in the
  # resampling dataset for future use in fairness checks
```

```r
  update_role(age, sex, race_eth, new_role = "bystander") |>
  # Single (linear regression) imputation for missing quantitative predictors
  step_impute_linear(
    map,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h
                       )
                     ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map
                       )
    ) |>
  step_impute_linear(
    pH,
    impute_with = vars(heart_failure, tx_unit, ssc_era, criteria_met,
                       sofa2_no_cv, neq, shock_dur_h, map, lactate
                       )
    )

pe_m3_rf_preproc <- list(pe_m3_rf = pe_m3_rf_rec)

pe_m3_rf_models <-
  workflow_set(pe_m3_rf_preproc,
               list(ranger = rand_forest(
                 engine = "ranger",
                 mode = "classification",
                 trees = 1000,      # Default is 500, 1000 is better
                 mtry = tune(),     # Number of randomly selected predictors
                 min_n = tune()     # Minimum node size
                 )),
               cross = TRUE
               )

set.seed(2026)
train_cv <- vfold_cv(pe, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

# Prep recipe and determine predictor count
pe_m3_rf_rec_prep <- prep(pe_m3_rf_rec)

x_train <- bake(pe_m3_rf_rec_prep, new_data = NULL) |>
  select(-c(response, age, sex, race_eth))

p <- ncol(x_train) # p == 10

rf_param_set <- parameters(
  mtry(range = c(1L, 10L)), #sqrt(p) ~= 3, (p / 2) == 5, which is too narrow
  min_n(range = c(2L, 100L)) # Expanded range (50L used for other medications)
```

```
                            # because optimal parameters were at boundary
                            # and performance was improving.
)

initial_candidates <- grid_space_filling(
  rf_param_set,
  size = 30
)

race_ctrl <- control_race(
  save_pred = TRUE,
  save_workflow = TRUE,
  parallel_over = "resamples",
  verbose = FALSE
)

pe_m3_rf_model_fit <-
  pe_m3_rf_models |>
  workflow_map(
    "tune_race_anova", seed = 2026, resamples = train_cv,
    grid = initial_candidates, metrics = class_metrics, control = race_ctrl
  )

# Get workflow IDs
workflow_ids <- pe_m3_rf_model_fit$wflow_id

# Finalize, fit, and collect results for the workflow
pe_m3_rf_final_fit <- map(workflow_ids, function(wf_id) {
  # Extract tuning (race) results for this workflow
  wf_result <- extract_workflow_set_result(
    pe_m3_rf_model_fit,
    id = wf_id
    )
  # Select best hyperparameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")
  # Extract the workflow itself
  wf <- extract_workflow(pe_m3_rf_model_fit, id = wf_id)
  # Finalize workflow with best hyperparameters
  final_wf <- finalize_workflow(wf, best_params)
  # Fit finalized workflow to derivation data
  final_fit <- fit(final_wf, data = pe)
  # Collect CV metrics for the selected configuration
  cv_metrics <- collect_metrics(wf_result) |>
    filter(.config == best_params$.config)
  # Collect CV predictions for the selected configuration
  cv_predictions <- collect_predictions(wf_result) |>
    filter(.config == best_params$.config)
  # Return results
  list(
    workflow_id     = wf_id,
    best_params     = best_params,
```

```
    final_workflow  = final_wf,
    final_fit       = final_fit,
    cv_metrics      = cv_metrics,
    cv_predictions  = cv_predictions
    )
})

names(pe_m3_rf_final_fit) <- map_chr(pe_m3_rf_final_fit, "workflow_id")
```

Show hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(pe_m3_rf_final_fit,
         ~ .x$best_params |> mutate(workflow_id = .y)
         ) |>
  select(workflow_id, mtry, min_n) |>
  gt() |>
  tab_header(
    title = "Random Forest Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    workflow_id = "Model",
    mtry = "Random Variables at Split (mtry)",
    min_n = "Min Terminal Node Size (min_n)"
    ) |>
  fmt_number(decimals = 0) |>
  cols_align(
    align = "left",
    columns = c(workflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mtry, min_n)
    )
```

Random Forest Best Hyperparameters Using 10-fold Cross-Validation (5 repeats)		
Model	Random Variables at Split (mtry)	Min Terminal Node Size (min_n)
pe_m3_rf_ranger	4	93

Although min_n is at the top of the range, performance did not continue to improve as min_n approached the top of the range (not shown). Therefore these were accepted as the final parameters.

Collect and display metrics for model.

▼ Code

```
pe_m3_rf_mets <-
 pe_m3_rf_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(pe_m1_lr_mets,
          pe_m2_glmnet_mets,
          pe_m3_rf_mets)  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Phenylephrine m3 (Random Forest) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Phenylephrine m3 (Random Forest) Metric Comparison Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
pe_m1_main_logistic	0.650 (0.634, 0.666)	0.221 (0.217, 0.224)
pe_m1_ns_logistic	0.647 (0.631, 0.664)	0.222 (0.218, 0.226)
pe_m2_main_glmnet	0.651 (0.635, 0.666)	0.222 (0.220, 0.224)
pe_m2_ns_glmnet	0.648 (0.632, 0.665)	0.221 (0.218, 0.225)
pe_m3_rf_ranger	0.657 (0.642, 0.673)	0.217 (0.214, 0.220)

The random forest model performed the best of those assessed at this point.

12.4.2.3 Boosted Trees

Generate a boosted tree model.

▼ Code

```
pe_m4_xgb_rec <-
  recipe(response ~ ., data = pe) |>
  update_role(age, sex, race_eth, new_role = "bystander") |>
  step_impute_linear(
    map,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h
    )
  ) |>
  step_impute_linear(
    lactate,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h, map
    )
  ) |>
  step_impute_linear(
    pH,
    impute_with = vars(
      heart_failure, tx_unit, ssc_era,
      criteria_met, sofa2_no_cv,
      neq, shock_dur_h, map, lactate
    )
```

```
  ) |>
  # Dummy code nominal predictors
  step_dummy(all_nominal_predictors())

pe_m4_xgb_preproc <- list(pe_m4_boost = pe_m4_xgb_rec)

pe_m4_xgb_model <-
  workflow_set(pe_m4_xgb_preproc,
               list(xgboost = boost_tree(
                 trees = 1000,               # Number of trees
                 tree_depth = tune(),        # Tree depth: # of "interactions"
                 min_n = tune(),             # Minimum node size
                 loss_reduction = tune(),    # Reduction in the loss function required to split
         further
                 sample_size = tune(),       # Proportion of data exposed to fitting routine
                 mtry = tune(),              # Random predictors at each split
                 learn_rate = tune()         # Rate at which the boosting algorithm adapts
                 ) |>
                   set_engine("xgboost") |>
                   set_mode("classification")
                 ),
               cross = FALSE
               )

set.seed(2026)
train_cv <- vfold_cv(pe, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

# Prep recipe and determine post-dummy predictor count
pe_m4_xgb_rec_prep <- prep(pe_m4_xgb_rec)

x_train <- bake(pe_m4_xgb_rec_prep, new_data = NULL) |>
  select(-c(response, age, sex, race_eth))

p <- ncol(x_train) # p == 10

# Select parameters (particularly depth) based p==10 and RF optimal parameters
xgb_params <- parameters(
  tree_depth(range = c(1L, 6L)),
  min_n(range = c(2L, 50L)),
  learn_rate(range = c(-3, -0.5)),
  loss_reduction(),
  sample_prop(range = c(0.5, 1.0)),
  finalize(mtry(), x_train)
  )

initial_candidates <- grid_space_filling(
  xgb_params,
  size = 30
  )
```

```r
race_ctrl <- control_race(
  save_pred = TRUE,
  save_workflow = TRUE,
  parallel_over = "resamples",
  verbose = FALSE
  )

pe_m4_xgb_model_fit <-
  pe_m4_xgb_model |>
  workflow_map(
    "tune_race_anova",
    seed = 2026,
    resamples = train_cv,
    grid = initial_candidates,
    metrics = class_metrics,
    control = race_ctrl
    )

workflow_ids <- pe_m4_xgb_model_fit$wflow_id

# Finalize, fit, and collect results the workflow
pe_m4_xgb_final_fit <- map(workflow_ids, function(wf_id) {
  # Extract tuning (race) results for this workflow
  wf_result <- extract_workflow_set_result(
    pe_m4_xgb_model_fit,
    id = wf_id
    )
  # Select best hyperparameters by AUROC
  best_params <- select_best(wf_result, metric = "roc_auc")
  # Extract the workflow itself
  wf <- extract_workflow(pe_m4_xgb_model_fit, id = wf_id)
  # Finalize workflow with best hyperparameters
  final_wf <- finalize_workflow(wf, best_params)
  # Fit finalized workflow to derivation data
  final_fit <- fit(final_wf, data = pe)
  # Collect CV metrics for the selected configuration
  cv_metrics <- collect_metrics(wf_result) |>
    filter(.config == best_params$.config)
  # Collect CV predictions for the selected configuration
  cv_predictions <- collect_predictions(wf_result) |>
    filter(.config == best_params$.config)
  # Return results
  list(
    workflow_id     = wf_id,
    best_params     = best_params,
    final_workflow  = final_wf,
    final_fit       = final_fit,
    cv_metrics      = cv_metrics,
    cv_predictions  = cv_predictions
    )
```

```
})

names(pe_m4_xgb_final_fit) <- map_chr(pe_m4_xgb_final_fit, "workflow_id")
```

Show hyperparameters resulting in the best fit.

▼ Code

```
imap_dfr(pe_m4_xgb_final_fit,
         ~ .x$best_params |> mutate(wflow_id = .y)
         ) |>
  mutate(trees = 1000) |>
  select(wflow_id, trees, tree_depth, min_n, learn_rate, loss_reduction,
         sample_size, mtry
         ) |>
  gt() |>
  fmt_number(columns = c(learn_rate, loss_reduction, sample_size),
             decimals = 3
             ) |>
  tab_header(
    title = "Gradient Boosted Model Best Hyperparameters",
    subtitle = "Using 10-fold Cross-Validation (5 repeats)"
    ) |>
  cols_label(
    wflow_id = "Model",
    trees = "Trees",
    tree_depth = "Tree Depth",
    min_n = "Node Size",
    learn_rate = "Learn Rate",
    loss_reduction = "Min Loss Reduction",
    sample_size = "Proportion of data exposed",
    mtry = "Predictors at Splits",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(trees, tree_depth, min_n, learn_rate, loss_reduction,
                sample_size, mtry
                )
    )
```

Gradient Boosted Model Best Hyperparameters

Using 10-fold Cross-Validation (5 repeats)

Model	Trees	Tree Depth	Node Size	Learn Rate	Min Loss Reduction	Proportion of data exposed	Predictors at Splits
pe_m4_boost_xgboost	1000	5	10	0.001	0.001	0.586	7

Optimal parameters are not at tuning boundaries.

Collect and display metrics for model.

▼ Code

```
pe_m4_xgb_mets <-
 pe_m4_xgb_final_fit |> map(function(x) {
    x$cv_metrics |>
    mutate(wflow_id = x$workflow_id)
  }) |>
  list_rbind() |>
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(pe_m1_lr_mets,
          pe_m2_glmnet_mets,
          pe_m3_rf_mets,
          pe_m4_xgb_mets
          )  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Phenylephrine m4 (XGBoost) Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
```

```
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Phenylephrine m4 (XGBoost) Metric Comparison Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
pe_m1_main_logistic	0.650 (0.634, 0.666)	0.221 (0.217, 0.224)
pe_m1_ns_logistic	0.647 (0.631, 0.664)	0.222 (0.218, 0.226)
pe_m2_main_glmnet	0.651 (0.635, 0.666)	0.222 (0.220, 0.224)
pe_m2_ns_glmnet	0.648 (0.632, 0.665)	0.221 (0.218, 0.225)
pe_m3_rf_ranger	0.657 (0.642, 0.673)	0.217 (0.214, 0.220)
pe_m4_boost_xgboost	0.658 (0.642, 0.673)	0.219 (0.217, 0.221)

The XGBoost performed similarly to the random forest model but with slightly worse overall performance (Brier score).

12.4.3 Discrimination/Accuracy Comparison

Now, show the discrimination and accuracy metrics for all models.

▼ Code

```
(pe_disc_perf_mets <- bind_rows(pe_m1_lr_mets,
          pe_m2_glmnet_mets,
          pe_m3_rf_mets,
          pe_m4_xgb_mets
          )  |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Phenylephrine Models Discrimination/Overall Performance Metric Comparison",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
```

```
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    ))
```

Phenylephrine Models Discrimination/Overall Performance Metric Comparison

Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).

Model	AUROC	Brier Score
pe_m1_main_logistic	0.650 (0.634, 0.666)	0.221 (0.217, 0.224)
pe_m1_ns_logistic	0.647 (0.631, 0.664)	0.222 (0.218, 0.226)
pe_m2_main_glmnet	0.651 (0.635, 0.666)	0.222 (0.220, 0.224)
pe_m2_ns_glmnet	0.648 (0.632, 0.665)	0.221 (0.218, 0.225)
pe_m3_rf_ranger	0.657 (0.642, 0.673)	0.217 (0.214, 0.220)
pe_m4_boost_xgboost	0.658 (0.642, 0.673)	0.219 (0.217, 0.221)

The random forest model (`pe_m3_rf_ranger`) had the best performance. The logistic regression model with main effects only had relatively similar performance, therefore will carry both models forward for calibration assessment prior to final model selection.

Export comparison table to Word (commented out for rendering)

▼ Code

```
#gtsave(
#  data = pe_disc_perf_mets,
#  filename = "pe_disc_perf_metrics.docx"
#)
```

12.4.4 Calibration

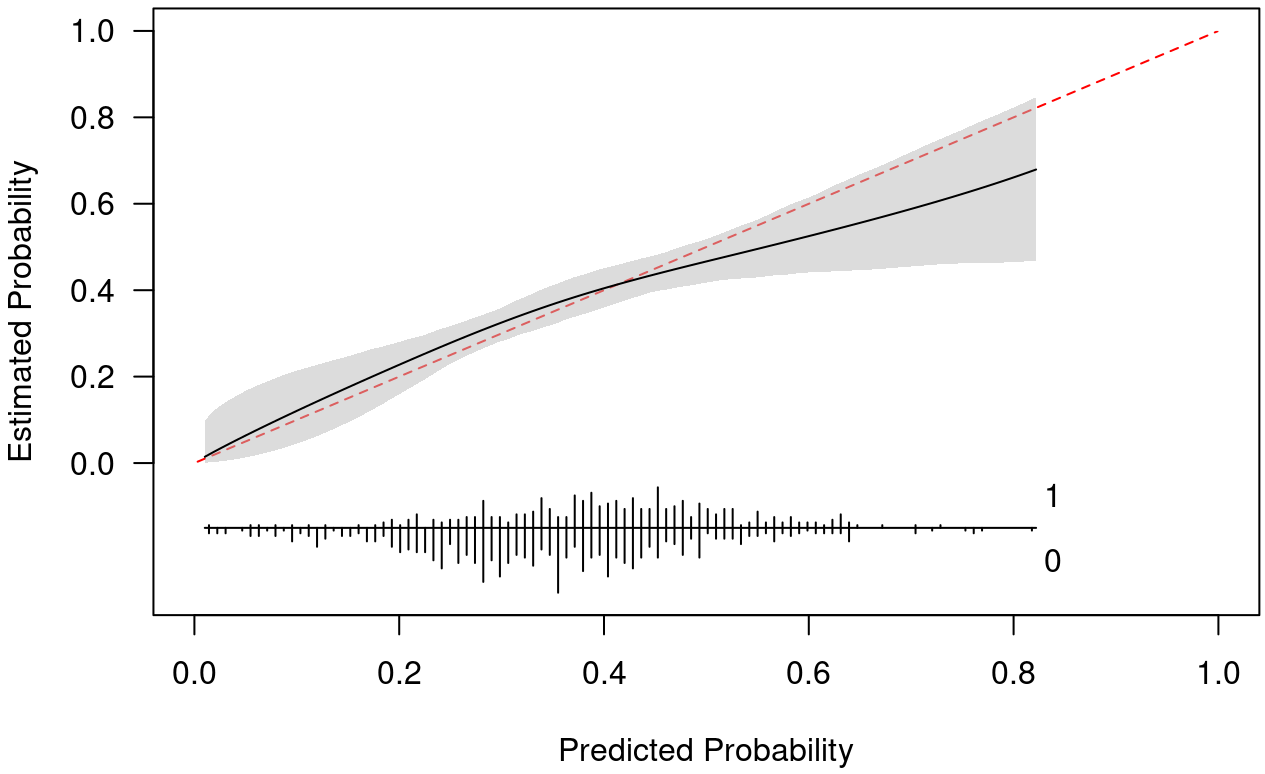
Use the same calibration approach as in Section 12.2.4 that was used for vasopressin (and for epinephrine).

Start with calibration for the logistic regression (main effects) model.

▼ Code

```
pe_m1_lr_preds_cv <- pe_m1_lr_preds |>
  filter(wflow_id == "pe_m1_main_logistic") |>
  group_by(.row) |>
  summarise(
    response = first(response),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
  ) |>
  mutate(
    response_num = if_else(response == "Response", 1, 0)
  )

set.seed(2026)
pe_lr_cal <- pmcalibration(
  y = pe_m1_lr_preds_cv$response_num,
  p = pe_m1_lr_preds_cv$.pred_Response,
  ci = "boot", # n = 1000 replicates is the default
  smooth = "rcs",
  nk = 3 # 5 knots is default, 3 has less bias
  )
plot(pe_lr_cal)
title("Calibration Plot: Phenylephrine Logistic Regression")
```



▼ Code

```
par(mfrow = c(1, 1))
invisible(dev.copy(pdf, "pe_logistic_calibration.pdf"))
invisible(dev.off())
```

Calibration with the logistic regression model is pretty good.

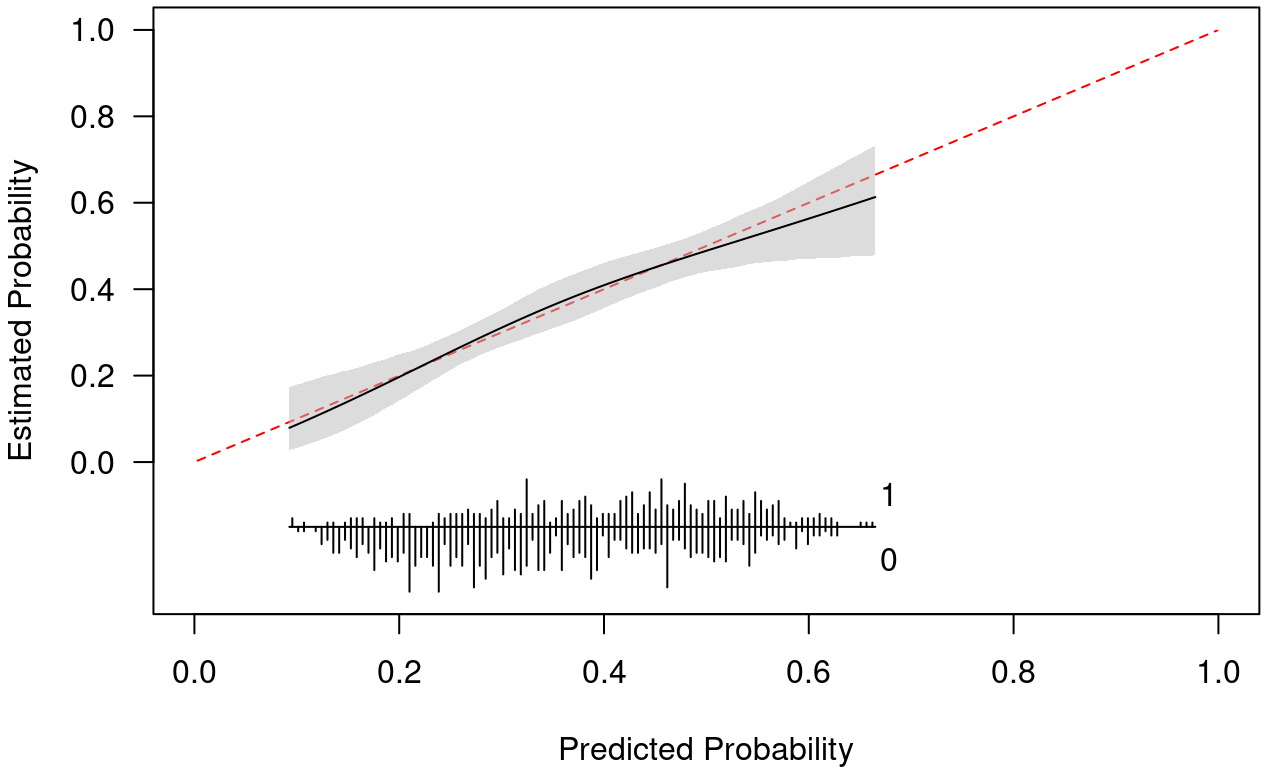
Next, assess calibration with the random forest model.

▼ Code

```
pe_m3_rf_preds <- pe_m3_rf_final_fit[[1]]$cv_predictions

pe_m3_rf_preds_cv <- pe_m3_rf_preds |>
  group_by(.row) |>
  summarise(
    response = first(response),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
  ) |>
  mutate(
    response_num = if_else(response == "Response", 1, 0)
  )
```

```r
set.seed(2026)
pe_rf_cal <- pmcalibration(
  y = pe_m3_rf_preds_cv$response_num,
  p = pe_m3_rf_preds_cv$.pred_Response,
  ci = "boot", # n = 1000 replicates is the default
  smooth = "rcs",
  nk = 3 # 5 knots is default, 3 has less bias
  )
```



▼ Code

```r
invisible(pdf("pe_rf_calibration.pdf"))
par(mfrow = c(1, 1),
    cex.axis = 1.4, cex.lab = 1.8, cex.main = 2,
    mar = c(5.1, 5.5, 1.4, 1.6), # bottom, left, top, right
    mgp = c(3.5, 1, 0)
    )
plot(pe_rf_cal)
invisible(dev.off())
```

Calibration of the phenylephrine random forest model is very good to excellent.

Save the two phenylephrine calibration plots as a panel figure.

▼ Code

```
invisible(ragg::agg_tiff("pe_calibration_panel.tiff",
                         width = 11, height = 5, units = "in",
                         res = 600, compression = "lzw"
                         ))

par(mfrow = c(1, 2), pty = "s")

plot(pe_lr_cal)
title("Phenylephrine Logistic Regression (Main)")

plot(pe_rf_cal)
title("Phenylephrine Random Forest")

par(mfrow = c(1, 1))
invisible(dev.off())
```

Show the quantitative calibration metrics for comparison.

▼ Code

```
(pe_cal_metrics_tbl <-
   bind_rows(
    as_tibble(pe_lr_cal$metrics, rownames = "metric") |>
      mutate(model = "Logistic (main)"),
    as_tibble(pe_rf_cal$metrics, rownames = "metric") |>
      mutate(model = "Random Forest")
  ) |>
  select(model, metric, value) |>
  pivot_wider(
    names_from = metric,
    values_from = value
  ) |>
  rename(
    Model = model,
    `Eavg (ICI)` = Eavg
  ) |>
  gt() |>
  tab_header(
    title = "Phenylephrine Models Calibration Metrics"
  ) |>
  fmt_number(
    columns = c(`Eavg (ICI)`, E50, E90, Emax, ECI),
    decimals = 4
  ) |>
  cols_align(
    align = "center"
  ))
```

Phenylephrine Models Calibration Metrics					
Model	Eavg (ICI)	E50	E90	Emax	ECI
Logistic (main)	0.0253	0.0233	0.0476	0.1427	0.1063
Random Forest	0.0109	0.0099	0.0227	0.0517	0.0195

The models both have good quantitative calibration metrics, but calibration is better for the phenylephrine random forest model.

Export comparison table to Word (commented out for rendering)

▼ Code

```
#gtsave(
#  data = pe_cal_metrics_tbl,
#  filename = "pe_cal_metric_comparison.docx"
#)
```

12.4.5 Decision Curve Analysis

Extract predictions from the logistic regression and random forest models, then combine.

▼ Code

```
# Extract the out-of-fold predictions for each model
# Then average predictions across the 5 CV repeats using .row
# Logistic model
pe_lr_dca <- pe_m1_lr_preds |>
  filter(wflow_id == "pe_m1_main_logistic") |>
  group_by(.row) |>
  summarise(
    response = first(response),
    p_logistic = mean(.pred_Response),
    .groups = "drop"
  )

# Random forest model
pe_rf_dca <- pe_m3_rf_final_fit[[1]]$cv_predictions |>
  group_by(.row) |>
  summarise(
    p_rf = mean(.pred_Response),
    .groups = "drop"
  )

# Combine
```

```
pe_dca_data <- pe_lr_dca |>
  left_join(pe_rf_dca, by = ".row")
```

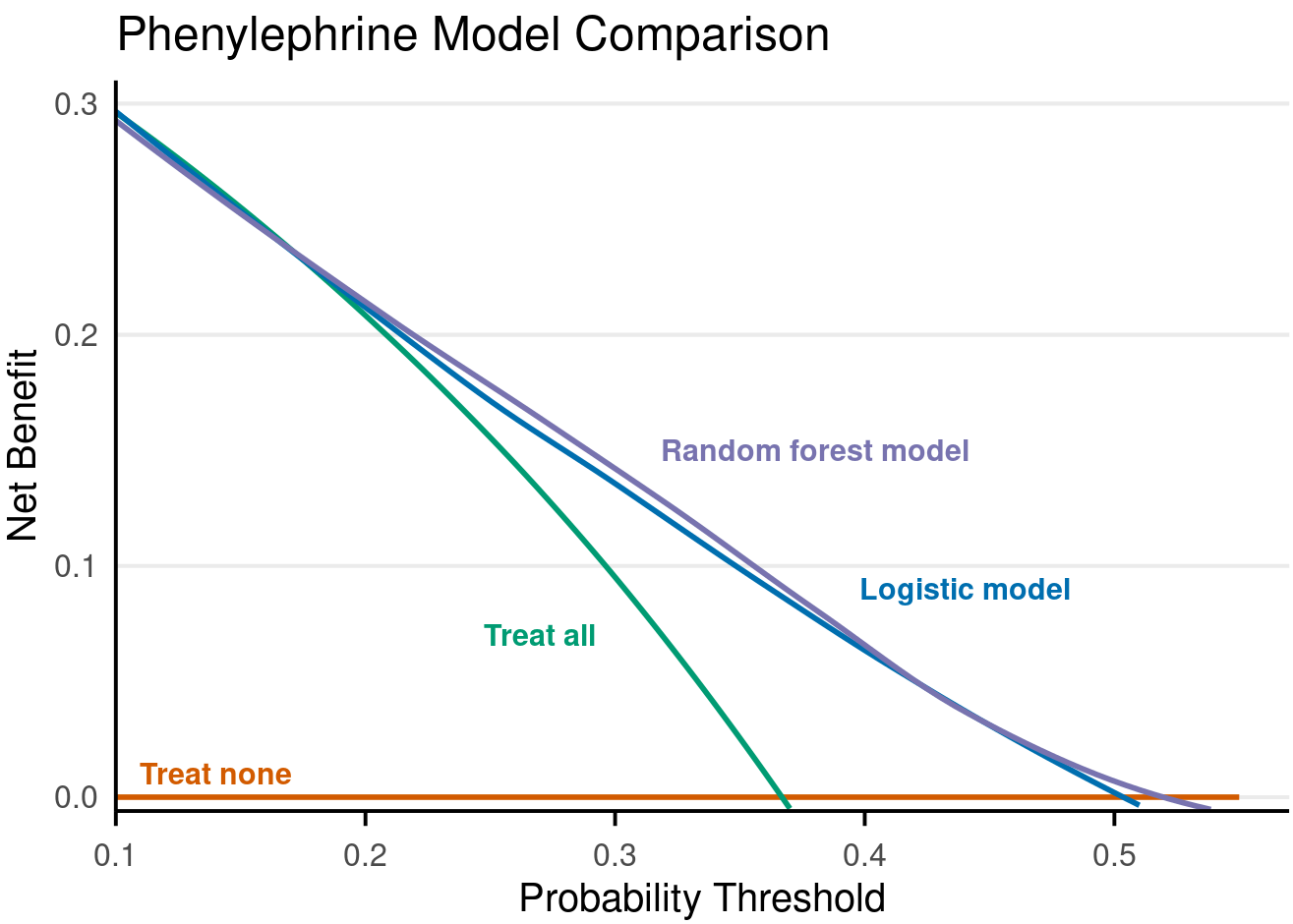
Then run DCA and plot.

▼ Code

```
pe_dca <- dca(
  response == "Response" ~ p_logistic + p_rf,
  data = pe_dca_data
  )

(pe_dca_plot <-
    ggplot(as_tibble(pe_dca) |>
             filter(net_benefit >= -0.006 &
                      net_benefit <= 0.31 &
                      threshold <=0.55
                    ) |>
             mutate(label = fct_relevel(label, "Treat None", "Treat All",
                                        "p_logistic", "p_rf"
                                        )),
           aes(x = threshold,
               y = net_benefit,
               color = label,
               group = label)) +
    geom_smooth(method = loess, formula = y ~ x, se = FALSE, linewidth = 1) +
    scale_color_manual(values = c(
      "p_logistic" = "#0072B2",
      "p_rf" = "#7876B1FF",
      "Treat All" = "#009E73",
      "Treat None" = "#D55E00"
      ),
      labels = c(
        "p_logistic" = "Logistic regression",
        "p_rf" = "Random forest",
        "Treat All" = "Treat all",
        "Treat None" = "Treat none"
      )) +
    scale_y_continuous(
      limits = c(-0.006, 0.31),
      expand = expansion(add = c(0, 0))
      ) +
    scale_x_continuous(
      limits = c(0.1, 0.55),
      breaks = seq(0,0.55, by = 0.1),
      expand = expansion(add = c(0, 0.02))
      ) +
    labs(
      title = "Phenylephrine Model Comparison",
      x = "Probability Threshold",
```

```
  y = "Net Benefit",
  color = NULL
  ) +
annotate("text", x = 0.44, y = 0.09, label = "Logistic model",
         color = "#0072B2", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.38, y = 0.15, label = "Random forest model",
         color = "#7876B1FF", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.27, y = 0.07, label = "Treat all",
         color = "#009E73", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.14, y = 0.01, label = "Treat none",
         color = "#D55E00", family = "sans", fontface = "bold",
         size = 4
         ) +
theme_minimal(base_size = 15) +
theme(
  legend.position = "none",
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x = element_line(),
  axis.line.y = element_line(),
  axis.ticks.x = element_line(),
  axis.ticks.length = unit(0.2, "cm")
))
```



Both models are better than “Treat none” and “Treat all” in the probability threshold range from about 0.2 to 0.5, the random forest model has slightly higher net benefit than the logistic regression model.

Obtain exact threshold values for positive net benefit with logistic regression and random forest models. Filter threshold values to those corresponding to the first instance the curve crosses zero net benefit.

▼ Code

```
as_tibble(pe_dca) |>
  select(threshold, label, net_benefit) |>
  pivot_wider(names_from = label, values_from = net_benefit) |>
  filter(!is.na(p_logistic), !is.na(p_rf),
         !is.na(`Treat All`), !is.na(`Treat None`),
         threshold <0.6
         ) |>
  summarise(
    min_prob_thresh_lr = min(threshold[p_logistic > `Treat All` + 0.001 &
                                       p_logistic > `Treat None` + 0.001]
                             ),
    max_prob_thresh_lr = max(threshold[p_logistic > `Treat All` + 0.001 &
                                       p_logistic > `Treat None` + 0.001]
                             ),
```

```
    min_prob_thresh_rf = min(threshold[p_rf > `Treat All` + 0.001 &
                                       p_rf > `Treat None` + 0.001]
                             ),
    max_prob_thresh_rf = max(threshold[p_rf > `Treat All` + 0.001 &
                                       p_rf > `Treat None` + 0.001]
                             )
  )
```

```
# A tibble: 1 × 4
  min_prob_thresh_lr max_prob_thresh_lr min_prob_thresh_rf
               <dbl>              <dbl>              <dbl>
1               0.16                0.5                0.2
# ℹ 1 more variable: max_prob_thresh_rf <dbl>
```

The phenylephrine response logistic regression model had higher net benefit than the “Treat all” and “Treat none” curves at threshold probabilities from 0.16 to 0.5. The phenylephrine response random forest model had higher net benefit than the “Treat all” and “Treat none” curves at threshold probabilities from 0.2 to 0.55. The range of these values are similar (0.34 and 0.35, respectively).

Save DCA plot

▼ Code

```
ggsave(
  "pe_dca_compare.tiff",
  plot = pe_dca_plot,
  device = ragg::agg_tiff,
  width = dev.size("in")[1],
  height = dev.size("in")[2],
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

12.4.6 Model Selection

Among the models assessed, the random forest machine learning model for phenylephrine response demonstrated the best discrimination (similar to XGBoost), overall performance, and calibration. It also had a slightly higher net benefit in decision curve analysis than the logistic regression model. In light of these findings, the random forest model (`pe_m3_rf_ranger`) was selected as the final model for phenylephrine.

12.4.6.1 Refit to Whole Derivation Dataset

Refit the final model (`pe_m3_rf_ranger`) in the whole epinephrine derivation dataset (`pe`).

▼ Code

```r
set.seed(2026)
pe_mod_final_wf <-
  pe_m3_rf_final_fit[[1]]$final_workflow

pe_mod_final_fit_dev <-
  fit(pe_mod_final_wf, data = pe)
```

Estimate apparent performance on derivation data.

▼ Code

```r
# Generate predictions
pe_dev_preds <-
  predict(
    pe_mod_final_fit_dev,
    new_data = pe,
    type = "prob"
    ) |>
  bind_cols(
    pe |> select(response)
    )

pe_dev_score_data <-
  pe_dev_preds |>
  transmute(
    response_num = if_else(response == "Response", 1, 0),
    p_rf = .pred_Response
  )

# Generate metrics
pe_dev_score <-
  Score(
    object = list(
      "Random Forest" = pe_dev_score_data$p_rf
    ),
    formula = response_num ~ 1,
    data = pe_dev_score_data,
    metrics = c("auc", "brier")
  )

# Extract and format results
auc_ci <-
  pe_dev_score$AUC$score |>
  as_tibble() |>
  transmute(
    auc = as.numeric(AUC),
    auc_lower = as.numeric(lower),
    auc_upper = as.numeric(upper)
  )
```

```
brier_ci <-
  pe_dev_score$Brier$score |>
  as_tibble() |>
  filter(model == "Random Forest") |>
  transmute(
    brier = as.numeric(Brier),
    brier_lower = as.numeric(lower),
    brier_upper = as.numeric(upper)
  )

pe_dev_mets <- cbind(auc_ci, brier_ci) |>
  mutate(
    AUROC = sprintf("%.3f (%.3f, %.3f)", auc, auc_lower, auc_upper),
    Brier = sprintf("%.3f (%.3f, %.3f)", brier, brier_lower, brier_upper)
  ) |>
  select(AUROC, Brier)
rm(auc_ci, brier_ci)

pe_dev_mets |>
  gt() |>
  tab_header(title = "Phenylephrine Random Forest Model Fit and Apparent Performance
         Parameters",
             subtitle = "In Derivation (Ohio) Dataset") |>
  cols_align(align = "center")
```

Phenylephrine Random Forest Model Fit and Apparent Performance Parameters

In Derivation (Ohio) Dataset

AUROC	Brier
0.840 (0.814, 0.867)	0.174 (0.165, 0.183)

Similar to epinephrine, the apparent performance metrics for the phenylephrine random forest model are very optimistic and substantially better than the cross-validated values (AUROC apparent 0.84 vs. CV 0.66, Brier apparent 0.17 vs. CV 0.22).

12.4.6.2 Compare to Simple Model

Compare the selected multivariable model (`pe_m3_rf_ranger`) with a simple logistic regression model that only includes `neq` with a spline term.

▼ Code

```
pe_m0_lr_rec_ns <-
  recipe(response ~ neq, data = pe) |>
  step_spline_natural(neq, deg_free = 5)
```

```r
pe_m0_lr_preproc <- list(pe_m0_neq_only = pe_m0_lr_rec_ns)

pe_m0_lr_model <-
  workflow_set(pe_m0_lr_preproc,
               list(logistic = logistic_reg(engine = "glm")),
               cross = FALSE
               )

set.seed(2026)
train_cv <- vfold_cv(pe, v = 10, repeats = 5, strata = response)
class_metrics <- metric_set(roc_auc, brier_class)
keep_pred <- control_resamples(save_pred = TRUE, save_workflow = FALSE)

pe_m0_lr_model_fit <-
  pe_m0_lr_model |>
  workflow_map("fit_resamples", seed = 2026, verbose = TRUE,
               resamples = train_cv, metrics = class_metrics,
               control = keep_pred
               )
```

```
i 1 of 1 resampling: pe_m0_neq_only_logistic

✓ 1 of 1 resampling: pe_m0_neq_only_logistic (10.9s)
```

▼ Code

```r
pe_m0_lr_preds <-
  collect_predictions(pe_m0_lr_model_fit, summarize = FALSE)

pe_m0_lr_mets <-
  collect_metrics(pe_m0_lr_model_fit) |>
  # Compute 95% Wald CIs across the 50 resamples
  mutate(
    conf.low.95 = mean - qt(0.975, df = 10*5-1) * std_err,
    conf.high.95 = mean + qt(0.975, df = 10*5-1) * std_err,
    mean_ci = sprintf("%.3f (%.3f, %.3f)", mean, conf.low.95, conf.high.95)
  ) |>
  select(wflow_id, .metric, mean, conf.low.95, conf.high.95, mean_ci) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean_ci, mean, conf.low.95, conf.high.95)
    )

bind_rows(pe_m0_lr_mets, pe_m3_rf_mets) |>
  select(wflow_id, mean_ci_roc_auc, mean_ci_brier_class) |>
  gt() |>
  tab_header(
    title = "Phenylephrine Final Model Performance vs. NE Alone",
    subtitle = "Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI)."
    ) |>
```

```
  cols_label(
    wflow_id = "Model",
    mean_ci_roc_auc = "AUROC",
    mean_ci_brier_class = "Brier Score",
    ) |>
  cols_align(
    align = "left",
    columns = c(wflow_id)
    ) |>
  cols_align(
    align = "center",
    columns = c(mean_ci_roc_auc, mean_ci_brier_class)
    )
```

Phenylephrine Final Model Performance vs. NE Alone Using 10-fold Cross-Validation (5 repeats). Presented as mean (95% CI).		
Model	AUROC	Brier Score
pe_m0_neq_only_logistic	0.496 (0.482, 0.510)	0.234 (0.233, 0.235)
pe_m3_rf_ranger	0.657 (0.642, 0.673)	0.217 (0.214, 0.220)

The multivariable model has better metrics than the simple model using `neq` alone. Using additional patient information beyond relying on norepinephrine dose alone provides meaningful improvement in the prediction of phenylephrine response.

13 Final Model Descriptors

13.1 Permutation Importance

Here, address the question "Which variables matter for the prediction models?". Approach this question with permutation importance.

First, define a prediction wrapper. Then, use the `vi()` function from the `{vip}` package to determine variable importance based on AUROC through 100 simulations. The output of the function can roughly be interpreted as the average decrease in AUROC when that variable is randomly permuted.

▼ Code

```
pred_fun <- function(object, newdata) {
  predict(
    object,
    new_data = newdata,
    type = "prob"
```

```
    )$.pred_Response
  }

avp_vi <- vi(
  avp_mod_final_fit_dev,
  method = "permute",
  target = "response",
  metric = roc_auc_vec,
  pred_wrapper = pred_fun,
  train = avp,
  nsim = 100,
  smaller_is_better = FALSE
  ) |>
  mutate(model = factor("Vasopressin Logistic Regression"))

epi_vi <- vi(
  epi_mod_final_fit_dev,
  method = "permute",
  target = "response",
  metric = roc_auc_vec,
  pred_wrapper = pred_fun,
  train = epi,
  nsim = 100,
  smaller_is_better = FALSE
  ) |>
  mutate(model = factor("Epinephrine XGBoost"))

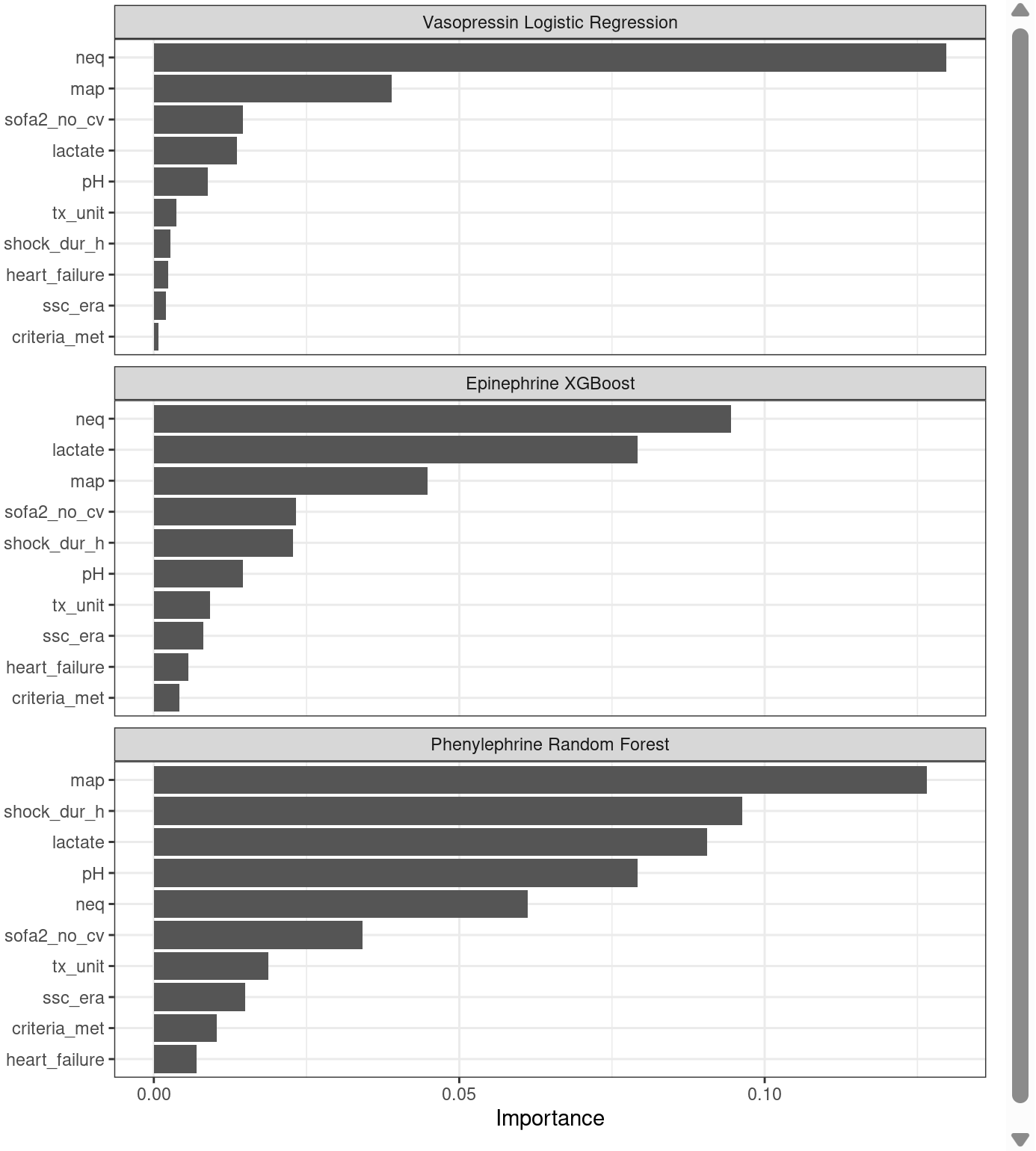
pe_vi <- vi(
  pe_mod_final_fit_dev,
  method = "permute",
  target = "response",
  metric = roc_auc_vec,
  pred_wrapper = pred_fun,
  train = pe,
  nsim = 100,
  smaller_is_better = FALSE
  ) |>
  mutate(model = factor("Phenylephrine Random Forest"))

all_vi <- bind_rows(avp_vi, epi_vi, pe_vi) |>
  filter(Variable != "age" & Variable != "sex" & Variable != "race_eth") |>
  group_by(model) |>
  arrange(desc(Importance), .by_group = TRUE) |>
  ungroup()
```

Then plot variable importance for each model.

▼ Code

```r
(vip_plot <- ggplot(all_vi,
                    aes(x = Importance,
                        y = reorder_within(Variable, Importance, model)
                        )
                    ) +
  geom_col() +
  facet_wrap(~ model, ncol = 1, scales = "free_y") +
  scale_y_reordered() +
  labs(y = NULL))
```



This plot shows that `neq` dominates the vasopressin model and has the highest importance in the epinephrine model, whereas `map` and `lactate` become relatively more important in the phenylephrine model.

Save plot.

▼ Code

```
ggsave(
  "variable_importance_plot.tiff",
  plot = vip_plot,
  device = ragg::agg_tiff,
  width = 5,
  height = 6,
  units = "in",
  dpi = 300
  )
```

13.2 Feature Importance

Here, address the question "How do the variables (features) influence individual predictions?". Approach this question with SHapley Additive exPlanations (SHAP) values. Use the `explain()` function from the `{fastshap}` package to determine SHAP values through 200 Monte Carlo simulations. Then, plot values with a beeswarm plot.

First, isolate predictor data used by each fitted model.

▼ Code

```
avp_x <- avp |> select(-response)
epi_x <- epi |> select(-response)
pe_x <- pe |> select(-response)
```

Then, build SHAP values and plot.

▼ Code

```
pred_fun <- function(object, newdata) {
  predict(
    object,
    new_data = newdata,
    type = "prob"
  )$.pred_Response
  }

shap_epi <- fastshap::explain(
  epi_mod_final_fit_dev,
  X = epi_x,
```

```r
  pred_wrapper = pred_fun,
  nsim = 200)
shap_epi <- shap_epi[, colnames(shap_epi) != "age"]
shap_epi <- shap_epi[, colnames(shap_epi) != "sex"]
shap_epi <- shap_epi[, colnames(shap_epi) != "race_eth"]

sv_epi <- shapviz(
  shap_epi,
  X = epi_x |> select(-c(age, sex, race_eth))
  )

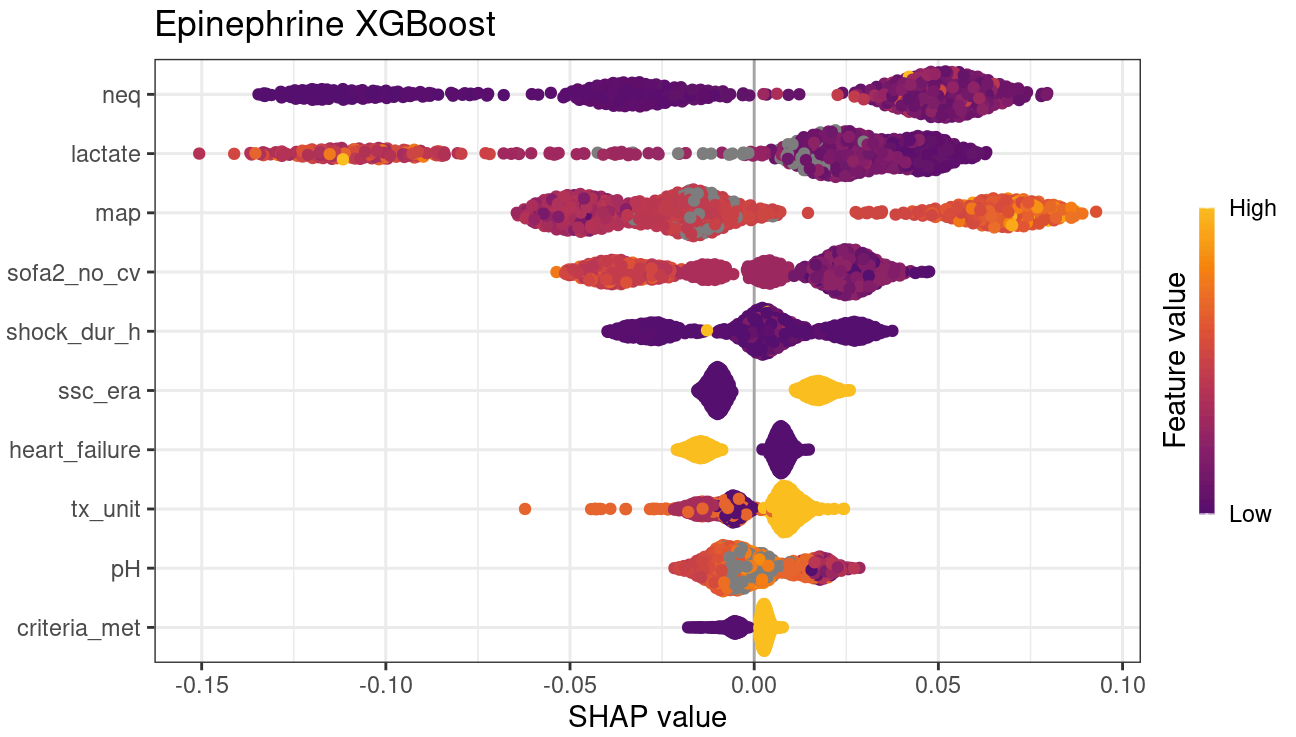
epi_shap <- sv_importance(sv_epi, kind = "beeswarm") +
  labs(title =  "Epinephrine XGBoost")

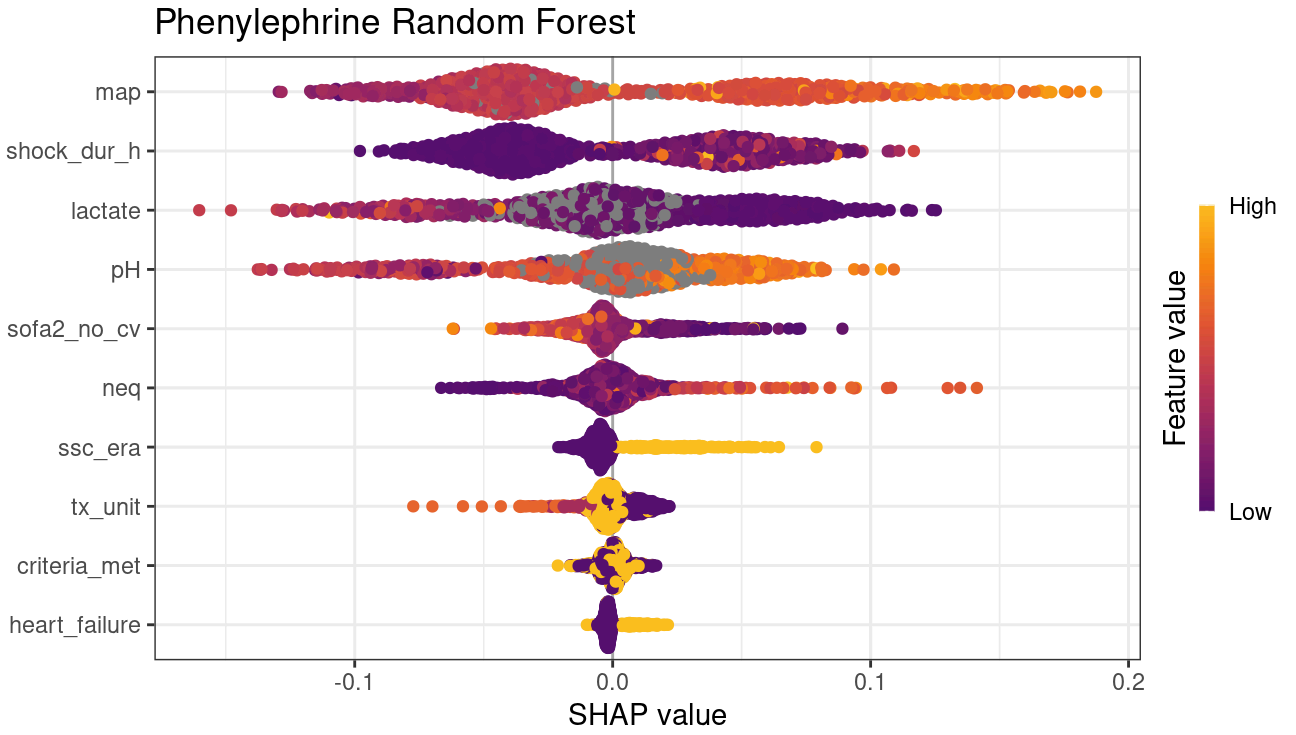
shap_pe <- fastshap::explain(
  pe_mod_final_fit_dev,
  X = pe_x,
  pred_wrapper = pred_fun,
  nsim = 200)
shap_pe <- shap_pe[, colnames(shap_pe) != "age"]
shap_pe <- shap_pe[, colnames(shap_pe) != "sex"]
shap_pe <- shap_pe[, colnames(shap_pe) != "race_eth"]

sv_pe <- shapviz(
  shap_pe,
  X = pe_x |> select(-c(age, sex, race_eth))
  )

pe_shap <- sv_importance(sv_pe, kind = "beeswarm") +
  labs(title =  "Phenylephrine Random Forest")

(shap_panel <- epi_shap / pe_shap)
```





▼ Code

```
rm(avp_x, epi_x, pe_x, shap_epi, sv_epi, epi_shap, shap_pe, sv_pe, pe_shap)
```

Save plot.

▼ Code

```
ggsave(
  "shap_beeswarm_panel.tiff",
  plot = shap_panel,
  device = ragg::agg_tiff,
  width = 5,
  height = 6,
  units = "in",
  dpi = 300
  )
```

13.3 Calibration

Generate alternative calibration plots to improve readability, particularly for prediction density distribution. These alternative calibration plots use the data derived with the `pmcalibration()` function but utilize `ggplot2()` for plotting instead of base plots that cannot be adjusted in the `pmcalibration()` function.

▼ Code

```
# Create reusable function for plotting curves
plot_calibration <- function(cal_obj, cal_dat, bin_width = 0.01) {

  #----------------------------------------
  # Distribution data
  #----------------------------------------
  dist_dat <- cal_obj$data |>
    mutate(bin = floor(p / bin_width) * bin_width + bin_width / 2) |>
    group_by(y, bin) |>
    summarise(n = n(), .groups = "drop")

  dist_dat <- dist_dat |>
    mutate(n_scaled = n / max(n))

  #----------------------------------------
  # Dynamic label position
  #----------------------------------------
  label_x <- min(max(cal_obj$data$p) + 0.05, 0.85)

  #----------------------------------------
  # Calibration plot
  #----------------------------------------
  p_cal <- ggplot(cal_dat, aes(x = p, y = p_c)) +
    geom_ribbon(aes(ymin = lower, ymax = upper),
                fill = "grey75", alpha = 0.5) +
    geom_abline(intercept = 0, slope = 1,
                linetype = 2, linewidth = 0.2, color = "red") +
    geom_line(linewidth = 0.6, color = "black") +
    scale_x_continuous(limits = c(0, 1), expand = c(0, 0)) +
```

```r
  scale_y_continuous(limits = c(0, 1), expand = c(0, 0)) +
  labs(y = "Estimated Probability", x = NULL) +
  theme_minimal(base_size = 18) +
  theme(
    text = element_text(family = "Arial"),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
    axis.ticks = element_line(color = "black"),
    axis.title.y = element_text(family = "Arial"),
    axis.text.y  = element_text(family = "Arial"),
    axis.title.x = element_blank(),
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank(),
    axis.line.x = element_blank()
    )

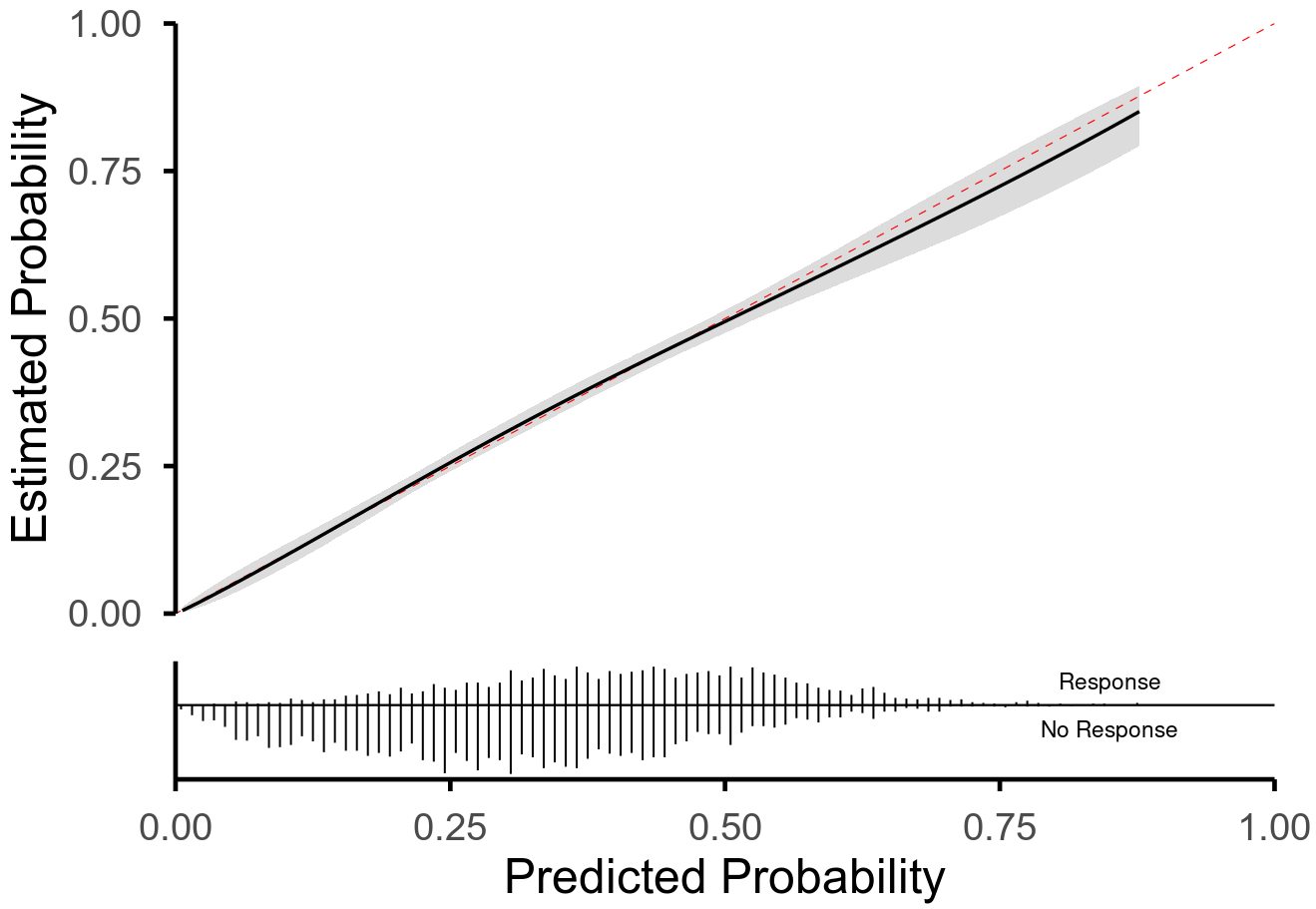
#----------------------------------------
# Prediction distribution plot
#----------------------------------------

p_dist <- ggplot() +
  geom_hline(yintercept = 0, linewidth = 0.4) +
  geom_segment(
    data = filter(dist_dat, y == 1),
    aes(x = bin, xend = bin, y = 0, yend = n_scaled),
    linewidth = 0.35
    ) +
  geom_segment(
    data = filter(dist_dat, y == 0),
    aes(x = bin, xend = bin, y = 0, yend = -n_scaled),
    linewidth = 0.35
    ) +
  annotate("text", x = label_x, y = 0.35,
           label = "Response", size = 3) +
  annotate("text", x = label_x, y = -0.35,
           label = "No Response", size = 3) +
  scale_x_continuous(limits = c(0, 1), expand = c(0, 0)) +
  labs(x = "Predicted Probability") +
  theme_minimal(base_size = 18) +
  theme(
    text = element_text(family = "Arial"),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
    axis.ticks = element_line(color = "black"),
    axis.title.x = element_text(family = "Arial"),
    axis.title.y = element_blank(),
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank()
    )
```

```
  p_cal / p_dist + plot_layout(heights = c(5, 1))
}

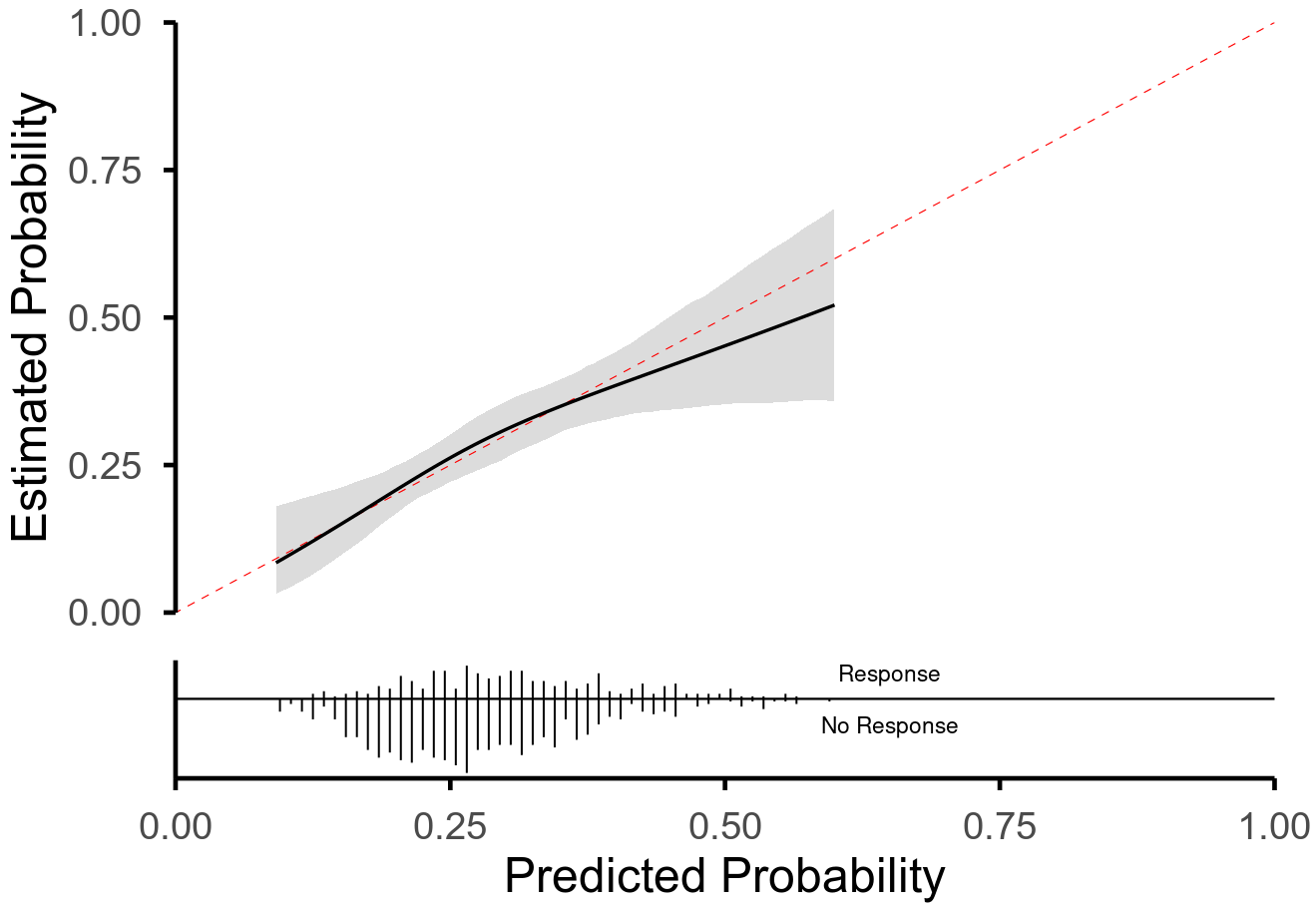
# Extract data from pmcalibration() objects
avp_cal_dat <- get_curve(avp_lr_cal)
epi_cal_dat <- get_curve(epi_xgb_cal)
pe_cal_dat <- get_curve(pe_rf_cal)

# Plot and save
(p_avp <- plot_calibration(avp_lr_cal, avp_cal_dat, bin_width = 0.01))
```



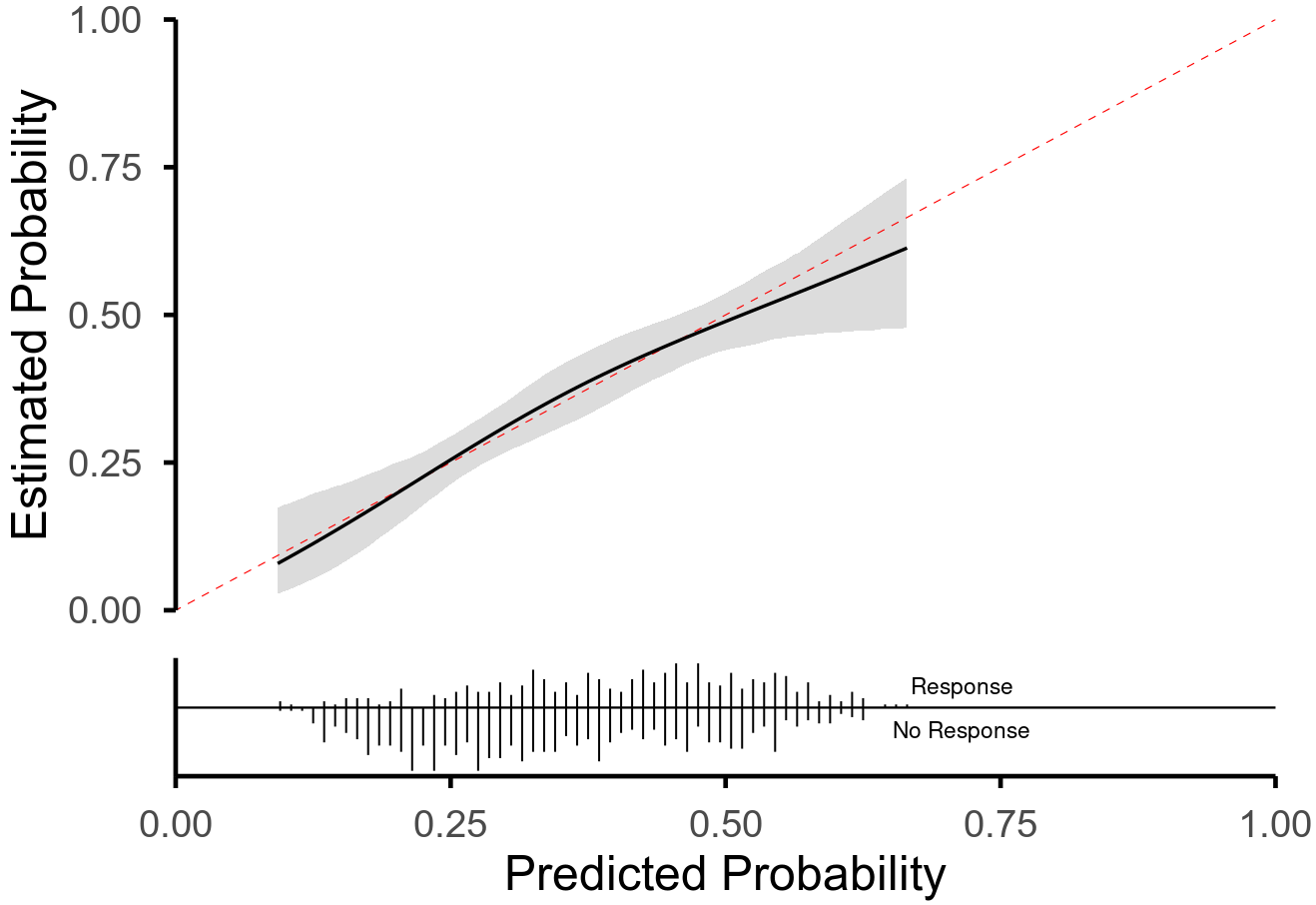
▼ Code

```
(p_epi <- plot_calibration(epi_xgb_cal, epi_cal_dat, bin_width = 0.01))
```



▼ Code

```
(p_pe  <- plot_calibration(pe_rf_cal, pe_cal_dat, bin_width = 0.01))
```



▼ Code

```
ggsave(
  "avp_calibration.tiff",
  plot = p_avp,
  device = ragg::agg_tiff,
  width = 6, height = 6,
  units = "in", dpi = 300
  )

ggsave(
  "epi_calibration.tiff",
  plot = p_epi,
  device = ragg::agg_tiff,
  width = 6, height = 6,
  units = "in", dpi = 300
  )

ggsave(
  "pe_calibration.tiff",
  plot = p_pe,
  device = ragg::agg_tiff,
  width = 6, height = 6,
```

```
  units = "in", dpi = 300
  )
```

13.4 Decision Curve Analyses

Show decision curve analyses of only the final/selected model by adjunctive vasopressor.

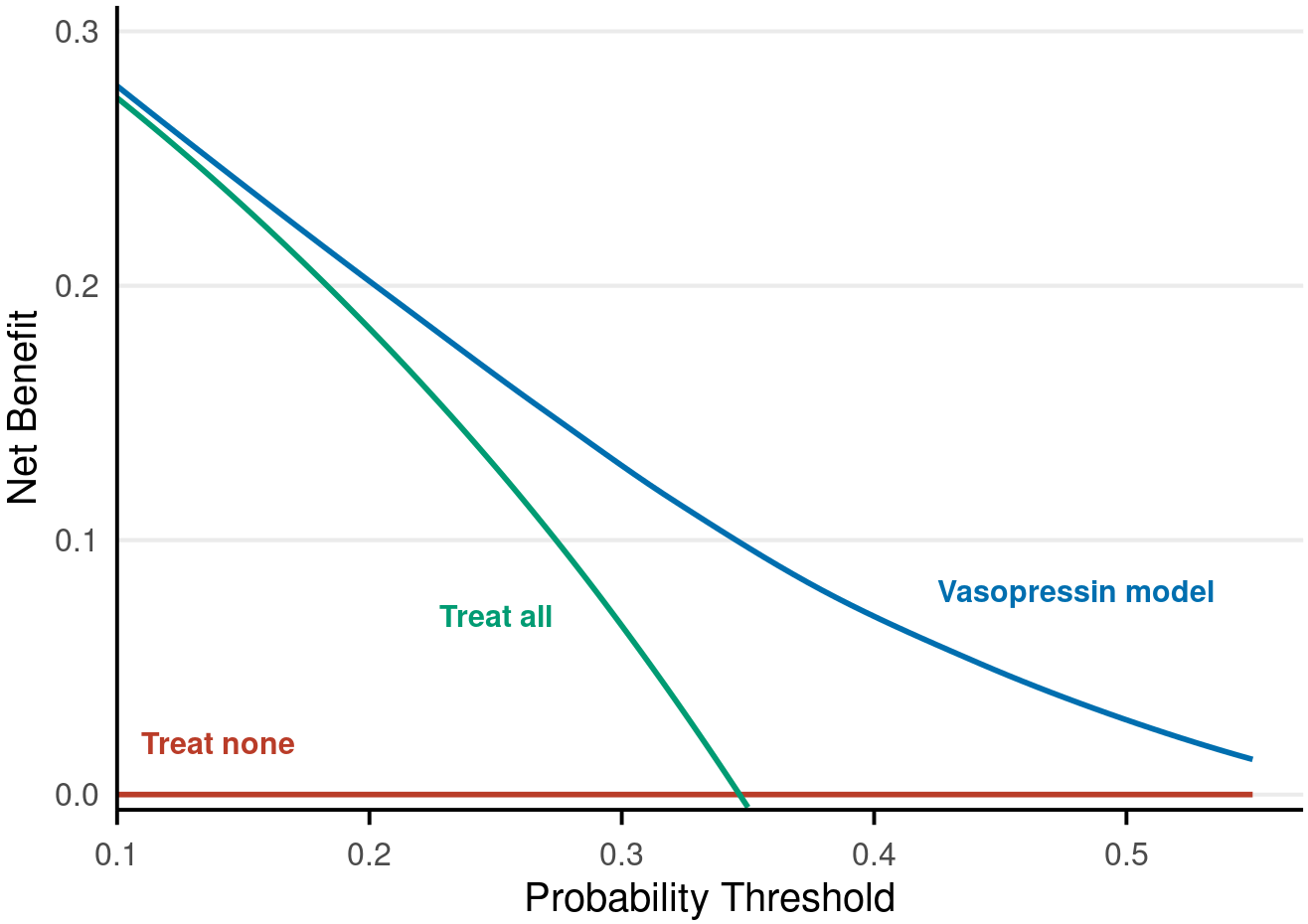
▼ Code

```
# Vasopressin
avp_dca_data <- avp_m1_lr_preds |>
  filter(wflow_id == "avp_m1_ns_logistic") |>
  group_by(.row) |>
  summarise(
    response = first(response),
    p_logistic = mean(.pred_Response),
    .groups = "drop"
  )

avp_dca_final <- dca(
  response == "Response" ~ p_logistic,
  data = avp_dca_data
  ) |>
  as_tibble()

(avp_dca_plot_final <-
  ggplot(avp_dca_final |>
           filter(net_benefit >= -0.006 &
                    net_benefit <= 0.31 &
                    threshold <=0.71
                  ) |>
           mutate(label = fct_relevel(label, "Treat None", "Treat All",
                                      "p_logistic"
                                      )),
         aes(x = threshold,
             y = net_benefit,
             color = label,
             group = label)) +
    geom_smooth(method = loess, formula = y ~ x,
                linewidth = 1, se = FALSE) +
    scale_color_manual(values = c(
      "p_logistic" = "#0072B2",
      "Treat All" = "#009E73",
      "Treat None" = "#BC3C29FF"
      ),
      labels = c(
        "p_logistic" = "Vasopressin model",
        "Treat All" = "Treat all",
        "Treat None" = "Treat none"
```

```
  )) +
scale_y_continuous(
  limits = c(-0.006, 0.31),
  expand = expansion(add = c(0, 0))
  ) +
scale_x_continuous(
  limits = c(0.1, 0.55),
  breaks = seq(0, 0.55, by = 0.1),
  expand = expansion(add = c(0, 0.02))
  ) +
labs(
  x = "Probability Threshold",
  y = "Net Benefit",
  color = NULL
  ) +
annotate("text", x = 0.48, y = 0.08, label = "Vasopressin model",
         color = "#0072B2", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.25, y = 0.07, label = "Treat all",
         color = "#009E73", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.14, y = 0.02, label = "Treat none",
         color = "#BC3C29FF", family = "sans", fontface = "bold",
         size = 4
         ) +
theme_minimal(base_size = 15) +
theme(
  text = element_text(family = "sans"),
  legend.position = "none",
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x = element_line(),
  axis.line.y = element_line(),
  axis.ticks.x = element_line(),
  axis.ticks.length = unit(0.2, "cm"),
  axis.title.x = element_text(family = "sans"),
  axis.title.y = element_text(family = "sans"),
  axis.text.x  = element_text(family = "sans"),
  axis.text.y  = element_text(family = "sans")
))
```

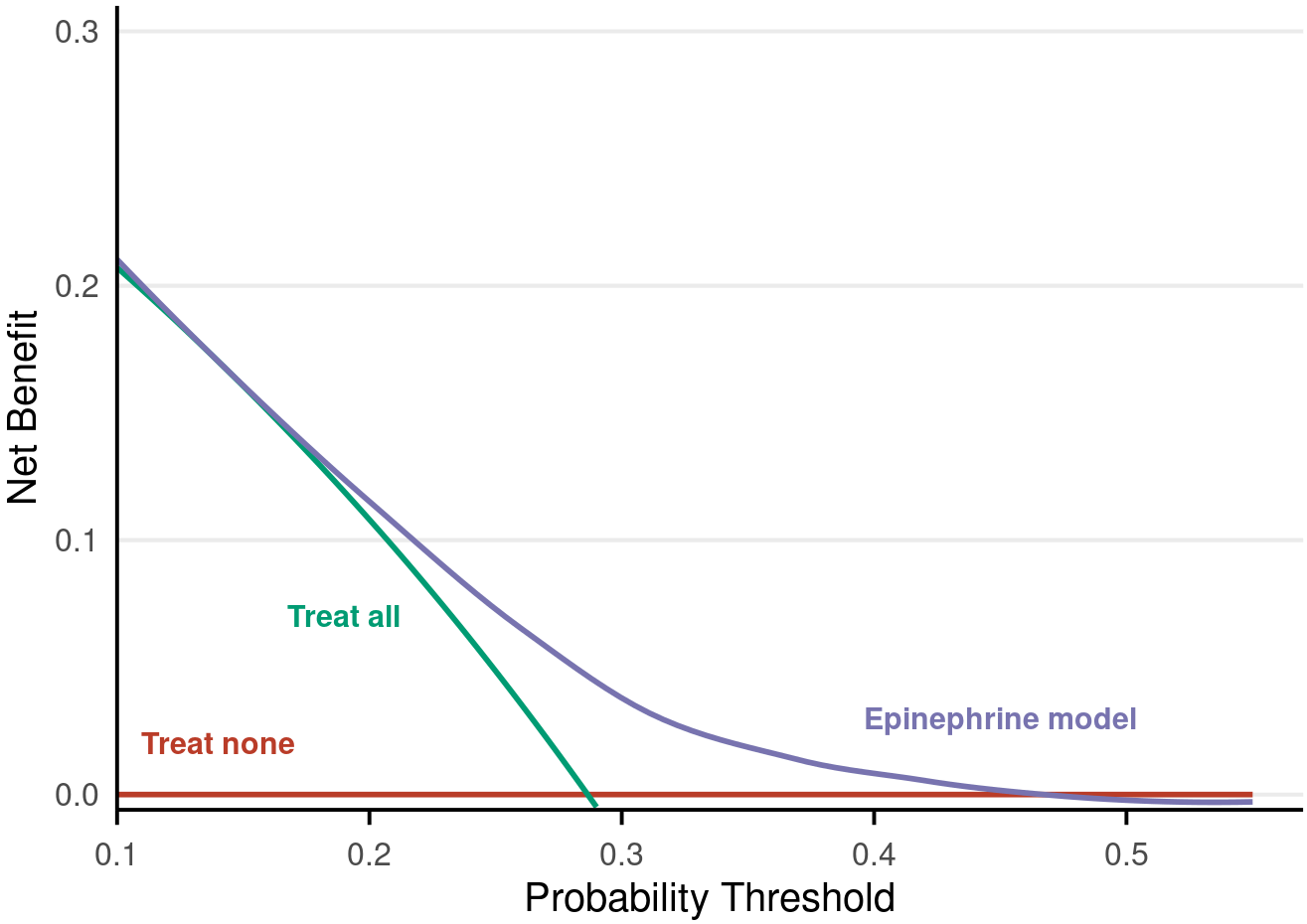


▼ Code

```
# Epinephrine
epi_dca_final <- dca(
  response == "Response" ~ p_xgb,
  data = epi_dca_data
  ) |>
  as_tibble()

(epi_dca_plot_final <-
    ggplot(as_tibble(epi_dca_final) |>
             filter(net_benefit >= -0.006 &
                      net_benefit <= 0.31 &
                    threshold <=0.71
                    ) |>
             mutate(label = fct_relevel(label, "Treat None", "Treat All",
                                        "p_xgb"
                                        )),
           aes(x = threshold,
               y = net_benefit,
               color = label,
               group = label)) +
    geom_smooth(method = loess, formula = y ~ x, se = FALSE, linewidth = 1) +
    scale_color_manual(values = c(
```

```
  "p_xgb" = "#7876B1FF",
  "Treat All" = "#009E73",
  "Treat None" = "#BC3C29FF"
  ),
  labels = c(
    "p_xgb" = "XGBoost",
    "Treat All" = "Treat all",
    "Treat None" = "Treat none"
  )) +
scale_y_continuous(
  limits = c(-0.006, 0.31),
  expand = expansion(add = c(0, 0))
  ) +
scale_x_continuous(
  limits = c(0.1, 0.55),
  breaks = seq(0,0.55, by = 0.1),
  expand = expansion(add = c(0, 0.02))
  ) +
labs(
  x = "Probability Threshold",
  y = "Net Benefit",
  color = NULL
  ) +
annotate("text", x = 0.45, y = 0.03, label = "Epinephrine model",
         color = "#7876B1FF", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.19, y = 0.07, label = "Treat all",
         color = "#009E73", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.14, y = 0.02, label = "Treat none",
         color = "#BC3C29FF", family = "sans", fontface = "bold",
         size = 4
         ) +
theme_minimal(base_size = 15) +
theme(
  legend.position = "none",
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x = element_line(),
  axis.line.y = element_line(),
  axis.ticks.x = element_line(),
  axis.ticks.length = unit(0.2, "cm"),
  axis.title.x = element_text(family = "sans"),
  axis.title.y = element_text(family = "sans"),
  axis.text.x  = element_text(family = "sans"),
  axis.text.y  = element_text(family = "sans")
))
```

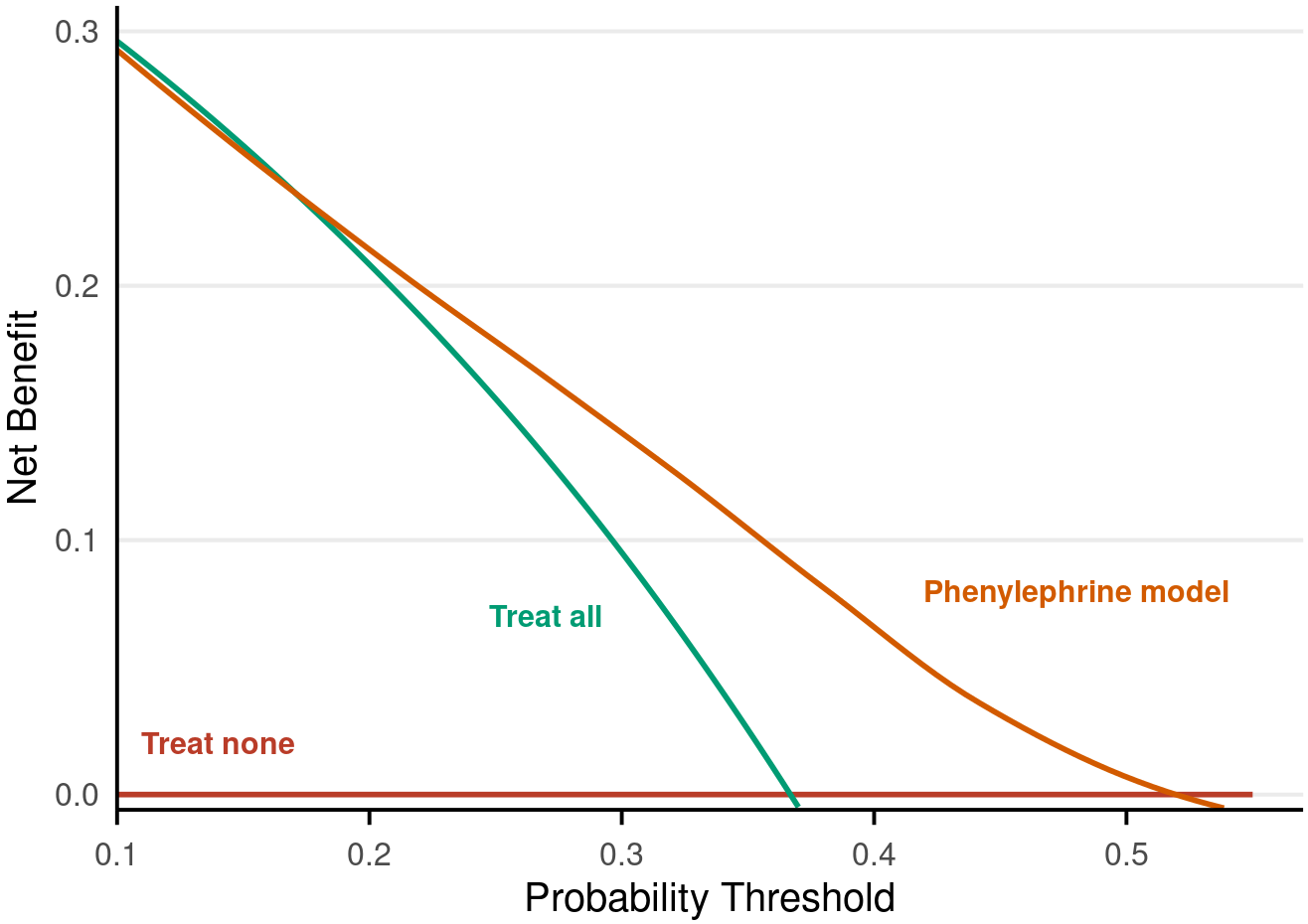


▼ Code

```
# Phenylephrine
pe_dca_final <- dca(
  response == "Response" ~ p_rf,
  data = pe_dca_data
  ) |>
  as_tibble()

(pe_dca_plot_final <-
    ggplot(as_tibble(pe_dca_final) |>
             filter(net_benefit >= -0.006 &
                      net_benefit <= 0.31 &
                      threshold <=0.71
                    ) |>
             mutate(label = fct_relevel(label, "Treat None", "Treat All",
                                        "p_rf"
                                        )),
           aes(x = threshold,
               y = net_benefit,
               color = label,
               group = label)) +
    geom_smooth(method = loess, formula = y ~ x, se = FALSE, linewidth = 1) +
    scale_color_manual(values = c(
```

```
  "p_rf" = "#D55E00",
  "Treat All" = "#009E73",
  "Treat None" = "#BC3C29FF"
  ),
  labels = c(
    "p_rf" = "Random forest",
    "Treat All" = "Treat all",
    "Treat None" = "Treat none"
  )) +
scale_y_continuous(
  limits = c(-0.006, 0.31),
  expand = expansion(add = c(0, 0))
  ) +
scale_x_continuous(
  limits = c(0.1, 0.55),
  breaks = seq(0,0.55, by = 0.1),
  expand = expansion(add = c(0, 0.02))
  ) +
labs(
  x = "Probability Threshold",
  y = "Net Benefit",
  color = NULL
  ) +
annotate("text", x = 0.48, y = 0.08, label = "Phenylephrine model",
         color = "#D55E00", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.27, y = 0.07, label = "Treat all",
         color = "#009E73", family = "sans", fontface = "bold",
         size = 4
         ) +
annotate("text", x = 0.14, y = 0.02, label = "Treat none",
         color = "#BC3C29FF", family = "sans", fontface = "bold",
         size = 4
         ) +
theme_minimal(base_size = 15) +
theme(
  legend.position = "none",
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x = element_line(),
  axis.line.y = element_line(),
  axis.ticks.x = element_line(),
  axis.ticks.length = unit(0.2, "cm"),
  theme_minimal(base_size = 15),
  axis.title.x = element_text(family = "sans"),
  axis.title.y = element_text(family = "sans"),
  axis.text.x  = element_text(family = "sans"),
  axis.text.y  = element_text(family = "sans")
  ))
```



Save plots for combining as a panel figure.

▼ Code

```
ggsave(
  "avp_dca_plot_final.tiff",
  plot = avp_dca_plot_final,
  device = ragg::agg_tiff,
  width = 6,
  height = 5,
  units = "in",
  dpi = 300
  )

ggsave(
  "epi_dca_plot_final.tiff",
  plot = epi_dca_plot_final,
  device = ragg::agg_tiff,
  width = 6,
  height = 5,
  units = "in",
  dpi = 300
  )
```

```
ggsave(
  "pe_dca_plot_final.tiff",
  plot = pe_dca_plot_final,
  device = ragg::agg_tiff,
  width = 6,
  height = 5,
  units = "in",
  dpi = 300
  )
```

13.5 Model Performance by Subgroup

Here, assess model performance (AUROC and Brier score) across clinically important subgroups. Utilize the out-of-fold predictions from each model, which corrects for optimism. Specifically, evaluate subgroup performance using out-of-fold predictions 10-fold cross-validation averaged over the 5 repeats. Then, obtain confidence intervals using 500 bootstrap resamples within each subgroup.

First, define a function to evaluate the models through bootstrap resampling (500 resamples). Then, define a function to create a reusable prediction helper.

▼ Code

```
boot_classification_by_group <- function(
    preds,
    outcome,
    prob_col,
    group_var,
    positive_level = "Response",
    B = 500,
    seed = 2026
    ) {

  set.seed(seed)

  split_data <- split(preds, preds[[group_var]])

  results <- map(
    split_data,
    function(dat) {
      n <- nrow(dat)
      group_level <- dat[[group_var]][1]
      boot_res <- replicate(
        B,
        {idx <- sample.int(n, replace = TRUE)
          boot_dat <- dat[idx, ]
          auc_val <-
            if (length(unique(boot_dat[[outcome]])) < 2) {
```

```
        NA_real_
        } else {
          roc_auc_vec(
            truth = boot_dat[[outcome]],
            estimate = boot_dat[[prob_col]],
            event_level = "first"
            )
          }
    brier_val <-
      mean(
        (as.integer(boot_dat[[outcome]] == positive_level) -
           boot_dat[[prob_col]]
         )^2
        )
    c(auc = auc_val, brier = brier_val)
    }
  )

boot_res <- t(boot_res) |> as_tibble()

auc_na_prop <-
  if (length(boot_res$auc) > 0) {
    mean(is.na(boot_res$auc))
    } else {
      NA_real_
      }
auc_hat <-
  roc_auc_vec(truth = dat[[outcome]], estimate = dat[[prob_col]],
              event_level = "first"
              )
brier_hat <-
  mean(
    (as.integer(dat[[outcome]] == positive_level) - dat[[prob_col]])^2
    )
auc_ci <-
  quantile(boot_res$auc, probs = c(0.025, 0.975), na.rm = TRUE)
brier_ci <-
  quantile(boot_res$brier, probs = c(0.025, 0.975), na.rm = TRUE)
list(
  summary =
    tibble(
      !!group_var := group_level,
      n = n,
      events = sum(dat[[outcome]] == positive_level),
      event_rate = events / n,
      auc_est = auc_hat,
      auc_low = auc_ci[1],
      auc_high = auc_ci[2],
      brier_est = brier_hat,
      brier_low = brier_ci[1],
      brier_high = brier_ci[2],
```

```
                        auc_na_prop = auc_na_prop
            ),
        boot =
          boot_res |>
          mutate(!!group_var := group_level)
        )
      }
    )

  list(
    summary = bind_rows(map(results, "summary")),
    boot = bind_rows(map(results, "boot"))
    )
  }

get_group_predictions <- function(
    fit_obj, data, outcome_var, group_var
    ) {

  probs <-
    predict(
      fit_obj,
      new_data = data,
      type = "prob"
      )
  bind_cols(
    data |> select(all_of(outcome_var), all_of(group_var)),
    probs
    )
  }
```

Apply the functions to relevant subgroups; age (<65 vs. ≥65 years), sex, and race/ethnicity (using collapsed categories to avoid small counts within levels).

▼ Code

```
# Add the variable back to predictions and collapse levels for interpretability
avp_fair_preds <-
  avp_m1_lr_preds |>
  left_join(
    avp |>
      mutate(.row = row_number()) |>
      select(.row, age, sex, race_eth),
    by = ".row"
    ) |>
  group_by(.row) |>
  summarise(
    response = first(response),
    age = first(age),
    sex = first(sex),
```

```
    race_eth = first(race_eth),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
    ) |>
  mutate(
    race_eth_collapsed = fct_collapse(
      race_eth,
      "Hispanic or Other race" = c("Hispanic or Latino", "Other, non-Hispanic",
                                   "Multiracial, non-Hispanic"
                                   )
      ),
    age_group = factor(if_else(age <65, "<65 years", "≥65 years"),
                       levels = c("<65 years", "≥65 years")
                       )
    ) |>
  filter(!is.na(race_eth_collapsed) & !is.na(sex))

epi_fair_preds <-
  epi_m4_xgb_final_fit[[1]]$cv_predictions |>
  left_join(
    epi |>
      mutate(.row = row_number()) |>
      select(.row, age, sex, race_eth),
    by = ".row"
    ) |>
  group_by(.row) |>
  summarise(
    response = first(response),
    age = first(age),
    sex = first(sex),
    race_eth = first(race_eth),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
    ) |>
  mutate(
    race_eth_collapsed = fct_collapse(
      race_eth,
      "Hispanic or Other race" = c("Hispanic or Latino", "Other, non-Hispanic",
                                   "Multiracial, non-Hispanic"
                                   )
      ),
    age_group = factor(if_else(age <65, "<65 years", "≥65 years"),
                       levels = c("<65 years", "≥65 years")
                       )
    ) |>
  filter(!is.na(race_eth_collapsed) & !is.na(sex))

pe_fair_preds <-
  pe_m3_rf_final_fit[[1]]$cv_predictions |>
  left_join(
    pe |>
```

```
      mutate(.row = row_number()) |>
      select(.row, age, sex, race_eth),
    by = ".row"
    ) |>
  group_by(.row) |>
  summarise(
    response = first(response),
    age = first(age),
    sex = first(sex),
    race_eth = first(race_eth),
    .pred_Response = mean(.pred_Response),
    .groups = "drop"
    ) |>
  mutate(
    race_eth_collapsed = fct_collapse(
      race_eth,
      "Hispanic or Other race" = c("Hispanic or Latino", "Other, non-Hispanic",
                                   "Multiracial, non-Hispanic"
                                   )
      ),
    age_group = factor(if_else(age <65, "<65 years", "≥65 years"),
                       levels = c("<65 years", "≥65 years")
                       )
    ) |>
  filter(!is.na(race_eth_collapsed) & !is.na(sex))

# Pass the predictions to the bootstrap function for each strata
avp_age_perf <-
  boot_classification_by_group(
    preds = avp_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
    group_var = "age_group",
    positive_level = "Response",
    B = 500
  )

avp_sex_perf <-
  boot_classification_by_group(
    preds = avp_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
    group_var = "sex",
    positive_level = "Response",
    B = 500
  )

avp_race_perf <-
  boot_classification_by_group(
    preds = avp_fair_preds,
    outcome = "response",
```

```
    prob_col = ".pred_Response",
    group_var = "race_eth_collapsed",
    positive_level = "Response",
    B = 500
    )

epi_age_perf <-
  boot_classification_by_group(
    preds = epi_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
    group_var = "age_group",
    positive_level = "Response",
    B = 500
  )

epi_sex_perf <-
  boot_classification_by_group(
    preds = epi_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
    group_var = "sex",
    positive_level = "Response",
    B = 500
  )

epi_race_perf <-
  boot_classification_by_group(
    preds = epi_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
    group_var = "race_eth_collapsed",
    positive_level = "Response",
    B = 500
    )

pe_age_perf <-
  boot_classification_by_group(
    preds = pe_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
    group_var = "age_group",
    positive_level = "Response",
    B = 500
  )

pe_sex_perf <-
  boot_classification_by_group(
    preds = pe_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
```

```
    group_var = "sex",
    positive_level = "Response",
    B = 500
  )

pe_race_perf <-
  boot_classification_by_group(
    preds = pe_fair_preds,
    outcome = "response",
    prob_col = ".pred_Response",
    group_var = "race_eth_collapsed",
    positive_level = "Response",
    B = 500
    )
```

Combine results into one tibble

▼ Code

```
perf_mets <- bind_rows(
  bind_rows(
    avp_age_perf$summary |>
      rename(group = age_group) |>
      mutate(variable = "Age"),
    avp_sex_perf$summary |>
      rename(group = sex) |>
      mutate(variable = "Sex"),
    avp_race_perf$summary |>
      rename(group = race_eth_collapsed) |>
      mutate(variable = "Race/Ethnicity")
    ) |>
    mutate(medication = "Vasopressin"),

  bind_rows(
    epi_age_perf$summary |>
      rename(group = age_group) |>
      mutate(variable = "Age"),
    epi_sex_perf$summary |>
      rename(group = sex) |>
      mutate(variable = "Sex"),
    epi_race_perf$summary |>
      rename(group = race_eth_collapsed) |>
      mutate(variable = "Race/Ethnicity")
    ) |>
    mutate(medication = "Epinephrine"),

  bind_rows(
    pe_age_perf$summary |>
      rename(group = age_group) |>
      mutate(variable = "Age"),
```

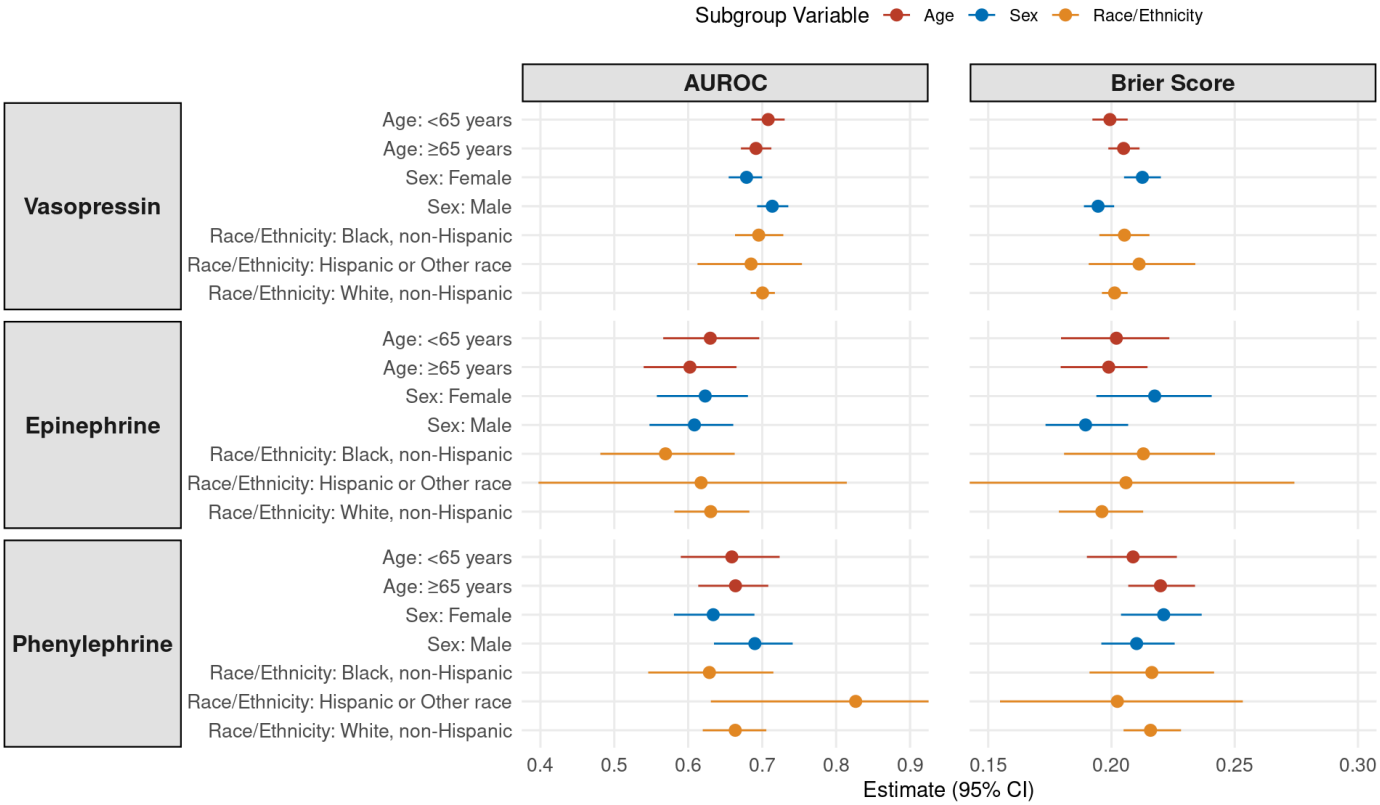
```
    pe_sex_perf$summary |>
      rename(group = sex) |>
      mutate(variable = "Sex"),
    pe_race_perf$summary |>
      rename(group = race_eth_collapsed) |>
      mutate(variable = "Race/Ethnicity")
    ) |>
    mutate(medication = "Phenylephrine")
  ) |>
  mutate(
    group_label = paste0(variable, ": ", group),
    group_order = factor(
      group_label,
      levels = c("Age: <65 years", "Age: ≥65 years", "Sex: Female",
                 "Sex: Male", "Race/Ethnicity: Black, non-Hispanic",
                 "Race/Ethnicity: Hispanic or Other race",
                 "Race/Ethnicity: White, non-Hispanic"
                 )
      )
  ) |>
  select(medication, variable, group_label, auc_est, auc_low, auc_high,
         brier_est, brier_low, brier_high, group_order
         ) |>
  pivot_longer(
    cols = matches("^(auc|brier)_"),
    names_to = c("metric", ".value"),
    names_pattern = "(auc|brier)_(est|low|high)"
    ) |>
  mutate(
    metric = recode(metric, auc = "AUROC", brier = "Brier Score"),
    medication = factor(
      medication,
      levels = c("Vasopressin", "Epinephrine", "Phenylephrine")
      ),
    variable = factor(variable, levels = c("Age", "Sex", "Race/Ethnicity"))
    )
```

Lastly, plot the results

▼ Code

```
perf_mets |>
  ggplot(aes(x = est, y = fct_rev(group_order), color = variable)) +
  geom_point(size = 2.5) +
  geom_errorbar(aes(xmin = low, xmax = high), width = 0, orientation = "y") +
  facet_grid(
    medication ~ metric,
    scales = "free_x",
    switch = "y"
    ) +
```

```
scale_color_nejm() +
labs(
  x = "Estimate (95% CI)",
  y = NULL,
  color = "Subgroup Variable"
  ) +
guides(color = guide_legend(nrow = 1)) +
theme_minimal() +
theme(
  strip.placement = "outside",
  strip.background = element_rect(
    fill = "grey90",
    color = "black", linewidth = 0.8
    ),
  strip.text.y.left = element_text(
    angle = 0,
    face = "bold",
    size = 12
    ),
  strip.text.x = element_text(
    face = "bold",
    size = 12
    ),
  axis.text.y = element_text(size = 10),
  axis.text.x = element_text(size = 10),
  panel.grid.minor = element_blank(),
  legend.position = "top",
  legend.direction = "horizontal",
  panel.spacing.x = unit(0.75, "cm")
  ) +
facetted_pos_scales(
  x = list(
    metric == "AUROC" ~
      scale_x_continuous(
        limits = c(0.4, 0.9),
        breaks = seq(0.4, 0.9, 0.1)
        ),
    metric == "Brier Score" ~
      scale_x_continuous(
        limits = c(0.15, 0.30),
        breaks = seq(0.15, 0.30, 0.05)
        )
    )
  )
```



▼ Code

```
# Save plot
ggsave(
  "fairness_plot.tiff",
  device = ragg::agg_tiff,
  width = 9,
  height = 6,
  units = "in",
  dpi = 300
  )
```

There are no concerns for model fairness across subgroups.

14 Decision Model Validation

The decision model will be validated in the validation (`fl`) dataset using a discordance analysis framework as described in Dalton 2016. First, the dataset will be described. Next, the final models for each vasopressor (vasopressin == logistic regression with spline term (`avp_m1_ns_logistic`), epinephrine == gradient boosted trees with XGBoost (`epi_m4_boost_xgboost`), phenylephrine == random forest (`pe_m3_rf_ranger`)) will be applied to each observation in the validation dataset. The vasopressor with the highest predicted probability of response will be considered the decision model-recommended vasopressor. Then, this recommended therapy will be compared to the observed treatment the patient received during usual care. Concordance will be defined as receipt of the adjunctive vasopressor recommended by the decision model, whereas discordance will be

defined as receipt of a different adjunctive vasopressor. Observed outcomes will be compared between concordant and discordant treatment decisions. The discordance analysis will estimate the global effectiveness of the decision model, defined as the odds ratio for hemodynamic response at 6 hours comparing concordant and discordant therapy using logistic regression.

Subgroup and sensitivity analyses for the decision model effectiveness will be performed. In the first subgroup analysis, concordance effectiveness will be evaluated according to the specific adjunctive vasopressor recommended by the decision model. In the second and third subgroup analyses, concordance effectiveness will be assessed in relation to decision model recommendation strength, defined by the difference in predicted response probabilities between the highest- and second-highest-ranked therapies and by quartiles of the corresponding decision model odds ratio, respectively. In the first sensitivity analysis, practical equivalence of probabilities will be considered for the medications with difference in predicted response probability zones of ≤0.05 or ≤0.10 compared with the highest-ranked therapy. Receipt of any therapy with a probability in the zone will be considered concordant. In the second sensitivity analysis, only vasopressin and epinephrine will be considered eligible adjunctive vasopressors in the decision model, consistent with current guideline recommendations.

14.1 Validation Cohort Patient Characteristics

Describe the characteristics of included patients in the validation (Florida) cohort by adjunctive vasopressor agent. Notably, the below table includes characteristics that were not used as model variables (predictors). Further, because antimicrobials are components of the clinical criteria (CDC ASE and Sepsis-3) used to identify patients with septic shock, each included patient received antimicrobials as part of their sepsis treatment course.

▼ Code

```
(val_char_by_adj <- incl |>
   filter(first_adj_hosp_state == "Florida") |>
   droplevels() |>
   mutate(first_adj_hosp_beds_cat =
            fct_relevel(first_adj_hosp_beds_cat, "<200", "200-499"),
          ssc_era = fct_relevel(ssc_era, "SSC 2008", "SSC 2012", "SSC 2016"),
          adv_resp_before_adj =
            fct_relevel(adv_resp_before_adj, "Adv Resp Before Adjunct"),
          mv_before_adj = fct_relevel(mv_before_adj, "MV Before Adjunct"),
          response = fct_relevel(response, "Response"),
          first_adj = fct_recode(first_adj, "Vasopressin" = "vasopressin",
                                 "Epinephrine" = "epinephrine",
                                 "Phenylephrine" = "phenylephrine"
                                 )
          ) |>
  select(age, sex, first_adj_wt_kg, race_eth, elixhauser_index,
         cerebrovascular_disease, hypertension, heart_failure,
         valvular_disease, chronic_pulmonary_disease,
         pulmonary_circulation_disease, liver_disease, kidney_disease,
```

```
      malignancy, diabetes, obesity, alcohol_abuse, first_adj_lvef,
      first_adj_hosp_beds_cat, first_adj_hosp_teaching_status,
      first_adj_tx_unit, ssc_era,  criteria_met, comm_onset, prior_surg,
      first_adj_sofa_2, first_adj_abx, adv_resp_before_adj, mv_before_adj,
      first_adj_pH, first_adj_aki, rrt_before_adj, first_adj_temp_c,
      first_adj_heart_rate, first_adj_resp_rate, first_adj_map,
      first_adj_lactate, first_adj_lvef, first_adj_fluid_mL,
      first_adj_fluid_mL_kg, adj_baseline_neq, adj_baseline_neq_mcg_min,
      dbt_before_adj, first_adj_steroid, first_adj, hours_to_first_adj,
      abx_days, mort_hosp, response
      ) |>
tbl_summary(
  by = first_adj,
  digits = list(
    all_categorical() ~ c(0, 1),
    first_adj_sofa_2 ~ 0,
    first_adj_temp_c ~ 1
    ),
  label = list(response = "Adjunct Response",
               age = "Age",
               sex = "Sex",
               race_eth = "Race/Ethnicity",
               prior_surg = "Prior surgery",
               elixhauser_index = "Elixhauser comorbidity index",
               ssc_era = "Surviving Sepsis Campaign guideline treatment era",
               wt_kg = "Weight (kg)",
               first_adj_hosp_beds_cat = "Hospital beds",
               first_adj_hosp_teaching_status = "Teaching status",
               first_adj_tx_unit = "Treatment unit",
               criteria_met = "Clinical criteria met",
               first_adj_lvef = "LVEF (%)",
               first_adj_sofa_2 = "SOFA-2",
               adv_resp_before_adj = "Advanced respiratory support",
               mv_before_adj = "Mechanical ventilation",
               rrt_before_adj = "Kidney replacement therapy",
               first_adj_fluid_mL = "Fluid bolus volume administered (mL)",
               first_adj_fluid_mL_kg = "Fluid bolus volume administered (mL/kg)",
               first_adj_steroid = "Corticosteroid administration",
               adj_baseline_neq = "NE dosage (mcg/kg/min)",
               adj_baseline_neq_mcg_min = "NE dosage (mcg/min)",
               pressors_dur_hours = "Vasopressor duration (hours)",
               abx_days = "Antibiotic duration (days)",
               mort_hosp = "In-Hospital mortality",
               first_adj = "First Adjunctive Vasopressor",
               dbt_before_adj = "Dobutamine administration"
               ),
 missing = "ifany"
  ) |>
 add_overall() |>
 modify_fmt_fun(
   all_stat_cols() ~ function(x) gsub("%", "", x)
```

```
  ) |>
  modify_caption("**Patient Characteristics in Validation (Florida) Dataset by Adjunct**")
)
```

Patient Characteristics in Validation (Florida) Dataset by Adjunct

Characteristic	Overall N = 1,128[1]	Vasopressin N = 976[1]	Epinephrine N = 95[1]	Phenylephrine N = 57[1]
Age	69 (59, 77)	69 (59, 77)	68 (58, 78)	71 (60, 79)
Sex				
Female	471 (41.8)	413 (42.3)	32 (33.7)	26 (45.6)
Male	657 (58.2)	563 (57.7)	63 (66.3)	31 (54.4)
first_adj_wt_kg	80 (67, 96)	80 (67, 96)	80 (68, 96)	75 (59, 92)
Unknown	13	8	2	3
Race/Ethnicity				
Black, non-Hispanic	150 (13.5)	139 (14.4)	8 (8.5)	3 (5.6)
Hispanic or Latino	200 (18.0)	172 (17.9)	22 (23.4)	6 (11.1)
Multiracial, non-Hispanic	19 (1.7)	18 (1.9)	1 (1.1)	0 (0.0)
Other, non-Hispanic	21 (1.9)	17 (1.8)	2 (2.1)	2 (3.7)
White, non-Hispanic	721 (64.9)	617 (64.1)	61 (64.9)	43 (79.6)
Unknown	17	13	1	3
Elixhauser comorbidity index	5 (0, 17)	5 (0, 17)	3 (0, 16)	2 (0, 23)
cerebrovascular_disease	71 (6.3)	58 (5.9)	8 (8.4)	5 (8.8)
hypertension	355 (31.5)	303 (31.0)	30 (31.6)	22 (38.6)
heart_failure	175 (15.5)	142 (14.5)	24 (25.3)	9 (15.8)
valvular_disease	104 (9.2)	81 (8.3)	15 (15.8)	8 (14.0)
chronic_pulmonary_disease	111 (9.8)	98 (10.0)	8 (8.4)	5 (8.8)
pulmonary_circulation_disease	66 (5.9)	61 (6.3)	3 (3.2)	2 (3.5)

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 1,128[1]	Vasopressin N = 976[1]	Epinephrine N = 95[1]	Phenylephrine N = 57[1]
liver_disease	117 (10.4)	104 (10.7)	5 (5.3)	8 (14.0)
kidney_disease	160 (14.2)	141 (14.4)	11 (11.6)	8 (14.0)
malignancy	231 (20.5)	209 (21.4)	12 (12.6)	10 (17.5)
diabetes	166 (14.7)	143 (14.7)	13 (13.7)	10 (17.5)
obesity	162 (14.4)	146 (15.0)	9 (9.5)	7 (12.3)
alcohol_abuse	35 (3.1)	28 (2.9)	3 (3.2)	4 (7.0)
LVEF (%)	56 (40, 64)	58 (41, 65)	55 (30, 60)	53 (39, 56)
Unknown	683	602	45	36
Hospital beds				
<200	92 (8.2)	73 (7.5)	7 (7.4)	12 (21.1)
200-499	1,036 (91.8)	903 (92.5)	88 (92.6)	45 (78.9)
Teaching status				
Teaching	783 (69.4)	693 (71.0)	63 (66.3)	27 (47.4)
Non-teaching	345 (30.6)	283 (29.0)	32 (33.7)	30 (52.6)
Treatment unit				
ED/Rapid Response	125 (11.1)	105 (10.8)	17 (17.9)	3 (5.3)
Medical ICU	529 (46.9)	501 (51.3)	15 (15.8)	13 (22.8)
Mixed Med/Surg ICU	202 (17.9)	169 (17.3)	9 (9.5)	24 (42.1)
Surgical ICU	272 (24.1)	201 (20.6)	54 (56.8)	17 (29.8)
Surviving Sepsis Campaign guideline treatment era				
SSC 2008	19 (1.7)	17 (1.7)	0 (0.0)	2 (3.5)
SSC 2012	162 (14.4)	148 (15.2)	11 (11.6)	3 (5.3)

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 1,128[1]	Vasopressin N = 976[1]	Epinephrine N = 95[1]	Phenylephrine N = 57[1]
SSC 2016	340 (30.1)	293 (30.0)	34 (35.8)	13 (22.8)
SSC 2021	607 (53.8)	518 (53.1)	50 (52.6)	39 (68.4)
Clinical criteria met				
Both	642 (56.9)	569 (58.3)	39 (41.1)	34 (59.6)
ASE only	389 (34.5)	335 (34.3)	33 (34.7)	21 (36.8)
Sep3 only	97 (8.6)	72 (7.4)	23 (24.2)	2 (3.5)
comm_onset				
Community	842 (74.6)	724 (74.2)	70 (73.7)	48 (84.2)
Hospital	286 (25.4)	252 (25.8)	25 (26.3)	9 (15.8)
Prior surgery				
No Surgery	921 (81.6)	831 (85.1)	48 (50.5)	42 (73.7)
Surgery	207 (18.4)	145 (14.9)	47 (49.5)	15 (26.3)
SOFA-2	8 (6, 10)	8 (6, 10)	7 (5, 9)	7 (5, 10)
first_adj_abx				
Antibiotics	889 (78.8)	786 (80.5)	56 (58.9)	47 (82.5)
No Antibiotics	239 (21.2)	190 (19.5)	39 (41.1)	10 (17.5)
Advanced respiratory support				
Adv Resp Before Adjunct	845 (74.9)	726 (74.4)	81 (85.3)	38 (66.7)
No Adv Resp Before Adjunct	283 (25.1)	250 (25.6)	14 (14.7)	19 (33.3)
Mechanical ventilation				
MV Before Adjunct	778 (69.0)	668 (68.4)	77 (81.1)	33 (57.9)
No MV Before Adjunct	350 (31.0)	308 (31.6)	18 (18.9)	24 (42.1)

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 1,128[1]	Vasopressin N = 976[1]	Epinephrine N = 95[1]	Phenylephrine N = 57[1]
first_adj_pH	7.31 (7.21, 7.38)	7.31 (7.21, 7.38)	7.29 (7.19, 7.38)	7.35 (7.26, 7.39)
Unknown	685	591	54	40
first_adj_aki				
No AKI	523 (46.9)	429 (44.5)	64 (68.8)	30 (52.6)
AKI Stage 1	255 (22.9)	228 (23.7)	15 (16.1)	12 (21.1)
AKI Stage 2	104 (9.3)	94 (9.8)	5 (5.4)	5 (8.8)
AKI Stage 3	169 (15.2)	154 (16.0)	6 (6.5)	9 (15.8)
ESKD	63 (5.7)	59 (6.1)	3 (3.2)	1 (1.8)
Unknown	14	12	2	0
Kidney replacement therapy				
No RRT Before Adjunct	1,055 (93.5)	908 (93.0)	92 (96.8)	55 (96.5)
RRT Before Adjunct	73 (6.5)	68 (7.0)	3 (3.2)	2 (3.5)
first_adj_temp_c	36.7 (36.3, 37.2)	36.7 (36.3, 37.3)	36.4 (35.8, 37.1)	36.7 (36.4, 37.1)
Unknown	110	81	16	13
first_adj_heart_rate	97 (80, 114)	98 (81, 114)	85 (73, 98)	123 (93, 135)
Unknown	3	1	2	0
first_adj_resp_rate	22 (18, 27)	22 (18, 27)	20 (18, 24)	23 (19, 27)
Unknown	3	1	2	0
first_adj_map	69 (62, 78)	69 (61, 78)	74 (64, 84)	69 (65, 77)
Unknown	28	21	5	2
first_adj_lactate	3.1 (1.7, 6.0)	3.1 (1.8, 5.9)	4.0 (1.5, 7.7)	3.1 (1.9, 5.5)
Unknown	113	89	16	8

[1] Median (Q1, Q3); n (%)

Characteristic	Overall N = 1,128[1]	Vasopressin N = 976[1]	Epinephrine N = 95[1]	Phenylephrine N = 57[1]
Fluid bolus volume administered (mL)	1,000 (0, 2,000)	1,000 (0, 2,250)	500 (0, 1,500)	1,000 (0, 2,000)
Fluid bolus volume administered (mL/kg)	11 (0, 30)	12 (0, 31)	7 (0, 17)	11 (0, 27)
Unknown	13	8	2	3
NE dosage (mcg/kg/min)	0.20 (0.12, 0.32)	0.22 (0.13, 0.32)	0.11 (0.06, 0.29)	0.13 (0.07, 0.23)
NE dosage (mcg/min)	16 (10, 25)	18 (10, 25)	10 (6, 20)	10 (5, 18)
Dobutamine administration				
Dobutamine Before Adjunct	28 (2.5)	26 (2.7)	2 (2.1)	0 (0.0)
No Dobutamine Before Adjunct	1,100 (97.5)	950 (97.3)	93 (97.9)	57 (100.0)
Corticosteroid administration				
No Steroids	750 (66.5)	629 (64.4)	79 (83.2)	42 (73.7)
Steroids	378 (33.5)	347 (35.6)	16 (16.8)	15 (26.3)
hours_to_first_adj	4 (2, 11)	4 (2, 11)	3 (1, 11)	4 (2, 12)
Antibiotic duration (days)	6 (3, 10)	6 (3, 11)	5 (2, 7)	6 (3, 10)
In-Hospital mortality				
Died	488 (43.5)	442 (45.5)	32 (33.7)	14 (25.0)
Survived	634 (56.5)	529 (54.5)	63 (66.3)	42 (75.0)
Unknown	6	5	0	1
Adjunct Response				
Response	315 (27.9)	274 (28.1)	21 (22.1)	20 (35.1)
No Response	813 (72.1)	702 (71.9)	74 (77.9)	37 (64.9)

[1] Median (Q1, Q3); n (%)

Export to Word

▼ Code

```
as_flex_table(val_char_by_adj) |>
  autofit() |>
  set_table_properties(layout = "autofit") |>
  font(fontname = "Arial", part = "all") |>
  fontsize(size = 10, part = "all") |>
  line_spacing(space = 1, part = "all") |>
  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
  save_as_docx(
    path = "val_characteristics_by_adjunct.docx",
    pr_section = prop_section(
      page_size = page_size(orient = "landscape", width = 11, height = 8.5),
      page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
      )
    )
```

14.2 Validation Data Adjustments and Assessment

14.2.1 Data Adjustments

Rename variables in the validation (`fl`) dataset to match those in the derivation (`oh`) dataset, then align categorical variable levels.

▼ Code

```
fl <- fl |>
  rename(sofa2_no_cv = first_adj_sofa_2_no_cv, pH = first_adj_pH,
         neq = adj_baseline_neq, shock_dur_h = hours_to_first_adj,
         map = first_adj_map, lactate = first_adj_lactate
         ) |>
  mutate(response = fct_relevel(response, "Response", "No Response"),
         tx_unit = factor(if_else(first_adj_tx_unit == "Surgical ICU" |
                                    first_adj_tx_unit == "Mixed Med/Surg ICU",
                                  "Surgical or Mixed Med/Surg ICU",
                                  first_adj_tx_unit
                                  ),
                          levels = c("Medical ICU", "Cardiac ICU",
                                     "ED/Rapid Response",
                                     "Surgical or Mixed Med/Surg ICU"
                                     )),
         ssc_era_orig = ssc_era,
         ssc_era = factor(if_else(ssc_era == "SSC 2021",
                                  "SSC 2021",
                                  "Before SSC 2021"
                                  ),
```

```
                        levels = c("Before SSC 2021", "SSC 2021")
                        ),
         criteria_met = factor(if_else(criteria_met == "ASE only",
                                       "ASE only",
                                       "Both or Sepsis-3 only"
                                       ),
                               levels = c("ASE only", "Both or Sepsis-3 only")
                               )
         )
```

Show a table of the observed first adjunctive vasopressor received as part of the therapy course in the validation dataset (`fl`).

▼ Code

```
fl |> tabyl(first_adj) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  kable(align = c('lcc'),
        caption = "First Adjunct Received in Validation (Florida) Dataset"
        ) |>
  kable_paper(full_width = FALSE)
```

First Adjunct Received in Validation (Florida) Dataset

FIRST_ADJ	N	PERCENT
vasopressin	976	86.5%
epinephrine	95	8.4%
phenylephrine	57	5.1%
Total	1128	100.0%

The majority (86.5%) of patients in the validation dataset received vasopressin as the first adjunctive vasopressor.

14.2.2 Dataset Description

Show date of final follow-up (maximum hospital discharge date).

▼ Code

```
max(incl$discharge_time[incl$first_adj_hosp_state == "Florida"])
```

```
[1] "2025-01-03 19:30:00 UTC"
```

Show response frequency by first adjunctive vasopressor.

▼ Code

```
fl |>
  tabyl(response, first_adj) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  kable(align = c('lcc'),
        caption = "Response by First Adjunctive Vasopressor (Validation Cohort)"
        ) |>
  kable_paper(full_width = FALSE)
```

Response by First Adjunctive Vasopressor (Validation Cohort)

RESPONSE/FIRST_ADJ	VASOPRESSIN	EPINEPHRINE	PHENYLEPHRINE	TOTAL
Response	274 (28.1%)	21 (22.1%)	20 (35.1%)	315 (27.9%)
No Response	702 (71.9%)	74 (77.9%)	37 (64.9%)	813 (72.1%)
Total	976 (100.0%)	95 (100.0%)	57 (100.0%)	1,128 (100.0%)

There are 976 included patients who received vasopressin as the first adjunctive vasopressor, 95 who received epinephrine, and 57 who received phenylephrine. Adjunctive vasopressor response was observed in 274 (28.1%) patients receiving vasopressin, 21 (22.1%) receiving epinephrine, and 20 (35.1%) receiving phenylephrine. Notably, these response frequencies are each lower than those observed in the derivation (Ohio) dataset.

Show a numerical description of all predictors in the validation dataset.

▼ Code

```
(val_predictors_by_adjunct <- fl |>
  select(first_adj, heart_failure, tx_unit, ssc_era, criteria_met, sofa2_no_cv,
           pH, neq, shock_dur_h, map, lactate
           ) |>
  mutate(first_adj = fct_recode(first_adj, "Vasopressin" = "vasopressin",
                                "Epinephrine" = "epinephrine",
                                "Phenylephrine" = "phenylephrine"
                                )
         ) |>
  tbl_summary(
    by = first_adj,
    digits = list(
      all_categorical() ~ c(0, 1),
      sofa2_no_cv ~ 0
      ),
    label = list(heart_failure = "History of heart failure",
                 tx_unit = "Treatment unit",
                 ssc_era = "Surviving Sepsis Campaign guideline treatment era",
```

```
                criteria_met = "Clinical criteria met",
                sofa2_no_cv = "SOFA-2 (without cardiovascular score)",
                pH = "Arterial pH",
                neq = "Norepinephrine dosage (mcg/kg/min)",
                shock_dur_h = "Shock duration (hours)",
                map = "MAP (mm Hg)",
                lactate = "Lactate (mmol/L)"
                ),
    missing = "no"
    ) |>
  add_overall() |>
  modify_fmt_fun(
    all_stat_cols() ~ function(x) gsub("%", "", x)
) |>
modify_caption("**Predictor Values in Validation (Florida) Dataset**"))
```

Predictor Values in Validation (Florida) Dataset

Characteristic	Overall N = 1,128[1]	Vasopressin N = 976[1]	Epinephrine N = 95[1]	Phenylephrine N = 57[1]
History of heart failure	175 (15.5)	142 (14.5)	24 (25.3)	9 (15.8)
Treatment unit				
Medical ICU	529 (46.9)	501 (51.3)	15 (15.8)	13 (22.8)
Cardiac ICU	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)
ED/Rapid Response	125 (11.1)	105 (10.8)	17 (17.9)	3 (5.3)
Surgical or Mixed Med/Surg ICU	474 (42.0)	370 (37.9)	63 (66.3)	41 (71.9)
Surviving Sepsis Campaign guideline treatment era				
Before SSC 2021	521 (46.2)	458 (46.9)	45 (47.4)	18 (31.6)
SSC 2021	607 (53.8)	518 (53.1)	50 (52.6)	39 (68.4)

[1] n (%); Median (Q1, Q3)

Characteristic	Overall N = 1,128[1]	Vasopressin N = 976[1]	Epinephrine N = 95[1]	Phenylephrine N = 57[1]
Clinical criteria met				
ASE only	389 (34.5)	335 (34.3)	33 (34.7)	21 (36.8)
Both or Sepsis-3 only	739 (65.5)	641 (65.7)	62 (65.3)	36 (63.2)
SOFA-2 (without cardiovascular score)	5 (3, 7)	5 (3, 7)	4 (3, 6)	4 (3, 7)
Arterial pH	7.31 (7.21, 7.38)	7.31 (7.21, 7.38)	7.29 (7.19, 7.38)	7.35 (7.26, 7.39)
Norepinephrine dosage (mcg/kg/min)	0.20 (0.12, 0.32)	0.22 (0.13, 0.32)	0.11 (0.06, 0.29)	0.13 (0.07, 0.23)
Shock duration (hours)	4 (2, 11)	4 (2, 11)	3 (1, 11)	4 (2, 12)
MAP (mm Hg)	69 (62, 78)	69 (61, 78)	74 (64, 84)	69 (65, 77)
Lactate (mmol/L)	3.1 (1.7, 6.0)	3.1 (1.8, 5.9)	4.0 (1.5, 7.7)	3.1 (1.9, 5.5)

[1] n (%); Median (Q1, Q3)

Export to Word

▼ Code

```
as_flex_table(val_predictors_by_adjunct) |>
  autofit() |>
  set_table_properties(layout = "autofit") |>
  font(fontname = "Arial", part = "all") |>
  fontsize(size = 10, part = "all") |>
  line_spacing(space = 1, part = "all") |>
  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
  save_as_docx(
    path = "val_predictors_by_adjunct.docx",
    pr_section = prop_section(
      page_size = page_size(orient = "landscape", width = 11, height = 8.5),
      page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
      )
    )
```

Next, evaluate for missing data in the outcome variable (`response`).

▼ Code

```
fl |> select(response) |>
  miss_var_summary() |> mutate(pct_miss = as.numeric(pct_miss)) |>
  kable(digits = 3,
        caption = "Missingness in Outcome in Validation (Florida) Data"
        ) |>
  kable_paper(full_width = FALSE)
```

Missingness in Outcome in Validation (Florida) Data

VARIABLE	N_MISS	PCT_MISS
response	0	0

There are no missing values in the outcome variable.

Evaluate for missing data in the model predictors.

▼ Code

```
fl |>
  group_by(first_adj) |>
  select(response, first_adj, heart_failure, tx_unit, ssc_era, criteria_met,
         sofa2_no_cv, pH, neq, shock_dur_h, map, lactate
         ) |>
  miss_var_summary() |>
  mutate(pct_miss = as.numeric(pct_miss)) |>
  kable(digits = 1,
        caption = "Missingness in Predictors in Validation (Florida) Data"
        ) |>
  kable_paper(full_width = FALSE)
```

Missingness in Predictors in Validation (Florida) Data

FIRST_ADJ	VARIABLE	N_MISS	PCT_MISS
vasopressin	pH	591	60.6
vasopressin	lactate	89	9.1
vasopressin	map	21	2.2
vasopressin	response	0	0.0
vasopressin	heart_failure	0	0.0
vasopressin	tx_unit	0	0.0
vasopressin	ssc_era	0	0.0

FIRST_ADJ	VARIABLE	N_MISS	PCT_MISS
vasopressin	criteria_met	0	0.0
vasopressin	sofa2_no_cv	0	0.0
vasopressin	neq	0	0.0
vasopressin	shock_dur_h	0	0.0
epinephrine	pH	54	56.8
epinephrine	lactate	16	16.8
epinephrine	map	5	5.3
epinephrine	response	0	0.0
epinephrine	heart_failure	0	0.0
epinephrine	tx_unit	0	0.0
epinephrine	ssc_era	0	0.0
epinephrine	criteria_met	0	0.0
epinephrine	sofa2_no_cv	0	0.0
epinephrine	neq	0	0.0
epinephrine	shock_dur_h	0	0.0
phenylephrine	pH	40	70.2
phenylephrine	lactate	8	14.0
phenylephrine	map	2	3.5
phenylephrine	response	0	0.0
phenylephrine	heart_failure	0	0.0
phenylephrine	tx_unit	0	0.0
phenylephrine	ssc_era	0	0.0
phenylephrine	criteria_met	0	0.0
phenylephrine	sofa2_no_cv	0	0.0
phenylephrine	neq	0	0.0
phenylephrine	shock_dur_h	0	0.0

There are missing data in the same predictors in the validation dataset as in the derivation dataset (`pH`, `lactate`, and `map`). Missing data in these variables is managed through imputation in a `recipes()` step for each model.

14.3 Create and Assess Decision Model

Generate predicted response probability in the validation dataset (`fl`) using the final clinical prediction models for response to vasopressin (logistic regression with a spline term for NEQ; `avp_m1_ns_logistic`), epinephrine (boosted trees with XGBoost; `epi_m4_boost_xgboost`), and phenylephrine (random forest; `pe_m3_rf_ranger`).

▼ Code

```
avp_val_preds <-
  predict(
    avp_mod_final_fit_dev,
    new_data = fl,
    type = "prob"
    ) |>
  select(avp_pred_response_prob = .pred_Response)

epi_val_preds <-
  predict(
    epi_mod_final_fit_dev,
    new_data = fl,
    type = "prob"
    ) |>
  select(epi_pred_response_prob = .pred_Response)

pe_val_preds <-
  predict(
    pe_mod_final_fit_dev,
    new_data = fl,
    type = "prob"
    ) |>
  select(pe_pred_response_prob = .pred_Response)

fl_aug <-
  cbind(fl, avp_val_preds, epi_val_preds, pe_val_preds)
# Remove objects that won't be used moving forward
rm(avp_val_preds, epi_val_preds, pe_val_preds)
```

Show a numerical summary of the predicted probabilities of response by adjunctive vasopressor.

▼ Code

```
describe(fl_aug |> select(avp_pred_response_prob, epi_pred_response_prob,
                          pe_pred_response_prob
                          )
         )
```

```
select(fl_aug, avp_pred_response_prob, epi_pred_response_prob, pe_pred_response_prob)

 3  Variables      1128  Observations
---------------------------------------------------------------------
```

```
avp_pred_response_prob
       n  missing distinct     Info     Mean  pMedian      Gmd
    1128        0     1128        1    0.298   0.2921     0.19
     .05      .10      .25      .50      .75      .90      .95
 0.06269  0.08733  0.15892  0.28556  0.41979  0.52270  0.57938

lowest : 0.00688296 0.014263   0.0182585  0.0193515  0.0219763
highest: 0.766558   0.777758   0.803707   0.808867   0.843636
------------------------------------------------------------------------
epi_pred_response_prob
       n  missing distinct     Info     Mean  pMedian      Gmd
    1128        0     1128        1   0.3288   0.3266    0.114
     .05      .10      .25      .50      .75      .90      .95
  0.1745   0.1985   0.2524   0.3252   0.3945   0.4738   0.5087

lowest : 0.0670611 0.0874844 0.104297  0.119839  0.121965
highest: 0.562223  0.568808  0.57232   0.584569  0.595376
------------------------------------------------------------------------
pe_pred_response_prob
       n  missing distinct     Info     Mean  pMedian      Gmd
    1128        0     1128        1   0.3492    0.348   0.1462
     .05      .10      .25      .50      .75      .90      .95
  0.1529   0.1776   0.2510   0.3506   0.4423   0.5176   0.5627

lowest : 0.0840998 0.0858361 0.0863435 0.0872859 0.092859
highest: 0.637145  0.638634  0.640324  0.6442    0.684944
------------------------------------------------------------------------
```

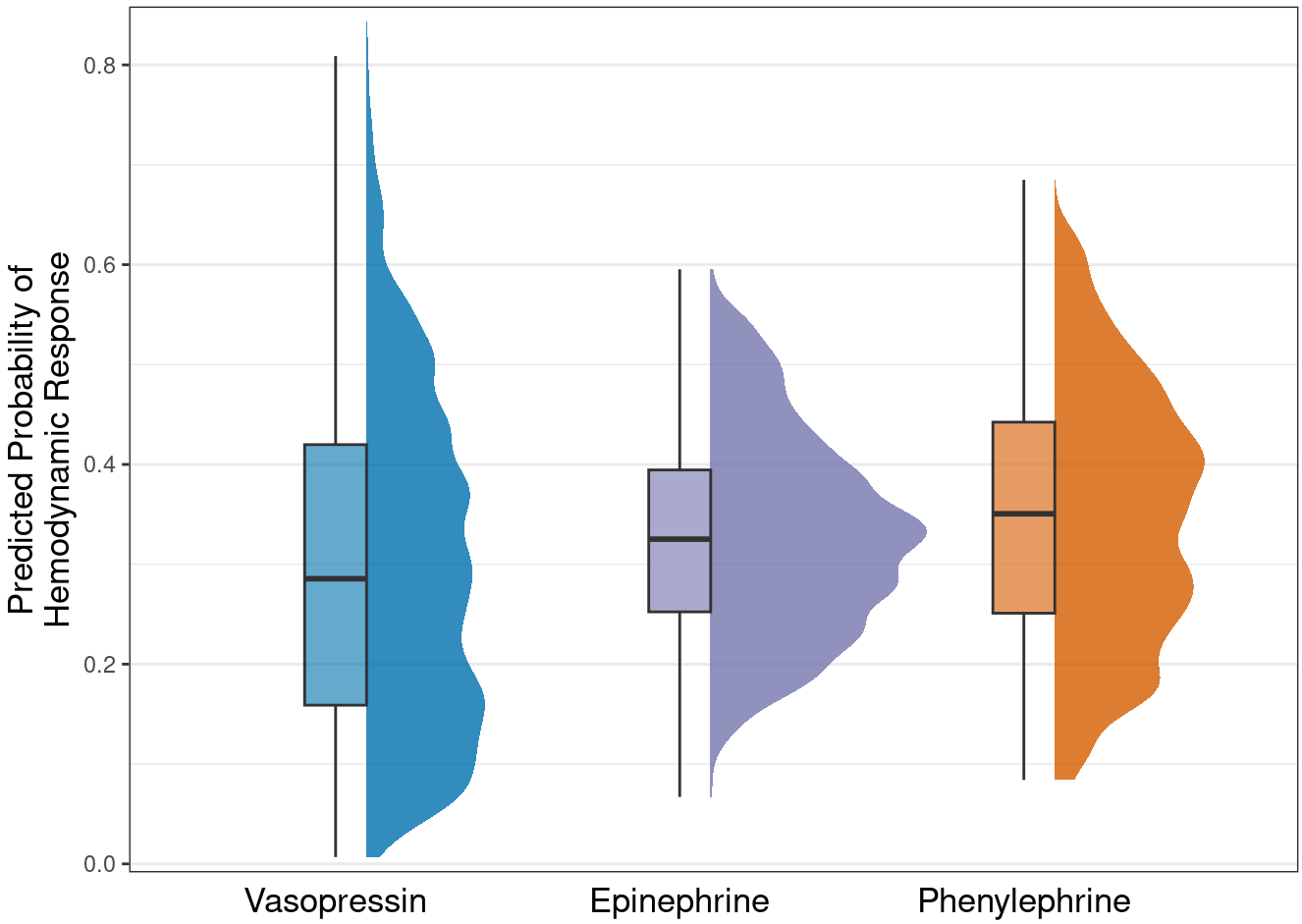
Show a raincloud plot of the predicted probability of response by adjunctive vasopressor.

▼ Code

```
pred_prob_long <-
  fl_aug |>
  select(
    avp_pred_response_prob,
    epi_pred_response_prob,
    pe_pred_response_prob
    ) |>
  pivot_longer(
    everything(),
    names_to = "vasopressor",
    values_to = "pred_prob"
    ) |>
  mutate(
    vasopressor = factor(
      vasopressor,
      levels = c("avp_pred_response_prob", "epi_pred_response_prob",
                 "pe_pred_response_prob"
                 ),
```

```
      labels = c("Vasopressin", "Epinephrine", "Phenylephrine")
      )
    )

(resp_pred_raincloud <- ggplot(
  pred_prob_long,
  aes(x = vasopressor, y = pred_prob, fill = vasopressor)
  ) +
  stat_halfeye(
    width = 0.7, justification = -0.14, point_interval = NULL, alpha = 0.8,
    adjust = 0.6 # Decrease smoothing in density plot (default = 1)
    ) +
  geom_boxplot(width = 0.18, alpha = 0.6, outlier.shape = NA) +
  scale_fill_manual(values = c(
      "Vasopressin" = "#0072B2",
      "Epinephrine" = "#7876B1FF",
      "Phenylephrine" = "#D55E00"
      )) +
  guides(fill = "none") +
  scale_y_continuous(
    limits = c(0, 0.85),
    breaks = seq(0, 0.8, by = 0.2),
    expand = expansion(mult = c(0.01, 0.01))
    ) +
  labs(
    x = NULL,
    y = "Predicted Probability of\nHemodynamic Response"
    ) +
  theme_bw() +
  theme(
    text = element_text(family = "sans"),
    axis.text.x = element_text(size = 13, color = "black",
                               margin = margin(t = 6)
                               ),
    axis.title.y = element_text(size = 13),
    axis.ticks.x = element_blank(),
    panel.grid.major.x = element_blank(),
    panel.grid.minor.x = element_blank()
    )
  )
```



▼ Code

```
rm(pred_prob_long)
```

Save raincloud plot

▼ Code

```
ggsave(
  "resp_pred_raincloud.tiff",
  plot = resp_pred_raincloud,
  device = ragg::agg_tiff,
  width = 7,
  height = 4,
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

14.4 Decision Model Recommendation

Determine decision model-recommended adjunctive vasopressor medication (`rec_first_adj`), defined as the first adjunctive vasopressor with the highest (maximum) predicted probability of response.

▼ Code

```
fl_aug <- fl_aug |>
  rowwise() |>
  mutate(
    best_prob = max(avp_pred_response_prob, epi_pred_response_prob,
        pe_pred_response_prob
        ),
    rec_first_adj = factor(case_when(
      best_prob == avp_pred_response_prob ~ "vasopressin",
      best_prob == epi_pred_response_prob ~ "epinephrine",
      best_prob == pe_pred_response_prob ~ "phenylephrine",
      .default = "Error"
      ),
      levels = c("vasopressin", "epinephrine", "phenylephrine")
      )
    ) |>
  ungroup()
```

Show a table of the decision model-recommended adjunctive vasopressor medication (`rec_first_adj`).

▼ Code

```
fl_aug |> tabyl(rec_first_adj) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  rename(Recommended = rec_first_adj) |>
  kable(align = c('lcc'),
        caption = "Decision Model-Recommended Adjunctive Vasopressor (Validation Cohort)"
        ) |>
  kable_paper(full_width = FALSE)
```

Decision Model-Recommended Adjunctive Vasopressor (Validation Cohort)

RECOMMENDED	N	PERCENT
vasopressin	323	28.6%
epinephrine	323	28.6%
phenylephrine	482	42.7%
Total	1128	100.0%

The most commonly recommended first adjunctive vasopressor medication in the validation (Florida) dataset was phenylephrine.

Show a table of recommended first adjunctive vasopressor medication vs. the first adjunctive vasopressor medication received during the patient's observed course of care.

▼ Code

```
fl_aug |> tabyl(first_adj, rec_first_adj) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  rename("Received (row)/Recommended (col)" = `first_adj/rec_first_adj`) |>
  kable(align = c('lcc'),
        caption = "Decision Model-Recommended Adjunctive Vasopressor vs. Received Adjunct"
        ) |>
  kable_paper(full_width = FALSE)
```

Decision Model-Recommended Adjunctive Vasopressor vs. Received Adjunct

RECEIVED (ROW)/RECOMMENDED (COL)	VASOPRESSIN	EPINEPHRINE	PHENYLEPHRINE	TOTAL
vasopressin	299 (92.6%)	281 (87.0%)	396 (82.2%)	976 (86.5%)
epinephrine	17 (5.3%)	26 (8.0%)	52 (10.8%)	95 (8.4%)
phenylephrine	7 (2.2%)	16 (5.0%)	34 (7.1%)	57 (5.1%)
Total	323 (100.0%)	323 (100.0%)	482 (100.0%)	1,128 (100.0%)

An adjunctive vasopressor other than the model-recommended vasopressor (discordance) was observed for 24 of 323 (7.4%) patients recommended to receive vasopressin, 297 of 323 (92%) patients recommended to receive epinephrine, and 448 of 482 (92.9%) patients recommended to receive phenylephrine.

Inversely, of the 976 patients who received vasopressin, 299 (30.6%) patients received the model recommendation-concordant adjunctive vasopressor (vasopressin) and 69.4% received a discordant adjunctive vasopressor. Of the 95 patients who received epinephrine, 26 (27.4%) patients received the model recommendation-concordant adjunctive vasopressor (epinephrine) and 69 patients (72.6%) received a discordant adjunctive vasopressor. Of the 57 patients who received phenylephrine, 34 (59.6%) patients received the model recommendation-concordant adjunctive vasopressor (phenylephrine) and 23 (40.4%) patients received a discordant adjunctive vasopressor.

Plot the concordance between model-recommended first adjunctive vasopressor and the treatment (adjunctive vasopressor) received using an alluvial plot.

▼ Code

```r
# Data for alluvial plot
alluvial_dat <-
  fl_aug |>
  count(rec_first_adj, first_adj) |>
  mutate(
    rec_first_adj = fct_recode(
      rec_first_adj,
      "Vasopressin" = "vasopressin",
      "Epinephrine" = "epinephrine",
      "Phenylephrine" = "phenylephrine"
      ),
    first_adj = fct_recode(
      first_adj,
      "Vasopressin" = "vasopressin",
      "Epinephrine" = "epinephrine",
      "Phenylephrine" = "phenylephrine"
      )
    )

# Counts for left and right strata
rec_counts <-
  fl_aug |>
  count(rec_first_adj) |>
  mutate(
    stratum = fct_recode(
      rec_first_adj,
      "Vasopressin" = "vasopressin",
      "Epinephrine" = "epinephrine",
      "Phenylephrine" = "phenylephrine"
      ),
    label = paste0(stratum, "\n(n = ", comma(n), ")")
    ) |>
  select(stratum, label)

first_counts <-
  fl_aug |>
  count(first_adj) |>
  mutate(
    stratum = fct_recode(
      first_adj,
      "Vasopressin" = "vasopressin",
      "Epinephrine" = "epinephrine",
      "Phenylephrine" = "phenylephrine"
      ),
    label = paste0(stratum, "\n(n = ", comma(n), ")")
    ) |>
  select(stratum, label)

# Base plot
p <-
```

```
  ggplot(
    alluvial_dat,
    aes(
      axis1 = rec_first_adj,
      axis2 = first_adj,
      y = n
      )
    ) +
  geom_alluvium(aes(fill = rec_first_adj), alpha = 0.65) +
  geom_stratum(width = 0.25, fill = "grey85", color = "black")

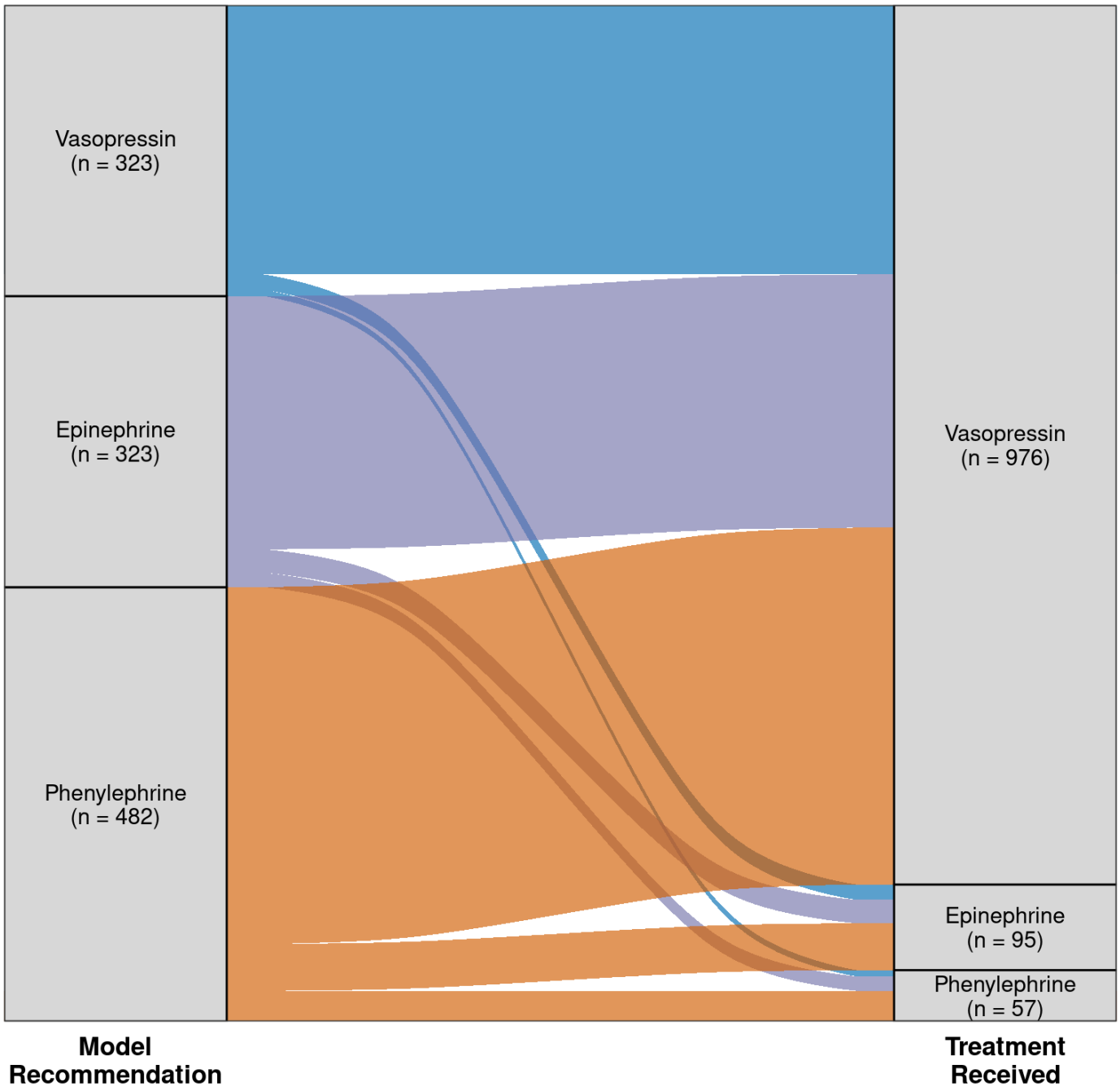
# Extract stratum positions
strata_pos <-
  ggplot_build(p)$data[[2]] |>
  distinct(x, stratum, ymin, ymax) |>
  mutate(y = (ymin + ymax) / 2)

# Labels for recommendation axis (left)
left_labels <-
  strata_pos |>
  filter(x == 1) |>
  left_join(rec_counts, by = "stratum")

# Labels for treatment axis (right)
right_labels <-
  strata_pos |>
  filter(x == 2) |>
  left_join(first_counts, by = "stratum")

# Final plot
(concord_alluv <- p +
    geom_text(data = left_labels, aes(x = x, y = y, label = label),
      inherit.aes = FALSE, size = 4, lineheight = 0.9
      ) +
    geom_text(data = right_labels, aes(x = x, y = y, label = label),
      inherit.aes = FALSE, size = 4, lineheight = 0.9
      ) +
    scale_fill_manual(values = c(
      "Vasopressin" = "#0072B2",
      "Epinephrine" = "#7876B1FF",
      "Phenylephrine" = "#D55E00"
      )) +
    scale_x_discrete(
      limits = c("Model\nRecommendation", "Treatment\nReceived"),
      expand = c(0.06, 0.06)
      ) +
    scale_y_continuous(
      expand = expansion(mult = c(0, 0)), breaks = NULL, labels = NULL) +
    guides(fill = "none") +
    labs(x = NULL, y = NULL) +
    theme(
```

```
    axis.text.x = element_text(size = 13, color = "black",
                               face = "bold", margin = margin(t = 8)
                               ),
    axis.text.y = element_blank(),
    axis.ticks.x = element_blank(),
    axis.ticks.y = element_blank(),
    panel.grid = element_blank(),
    plot.margin = margin(t = 5.5, r = 5.5, b = 20, l = 5.5)
    )
)
```



▼ Code

```
rm(rec_counts, first_counts, alluvial_dat, strata_pos,
   left_labels, right_labels, p
   )
```

Save alluvial plot

▼ Code

```
ggsave(
  "concord_alluv.tiff",
  plot = concord_alluv,
  device = ragg::agg_tiff,
  width = 9,
  height = 9,
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

14.5 Discordance analysis

14.5.1 Primary analysis

Perform a discordance analysis and sensitivity analyses as described in Dalton 2016.

Determine concordance between the decision model-recommended therapy (`rec_first_adj`) and the adjunctive vasopressor the patient received during the normal course of care (`first_adj`).

▼ Code

```
fl_aug <- fl_aug |>
  mutate(first_adj_concordance =
           factor(if_else(first_adj == rec_first_adj,
                          "concordant",
                          "discordant"
                          ),
                  levels = c("discordant", "concordant")
                  )
         )

fl_aug |> tabyl(first_adj_concordance) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  rename(Concordance = first_adj_concordance) |>
  kable(align = c('lcc'),
        caption = "Concordance of Decision Model Recommended and Observed Adjunct"
```

```
      ) |>
  kable_paper(full_width = FALSE)
```

Concordance of Decision Model Recommended and Observed Adjunct

CONCORDANCE	N	PERCENT
discordant	769	68.2%
concordant	359	31.8%
Total	1128	100.0%

Only 31.8% of patients received the decision model-recommended adjunctive vasopressor (the medication with the highest predicted probability of response) during the standard course of care.

Show the observed incidence of response among concordant and discordant cases.

▼ Code

```
fl_aug |> tabyl(response, first_adj_concordance) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col") |>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  rename("Response (row)/Concordance (col)" = `response/first_adj_concordance`) |>
  kable(align = c('lcc'),
        caption = "Observed Response Incidence by Decision Model-Recommendation Concordance"
        ) |>
  kable_paper(full_width = FALSE)
```

Observed Response Incidence by Decision Model-Recommendation Concordance

RESPONSE (ROW)/CONCORDANCE (COL)	DISCORDANT	CONCORDANT	TOTAL
Response	178 (23.1%)	137 (38.2%)	315 (27.9%)
No Response	591 (76.9%)	222 (61.8%)	813 (72.1%)
Total	769 (100.0%)	359 (100.0%)	1,128 (100.0%)

Show data similarly in a contingency table (with interpretation below).

▼ Code

```
Epi::twoby2(fl_aug$first_adj_concordance, fl_aug$response)
```

```
2 by 2 table analysis:
------------------------------------------------------
Outcome   : Response
```

```
Comparing : discordant vs. concordant

           Response No Response    P(Response) 95% conf. interval
discordant      178         591         0.2315    0.2030   0.2626
concordant      137         222         0.3816    0.3328   0.4330

                                    95% conf. interval
            Relative Risk:  0.6066    0.5045   0.7292
         Sample Odds Ratio:  0.4881    0.3722   0.6399
Conditional MLE Odds Ratio:  0.4884    0.3690   0.6465
    Probability difference: -0.1501   -0.2088  -0.0923

             Exact P-value: 0.0000
         Asymptotic P-value: 0.0000
------------------------------------------------------
```

Adjunctive vasopressor response was observed in 178 of 769 (23.1%) patients receiving discordant vasopressor therapy and 137 of 359 (38.2%) patients receiving concordant vasopressor therapy (difference in percentages 15.0% [95% CI; 9.2%, 20.9%]).

Create a numeric response outcome variable, then determine the association of concordance with adjunctive vasopressor response using logistic regression.

▼ Code

```
fl_aug <- fl_aug |>
  mutate(response_bin = as.integer(response == "Response"))

discord_analysis <- glm(response_bin ~ first_adj_concordance,
                        data = fl_aug, family = binomial(link = "logit")
                        )

tidy(discord_analysis, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  mutate(term = sub("^first_adj_concordance", "", term)) |>
  select("Term" = term, "OR" = estimate,
         "95% CI, Lower" = conf.low, "95% CI, Upper" = conf.high) |>
  kable(align = c('lcccc'),
        caption = "Logistic Regression for Adjunctive Vasopressor Response by Decision Model
         Concordance",
        digits = 2
        ) |>
  kable_paper(full_width = FALSE)
```

Logistic Regression for Adjunctive Vasopressor Response by Decision Model Concordance

TERM	OR	95% CI, LOWER	95% CI, UPPER
concordant	2.05	1.56	2.69

Decision model-concordant adjunctive vasopressor therapy was associated with 2-fold higher odds of response (OR 2.05 [95% CI; 1.56, 2.69]).

14.5.2 Subgroup: Concordance by Recommended Vasopressor

To gain further insight into the nature of discordance, show concordance by decision model-recommended first adjunctive vasopressor.

▼ Code

```
fl_aug |>
  mutate(first_adj_concordance =
           fct_relevel(first_adj_concordance, "concordant")
         ) |>
  tabyl(first_adj_concordance, rec_first_adj) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  rename("Concordance/Recommended" = `first_adj_concordance/rec_first_adj`) |>
  kable(align = c('lcc'),
        caption = "Concordance by Decision Model-Recommended First Adjunct"
        ) |>
  kable_paper(full_width = FALSE)
```

Concordance by Decision Model-Recommended First Adjunct

CONCORDANCE/RECOMMENDED	VASOPRESSIN	EPINEPHRINE	PHENYLEPHRINE	TOTAL
concordant	299 (92.6%)	26 (8.0%)	34 (7.1%)	359 (31.8%)
discordant	24 (7.4%)	297 (92.0%)	448 (92.9%)	769 (68.2%)
Total	323 (100.0%)	323 (100.0%)	482 (100.0%)	1,128 (100.0%)

Next, assess whether the association between concordance and response varies according to the decision model-recommended first adjunct by fitting a logistic regression model with an interaction term between recommended first adjunct and concordance. This addresses the question: “Within each recommended first adjunct, what is the odds ratio for response comparing concordant versus discordant therapy?”. Then, formally test for effect modification/interaction (“Does the effect of concordance on response differ across recommended first adjunct groups?”).

▼ Code

```
fl_aug <- fl_aug |>
  mutate(first_adj_conc_relevl =
           fct_relevel(first_adj_concordance, "concordant")
         )
```

```
fit <- glm(response_bin ~ rec_first_adj * first_adj_conc_relevl,
           family = binomial(link = "logit"), data = fl_aug
           )

(emm <- emmeans(fit,
                pairwise ~ first_adj_conc_relevl | rec_first_adj,
                type = "response"
                ))
```

```
$emmeans
rec_first_adj = vasopressin:
 first_adj_conc_relevl  prob     SE  df asymp.LCL asymp.UCL
 concordant            0.398 0.0283 Inf    0.3440     0.455
 discordant            0.417 0.1010 Inf    0.2409     0.617

rec_first_adj = epinephrine:
 first_adj_conc_relevl  prob     SE  df asymp.LCL asymp.UCL
 concordant            0.192 0.0773 Inf    0.0824     0.387
 discordant            0.256 0.0253 Inf    0.2095     0.309

rec_first_adj = phenylephrine:
 first_adj_conc_relevl  prob     SE  df asymp.LCL asymp.UCL
 concordant            0.382 0.0833 Inf    0.2366     0.553
 discordant            0.205 0.0191 Inf    0.1705     0.245

Confidence level used: 0.95
Intervals are back-transformed from the logit scale

$contrasts
rec_first_adj = vasopressin:
 contrast               odds.ratio    SE  df null z.ratio p.value
 concordant / discordant      0.926 0.399 Inf    1  -0.180  0.8574

rec_first_adj = epinephrine:
 contrast               odds.ratio    SE  df null z.ratio p.value
 concordant / discordant      0.692 0.357 Inf    1  -0.714  0.4754

rec_first_adj = phenylephrine:
 contrast               odds.ratio    SE  df null z.ratio p.value
 concordant / discordant      2.395 0.891 Inf    1   2.350  0.0188

Tests are performed on the log odds ratio scale
```

▼ Code

```
emmeans(fit, pairwise ~ first_adj_conc_relevl | rec_first_adj,
        type = "response"
        )$contrasts |>
  summary(infer = c(TRUE, TRUE))
```

```
rec_first_adj = vasopressin:
 contrast               odds.ratio    SE  df asymp.LCL asymp.UCL
 concordant / discordant      0.926 0.399 Inf     0.398      2.15
 null z.ratio p.value
    1  -0.180  0.8574

rec_first_adj = epinephrine:
 contrast               odds.ratio    SE  df asymp.LCL asymp.UCL
 concordant / discordant      0.692 0.357 Inf     0.252      1.90
 null z.ratio p.value
    1  -0.714  0.4754

rec_first_adj = phenylephrine:
 contrast               odds.ratio    SE  df asymp.LCL asymp.UCL
 concordant / discordant      2.395 0.891 Inf     1.156      4.96
 null z.ratio p.value
    1   2.350  0.0188

Confidence level used: 0.95
Intervals are back-transformed from the log odds ratio scale
Tests are performed on the log odds ratio scale
```

▼ Code

```
anova(fit, test = "Chisq")
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: response_bin

Terms added sequentially (first to last)

                                  Df Deviance Resid. Df Resid. Dev
NULL                                                1127     1336.1
rec_first_adj                      2   32.400      1125     1303.7
first_adj_conc_relevl              1    0.865      1124     1302.8
rec_first_adj:first_adj_conc_relevl 2    4.834      1122     1298.0
                                   Pr(>Chi)
NULL
rec_first_adj                     9.215e-08 ***
first_adj_conc_relevl               0.35233
rec_first_adj:first_adj_conc_relevl   0.08917 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

▼ Code

```
rm(fit)
```

There was some evidence that the association between concordance and response differed by the decision model-recommended first adjunctive vasopressor, but the evidence for effect modification was not substantial (interaction p = 0.09). Concordant therapy was associated with higher odds of response when phenylephrine was recommended (OR 2.40, 95% CI 1.13 to 4.91), whereas no evidence of an association was observed when vasopressin (OR 0.93, 95% CI 0.40 to 2.21) or epinephrine (OR 0.69, 95% CI 0.23 to 1.77) was recommended.

14.5.2.1 Subgroup: Recommended vs. Received

In a corollary analysis, further assess the nature of discordance by evaluating response by the recommended adjunct and the received adjunct - beyond just concordant/discordant therapy receipt and rather the specific agents. This will be accomplished by fitting a model with an interaction term between the first adjunct recommended (`rec_first_adj`) and the received first adjunct (`first_adj`). Then, assess within each recommended adjunct subgroup how the response odds compare with the concordant therapy.

▼ Code

```
# Fit model to estimate response probabilities (these estimates are the same
# as observed because the model is saturated)
fit <- glm(response_bin ~ rec_first_adj * first_adj,
           family = binomial(link = "logit"), data = fl_aug
           )

# Estimated response percentages
prob_tbl <- as.data.frame(
  summary(emmeans(fit, ~ rec_first_adj * first_adj, type = "response"))
  ) |>
  transmute(
    rec_first_adj = as.character(rec_first_adj),
    first_adj = as.character(first_adj),
    response_ci = sprintf("%.1f (%.1f, %.1f)", 100 * prob, 100 * asymp.LCL,
                          100 * asymp.UCL
                          )
    )

# ORs relative to concordant therapy within each recommendation stratum

or_tbl <- map_dfr(
  c("vasopressin", "epinephrine", "phenylephrine"),
  \(ref_lvl) {

    fl_aug |>
      filter(rec_first_adj == ref_lvl) |>
      mutate(first_adj = relevel(factor(first_adj), ref = ref_lvl)) |>
      glm(response_bin ~ first_adj, family = binomial, data = _) |>
```

```
      tidy(exponentiate = TRUE, conf.int = TRUE) |>
      filter(term != "(Intercept)") |>
      transmute(
        rec_first_adj = ref_lvl,
        first_adj = sub("^first_adj", "", term),
        or_ci = sprintf("%.2f (%.2f, %.2f)", estimate, conf.low, conf.high))
    }
  )

# Add concordant rows
or_tbl <- bind_rows(
  tibble(rec_first_adj = c("vasopressin", "epinephrine", "phenylephrine"),
         first_adj = c("vasopressin", "epinephrine", "phenylephrine"),
         or_ci = "1.00 (Reference)"),
  or_tbl
  )

# Final table
discord_analysis_sub2 <- prob_tbl |>
  left_join(or_tbl,
            by = c("rec_first_adj", "first_adj")
            ) |>
  mutate(
    rec_first_adj = factor(rec_first_adj,
                           levels = c("vasopressin", "epinephrine",
                                      "phenylephrine"
                                      ),
                           labels = c("Vasopressin", "Epinephrine",
                                      "Phenylephrine"
                                      )
                           ),
    first_adj = factor(first_adj,
                       levels = c("vasopressin", "epinephrine",
                                  "phenylephrine"
                                  ),
                       labels = c("Vasopressin", "Epinephrine",
                                  "Phenylephrine"
                                  )
                       )
    ) |>
  arrange(rec_first_adj, first_adj) |>
  rename(`Recommended Adjunct` = rec_first_adj,
         `Received Adjunct` = first_adj,
         `Response, % (95% CI)` = response_ci,
         `OR vs Concordant (95% CI)` = or_ci
         )

(discord_analysis_sub2 |>
  kable(align = c('llcc')) |>
  kable_paper(full_width = FALSE))
```

RECOMMENDED ADJUNCT	RECEIVED ADJUNCT	RESPONSE, % (95% CI)	OR VS CONCORDANT (95% CI)
Vasopressin	Vasopressin	39.8 (34.4, 45.5)	1.00 (Reference)
Vasopressin	Epinephrine	41.2 (21.0, 64.8)	1.06 (0.38, 2.83)
Vasopressin	Phenylephrine	42.9 (14.4, 77.0)	1.13 (0.22, 5.23)
Epinephrine	Vasopressin	25.6 (20.9, 31.0)	1.45 (0.57, 4.46)
Epinephrine	Epinephrine	19.2 (8.2, 38.7)	1.00 (Reference)
Epinephrine	Phenylephrine	25.0 (9.7, 50.8)	1.40 (0.30, 6.32)
Phenylephrine	Vasopressin	21.0 (17.2, 25.2)	0.43 (0.21, 0.91)
Phenylephrine	Epinephrine	17.3 (9.3, 30.0)	0.34 (0.12, 0.91)
Phenylephrine	Phenylephrine	38.2 (23.7, 55.3)	1.00 (Reference)

▼ Code

```
#rm(fit, prob_tbl, or_tbl)
```

First, when the model recommends vasopressin the rate (percentage) of response is relatively similar (~40%) regardless of which adjunct was administered. When the model recommends epinephrine and the patient receives epinephrine (concordant therapy), the response rate appears slightly lower compared with receipt of another adjunct. The most intriguing finding is when the model recommends phenylephrine - when the patient receives phenylephrine (concordant therapy) response appears substantially higher compared with receipt of another agent.

When compared to concordant therapy, there was no convincing evidence that following the decision model-recommended therapy improved response when vasopressin or epinephrine was recommended (discordant confidence intervals are relatively wide and include 1). However, among patients in whom phenylephrine was recommended, receipt of discordant therapy with vasopressin (OR 0.43, 95% CI 0.21-0.91) or epinephrine (OR 0.34, 95% CI 0.12-0.91) was associated with lower odds of response compared with receipt of phenylephrine.

Export to Word

▼ Code

```
discord_analysis_sub2 |>
flextable() |>
  autofit() |>
  set_table_properties(layout = "autofit") |>
  font(fontname = "Arial", part = "all") |>
  fontsize(size = 10, part = "all") |>
  line_spacing(space = 1, part = "all") |>
  padding(padding.top = 0, padding.bottom = 0, part = "all") |>
  save_as_docx(
    path = "resp_rec_vs_received.docx",
```

```
    pr_section = prop_section(
      page_size = page_size(orient = "portrait", width = 8.5, height = 11),
      page_margins = page_mar(top = 0.5, bottom = 0.5, left = 0.5, right = 0.5)
      )
    )
```

14.5.3 Subgroup: Concordance Effect by Decision Model Recommendation Strength

14.5.3.1 Model Probability Difference by Predicted Response

In this analysis assess if the association of concordance with response differs when the model is more certain about the recommended first adjunctive vasopressor. To address this question create a plot with the difference between the adjunct with the highest predicted probability for response and the adjunct with the second-highest predicted probability (`margin`) on the y-axis and the predicted probability for the highest adjunct (`best_prob`) on the x-axis. Create bins (strata) for these two variables where the odds ratio for response will be calculated and plotted in a grid.

First, calculate the `margin` as the difference between the vasopressor with the highest predicted probability of response and the vasopressor with the second-highest predicted probability of response.

▼ Code

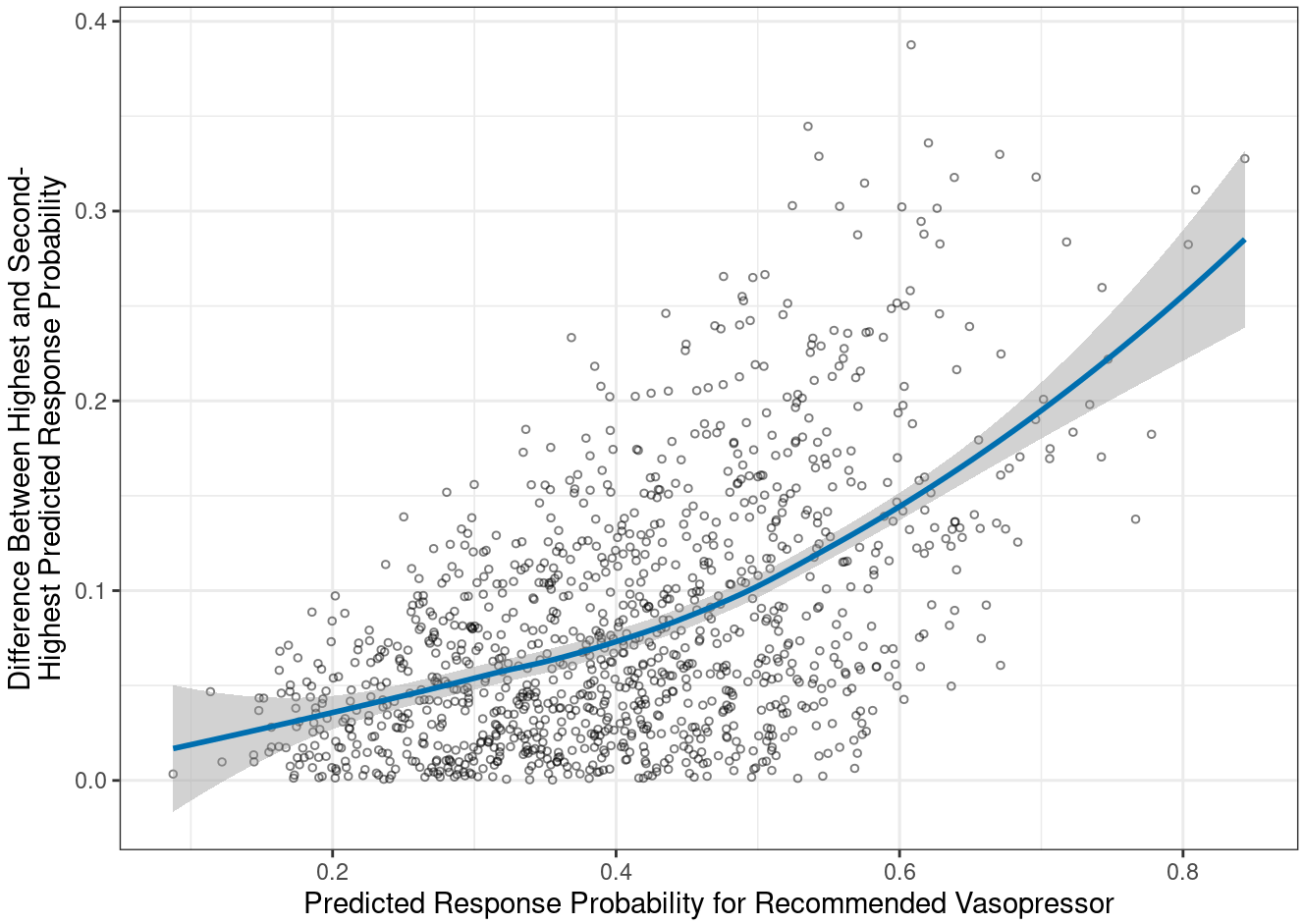
```
fl_aug <-
  fl_aug |>
  rowwise() |>
  mutate(
    best_prob = max(avp_pred_response_prob, epi_pred_response_prob,
                    pe_pred_response_prob
                    ),
    second_prob = sort(
      c(avp_pred_response_prob, epi_pred_response_prob, pe_pred_response_prob),
      decreasing = TRUE
      )[2],
    margin = best_prob - second_prob
    ) |>
  ungroup()
```

Then, show scatter plot of `margin` by `best_prob` with a LOESS smooth for the relationship.

▼ Code

```
ggplot(fl_aug, aes(y = margin, x = best_prob)) +
  geom_point(alpha = 0.5, size = 1, shape = 1, color = "black") +
  geom_smooth(method = loess, formula = y ~ x, color = "#0072B2") +
  labs(
    x = "Predicted Response Probability for Recommended Vasopressor",
```

```
    y = "Difference Between Highest and Second-\nHighest Predicted Response Probability"
    )
```



There appears to be a positive, non-linear relationship between `margin` and `best_prob`.

Save plot.

▼ Code

```
ggsave(
  "margin_by_best_pred.tiff",
  device = ragg::agg_tiff,
  width = 6,
  height = 4,
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

Create the bins (strata) for `margin` (by intervals of 0.05) and `best_prob` (by intervals of 0.1).

▼ Code

```r
fl_aug <- fl_aug |>
  mutate(
    margin_strata = cut(margin,
                        breaks = c(0, 0.05, 0.10, 0.15, 0.20, Inf),
                        include.lowest = TRUE,
                        right = FALSE,
                        labels = c("<0.05", "0.05-0.10", "0.10-0.15",
                                   "0.15-0.20", "≥0.20"
                                   )
                        ),
    prob_bin = cut(best_prob,
                   breaks = c(0, 0.20, 0.30, 0.40, 0.50, 0.60, Inf),
                   include.lowest = TRUE,
                   right = FALSE,
                   labels = c("<0.20", "0.20-0.30", "0.30-0.40",
                              "0.40-0.50", "0.50-0.60", "≥0.60"
                              )
                   )
    )
```

Next, examine counts of concordance by bin to ensure there are adequate data for estimating ORs.

▼ Code

```r
fl_aug |>
  count(prob_bin, margin_strata, first_adj_concordance)
```

```
# A tibble: 50 × 4
   prob_bin  margin_strata first_adj_concordance     n
   <fct>     <fct>         <fct>                 <int>
 1 <0.20     <0.05         discordant               37
 2 <0.20     <0.05         concordant                9
 3 <0.20     0.05-0.10     discordant               10
 4 <0.20     0.05-0.10     concordant                1
 5 0.20-0.30 <0.05         discordant               93
 6 0.20-0.30 <0.05         concordant               22
 7 0.20-0.30 0.05-0.10     discordant               58
 8 0.20-0.30 0.05-0.10     concordant               11
 9 0.20-0.30 0.10-0.15     discordant               13
10 0.20-0.30 0.10-0.15     concordant                2
# ℹ 40 more rows
```

There are some cells with sparse data (as expected), but the counts look reasonable enough to proceed with these bins.

Check counts for response by concordant and discordant observations by cell to check if data will be stable for logistic regression.

▼ Code

```
fl_aug |>
  count(prob_bin, margin_strata, first_adj_concordance, response)
```

```
# A tibble: 94 × 5
   prob_bin  margin_strata first_adj_concordance response        n
   <fct>     <fct>         <fct>                 <fct>       <int>
 1 <0.20     <0.05         discordant            Response        3
 2 <0.20     <0.05         discordant            No Response    34
 3 <0.20     <0.05         concordant            Response        2
 4 <0.20     <0.05         concordant            No Response     7
 5 <0.20     0.05-0.10     discordant            Response        2
 6 <0.20     0.05-0.10     discordant            No Response     8
 7 <0.20     0.05-0.10     concordant            No Response     1
 8 0.20-0.30 <0.05         discordant            Response       20
 9 0.20-0.30 <0.05         discordant            No Response    73
10 0.20-0.30 <0.05         concordant            Response        6
# ℹ 84 more rows
```

Several cells have 0 or few (1-3) responders or nonresponders in a group, therefore estimates with logistic regression may be unstable.

Instead of using logistic regression, use a function to estimate odds ratios within contingency tables. As part of the function apply a correction factor of 0.5 to each cell (this approach is also known as “continuity correction” or “Haldane-Anscombe correction”) due to the sparse data. Then generate the dataset with OR and 95% CI.

▼ Code

```
# Create function
compute_or <- function(dat) {
  tab <- table(dat$first_adj_concordance, dat$response)

  a <- if_else(
    "concordant" %in% rownames(tab) &
      "Response" %in% colnames(tab),
    tab["concordant", "Response"],
    0
    )

  b <- if_else(
    "concordant" %in% rownames(tab) &
      "No Response" %in% colnames(tab),
    tab["concordant", "No Response"],
    0
    )

  c <- if_else(
    "discordant" %in% rownames(tab) &
      "Response" %in% colnames(tab),
```

```
    tab["discordant", "Response"],
    0
    )

  d <- if_else(
    "discordant" %in% rownames(tab) &
      "No Response" %in% colnames(tab),
    tab["discordant", "No Response"],
    0
    )

  a_adj <- a + 0.5
  b_adj <- b + 0.5
  c_adj <- c + 0.5
  d_adj <- d + 0.5

  OR <-
    (a_adj * d_adj) /
    (b_adj * c_adj)

  tibble(
    OR = OR,
    a = a,
    b = b,
    c = c,
    d = d,
    concordant_n = a + b,
    discordant_n = c + d,
    total_n = a + b + c + d
  )
}

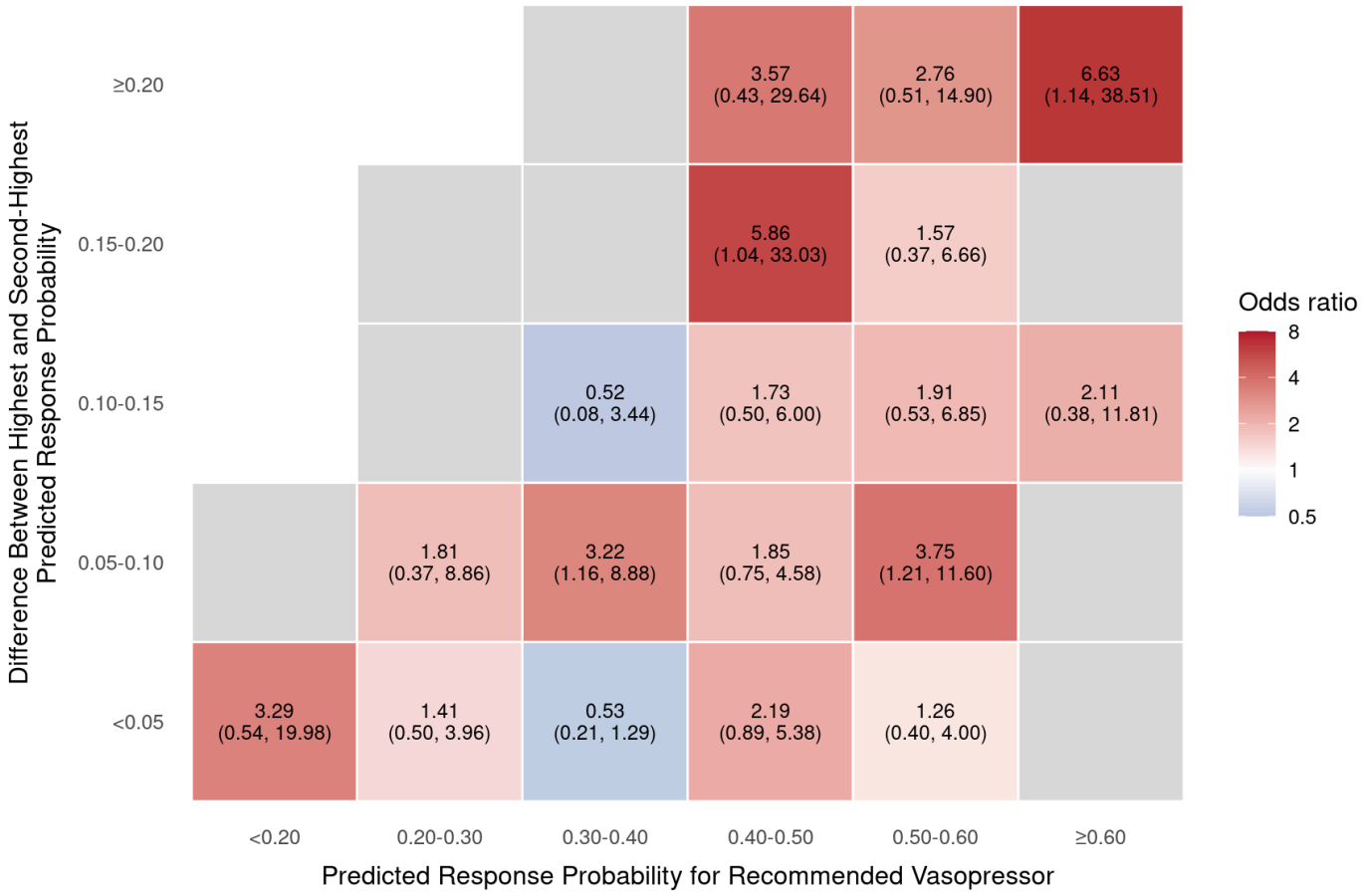
# Generate dataset
plot_dat <- fl_aug |>
  group_by(prob_bin, margin_strata) |>
  group_modify(~compute_or(.x)) |>
  ungroup() |>
  mutate(
    se_log_or = sqrt(
      1/(a + 0.5) +
      1/(b + 0.5) +
      1/(c + 0.5) +
      1/(d + 0.5)
      ),
    lower =
      exp(log(OR) - 1.96 * se_log_or),
    upper =
      exp(log(OR) + 1.96 * se_log_or),
    stable = concordant_n >= 5 & discordant_n >= 5,
    fill_value = if_else(stable, log(OR), NA_real_),
    label = if_else(stable,
```

```
                    paste0(sprintf("%.2f\n(%0.2f, %0.2f)", OR, lower, upper)),
                    ""
                    )
    )
```

Now generate and save the plot.

▼ Code

```
ggplot(plot_dat, aes(x = prob_bin, y = margin_strata, fill = fill_value)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(
    aes(label = label),
    size = 3.3,
    lineheight = 0.9
    ) +
  scale_fill_gradient2(
    low = "#2166AC",
    mid = "white",
    high = "#B2182B",
    midpoint = 0,
    na.value = "grey85",
    name = "Odds ratio",
    limits = c(log(0.5), log(8)),
    breaks = log(c(0.5, 1, 2, 4, 8)),
    labels = c("0.5", "1", "2", "4", "8")
    ) +
  labs(
    x = "Predicted Response Probability for Recommended Vasopressor",
    y = "Difference Between Highest and Second-Highest\nPredicted Response Probability"
    ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid = element_blank(),
    axis.title.x = element_text(margin = margin(t = 8)),
    axis.title.y = element_text(margin = margin(r = 8)),
    legend.title = element_text(margin = margin(b = 8))
    )
```



▼ Code

```
ggsave(
  "resp_or_by_pred_and_margin.tiff",
  device = ragg::agg_tiff,
  width = 9,
  height = 5,
  units = "in",
  dpi = 600,
  compression = "lzw"
)

rm(plot_dat)
```

Figure caption: Heat map showing the odds ratio for response among patients receiving the decision model-recommended adjunctive vasopressor (concordant therapy) versus a non-recommended adjunctive vasopressor (discordant therapy), stratified by the predicted probability of response to the recommended vasopressor and the margin between the highest and second-highest predicted response probabilities. Values within cells represent the odds ratio and corresponding 95% confidence interval. Cell color corresponds to the odds ratio on a logarithmic scale. Gray cells indicate strata with fewer than five concordant or discordant patients, for which odds ratios were not estimated. White cells indicate strata with no observations.

14.5.3.2 Model Probability Difference Strata

In this subgroup analysis create strata for the absolute difference of response probabilities between the adjunctive vasopressor with the highest predicted probability of response and the vasopressor with the second-highest predicted probability. This difference reflects the decision model probability difference and was created above as `margin_strata`.

Examine counts and concordance frequency (rate) within the probability difference strata.

▼ Code

```
fl_aug |>
  group_by(margin_strata) |>
  summarize(
    n = n(),
    concordance_pct =
      round_half_up(mean(first_adj_concordance == "concordant")*100, 1),
    .groups = "drop"
  )
```

```
# A tibble: 5 × 3
  margin_strata     n concordance_pct
  <fct>         <int>           <dbl>
1 <0.05           452            26.8
2 0.05-0.10       315            34.3
3 0.10-0.15       183            36.6
4 0.15-0.20       100            41
5 ≥0.20            78            28.2
```

The strata with larger probability differences have substantially fewer observations than the lowest two strata. Therefore, collapse to only three categories and examine data again.

▼ Code

```
fl_aug <- fl_aug |>
  mutate(margin_strata = cut(margin,
                             breaks = c(0, 0.05, 0.10, Inf),
                             include.lowest = TRUE,
                             right = FALSE,
                             labels = c("<0.05", "0.05-0.10", "≥0.10")
                             )
         )

fl_aug |>
  group_by(margin_strata) |>
  summarize(
    n = n(),
    concordance_pct =
      round_half_up(mean(first_adj_concordance == "concordant")*100, 1),
```

```
    .groups = "drop"
  )
```

```
# A tibble: 3 × 3
  margin_strata     n concordance_pct
  <fct>         <int>           <dbl>
1 <0.05           452            26.8
2 0.05-0.10       315            34.3
3 ≥0.10           361            36
```

These adjusted strata cuts result in more homogenous observation counts across strata, therefore continue with this approach.

Estimate the stratum-specific odds ratio for response with logistic regression. Notably, this approach with `group_by()` can produce the overall stratum-specific odds ratio without the need for multiplying odds ratio estimates for the individual components (which would be needed in a simple call to `glm()` with predictor `first_adj_concordance * margin_strata`). Further, as in the primary analysis, the reference category for `first_adj_concordance` is "discordant", therefore the odds ratio represents the comparison of "concordant" vs. "discordant" treatment.

▼ Code

```
discord_analysis_sens2 <- fl_aug |>
  group_by(margin_strata) |>
  do(
    tidy(
      glm(
        response_bin ~ first_adj_concordance,
        family = binomial,
        data = .
      ),
      exponentiate = TRUE,
      conf.int = TRUE
    )
  ) |>
  filter(term != "(Intercept)") |>
  transmute(
    `Prob Difference` = margin_strata,
    `Response OR` = estimate,
    Lower = conf.low,
    Upper = conf.high
  ) |>
  ungroup() |>
  select(-margin_strata)

discord_analysis_sens2 |>
  kable(align = c('lccc'),
        caption = "Sensitivity: Response Odds Ratio for Concordance by Decision Model
         Probability Difference Strata",
        digits = 2
```

```
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Response Odds Ratio for Concordance by Decision Model Probability Difference Strata

PROB DIFFERENCE	RESPONSE OR	LOWER	UPPER
<0.05	1.30	0.82	2.05
0.05-0.10	3.00	1.80	5.03
≥0.10	2.43	1.52	3.89

There appears to be some evidence for greater odds ratios for response with concordant adjunctive vasopressor therapy with higher decision model probability difference strata. There is no evidence concordance within the lowest stratum (where there is little separation in predicted probability of response between the first- and second-highest vasopressor) is associated with response, but concordance within higher probability difference strata is associated with higher odds for response.

Fit a logistic regression model with an interaction term between concordance and decision model probability difference strata and a model fit with main effects for these two variables alone. Then, compare the two models with AIC and ANOVA, the latter of which will provide an overall (global) p-value for interaction (effect modification) by decision probability difference strata.

▼ Code

```
fit_int <- glm(
  response_bin ~ first_adj_concordance * factor(margin_strata),
  family = binomial,
  data = fl_aug
  )

fit_no_int <- glm(
  response_bin ~ first_adj_concordance + factor(margin_strata),
  family = binomial,
  data = fl_aug
  )

AIC(fit_no_int, fit_int)
```

```
           df      AIC
fit_no_int  4 1317.389
fit_int     6 1314.877
```

▼ Code

```
anova(fit_no_int, fit_int, test = "LRT")
```

```
Analysis of Deviance Table

Model 1: response_bin ~ first_adj_concordance + factor(margin_strata)
Model 2: response_bin ~ first_adj_concordance * factor(margin_strata)
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1      1124     1309.4
2      1122     1302.9  2   6.5117  0.03855 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

▼ Code

```
rm(fit_int, fit_no_int)
```

There is evidence for an interaction by decision model probability difference strata (that the association between concordance and response differs across decision model probability difference strata) with a lower AIC value and likelihood ratio test p = 0.04.

14.5.3.3 Decision Model Odds Ratio Strata

In this subgroup analysis create strata (quartiles) for the relative comparison of response probabilities between the adjunctive vasopressor with the highest predicted probability of response and the vasopressor with the second-highest predicted probability, expressed as a decision model odds ratio.

First, calculate the patient-specific decision model odds ratio (`dm_or_best_vs_second`) as response to the adjunctive vasopressor with the highest predicted probability of response (`best_prob`) versus the second-best option (`second_prob`).

▼ Code

```
fl_aug <- fl_aug |>
  mutate(
    dm_or_best_vs_second =
      (best_prob / (1 - best_prob)) /
      (second_prob / (1 - second_prob))
  )
```

Then, stratify the decision model odds ratio into quartiles (`dm_or_quartile`).

▼ Code

```
fl_aug <- fl_aug |>
  mutate(dm_or_quartile = ntile(dm_or_best_vs_second, 4))
```

Show the decision model odds ratio cutpoints defining the quartiles.

▼ Code

```
round_half_up(
  quantile(
    fl_aug$dm_or_best_vs_second,
    probs = seq(0, 1, by = 0.25)
    ),
  2)
```

```
  0%  25%  50%  75% 100% 
1.00 1.15 1.36 1.72 5.48 
```

Quartile cutpoints were 1.15, 1.36, and 1.72, therefore quartiles are represented by decision model odds ratios as follows: Q1 (1.00-1.15), Q2 (>1.15-1.36), Q3 (>1.36-1.72), and Q4 (>1.72-5.48).

Methods wording: Patient-specific decision-model odds ratios were calculated as the predicted odds of response for the highest-ranked adjunctive vasopressor divided by the predicted odds of response for the second-highest-ranked adjunctive vasopressor. Patients were then stratified into quartiles of the decision-model odds ratio (Q1: 1.00-1.15, Q2: >1.15-1.36, Q3: >1.36-1.72, Q4: >1.72-5.48).

Potential results wording (but this has a lot of overlap with the wording above for the methods): The decision-model odds ratio was generally modest, with quartile boundaries at 1.15, 1.36, and 1.72. Thus, patients in the lowest quartile had little separation between the top two treatment recommendations (decision model odds ratio 1.00-1.15), whereas those in the highest quartile had substantially greater separation (decision model odds ratio >1.72) and stronger model preference for the recommended adjunctive vasopressor.”

Next, show the connection between the decision model odds ratio quartiles (a relative approach) and the absolute value approach for the difference in predicted probability <0.05 (the zone of practical equivalence explored above).

▼ Code

```
fl_aug |>
  group_by(dm_or_quartile) |>
  summarize(
    pct_margin_lt_005 = round_half_up(
      mean(margin < 0.05)*100,
      1)
  )
```

```
# A tibble: 4 × 2
  dm_or_quartile pct_margin_lt_005
           <int>             <dbl>
1              1             100  
2              2              56.4
3              3               3.5
4              4               0.4
```

There is substantial alignment between the absolute and relative approaches. All patients in the lowest quartile of the decision-model odds ratio had an absolute probability difference of less than 0.05, whereas only 3.5% and 0.4% of patients in the third and fourth quartiles, respectively, fell below this threshold.

Show the median (IQR) decision model odds ratio and range by quartile.

▼ Code

```
fl_aug |>
  group_by(dm_or_quartile) |>
  summarize(
    n = n(),
    `Decision-model OR` =
      sprintf(
        "%.2f (%.2f, %.2f)",
        median(dm_or_best_vs_second),
        quantile(dm_or_best_vs_second, 0.25),
        quantile(dm_or_best_vs_second, 0.75)
      ),
    Range =
      sprintf(
        "%.2f to %.2f",
        min(dm_or_best_vs_second),
        max(dm_or_best_vs_second)
      )
  ) |>
  rename(quartile = dm_or_quartile)
```

```
# A tibble: 4 × 4
  quartile     n `Decision-model OR` Range
     <int> <int> <chr>               <chr>
1        1   282 1.07 (1.04, 1.11)   1.00 to 1.15
2        2   282 1.24 (1.20, 1.29)   1.15 to 1.36
3        3   282 1.51 (1.43, 1.62)   1.36 to 1.72
4        4   282 2.07 (1.85, 2.50)   1.73 to 5.48
```

The median (IQR) decision-model odds ratio increased across quartiles from 1.07 (1.04, 1.11) in quartile 1 to 2.07 (1.85, 2.50) in Q4, indicating progressively greater separation between the highest- and second-ranked treatment options.

Examine concordance frequency (rate) within the quartiles.

▼ Code

```
fl_aug |>
  group_by(dm_or_quartile) |>
  summarize(
    n = n(),
```

```
    concordance_freq =
      mean(first_adj_concordance == "concordant"),
    .groups = "drop"
  )
```

```
# A tibble: 4 × 3
  dm_or_quartile     n concordance_freq
           <int> <int>            <dbl>
1              1   282            0.280
2              2   282            0.312
3              3   282            0.351
4              4   282            0.330
```

Examine response frequency (rate) within the quartiles.

▼ Code

```
fl_aug |>
  group_by(dm_or_quartile, first_adj_concordance) |>
  summarize(
    n = n(),
    response_prop =
      mean(response_bin),
    .groups = "drop"
  )
```

```
# A tibble: 8 × 4
  dm_or_quartile first_adj_concordance     n response_prop
           <int> <fct>                 <int>         <dbl>
1              1 discordant              203         0.300
2              1 concordant               79         0.316
3              2 discordant              194         0.237
4              2 concordant               88         0.420
5              3 discordant              183         0.169
6              3 concordant               99         0.343
7              4 discordant              189         0.212
8              4 concordant               93         0.441
```

Estimate the stratum-specific (quartile-specific) odds ratio for response with logistic regression. Notably, this approach with `group_by()` can produce the overall stratum-specific odds ratio without the need for multiplying odds ratio estimates for the individual components (which would be needed in a simple call to `glm()` with predictor `first_adj_concordance * dm_or_quartile`). Further, as in the primary analysis, the reference category for `first_adj_concordance` is “discordant”, therefore the odds ratio represents the comparison of “concordant” vs. “discordant” treatment.

▼ Code

```
discord_analysis_sens3 <- fl_aug |>
  group_by(dm_or_quartile) |>
```

```
  do(
    tidy(
      glm(
        response_bin ~ first_adj_concordance,
        family = binomial,
        data = .
      ),
      exponentiate = TRUE,
      conf.int = TRUE
    )
  ) |>
  filter(term != "(Intercept)") |>
  transmute(
    Quartile = dm_or_quartile,
    `Response OR` = estimate,
    Lower = conf.low,
    Upper = conf.high
  ) |>
  ungroup() |>
  select(-dm_or_quartile)

discord_analysis_sens3 |>
  kable(align = c('lccc'),
        caption = "Sensitivity: Response Odds Ratio for Concordance by Decision Model Odds Ratio
         Stratum",
        digits = 2
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Response Odds Ratio for Concordance by Decision Model Odds Ratio Stratum

QUARTILE	RESPONSE OR	LOWER	UPPER
1	1.08	0.61	1.88
2	2.33	1.36	4.00
3	2.56	1.46	4.54
4	2.94	1.72	5.05

There appears to be some evidence for greater odds ratios for response with concordant adjunctive vasopressor therapy with higher decision model odds ratio quartiles. There is no evidence concordance within the lowest decision model odds ratio quartile (where there is little separation in predicted probability of response between the first- and second-highest vasopressor) is associated with observed response, but concordance within higher quartiles is associated with higher odds for response. Notably, while the odds ratio point estimates for response with concordance increases with increasing decision model odds ratio quartile, the point estimates and 95% confidence intervals are relatively similar within quartiles 2-4, suggesting a potential threshold effect (there does not appear to be a linear effect).

Fit a logistic regression model with an interaction term between concordance and decision model odds ratio quartile (stratum) and a model fit with main effects for these two variables alone. Then, compare the two models with AIC and ANOVA, the latter of which will provide an overall (global) p-value for interaction (effect modification) by decision model odds ratio.

▼ Code

```
fit_int <- glm(
  response_bin ~ first_adj_concordance * factor(dm_or_quartile),
  family = binomial,
  data = fl_aug
  )

fit_no_int <- glm(
  response_bin ~ first_adj_concordance + factor(dm_or_quartile),
  family = binomial,
  data = fl_aug
  )

AIC(fit_no_int, fit_int)
```

```
           df      AIC
fit_no_int  5 1313.371
fit_int     8 1311.666
```

▼ Code

```
anova(fit_no_int, fit_int, test = "LRT")
```

```
Analysis of Deviance Table

Model 1: response_bin ~ first_adj_concordance + factor(dm_or_quartile)
Model 2: response_bin ~ first_adj_concordance * factor(dm_or_quartile)
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1      1123     1303.4
2      1120     1295.7  3   7.7048  0.05252 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

▼ Code

```
rm(fit_int, fit_no_int)
```

There is some evidence for an interaction by quartile (that the association between concordance and response differs across decision-model odds ratio quartiles) with a lower AIC value and likelihood ratio test p = 0.053.

14.5.4 Sensitivity: Practical Equivalence

Incorporate zones of practical equivalence for the recommended adjunctive vasopressor (compared with the second-best) based on the predicted probabilities of response from the models. Explore possible thresholds for the zone(s) based on the distribution (proportion with `margin` below exploratory thresholds).

▼ Code

```
fl_aug |>
  summarize(
    pct_lt_001 = round_half_up(mean(margin < 0.01)*100, 1),
    pct_lt_005 = round_half_up(mean(margin < 0.05)*100, 1),
    pct_lt_010 = round_half_up(mean(margin < 0.10)*100, 1),
    pct_lt_015 = round_half_up(mean(margin < 0.15)*100, 1),
    pct_lt_020 = round_half_up(mean(margin < 0.20)*100, 1),
    pct_lt_025 = round_half_up(mean(margin < 0.25)*100, 1)
  )
```

```
# A tibble: 1 × 6
  pct_lt_001 pct_lt_005 pct_lt_010 pct_lt_015 pct_lt_020 pct_lt_025
       <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
1        8.5       40.1         68       84.2       93.1       97.3
```

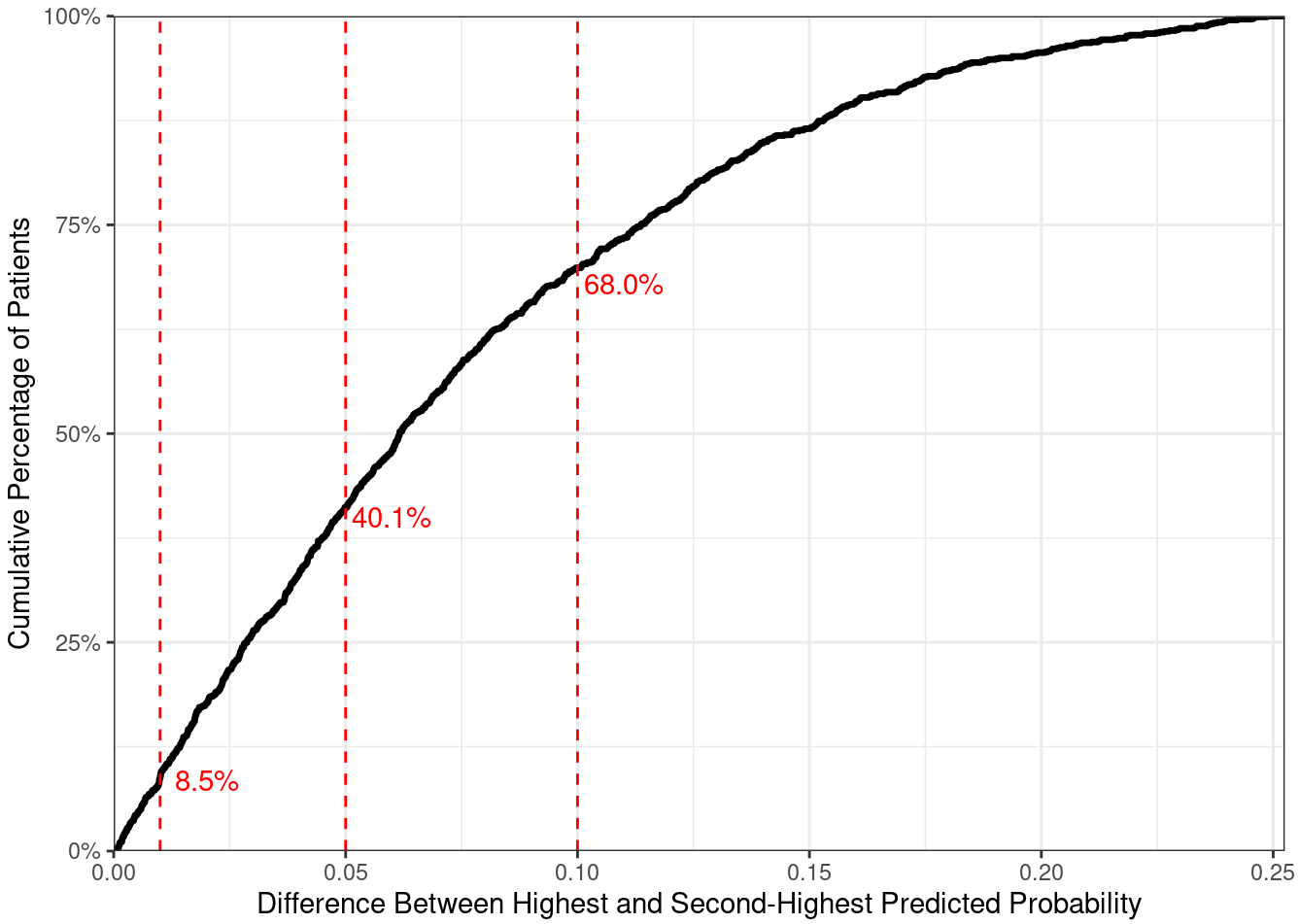
There were 40% of patients with a difference within 0.05 in the predicted probability (5 percentage points) between the adjunctive vasopressor with the highest and second-highest predicted probability of response. As shown in Section 14.3, the median predicted response probabilities for the adjunctive vasopressors ranged from 0.286 to 0.35, therefore a difference in probabilities of 0.05 is approximately a 15% relative difference.

Visualize `margin` by plotting the empirical cumulative distribution, showing the percentage of data points (y-axis) less than or equal to any given value of `margin` (x-axis). Include vertical lines at the empirical thresholds with the percentage of values below the threshold.

▼ Code

```
ggplot(fl_aug, aes(x = margin)) +
  stat_ecdf(linewidth = 1.2) +
  geom_vline(xintercept = c(0.01, 0.05, 0.10), linetype = 2, color = "red") +
  annotate("text", x = 0.02, y = 0.085, label = "8.5%", color = "red") +
  annotate("text", x = 0.06, y = 0.40, label = "40.1%", color = "red") +
  annotate("text", x = 0.11, y = 0.68, label = "68.0%", color = "red") +
  scale_y_continuous(labels = percent, expand = c(0, 0)) +
  scale_x_continuous(limits = c(0,0.25), breaks = seq(0, 0.25, by = 0.05),
                     expand = expansion(mult = c(0, 0.01))
                     ) +
  labs(
    x = "Difference Between Highest and Second-Highest Predicted Probability",
    y = "Cumulative Percentage of Patients"
```

```
  ) +
  theme_bw()
```



Show the median (IQR) `margin` between highest and second-highest predicted probability by recommended first adjunct.

▼ Code

```
fl_aug |>
  group_by(rec_first_adj) |>
  summarize(
    median_margin = round_half_up(median(margin),3),
    q1 = round_half_up(quantile(margin, 0.25),3),
    q3 = round_half_up(quantile(margin, 0.75),3)
  )
```

```
# A tibble: 3 × 4
  rec_first_adj median_margin    q1    q3
  <fct>                <dbl> <dbl> <dbl>
1 vasopressin          0.073 0.035 0.128
2 epinephrine          0.043 0.018 0.072
3 phenylephrine        0.081 0.039 0.144
```

When epinephrine was the recommended adjunctive vasopressor the median (IQR) difference (`margin`) between epinephrine and the second-highest recommended adjunctive vasopressor was smaller (0.043 [0.018, 0.072]) compared with cases when the recommended adjunctive vasopressor was vasopressin (0.073 [0.035, 0.128]) or phenylephrine (0.081 [0.039, 0.144]). This suggests that recommendations were weaker (less strongly in favor versus an alternative) for epinephrine as the preferred adjunctive vasopressor as compared with when the other two adjunctive vasopressors were recommended.

Show a scatterplot panel of the predicted probabilities of response by adjunctive vasopressor, incorporating equivalence bands of +/- 0.05 and +/- 0.1.

▼ Code

```
# Common equivalence bands, plotting theme, and limits
eq_band_10 <- tibble(
  x = seq(0, 0.85, by = 0.01),
  ymin = pmax(0, x - 0.10),
  ymax = pmin(0.85, x + 0.10)
)

eq_band_05 <- tibble(
  x = seq(0, 0.85, by = 0.01),
  ymin = pmax(0, x - 0.05),
  ymax = pmin(0.85, x + 0.05)
)

plot_theme <- theme_bw() +
  theme(panel.grid.minor = element_blank(),
        axis.title = element_text(size = 11),
        axis.text = element_text(size = 10)
        )

make_plot <- function(data, xvar, yvar, xlab, ylab) {
  ggplot(data, aes({{ xvar }}, {{ yvar }})) +
    geom_ribbon(data = eq_band_10,
                aes(x = x, ymin = ymin, ymax = ymax),
                inherit.aes = FALSE,
                fill = "grey80",
                alpha = 0.6
                ) +
    geom_ribbon(data = eq_band_05,
                aes(x = x, ymin = ymin, ymax = ymax),
                inherit.aes = FALSE,
                fill = "#A9CCE3",
                alpha = 0.7
                ) +
    geom_abline(slope = 1, intercept = 0,
                color = "navyblue", linetype = 2, linewidth = 0.8, alpha = 0.7
                ) +
```

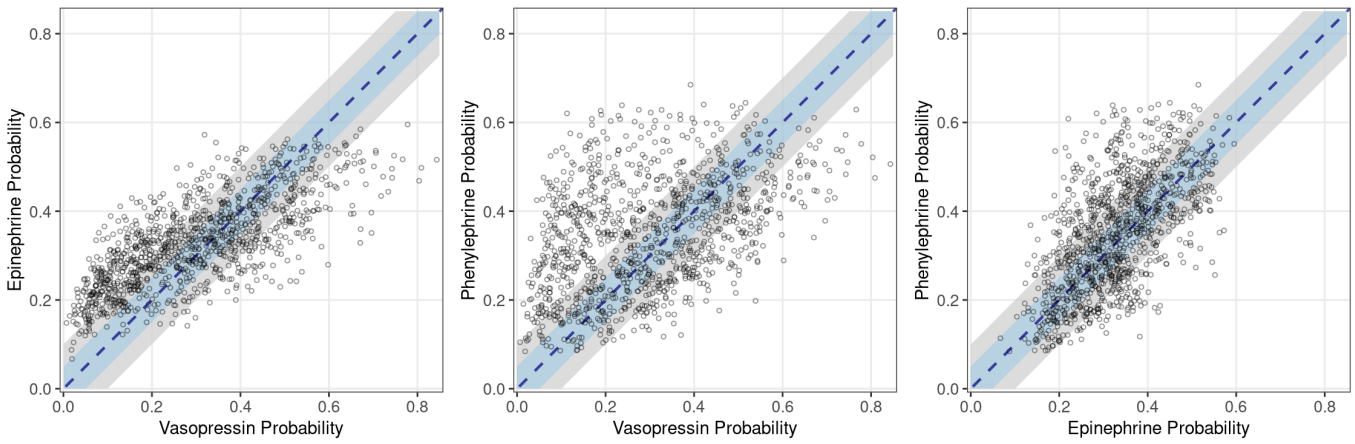
```
    geom_point(alpha = 0.4, size = 1, shape = 1, color = "black") +
    coord_equal() +
    scale_x_continuous(limits = c(0, 0.85), breaks = seq(0, 0.8, 0.2),
                       expand = expansion(mult = c(0.01, 0.01))
                       ) +
    scale_y_continuous(limits = c(0, 0.85), breaks = seq(0, 0.8, 0.2),
                       expand = expansion(mult = c(0.01, 0.01))
                       ) +
    labs(x = xlab, y = ylab) +
    plot_theme
}

p1 <- make_plot(fl_aug,
                avp_pred_response_prob, epi_pred_response_prob,
                "Vasopressin Probability", "Epinephrine Probability"
                )

p2 <- make_plot(fl_aug,
                avp_pred_response_prob, pe_pred_response_prob,
                "Vasopressin Probability", "Phenylephrine Probability"
                )

p3 <- make_plot(fl_aug,
                epi_pred_response_prob, pe_pred_response_prob,
                "Epinephrine Probability", "Phenylephrine Probability"
                )

(scatter_pred_prob_panel <- (p1 | p2 | p3))
```



▼ Code

```
rm(eq_band_10, eq_band_05, plot_theme, make_plot, p1, p2, p3)
```

In this plot each point represents an individual patient, with predicted probability of response to the vasopressor listed on the axes contrasted. The dashed line represents equal predicted probabilities of response for the two vasopressors being compared in the panel, and the shaded area

surrounding the dashed line represents probability ±0.05 (considered a “zone of practical equivalence”). Therefore, points within the shaded area may be considered clinically equivalent predicted probability for response to the two vasopressors. Points above the shaded area indicate the y-axis adjunctive vasopressor has a meaningfully higher response predicted probability, while points below the shaded area indicate the x-axis adjunctive vasopressor has a meaningfully higher response predicted probability.

The figures indicate that while predictions for all three models are positively associated, there is substantial patient-level heterogeneity in predictions that creates individualized treatment recommendations. Agreement was strongest between the vasopressin and epinephrine model, with points clustered near the dashed line of identity. Yet, at low probabilities (<0.3) epinephrine appeared to be favored more commonly. The comparison between vasopressin and phenylephrine showed the largest heterogeneity, with vertical spread across the range of vasopressin probabilities. This indicates the vasopressin and phenylephrine models generated the greatest treatment differentiation - patients with similar predicted response to vasopressin may have markedly different predicted response to phenylephrine. In the epinephrine vs. phenylephrine comparison the spread is more than in the vasopressin vs. epinephrine comparison but less than in the vasopressin vs. phenylephrine comparison (particularly when considering horizontal spread in addition to vertical spread). This indicates the epinephrine and phenylephrine predictions are related but still distinguish clinically meaningful subgroups of patients.

Potential manuscript interpretation: Pairwise comparisons of predicted probabilities of response demonstrated positive associations among the vasopressin, epinephrine, and phenylephrine response models. However, substantial dispersion around the line of equality was observed, particularly for comparisons involving phenylephrine, indicating meaningful patient-level differences in predicted treatment response.

Save the scatter plot

▼ Code

```
ggsave(
  "scatter_pred_prob_panel.tiff",
  plot = scatter_pred_prob_panel,
  device = ragg::agg_tiff,
  width = 12,
  height = 4,
  units = "in",
  dpi = 600,
  compression = "lzw"
)
```

14.5.4.1 Equivalence as Difference <=0.05

Consider an adjunctive vasopressor a recommended adjunct if the predicted probability for response for that vasopressor is within 0.05 of the adjunctive vasopressor with the highest

probability (`best_prob`). Then consider concordance as receipt of a recommended adjunct based on the zone of practical equivalence (difference <=0.05).

▼ Code

```
fl_aug <-
  fl_aug |>
  rowwise() |>
  mutate(rec_avp_sens = avp_pred_response_prob > (best_prob - 0.05),
         rec_epi_sens = epi_pred_response_prob > (best_prob - 0.05),
         rec_pe_sens  = pe_pred_response_prob  > (best_prob - 0.05)
         ) |>
  ungroup() |>
  mutate(
    first_adj_concordance_sens = factor(
      (first_adj == "vasopressin" & rec_avp_sens == TRUE) |
        (first_adj == "epinephrine" & rec_epi_sens == TRUE) |
        (first_adj == "phenylephrine" & rec_pe_sens == TRUE),
      levels = c(FALSE, TRUE),
      labels = c("discordant", "concordant")
      )
    )
```

Show a count of the number of recommended adjunctive vasopressors (number within the zone of practical equivalence for being highest/recommended).

▼ Code

```
fl_aug |>
  mutate(n_recommended = rec_avp_sens + rec_epi_sens + rec_pe_sens) |>
  tabyl(n_recommended) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  kable(align = c('lcc'),
        caption = "Sensitivity (Difference <=0.05 as Equivalent): Number of Recommended
         Adjunctive Vasopressors"
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity (Difference <=0.05 as Equivalent): Number of Recommended Adjunctive Vasopressors

N_RECOMMENDED	N	PERCENT
1	676	59.9%
2	384	34.0%
3	68	6.0%
Total	1128	100.0%

In the primary analysis only 1 adjunctive vasopressor was recommended. In this sensitivity analysis 34% of patients had 2 acceptable adjunctive vasopressor regimens and in 6% all 3 adjunctive vasopressor regimens were considered acceptable/clinically equivalent.

Show a table of concordance based on this sensitivity criterion.

▼ Code

```
fl_aug |> tabyl(first_adj_concordance_sens) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  rename(Concordance = first_adj_concordance_sens) |>
  kable(align = c('lcc'),
        caption = "Sensitivity: Concordance of Clinically Equivalent Decision Model
         Recommendation and Observed Adjunct"
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Concordance of Clinically Equivalent Decision Model Recommendation and Observed Adjunct

CONCORDANCE	N	PERCENT
discordant	604	53.5%
concordant	524	46.5%
Total	1128	100.0%

In the primary analysis, where only one adjunctive vasopressor was recommended, 31.8% of patients received decision model-concordant therapy. When recommendations within a probability of 0.05 (5 percentage points) of the highest predicted response probability were considered clinically equivalent, concordance increased to 46.5%.

Show a table of response by concordance.

▼ Code

```
fl_aug |> tabyl( response, first_adj_concordance_sens) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  kable(align = c('lcc')) |>
  kable_paper(full_width = FALSE)
```

RESPONSE	DISCORDANT	CONCORDANT	TOTAL
Response	128 (21.2%)	187 (35.7%)	315 (27.9%)
No Response	476 (78.8%)	337 (64.3%)	813 (72.1%)

RESPONSE	DISCORDANT	CONCORDANT	TOTAL
Total	604 (100.0%)	524 (100.0%)	1,128 (100.0%)

Determine the association of concordance per the sensitivity definition with first adjunctive vasopressor response.

▼ Code

```
discord_analysis_sens1 <- glm(response_bin ~ first_adj_concordance_sens,
                        data = fl_aug, family = binomial(link = "logit")
                        )

tidy(discord_analysis_sens1, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  mutate(term = sub("^first_adj_concordance_sens", "", term)) |>
  select("Term" = term, "OR" = estimate,
         "95% CI, Lower" = conf.low, "95% CI, Upper" = conf.high) |>
  kable(align = c('lcccc'),
        caption = "Sensitivity: Logistic Regression for Adjunctive Vasopressor Response by
         Concordance (Practical Equivalence <=0.05)",
        digits = 2
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Logistic Regression for Adjunctive Vasopressor Response by Concordance (Practical Equivalence <=0.05)

TERM	OR	95% CI, LOWER	95% CI, UPPER
concordant	2.06	1.59	2.69

In this sensitivity analysis, we acknowledged that the adjunctive vasopressor with the highest predicted probability of response may be essentially tied with the next-best adjunctive vasopressor. Therefore we incorporated a zone of practical equivalence (absolute probability within 0.05 [5 percentage points], or about a 15% relative difference) for the model-recommended adjunctive vasopressor. With this sensitivity analysis criterion, 40% of patients had more than one recommended adjunctive vasopressor. Accordingly, concordance between a model-recommended therapy and the observed therapy increased from 31.8% in the primary analysis to 46.5% in this sensitivity analysis. However, the association between concordance and response was virtually unchanged (OR 2.05 [95% CI 1.56, 2.69] in the primary analysis versus OR 2.06 [95% CI 1.59, 2.69] in the sensitivity analysis). As such, the decision model appears robust to a reasonable definition of practical equivalence.

14.5.4.1.1 Corollary Analysis

As a corollary analysis, show a table of response by concordance within strata of practical equivalence.

▼ Code

```
fl_aug <- fl_aug |>
  mutate(margin_equiv_005 = factor(if_else(margin <=0.05,
                                           "Difference <=0.05",
                                           "Difference >0.05"
                                           )
                                   )
         )

fl_aug |> tabyl(first_adj_concordance_sens, response, margin_equiv_005) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "row")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front")
```

```
$`Difference <=0.05`
 first_adj_concordance_sens    Response No Response        Total
                 discordant  36 (21.7%) 130 (78.3%) 166 (100.0%)
                 concordant  88 (30.8%) 198 (69.2%) 286 (100.0%)
                      Total 124 (27.4%) 328 (72.6%) 452 (100.0%)

$`Difference >0.05`
 first_adj_concordance_sens    Response No Response        Total
                 discordant  92 (21.0%) 346 (79.0%) 438 (100.0%)
                 concordant  99 (41.6%) 139 (58.4%) 238 (100.0%)
                      Total 191 (28.3%) 485 (71.7%) 676 (100.0%)
```

Then, estimate the stratum-specific odds ratio for response with concordance (using the sensitivity criteria of concordance as difference <=0.05).

▼ Code

```
fl_aug |>
  group_by(margin_equiv_005) |>
  do(
    tidy(
      glm(
        response_bin ~ first_adj_concordance_sens,
        family = binomial,
        data = .
      ),
      exponentiate = TRUE,
      conf.int = TRUE
    )
  ) |>
  filter(term != "(Intercept)") |>
  transmute(
    `Prob Difference` = margin_equiv_005,
    `Response OR` = estimate,
```

```
    Lower = conf.low,
    Upper = conf.high
  ) |>
  ungroup() |>
  select(-margin_equiv_005) |>
  kable(align = c('lccc'), digits = 2,
        caption = "Sensitivity: Response for Concordance (With Difference <=0.05 as Concordant)
         by Difference") |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Response for Concordance (With Difference <=0.05 as Concordant) by Difference

PROB DIFFERENCE	RESPONSE OR	LOWER	UPPER
Difference <=0.05	1.60	1.03	2.53
Difference >0.05	2.68	1.90	3.79

14.5.4.2 Equivalence as Difference <=0.1

Because the definition of (criteria for) practical equivalence is subjective, alternatively consider a difference <=0.10 as equivalent. Use the same approach as for the above section (with <=0.05 as equivalent) with minimal annotation.

▼ Code

```
fl_aug <-
  fl_aug |>
  rowwise() |>
  mutate(rec_avp_sens2 = avp_pred_response_prob > (best_prob - 0.1),
         rec_epi_sens2 = epi_pred_response_prob > (best_prob - 0.1),
         rec_pe_sens2  = pe_pred_response_prob  > (best_prob - 0.1)
         ) |>
  ungroup() |>
  mutate(
    first_adj_concordance_sens2 = factor(
      (first_adj == "vasopressin" & rec_avp_sens2 == TRUE) |
        (first_adj == "epinephrine" & rec_epi_sens2 == TRUE) |
        (first_adj == "phenylephrine" & rec_pe_sens2 == TRUE),
      levels = c(FALSE, TRUE),
      labels = c("discordant", "concordant")
      )
    )
```

Show a count of the number of recommended adjunctive vasopressors (number within the zone of practical equivalence for being highest/recommended).

▼ Code

```r
fl_aug |>
  mutate(n_recommended = rec_avp_sens2 + rec_epi_sens2 + rec_pe_sens2) |>
  tabyl(n_recommended) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  kable(align = c('lcc'),
        caption = "Sensitivity (Difference <=0.1 as Equivalent): Number of Recommended
         Adjunctive Vasopressors"
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity (Difference <=0.1 as Equivalent): Number of Recommended Adjunctive Vasopressors

N_RECOMMENDED	N	PERCENT
1	361	32.0%
2	438	38.8%
3	329	29.2%
Total	1128	100.0%

In this sensitivity analysis, 39% of patients had 2 acceptable adjunctive vasopressor regimens and in 29% all 3 adjunctive vasopressor regimens were considered acceptable/clinically equivalent.

Show a table of concordance based on this sensitivity criterion.

▼ Code

```r
fl_aug |> tabyl(first_adj_concordance_sens2) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  rename(Concordance = first_adj_concordance_sens2) |>
  kable(align = c('lcc'),
        caption = "Sensitivity (Difference <=0.1 as Equivalent): Concordance of Clinically
         Equivalent Decision Model Recommendation and Observed Adjunct"
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity (Difference <=0.1 as Equivalent): Concordance of Clinically Equivalent Decision Model Recommendation and Observed Adjunct

CONCORDANCE	N	PERCENT
discordant	432	38.3%
concordant	696	61.7%
Total	1128	100.0%

When recommendations within a probability of 0.1 (10 percentage points) of the highest predicted response probability were considered clinically equivalent, concordance increased to 61.7%.

Show a table of response by concordance.

▼ Code

```
fl_aug |> tabyl(response, first_adj_concordance_sens2) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  kable(align = c('lcc')) |>
  kable_paper(full_width = FALSE)
```

RESPONSE	DISCORDANT	CONCORDANT	TOTAL
Response	85 (19.7%)	230 (33.0%)	315 (27.9%)
No Response	347 (80.3%)	466 (67.0%)	813 (72.1%)
Total	432 (100.0%)	696 (100.0%)	1,128 (100.0%)

Determine the association of concordance per the sensitivity definition with first adjunctive vasopressor response.

▼ Code

```
discord_analysis_sens1b <- glm(response_bin ~ first_adj_concordance_sens2,
                        data = fl_aug, family = binomial(link = "logit")
                        )

tidy(discord_analysis_sens1b, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  mutate(term = sub("^first_adj_concordance_sens2", "", term)) |>
  select("Term" = term, "OR" = estimate,
         "95% CI, Lower" = conf.low, "95% CI, Upper" = conf.high) |>
  kable(align = c('lcccc'),
        caption = "Sensitivity: Logistic Regression for Adjunctive Vasopressor Response by
         Concordance (Practical Equivalence <=0.1)",
        digits = 2
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Logistic Regression for Adjunctive Vasopressor Response by Concordance (Practical Equivalence <=0.1)

TERM	OR	95% CI, LOWER	95% CI, UPPER
concordant	2.01	1.52	2.69

In this sensitivity analysis, we acknowledged that the adjunctive vasopressor with the highest predicted probability of response may be essentially tied with the next-best adjunctive vasopressor. Therefore we incorporated a zone of practical equivalence (absolute probability within 0.1 [10 percentage points], or about a 30% relative difference) for the model-recommended adjunctive vasopressor. With this sensitivity analysis criterion, 68% of patients had more than one recommended adjunctive vasopressor. Accordingly, concordance between a model-recommended therapy and the observed therapy increased to 61.7% (from 31.8% in the primary analysis and 46.5% in the sensitivity analysis with difference <=0.05 considered equivalent). However, the association between concordance and response was virtually unchanged (OR 2.01 [95% CI 1.52, 2.69] in this sensitivity analysis with difference <=0.1 as equivalent, primary analysis: OR 2.05 [95% CI 1.56, 2.69], and sensitivity with difference <=0.05 as equivalent: OR 2.06 [95% CI 1.59, 2.69]). As such, the decision model appears robust to differing definitions of practical equivalence.

#####Corollary Analysis

As a corollary analysis, show a table of response by concordance within strata of practical equivalence.

▼ Code

```
fl_aug <- fl_aug |>
  mutate(margin_equiv_01 = factor(if_else(margin <=0.1,
                                          "Difference <=0.1",
                                          "Difference >0.1"
                                          )
                                  )
         )

fl_aug |> tabyl(first_adj_concordance_sens2, response, margin_equiv_01) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "row")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front")
```

```
$`Difference <=0.1`
 first_adj_concordance_sens2    Response No Response        Total
                  discordant  34 (16.9%) 167 (83.1%) 201 (100.0%)
                  concordant 177 (31.3%) 389 (68.7%) 566 (100.0%)
                       Total 211 (27.5%) 556 (72.5%) 767 (100.0%)

$`Difference >0.1`
 first_adj_concordance_sens2    Response No Response        Total
                  discordant  51 (22.1%) 180 (77.9%) 231 (100.0%)
                  concordant  53 (40.8%)  77 (59.2%) 130 (100.0%)
                       Total 104 (28.8%) 257 (71.2%) 361 (100.0%)
```

Then, estimate the stratum-specific odds ratio for response with concordance (using the sensitivity criteria of concordance as difference <=0.1).

▼ Code

```
fl_aug |>
  group_by(margin_equiv_01) |>
  do(
    tidy(
      glm(
        response_bin ~ first_adj_concordance_sens2,
        family = binomial,
        data = .
      ),
      exponentiate = TRUE,
      conf.int = TRUE
    )
  ) |>
  filter(term != "(Intercept)") |>
  transmute(
    `Prob Difference` = margin_equiv_01,
    `Response OR` = estimate,
    Lower = conf.low,
    Upper = conf.high
  ) |>
  ungroup() |>
  select(-margin_equiv_01) |>
  kable(align = c('lccc'), digits = 2,
        caption = "Sensitivity: Response for Concordance (With Difference <=0.1 as Concordant)
         by Difference") |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Response for Concordance (With Difference <=0.1 as Concordant) by Difference

PROB DIFFERENCE	RESPONSE OR	LOWER	UPPER
Difference <=0.1	2.23	1.50	3.41
Difference >0.1	2.43	1.52	3.89

14.5.4.3 Alternative Method

As an alternative approach for this sensitivity analysis, define observations (patients) as having predicted probability within the equivalence band or outside of the equivalence band. Then, determine the association of concordance with response by presence (or not) of probabilities within the zone of practical equivalence.

First, define the variable and show the number of observations meeting or not meeting the criteria.

▼ Code

```
fl_aug <- fl_aug |>
  mutate(margin_equiv_005 =
           factor(if_else(margin <=0.05,
                          "Difference <=0.05",
                          "Difference >0.05"))
         )

fl_aug |> tabyl(margin_equiv_005)
```

```
  margin_equiv_005   n   percent
Difference <=0.05 452 0.4007092
 Difference >0.05 676 0.5992908
```

Show the frequency of concordance within the difference strata.

▼ Code

```
fl_aug |> tabyl(margin_equiv_005, first_adj_concordance) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "row")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  rename("Difference (row)/Concordance (col)" = `margin_equiv_005/first_adj_concordance`) |>
  kable(align = c('lcc')) |>
  kable_paper(full_width = FALSE)
```

DIFFERENCE (ROW)/CONCORDANCE (COL)	DISCORDANT	CONCORDANT	TOTAL
Difference <=0.05	331 (73.2%)	121 (26.8%)	452 (100.0%)
Difference >0.05	438 (64.8%)	238 (35.2%)	676 (100.0%)
Total	769 (68.2%)	359 (31.8%)	1,128 (100.0%)

Show the frequency of response with concordant therapy by difference stratum.

▼ Code

```
fl_aug |> tabyl(first_adj_concordance, response, margin_equiv_005) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "row")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front")
```

```
$`Difference <=0.05`
 first_adj_concordance    Response No Response        Total
           discordant  86 (26.0%) 245 (74.0%) 331 (100.0%)
           concordant  38 (31.4%)  83 (68.6%) 121 (100.0%)
                Total 124 (27.4%) 328 (72.6%) 452 (100.0%)
```

```
$`Difference >0.05`
 first_adj_concordance    Response No Response        Total
            discordant  92 (21.0%) 346 (79.0%) 438 (100.0%)
            concordant  99 (41.6%) 139 (58.4%) 238 (100.0%)
                 Total 191 (28.3%) 485 (71.7%) 676 (100.0%)
```

Then, estimate the stratum-specific odds ratio for response with logistic regression.

▼ Code

```
discord_analysis_sens1_alt <- fl_aug |>
  group_by(margin_equiv_005) |>
  do(
    tidy(
      glm(
        response_bin ~ first_adj_concordance,
        family = binomial,
        data = .
      ),
      exponentiate = TRUE,
      conf.int = TRUE
    )
  ) |>
  filter(term != "(Intercept)") |>
  transmute(
    `Prob Difference` = margin_equiv_005,
    `Response OR` = estimate,
    Lower = conf.low,
    Upper = conf.high
  ) |>
  ungroup() |>
  select(-margin_equiv_005)

discord_analysis_sens1_alt |>
  kable(align = c('lccc'),
        caption = "Sensitivity Alternative Approach: Response for Concordance by Difference
         <=0.05",
        digits = 2
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity Alternative Approach: Response for Concordance by Difference <=0.05

PROB DIFFERENCE	RESPONSE OR	LOWER	UPPER
Difference <=0.05	1.30	0.82	2.05
Difference >0.05	2.68	1.90	3.79

Use a similar approach as above except defining the criteria for equivalence as a probability difference <=0.1.

First, define the variable and show the number of observations meeting or not meeting the criteria.

▼ Code

```
fl_aug <- fl_aug |>
  mutate(margin_equiv_01 =
           factor(if_else(margin <=0.1,
                          "Difference <=0.1",
                          "Difference >0.1"))
         )

fl_aug |> tabyl(margin_equiv_01)
```

```
 margin_equiv_01   n   percent
Difference <=0.1 767 0.6799645
 Difference >0.1 361 0.3200355
```

Show the frequency of concordance within the difference strata.

▼ Code

```
fl_aug |> tabyl(margin_equiv_01, first_adj_concordance) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "row")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  rename("Difference (row)/Concordance (col)" = `margin_equiv_01/first_adj_concordance`) |>
  kable(align = c('lcc')) |>
  kable_paper(full_width = FALSE)
```

DIFFERENCE (ROW)/CONCORDANCE (COL)	DISCORDANT	CONCORDANT	TOTAL
Difference <=0.1	538 (70.1%)	229 (29.9%)	767 (100.0%)
Difference >0.1	231 (64.0%)	130 (36.0%)	361 (100.0%)
Total	769 (68.2%)	359 (31.8%)	1,128 (100.0%)

Show the frequency of response with concordant therapy by difference stratum.

▼ Code

```
fl_aug |> tabyl(first_adj_concordance, response, margin_equiv_01) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "row")|>
```

```
  adorn_pct_formatting() |>
  adorn_ns(position = "front")
```

```
$`Difference <=0.1`
 first_adj_concordance    Response No Response        Total
            discordant 127 (23.6%) 411 (76.4%) 538 (100.0%)
            concordant  84 (36.7%) 145 (63.3%) 229 (100.0%)
                 Total 211 (27.5%) 556 (72.5%) 767 (100.0%)

$`Difference >0.1`
 first_adj_concordance    Response No Response        Total
            discordant  51 (22.1%) 180 (77.9%) 231 (100.0%)
            concordant  53 (40.8%)  77 (59.2%) 130 (100.0%)
                 Total 104 (28.8%) 257 (71.2%) 361 (100.0%)
```

Then, estimate the stratum-specific odds ratio for response with logistic regression.

▼ Code

```
discord_analysis_sens1b_alt <- fl_aug |>
  group_by(margin_equiv_01) |>
  do(
    tidy(
      glm(
        response_bin ~ first_adj_concordance,
        family = binomial,
        data = .
      ),
      exponentiate = TRUE,
      conf.int = TRUE
    )
  ) |>
  filter(term != "(Intercept)") |>
  transmute(
    `Prob Difference` = margin_equiv_01,
    `Response OR` = estimate,
    Lower = conf.low,
    Upper = conf.high
  ) |>
  ungroup() |>
  select(-margin_equiv_01)

discord_analysis_sens1b_alt |>
  kable(align = c('lccc'),
        caption = "Sensitivity Alternative Approach: Response for Concordance by Difference
         <=0.1",
        digits = 2
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity Alternative Approach: Response for Concordance by Difference <=0.1

PROB DIFFERENCE	RESPONSE OR	LOWER	UPPER
Difference <=0.1	1.87	1.34	2.62
Difference >0.1	2.43	1.52	3.89

14.5.5 Sensitivity: Only Vasopressin and Epinephrine Considered

As noted in Section 7, the 2026 Surviving Sepsis Campaign guidelines do not recommend for or against phenylephrine. Therefore, this sensitivity analysis will only consider vasopressin and epinephrine as eligible decision model-recommended adjunctive vasopressors. The steps in this sensitivity analysis will mirror the approach used in the primary analysis.

First, restrict the sample to only patients receiving vasopressin or epinephrine as the first adjunctive vasopressor (excluding the 57 patients who received phenylephrine).

▼ Code

```
fl_aug_sens <- fl_aug |>
  filter(first_adj == "vasopressin" | first_adj == "epinephrine") |>
  droplevels()
```

Then, determine the decision model-recommended adjunctive vasopressor medication (`rec_first_adj_sens`), defined as the adjunctive vasopressor, whether vasopressin or epinephrine, with the higher predicted probability of response.

▼ Code

```
fl_aug_sens <- fl_aug_sens |>
  rowwise() |>
  mutate(
    best_prob_sens = max(avp_pred_response_prob, epi_pred_response_prob),
    rec_first_adj_sens = factor(case_when(
      best_prob_sens == avp_pred_response_prob ~ "vasopressin",
      best_prob_sens == epi_pred_response_prob ~ "epinephrine",
      .default = "Error"
      ),
      levels = c("vasopressin", "epinephrine")
      )
    ) |>
  ungroup()
```

Show a table of this sensitivity analysis decision model-recommended adjunctive vasopressor medication (`rec_first_adj_sens`).

▼ Code

```
fl_aug_sens |> tabyl(rec_first_adj_sens) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  rename(Recommended = rec_first_adj_sens) |>
  kable(align = c('lcc'),
        caption = "Sensitivity: Decision Model-Recommended Adjunctive Vasopressor"
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Decision Model-Recommended Adjunctive Vasopressor

RECOMMENDED	N	PERCENT
vasopressin	407	38.0%
epinephrine	664	62.0%
Total	1071	100.0%

In this sensitivity analysis, the decision model more commonly recommended epinephrine (62% of cases) than vasopressin (38% of cases). This finding is not surprising in light of the analyses in Section 14.3, which showed higher predicted probability for response with epinephrine compared with vasopressin.

Next, determine concordance between the decision model-recommended therapy (`rec_first_adj_sens`) and the adjunctive vasopressor the patient received during the normal course of care (`first_adj`).

▼ Code

```
fl_aug_sens <- fl_aug_sens |>
  mutate(first_adj_concordance_sens =
           factor(if_else(first_adj == rec_first_adj_sens,
                          "concordant",
                          "discordant"
                          ),
                  levels = c("discordant", "concordant")
                  )
         )

fl_aug_sens |> tabyl(first_adj_concordance_sens) |>
  adorn_totals() |>
  adorn_pct_formatting() |>
  rename(Concordance = first_adj_concordance_sens) |>
  kable(align = c('lcc'),
        caption = "Sensitivity: Concordance of Decision Model Recommended and Observed Adjunct
         (AVP and Epi Only)"
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Concordance of Decision Model Recommended and Observed Adjunct (AVP and Epi Only)

CONCORDANCE	N	PERCENT
discordant	605	56.5%
concordant	466	43.5%
Total	1071	100.0%

In this sensitivity analysis, 43.5% of patients received adjunctive vasopressor therapy concordant with the decision model-recommended therapy (compared with 31.8% of patients in the primary analysis).

Show the observed incidence of response among concordant and discordant cases.

▼ Code

```
fl_aug_sens |> tabyl(response, first_adj_concordance_sens) |>
  adorn_totals(where = c("row", "col")) |>
  adorn_percentages(denominator = "col")|>
  adorn_pct_formatting() |>
  adorn_ns(position = "front") |>
  adorn_title(placement = "combined") |>
  rename("Response (row)/Concordance (col)" = `response/first_adj_concordance_sens`) |>
  kable(align = c('lcc'),
        caption = "Sensitivity: Observed Response Incidence by Decision Model-Recommendation
         Concordance (AVP and Epi Only)"
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Observed Response Incidence by Decision Model-Recommendation Concordance (AVP and Epi Only)

RESPONSE (ROW)/CONCORDANCE (COL)	DISCORDANT	CONCORDANT	TOTAL
Response	137 (22.6%)	158 (33.9%)	295 (27.5%)
No Response	468 (77.4%)	308 (66.1%)	776 (72.5%)
Total	605 (100.0%)	466 (100.0%)	1,071 (100.0%)

Show data similarly in a contingency table (with interpretation below).

▼ Code

```
Epi::twoby2(fl_aug_sens$first_adj_concordance_sens, fl_aug_sens$response)
```

```
2 by 2 table analysis:
------------------------------------------------------
Outcome   : Response
```

```
Comparing : discordant vs. concordant

           Response No Response    P(Response) 95% conf. interval
discordant      137         468         0.2264    0.1948   0.2615
concordant      158         308         0.3391    0.2975   0.3833

                                  95% conf. interval
             Relative Risk:  0.6679    0.5499   0.8111
         Sample Odds Ratio:  0.5706    0.4355   0.7477
Conditional MLE Odds Ratio:  0.5709    0.4315   0.7547
    Probability difference: -0.1126   -0.1669  -0.0583

             Exact P-value: 0.0000
         Asymptotic P-value: 0.0000
------------------------------------------------------
```

Adjunctive vasopressor response was observed in 137 of 605 (22.6%) patients receiving discordant vasopressor therapy and 158 of 466 (33.9%) patients receiving concordant vasopressor therapy (difference in percentages 11.3% [95% CI; 5.8%, 16.7%]).

Determine the association of concordance with adjunctive vasopressor response using logistic regression.

▼ Code

```
discord_analysis_sens4 <-
  glm(response_bin ~ first_adj_concordance_sens,
      data = fl_aug_sens, family = binomial(link = "logit")
      )

tidy(discord_analysis_sens4, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  mutate(term = sub("^first_adj_concordance_sens", "", term)) |>
  select("Term" = term, "OR" = estimate,
         "95% CI, Lower" = conf.low, "95% CI, Upper" = conf.high) |>
  kable(align = c('lcccc'),
        caption = "Sensitivity: Logistic Regression for Adjunctive Vasopressor Response by
         Decision Model Concordance (AVP and Epi Only)",
        digits = 2
        ) |>
  kable_paper(full_width = FALSE)
```

Sensitivity: Logistic Regression for Adjunctive Vasopressor Response by Decision Model Concordance (AVP and Epi Only)

TERM	OR	95% CI, LOWER	95% CI, UPPER
concordant	1.75	1.34	2.3

In this sensitivity analysis where only vasopressin and epinephrine were considered, decision model-concordant adjunctive vasopressor therapy was associated with 75% higher odds of response (OR 1.75 [95% CI; 1.34, 2.30]), which is similar to the findings from the primary analysis (OR 2.05 [95% CI; 1.56, 2.69]).

This analysis complements the subgroup analysis where there was some evidence for effect modification of the association between concordance and response by the decision model-recommended first adjunctive vasopressor, with phenylephrine concordance higher than other agents. When phenylephrine was removed as an adjunctive vasopressor option, concordance remained associated with response.

Assess for effect modification by the recommended adjunct in this sensitivity analysis.

▼ Code

```
fit <- glm(response == "Response" ~ rec_first_adj * first_adj_concordance_sens,
           family = binomial(link = "logit"), data = fl_aug_sens
           )

anova(fit, test = "Chisq")
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: response == "Response"

Terms added sequentially (first to last)

                                        Df Deviance Resid. Df
NULL                                                     1070
rec_first_adj                            2   35.004      1068
first_adj_concordance_sens               1    0.449      1067
rec_first_adj:first_adj_concordance_sens  2    1.690      1065
                                        Resid. Dev  Pr(>Chi)
NULL                                        1260.8
rec_first_adj                               1225.8 2.507e-08 ***
first_adj_concordance_sens                  1225.3    0.5029
rec_first_adj:first_adj_concordance_sens    1223.6    0.4295
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

▼ Code

```
rm(fit)
```

There is no evidence for effect modification of the association between concordance and response by the decision model-recommended first adjunctive vasopressor in this sensitivity analysis of only

vasopressin and phenylephrine (interaction p = 0.43).

14.5.6 Summary Figure

Create a summary table and figure to show the response percentage for concordant and discordant adjunctive vasopressor use and the discordance analysis odds ratio (with 95% CI) in the primary analysis and sensitivity analyses.

▼ Code

```
t1 <- fl_aug |>
  tabyl(first_adj_concordance) |>
  adorn_pct_formatting() |>
  mutate(Analysis = "Primary", .before = percent) |>
  filter(first_adj_concordance == "concordant") |>
  select(Analysis, `Concordance` = percent)

t1_resp <- fl_aug |>
  mutate(first_adj_concordance =
           fct_relevel(first_adj_concordance, "concordant")
         ) |>
  tabyl(response, first_adj_concordance) |>
  adorn_percentages(denominator = "col") |>
  adorn_pct_formatting() |>
  filter(response == "Response") |>
  select("Response if Concordant" = concordant, "Response if Discordant" = discordant)

t1_or <- tidy(discord_analysis, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  select("OR" = estimate, "95% CI, Lower" = conf.low,
         "95% CI, Upper" = conf.high
         )

prim <- cbind(t1, t1_resp, t1_or)

sub1_conc <- fl_aug |>
  group_by(rec_first_adj) |>
  summarize(
    `Concordance` =
      mean(first_adj_concordance == "concordant")*100,
    .groups = "drop"
  ) |>
  mutate(rec_first_adj = case_when(
    rec_first_adj == "vasopressin" ~ "      Vasopressin",
    rec_first_adj == "epinephrine" ~ "      Epinephrine",
    rec_first_adj == "phenylephrine" ~ "      Phenylephrine"
    ),
    `Concordance` = sprintf("%.1f%%", round_half_up(Concordance, 1))
    ) |>
  select(Analysis = rec_first_adj, `Concordance`)
```

```r
sub1_resp <- summary(emm$emmeans) |>
  as.data.frame() |>
  mutate(
    response = sprintf("%.1f%%", 100 * prob)
    ) |>
  pivot_wider(id_cols = rec_first_adj,
              names_from = first_adj_conc_relevl,
              values_from = response
              ) |>
  select(`Response if Concordant` = concordant, `Response if Discordant`= discordant)

sub1_or <- summary(emm$contrasts, infer = c(TRUE, TRUE)) |>
  as.data.frame() |>
  select("OR" = odds.ratio, "95% CI, Lower" = asymp.LCL,
         "95% CI, Upper" = asymp.UCL
         )

sub1 <- cbind(sub1_conc, sub1_resp, sub1_or) |>
  add_row(Analysis = "Subgroup", .before = 1) |>
  add_row(Analysis = paste0("   Recommended Adjunct", .after = 1))

t2 <- fl_aug |>
  tabyl(first_adj_concordance_sens) |>
  adorn_pct_formatting() |>
  mutate(Analysis = "      Difference <=0.05 as Equivalent", .before = percent) |>
  filter(first_adj_concordance_sens == "concordant") |>
  select(Analysis, "Concordance" = percent)

t2_resp <- fl_aug |>
  mutate(first_adj_concordance_sens =
           fct_relevel(first_adj_concordance_sens, "concordant")
         ) |>
  tabyl(response, first_adj_concordance_sens) |>
  adorn_percentages(denominator = "col") |>
  adorn_pct_formatting() |>
  filter(response == "Response") |>
  select("Response if Concordant" = concordant, "Response if Discordant" = discordant)

t2_or <- tidy(discord_analysis_sens1, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  select("OR" = estimate, "95% CI, Lower" = conf.low,
         "95% CI, Upper" = conf.high
         )

sens1 <- cbind(t2, t2_resp, t2_or) |>
  add_row(Analysis = "   Practical Equivalence of Probabilities",
          .before = 1
          )

t2b <- fl_aug |>
```

```
  tabyl(first_adj_concordance_sens2) |>
  adorn_pct_formatting() |>
  mutate(Analysis = "      Difference <=0.1 as Equivalent", .before = percent) |>
  filter(first_adj_concordance_sens2 == "concordant") |>
  select(Analysis, "Concordance" = percent)

t2b_resp <- fl_aug |>
  mutate(first_adj_concordance_sens2 =
           fct_relevel(first_adj_concordance_sens2, "concordant")
         ) |>
  tabyl(response, first_adj_concordance_sens2) |>
  adorn_percentages(denominator = "col") |>
  adorn_pct_formatting() |>
  filter(response == "Response") |>
  select("Response if Concordant" = concordant, "Response if Discordant" = discordant)

t2b_or <- tidy(discord_analysis_sens1b, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  select("OR" = estimate, "95% CI, Lower" = conf.low,
         "95% CI, Upper" = conf.high
         )

sens1b <- cbind(t2b, t2b_resp, t2b_or)

t3 <- fl_aug |>
  group_by(margin_strata) |>
  summarize(
    `Concordance` =
      mean(first_adj_concordance == "concordant")*100,
    .groups = "drop"
  ) |>
  mutate(
    margin_strata = case_when(
      margin_strata == "<0.05"     ~ paste0("      <0.05"),
      margin_strata == "0.05-0.10" ~ paste0("      0.05 to <0.10"),
      margin_strata == "≥0.10"     ~ paste0("      >=0.10")
      ),
    `Concordance` = sprintf("%.1f%%", round_half_up(Concordance, 1))
        ) |>
  select(Analysis = margin_strata, `Concordance`)

t3_resp <- fl_aug |>
  group_by(margin_strata, first_adj_concordance) |>
  summarize(
    response_pct =
      mean(response == "Response")*100,
    .groups = "drop"
  ) |>
  mutate(response_pct = sprintf("%.1f%%", round_half_up(response_pct, 1))) |>
  pivot_wider(id_cols = margin_strata,
              names_from = first_adj_concordance,
```

```
              values_from = response_pct
              ) |>
  select(`Response if Concordant` = concordant, `Response if Discordant`= discordant)

t3_or <- discord_analysis_sens2 |>
  select("OR" = `Response OR`, "95% CI, Lower" = Lower,
         "95% CI, Upper" = Upper
         )

sens2 <- cbind(t3, t3_resp, t3_or) |>
  add_row(Analysis = "   Probability Difference Strata",
          .before = 1
          )

t4 <- fl_aug |>
  group_by(dm_or_quartile) |>
  summarize(
    `Concordance` =
      mean(first_adj_concordance == "concordant")*100,
    .groups = "drop"
  ) |>
  mutate(
    dm_or_quartile = case_when(
      dm_or_quartile == 1 ~ paste0("      Q1 (OR range 1.00 to 1.15)"),
      dm_or_quartile == 2 ~ paste0("      Q2 (OR range >1.15 to 1.36)"),
      dm_or_quartile == 3 ~ paste0("      Q3 (OR range >1.36 to 1.72)"),
      dm_or_quartile == 4 ~ paste0("      Q4 (OR range >1.72 to 5.48)")
      ),
    `Concordance` = sprintf("%.1f%%", round_half_up(Concordance, 1))
        ) |>
  select(Analysis = dm_or_quartile, `Concordance`)

t4_resp <- fl_aug |>
  group_by(dm_or_quartile, first_adj_concordance) |>
  summarize(
    response_pct =
      mean(response == "Response")*100,
    .groups = "drop"
  ) |>
  mutate(response_pct = sprintf("%.1f%%", round_half_up(response_pct, 1))) |>
  pivot_wider(id_cols = dm_or_quartile,
              names_from = first_adj_concordance,
              values_from = response_pct
              ) |>
  select(`Response if Concordant` = concordant, `Response if Discordant`= discordant)

t4_or <- discord_analysis_sens3 |>
  select("OR" = `Response OR`, "95% CI, Lower" = Lower,
         "95% CI, Upper" = Upper
         )
```

```r
sens3 <- cbind(t4, t4_resp, t4_or) |>
  add_row(Analysis = "   Decision Model OR Quartiles",
          .before = 1
          )

t5 <- fl_aug_sens |>
  tabyl(first_adj_concordance_sens) |>
  adorn_pct_formatting() |>
  mutate(Analysis = "   Vasopressin and Epinephrine Only",
         .before = percent
         ) |>
  filter(first_adj_concordance_sens == "concordant") |>
  select(Analysis, "Concordance" = percent)

t5_resp <- fl_aug_sens |>
  mutate(first_adj_concordance_sens =
           fct_relevel(first_adj_concordance_sens, "concordant")
         ) |>
  tabyl(response, first_adj_concordance_sens) |>
  adorn_percentages(denominator = "col") |>
  adorn_pct_formatting() |>
  filter(response == "Response") |>
  select("Response if Concordant" = concordant, "Response if Discordant" = discordant)

t5_or <- tidy(discord_analysis_sens4, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  select("OR" = estimate, "95% CI, Lower" = conf.low,
         "95% CI, Upper" = conf.high
         )

sens4 <- cbind(t5, t5_resp, t5_or)

(sum_fig <- rbind(prim, sub1, sens2, sens3, sens1, sens1b, sens4) |>
  add_row(Analysis = "Sensitivity", .after = 15))
```

	Analysis	Concordance
1	Primary	31.8%
2	Subgroup	<NA>
3	Vasopressin	92.6%
4	Epinephrine	8.0%
5	Phenylephrine	7.1%
6	Recommended Adjunct1	<NA>
7	Probability Difference Strata	<NA>
8	<0.05	26.8%
9	0.05 to <0.10	34.3%
10	>=0.10	36.0%
11	Decision Model OR Quartiles	<NA>
12	Q1 (OR range 1.00 to 1.15)	28.0%
13	Q2 (OR range >1.15 to 1.36)	31.2%
14	Q3 (OR range >1.36 to 1.72)	35.1%

```
15              Q4 (OR range >1.72 to 5.48)       33.0%
16                                Sensitivity        <NA>
17    Practical Equivalence of Probabilities        <NA>
18           Difference <=0.05 as Equivalent       46.5%
19            Difference <=0.1 as Equivalent       61.7%
20          Vasopressin and Epinephrine Only       43.5%
   Response if Concordant Response if Discordant        OR
1                   38.2%                  23.1% 2.0489675
2                    <NA>                   <NA>        NA
3                   39.8%                  41.7% 0.9255556
4                   19.2%                  25.6% 0.6923559
5                   38.2%                  20.5% 2.3954451
6                    <NA>                   <NA>        NA
7                    <NA>                   <NA>        NA
8                   31.4%                  26.0% 1.3042869
9                   42.6%                  19.8% 3.0039339
10                  40.8%                  22.1% 2.4293354
11                   <NA>                   <NA>        NA
12                  31.6%                  30.0% 1.0777171
13                  42.0%                  23.7% 2.3341858
14                  34.3%                  16.9% 2.5647643
15                  44.1%                  21.2% 2.9370192
16                   <NA>                   <NA>        NA
17                   <NA>                   <NA>        NA
18                  35.7%                  21.2% 2.0635200
19                  33.0%                  19.7% 2.0148952
20                  33.9%                  22.6% 1.7523936
   95% CI, Lower 95% CI, Upper
1      1.5621474      2.686711
2             NA            NA
3      0.3980189      2.152293
4      0.2522883      1.900035
5      1.1559157      4.964166
6             NA            NA
7             NA            NA
8      0.8218400      2.048819
9      1.8040196      5.034507
10     1.5230489      3.889368
11            NA            NA
12     0.6088056      1.876679
13     1.3633392      4.001661
14     1.4568242      4.540863
15     1.7183091      5.050365
16            NA            NA
17            NA            NA
18     1.5855542      2.692547
19     1.5202044      2.689440
20     1.3380609      2.298001
```

▼ Code

```
rm(t1, t1_resp, t1_or, prim, sub1_conc, sub1_resp, sub1_or, sub1,
   t2, t2_resp, t2_or, sens1, t2b, t2b_resp, t2b_or, sens1b,
   t3, t3_resp, t3_or, sens2, t4, t4_resp, t4_or, sens3,
   t5, t5_resp, t5_or, sens4
   )
```

Lastly, plot the summary forest plot with `forester()` (exporting as a pdf [not shown]).

▼ Code

```
forester(left_side_data = sum_fig[,1:4],
         estimate = sum_fig$OR,
         ci_low = sum_fig$`95% CI, Lower`,
         ci_high = sum_fig$`95% CI, Upper`,
         ci_sep = ", ",
         estimate_precision = 2,
         estimate_col_name = "Response OR (95% CI)",
         null_line_at = 1,
         ggplot_width = 20,
         display = FALSE,
         dpi = 600,
         xlim = c(0, 6),
         xbreaks = seq(0,6,1),
         font_family = "Helvetica",
         justify = c(0, 0.5, 0.5, 0.5, 0.5),
         file_path = "summary_discord_analysis.pdf",
         render_as = "pdf"
         )
```

```
`height` was translated to `width`.
```

15 Snapshot and Session Info

Call `snapshot()` to update the lockfile with metadata about the currently-used packages in the project library. Then produce a summary report of the session information.

▼ Code

```
#snapshot()
summary(report(sessionInfo()))
```

```
The analysis was done using the R Statistical language (v4.6.0; R
Core Team, 2026) on Red Hat Enterprise Linux 8.10 (Ootpa), using the
packages forester (v0.3.0), ggalluvial (v0.12.6), tailor (v0.1.0),
infer (v1.1.0), DataExplorer (v0.9.0), pmsampsize (v1.1.3), janitor
(v2.2.1), rsample (v1.3.2), workflowsets (v1.1.1), riskRegression
(v2026.3.11), officer (v0.7.4), flextable (v0.9.11), fastshap
(v0.1.1), vip (v0.4.6), lubridate (v1.9.5), Hmisc (v5.2.5), rms
```

(v8.1.1), vroom (v1.7.1), gt (v1.3.0), ggpubr (v0.6.3), survminer (v0.5.2), ggdist (v3.3.3), modeldata (v1.5.1), finetune (v1.3.0), tune (v2.1.0), dials (v1.4.3), parsnip (v1.5.0), yardstick (v1.4.0), tidymodels (v1.5.0), recipes (v1.3.2), emmeans (v2.0.3), report (v0.6.3), shapviz (v0.10.3), tibble (v3.3.1), patchwork (v1.3.2), pmcalibration (v0.2.0), broom (v1.0.12), GGally (v2.4.0), tidytext (v0.4.3), dcurves (v0.5.1), gtsummary (v2.5.0), ggrepel (v0.9.8), survival (v3.8.6), naniar (v1.1.0), renv (v1.2.2), ggh4x (v0.3.1), simputation (v0.2.9), ivs (v0.2.0), workflows (v1.3.0), nnet (v7.3.20), splines2 (v0.5.4), ggplot2 (v4.0.3), conflicted (v1.2.0), forcats (v1.0.1), stringr (v1.6.0), tidyverse (v2.0.0), dplyr (v1.2.1), purrr (v1.2.2), readr (v2.2.0), scales (v1.4.0), tidyr (v1.3.2), ggsci (v5.0.0) and kableExtra (v1.4.0).

16 References

▼ Code

```
cite_packages()
```


- Boyes R (2021). _Forester: An R package for creating publication-ready forest plots._. R package version 0.3.0, <https://github.com/rdboyes/forester>.
- Brunson JC, Read QD (2026). "ggalluvial: Alluvial Plots in 'ggplot2'." R package version 0.12.6, <http://corybrunson.github.io/ggalluvial/>. Brunson JC (2020). "ggalluvial: Layered Grammar for Alluvial Plots." _Journal of Open Source Software_, *5*(49), 2017. doi:10.21105/joss.02017 <https://doi.org/10.21105/joss.02017>.
- Couch S, Frick H, HvitFeldt E, Kuhn M (2025). _tailor: Iterative Steps for Postprocessing Model Predictions_. R package version 0.1.0, <https://github.com/tidymodels/tailor>.
- Couch SP, Bray AP, Ismay C, Chasnovski E, Baumer BS, Çetinkaya-Rundel M (2021). "infer: An R package for tidyverse-friendly statistical inference." _Journal of Open Source Software_, *6*(65), 3661. doi:10.21105/joss.03661 <https://doi.org/10.21105/joss.03661>.
- Cui B (2026). _DataExplorer: Automate Data Exploration and Treatment_. R package version 0.9.0, <http://boxuancui.github.io/DataExplorer/>.
- Ensor J (2023). _pmsampsize: Sample Size for Development of a Prediction Model_. R package version 1.1.3.
- Firke S (2024). _janitor: Simple Tools for Examining and Cleaning Dirty Data_. R package version 2.2.1, <https://github.com/sfirke/janitor>.
- Frick H, Chow F, Kuhn M, Mahoney M, Silge J, Wickham H (2026). _rsample: General Resampling Infrastructure_. R package version 1.3.2, <https://rsample.tidymodels.org>.
- Frick H, Kuhn M, Couch S (2025). _workflowsets: Create a Collection of 'tidymodels' Workflows_. R package version 1.1.1, <https://github.com/tidymodels/workflowsets>.
- Gerds T, Ohlendorff J, Ozenne B (2026). _riskRegression: Risk Regression Models and Prediction Scores for Survival Analysis with Competing Risks_. R package version 2026.03.11, <https://github.com/tagteam/riskRegression>. Gerds T, Kattan M (2021). _Medical Risk Prediction Models: With Ties to Machine Learning (1st ed.)_. Chapman and Hall/CRC. doi:10.1201/9781138384484 <https://doi.org/10.1201/9781138384484>.
- Gohel D, Moog S, Heckmann M (2026). _officer: Manipulation of Microsoft Word and PowerPoint Documents_. R package version 0.7.4, <https://ardata-fr.github.io/officeverse/>.
- Gohel D, Skintzos P (2026). _flextable: Functions for Tabular Reporting_. R package version

0.9.11, <https://ardata-fr.github.io/flextable-book/>.
- Greenwell B (2024). _fastshap: Fast Approximate Shapley Values_. R package version 0.1.1, <https://github.com/bgreenwell/fastshap>.
- Greenwell BM, Boehmke BC (2020). "Variable Importance Plots-An Introduction to the vip Package." _The R Journal_, *12*(1), 343-366. <https://doi.org/10.32614/RJ-2020-013>.
- Grolemund G, Wickham H (2011). "Dates and Times Made Easy with lubridate." _Journal of Statistical Software_, *40*(3), 1-25. <https://www.jstatsoft.org/v40/i03/>.
- Harrell Jr F (2026). _Hmisc: Harrell Miscellaneous_. R package version 5.2-5, <https://hbiostat.org/R/Hmisc/>.
- Harrell Jr F (2026). _rms: Regression Modeling Strategies_. R package version 8.1-1, <https://hbiostat.org/R/rms/>.
- Hester J, Wickham H, Bryan J (2026). _vroom: Read and Write Rectangular Text Data Quickly_. R package version 1.7.1, <https://vroom.tidyverse.org>.
- Iannone R, Cheng J, Schloerke B, Haughton S, Hughes E, Lauer A, François R, Seo J, Brevoort K, Roy O (2026). _gt: Easily Create Presentation-Ready Display Tables_. R package version 1.3.0, <https://gt.rstudio.com>.
- Kassambara A (2026). _ggpubr: 'ggplot2' Based Publication Ready Plots_. R package version 0.6.3, <https://rpkgs.datanovia.com/ggpubr/>.
- Kassambara A, Kosinski M, Biecek P (2026). _survminer: Drawing Survival Curves using 'ggplot2'_. R package version 0.5.2, <https://rpkgs.datanovia.com/survminer/index.html>.
- Kay M (2024). "ggdist: Visualizations of Distributions and Uncertainty in the Grammar of Graphics." _IEEE Transactions on Visualization and Computer Graphics_, *30*(1), 414-424. doi:10.1109/TVCG.2023.3327195 <https://doi.org/10.1109/TVCG.2023.3327195>. Kay M (2025). _ggdist: Visualizations of Distributions and Uncertainty_. doi:10.5281/zenodo.3879620 <https://doi.org/10.5281/zenodo.3879620>. R package version 3.3.3, <https://mjskay.github.io/ggdist/>.
- Kuhn M (2025). _modeldata: Data Sets Useful for Modeling Examples_. R package version 1.5.1, <https://modeldata.tidymodels.org>.
- Kuhn M (2026). _finetune: Additional Functions for Model Tuning_. R package version 1.3.0, <https://github.com/tidymodels/finetune>.
- Kuhn M (2026). _tune: Tidy Tuning Tools_. R package version 2.1.0, <https://tune.tidymodels.org/>.
- Kuhn M, Frick H (2026). _dials: Tools for Creating Tuning Parameter Values_. R package version 1.4.3, <https://dials.tidymodels.org>.
- Kuhn M, Vaughan D (2026). _parsnip: A Common API to Modeling and Analysis Functions_. R package version 1.5.0, <https://github.com/tidymodels/parsnip>.
- Kuhn M, Vaughan D, Hvitfeldt E (2026). _yardstick: Tidy Characterizations of Model Performance_. R package version 1.4.0, <https://github.com/tidymodels/yardstick>.
- Kuhn M, Wickham H (2020). _Tidymodels: a collection of packages for modeling and machine learning using tidyverse principles._. <https://www.tidymodels.org>.
- Kuhn M, Wickham H, Hvitfeldt E (2026). _recipes: Preprocessing and Feature Engineering Steps for Modeling_. R package version 1.3.2, <https://github.com/tidymodels/recipes>.
- Lenth R, Piaskowski J (2026). _emmeans: Estimated Marginal Means, aka Least-Squares Means_. R package version 2.0.3, <https://rvlenth.github.io/emmeans/>.
- Makowski D, Lüdecke D, Patil I, Thériault R, Ben-Shachar M, Wiernik B (2023). "Automated Results Reporting as a Practical Tool to Improve Reproducibility and Methodological Best Practices Adoption." _CRAN_. doi:10.32614/CRAN.package.report <https://doi.org/10.32614/CRAN.package.report>. <https://easystats.github.io/report/>.
- Mayer M (2025). _shapviz: SHAP Visualizations_. R package version 0.10.3, <https://github.com/ModelOriented/shapviz>.
- Müller K, Wickham H (2026). _tibble: Simple Data Frames_. R package version 3.3.1,

<https://tibble.tidyverse.org/>.

- Pedersen T (2025). _patchwork: The Composer of Plots_. R package version 1.3.2, <https://patchwork.data-imaginist.com>.

- R Core Team (2026). _R: A Language and Environment for Statistical Computing_. R Foundation for Statistical Computing, Vienna, Austria. doi:10.32614/R.manuals <https://doi.org/10.32614/R.manuals>. <https://www.R-project.org/>.

- Rhodes S (2025). _pmcalibration: Calibration curves for clinical prediction models_. R package version 0.2.0, <https://github.com/stephenrho/pmcalibration>.

- Robinson D, Hayes A, Couch S, Hvitfeldt E (2026). _broom: Convert Statistical Objects into Tidy Tibbles_. R package version 1.0.12, <https://broom.tidymodels.org/>.

- Schloerke B, Cook D, Larmarange J, Briatte F, Marbach M, Thoen E, Elberg A, Crowley J (2025). _GGally: Extension to 'ggplot2'_. R package version 2.4.0, <https://ggobi.github.io/ggally/>.

- Silge J, Robinson D (2016). "tidytext: Text Mining and Analysis Using Tidy Data Principles in R." _JOSS_, *1*(3). doi:10.21105/joss.00037 <https://doi.org/10.21105/joss.00037>. <http://dx.doi.org/10.21105/joss.00037>.

- Sjoberg D (2025). _dcurves: Decision Curve Analysis for Model Evaluation_. R package version 0.5.1, <https://github.com/ddsjoberg/dcurves>.

- Sjoberg D, Whiting K, Curry M, Lavery J, Larmarange J (2021). "Reproducible Summary Tables with the gtsummary Package." _The R Journal_, *13*, 570-580. doi:10.32614/RJ-2021-053 <https://doi.org/10.32614/RJ-2021-053>. <https://doi.org/10.32614/RJ-2021-053>.

- Slowikowski K (2026). _ggrepel: Automatically Position Non-Overlapping Text Labels with 'ggplot2'_. R package version 0.9.8, <https://ggrepel.slowkow.com/>.

- Therneau T (2026). _A Package for Survival Analysis in R_. R package version 3.8-6, <https://CRAN.R-project.org/package=survival>. Terry M. Therneau, Patricia M. Grambsch (2000). _Modeling Survival Data: Extending the Cox Model_. Springer, New York. ISBN 0-387-98784-3.

- Tierney N, Cook D (2023). "Expanding Tidy Data Principles to Facilitate Missing Data Exploration, Visualization and Assessment of Imputations." _Journal of Statistical Software_, *105*(7), 1-31. doi:10.18637/jss.v105.i07 <https://doi.org/10.18637/jss.v105.i07>.

- Ushey K, Wickham H (2026). _renv: Project Environments_. R package version 1.2.2, <https://rstudio.github.io/renv/>.

- van den Brand T (2025). _ggh4x: Hacks for 'ggplot2'_. R package version 0.3.1, <https://github.com/teunbrand/ggh4x>.

- van der Loo M (2024). _simputation: Simple Imputation_. R package version 0.2.9, <https://github.com/markvanderloo/simputation>.

- Vaughan D (2023). _ivs: Interval Vectors_. R package version 0.2.0, <https://github.com/DavisVaughan/ivs>.

- Vaughan D, Couch S, Frick H (2025). _workflows: Modeling Workflows_. R package version 1.3.0, <https://github.com/tidymodels/workflows>.

- Venables WN, Ripley BD (2002). _Modern Applied Statistics with S_, Fourth edition. Springer, New York. ISBN 0-387-95457-0, <https://www.stats.ox.ac.uk/pub/MASS4/>.

- Wang W, Yan J (2025). _splines2: Regression Spline Functions and Classes_. R package version 0.5.4, <https://CRAN.R-project.org/package=splines2>. Wang W, Yan J (2021). "Shape-Restricted Regression Splines with R Package splines2." _Journal of Data Science_, *19*(3), 498-517. doi:10.6339/21-JDS1020 <https://doi.org/10.6339/21-JDS1020>.

- Wickham H (2016). _ggplot2: Elegant Graphics for Data Analysis_. Springer-Verlag New York. ISBN 978-3-319-24277-4. <https://ggplot2.tidyverse.org>.

- Wickham H (2023). _conflicted: An Alternative Conflict Resolution Strategy_. R package version 1.2.0, <https://conflicted.r-lib.org/>.

- Wickham H (2025). _forcats: Tools for Working with Categorical Variables (Factors)_. R package version 1.0.1, <https://forcats.tidyverse.org/>.

- Wickham H (2025). _stringr: Simple, Consistent Wrappers for Common String Operations_. R

package version 1.6.0, <https://stringr.tidyverse.org>.
  - Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R, Grolemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E, Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi K, Vaughan D, Wilke C, Woo K, Yutani H (2019). "Welcome to the tidyverse." _Journal of Open Source Software_, *4*(43), 1686. doi:10.21105/joss.01686 <https://doi.org/10.21105/joss.01686>.
  - Wickham H, François R, Henry L, Müller K, Vaughan D (2026). _dplyr: A Grammar of Data Manipulation_. R package version 1.2.1, <https://dplyr.tidyverse.org>.
  - Wickham H, Henry L (2026). _purrr: Functional Programming Tools_. R package version 1.2.2, <https://purrr.tidyverse.org/>.
  - Wickham H, Hester J, Bryan J (2026). _readr: Read Rectangular Text Data_. R package version 2.2.0, <https://readr.tidyverse.org>.
  - Wickham H, Pedersen T, Seidel D (2025). _scales: Scale Functions for Visualization_. R package version 1.4.0, <https://scales.r-lib.org>.
  - Wickham H, Vaughan D, Girlich M (2025). _tidyr: Tidy Messy Data_. R package version 1.3.2, <https://tidyr.tidyverse.org>.
  - Xiao N (2026). _ggsci: Scientific Journal and Sci-Fi Themed Color Palettes for 'ggplot2'_. R package version 5.0.0, <https://nanx.me/ggsci/>.
  - Zhu H (2024). _kableExtra: Construct Complex Table with 'kable' and Pipe Syntax_. R package version 1.4.0, <http://haozhu233.github.io/kableExtra/>.